

The Tangled Web. Руководство по защите современных веб-приложений

Русский перевод книги The Tangled Web о безопасности браузеров и защите современных веб-приложений.

Больше крутых материалов в моём телеграм-канале [@grogrigs](#)

Предварительные материалы

Отзывы о *The Tangled Web*

"Подробный и всесторонний охват от одного из ведущих экспертов по безопасности браузеров."

Tavis Ormandy, Google Inc.

"Обязательное чтение для каждого, кто ценит свою безопасность и приватность в интернете."

Collin Jackson, исследователь в Carnegie Mellon Web Security Group.

"Возможно, самый подробный и проницательный трактат о состоянии безопасности технологий, движимых Web, на сегодняшний день. Необходимо иметь!"

Mark Dowd, Azimuth Security, автор *The Art of Software Security Assessment*.

Отзывы о *Silence on the Wire* Михала Залевски

"Одна из самых инновационных и оригинальных книг о компьютерах из доступных."

Richard Bejtlich, TaoSecurity.

"Для настоящего специалиста по информационной безопасности эта книга - чистое золото."

Mitch Tulloch, Windows Security.

"Объяснения Залевски ясно показывают, что он один из лучших в отрасли."

Computerworld.

"Количество деталей поразительно для такого небольшого тома, а примеры великолепны... После прочтения этой книги вы точно будете думать иначе."

(IN)SECURE Magazine.

"Безусловно на голову выше других подобных книг о безопасности."

Linux User & Developer.

Титульная страница

The Tangled Web

A Guide to Securing Modern Web Applications

Michal Zalewski

San Francisco

Выходные сведения

The Tangled Web. Copyright © 2012 by Michal Zalewski.

Все права защищены. Никакая часть этой работы не может быть воспроизведена или передана в какой-либо форме и какими-либо средствами, электронными или механическими, включая фотокопирование, запись или любые системы хранения и извлечения информации, без предварительного письменного разрешения владельца авторских прав и издателя.

15 14 13 12 11 1 2 3 4 5 6 7 8 9

ISBN-10: 1-59327-388-6

ISBN-13: 978-1-59327-388-0

Издатель: William Pollock

Выпускающий редактор: Serena Yang

Иллюстрация на обложке: Hugh D'Andrade

Дизайн внутреннего оформления: Octopod Studios

Редактор-разработчик: William Pollock

Технический рецензент: Chris Evans

Литературный редактор: Paula L. Fleming

Верстальщик: Serena Yang

Корректор: Ward Webber

Составитель указателя: Nancy Guenther

За информацией о распространителях книги или переводах обращайтесь непосредственно в No Starch Press, Inc.:

No Starch Press, Inc.

38 Ringold Street, San Francisco, CA 94103

телефон: 415.863.9900; факс: 415.863.9950; info@nostarch.com; www.nostarch.com

Каталожные данные Библиотеки Конгресса

Zalewski, Michal.

The Tangled Web: a guide to securing modern Web applications / Michal Zalewski.

p. cm.

Включает библиографические ссылки и указатель.

ISBN-13: 978-1-59327-388-0 (pbk.)

ISBN-10: 1-59327-388-6 (pbk.)

1. Компьютерные сети - меры безопасности. 2. Браузеры (компьютерные программы). 3.
Компьютерная безопасность. I. Заглавие.

TK5105.59.Z354 2011

005.8--dc23

2011039636

No Starch Press и логотип No Starch Press являются зарегистрированными товарными знаками No Starch Press, Inc. "The Book of" является товарным знаком No Starch Press, Inc. Другие названия продуктов и компаний, упомянутые здесь, могут быть товарными знаками соответствующих владельцев. Вместо того чтобы использовать символ товарного знака при каждом появлении охраняемого имени, мы используем эти имена только редакционно и в интересах владельца товарного знака, без намерения нарушить права на товарный знак.

Информация в этой книге распространяется на условиях "as is", без гарантии. Хотя при подготовке этой работы были приняты все меры предосторожности, ни автор, ни No Starch Press, Inc. не несут ответственности перед каким-либо лицом или организацией за любые потери или ущерб, вызванные или предположительно вызванные прямо или косвенно информацией, содержащейся в ней.

Посвящение

Моему сыну.

Предисловие

Всего пятнадцать лет назад Web был настолько же прост, насколько и неважен: причудливый механизм, позволявший горстке студентов и кучке асоциальных обитателей подвалов заходить на домашние страницы друг друга, посвященные науке, домашним животным или поэзии. Сегодня это предпочтительная платформа для написания сложных интерактивных приложений - от почтовых клиентов до редакторов изображений и компьютерных игр - и среда, достигающая сотен миллионов обычных пользователей по всему миру. Это также важнейший инструмент коммерции, настолько значимый, что ему приписывают вклад в рецессию, когда в 1999-2001 годах лопнул пузырь доткомов.

Этот переход от неизвестности к повсеместности был удивительно быстрым даже по меркам, к которым мы привыкли в нынешнюю информационную эпоху, и скорость этого взлета принесла с собой неожиданную проблему. Проектные изъяны и недостатки реализации World Wide Web - это изъяны технологии, которая никогда не стремилась к своему нынешнему статусу и никогда не имела возможности остановиться и оглянуться на прошлые ошибки. Возникшие проблемы быстро стали одними из самых значительных и распространенных угроз безопасности данных сегодня: как оказалось, стандарты проектирования протоколов, которые можно применить к черно-серой домашней странице, полной танцующих хомячков, не обязательно подходят интернет-магазину, обрабатывающему миллионы транзакций по кредитным картам каждый год.

Оглядываясь на последнее десятилетие, трудно не испытывать легкого разочарования: почти каждое заметное онлайн-приложение, придуманное до сих пор, вынуждено было заплатить цену за

углы, срезанные в ранние дни Web. Черт возьми, xssed.com, сайт, посвященный отслеживанию узкого подмножества связанных с Web сбоев безопасности, накопил около 50 000 записей примерно за три года работы. Тем не менее поставщики браузеров в основном невозмутимы, а само сообщество безопасности предложило мало понимания или советов о том, как справляться с этим повсеместным бедствием. Вместо этого многие эксперты по безопасности продолжают строить византийские таксономии уязвимостей и привычно, но расплывчато заламывать руки из-за предполагаемых причин этой неразберихи.

Отчасти проблема в том, что эти эксперты долго пренебрежительно относились ко всей шумихе вокруг веб-безопасности, не понимая, в чем ее суть. Они быстро приклеивали к изъяснам веб-безопасности ярлык тривиальных проявлений проблемы confused deputy^[1] или какого-нибудь другого броского термина, описанного в отраслевом журнале три десятилетия назад. И зачем им вообще заботиться о веб-безопасности? Каково значение непристойного комментария, внедренного на скучную домашнюю страницу о домашних питомцах, по сравнению с серьезностью традиционной уязвимости компрометации системы?

Оглядываясь назад, я почти уверен, что большинство из нас теперь прикусывает язык. Web не только оказался намного важнее, чем ожидалось изначально, но мы еще и не обратили внимания на несколько фундаментальных характеристик, которые выводят его далеко за пределы нашей зоны комфорта. В конце концов, даже лучшие по дизайну и самые тщательно проверенные веб-приложения имеют намного больше проблем и намного чаще, чем их не-веб аналоги.

Мы все напортачили, и пора покаяться. В интересах покаяния *The Tangled Web* пытается сделать небольшой шаг к столь необходимой нормальности и, как таковая, может быть первой публикацией, предлагающей систематический и подробный анализ нынешнего положения дел в мире безопасности веб-приложений. В процессе книга стремится пролить свет на уникальность вызовов безопасности, с которыми мы - инженеры безопасности, веб-разработчики и пользователи - сталкиваемся каждый день.

Композиция этой книги сосредоточена на исследовании некоторых наиболее заметных высокоуровневых строительных блоков браузера и различных тем, важных для безопасности, вытекающих из этого рассказа. Я выбрал такой подход, потому что он кажется более информативным и интуитивным, чем простое перечисление проблем с помощью произвольно выбранной таксономии, как это делается во многих других книгах по информационной безопасности. Надеюсь также, что этот подход сделает *The Tangled Web* более приятным чтением.

Для читателей, которым нужны быстрые ответы, я решил включить краткие инженерные памятки в конце многих глав. Эти памятки описывают разумные подходы к некоторым из самых распространенных проблем, встречающихся при проектировании веб-приложений. Кроме того, заключительная часть книги предлагает краткий глоссарий известных уязвимостей реализации, с которыми можно столкнуться.

Благодарности

Многие части *The Tangled Web* уходят корнями в исследование, выполненное для *Browser Security Handbook* от Google, технической wiki, которую я собрал в 2008 году и публично выпустил под лицензией Creative Commons. Оригинальный документ можно посмотреть онлайн по адресу <http://code.google.com/p/browsersec/>.

Мне повезло работать в компании, которая позволила мне заняться этим проектом, и я рад работать с рядом талантливых коллег, которые дали отличные замечания, чтобы сделать *Browser Security Handbook* полезнее и точнее. В частности, спасибо Filipe Almeida, Drew Hintz, Marius Schilder и Parisa Tabriz за помощь.

Я также горжусь тем, что стою на плечах гигантов. Эта книга многим обязана исследованиям браузерной безопасности, выполненным участниками сообщества информационной безопасности. Особая благодарность Adam Barth, Collin Jackson, Chris Evans, Jesse Ruderman, Billy Rios и Eduardo Vela Nava за продвижение нашего понимания этой области.

Спасибо всем - и продолжайте хорошую работу.

Сноски

[1] [\[назад\]](#) Проблема confused deputy - общий концепт информационной безопасности, используемый для обозначения широкого класса изъянов проектирования или реализации. Термин описывает любой вектор, позволяющий злоумышленнику обманом заставить программу неправильно использовать некие "полномочия" (привилегии доступа) для манипулирования ресурсом непредусмотренным образом, предположительно выгодным злоумышленнику, как бы эта выгода ни определялась. Выражение "confused deputy" регулярно используют исследователи безопасности в академической среде, но поскольку практически все реальные проблемы безопасности можно поместить в эту корзину на некотором уровне абстракции, термин почти лишен смысла.

Часть I. Анатомия Web

Первая часть этой книги сосредоточена на основных концепциях, управляющих работой веб-браузеров, а именно на протоколах, форматах документов и языках программирования, благодаря которым все это работает. Поскольку все знакомые пользователю механизмы безопасности, применяемые в современных браузерах, глубоко переплетены с этим внутренним устройством, базовые внутренние механизмы заслуживают немалого внимания, прежде чем мы уйдем глубже в лес.

Глава 1. Безопасность в мире веб-приложений

Чтобы задать правильный контекст для технических обсуждений в последующих главах, сначала разумно объяснить, чего пытаются добиться область инженерии безопасности, а затем очертить, почему в этом, казалось бы, хорошо изученном контексте веб-приложения заслуживают особого отношения. Итак, начнем?

Информационная безопасность в двух словах

На первый взгляд область информационной безопасности кажется зрелой, четко определенной и успешной ветвью компьютерных наук. Постоянные эксперты охотно подчеркивают важность своей области знаний, указывая на большие наборы аккуратно каталогизированных уязвимостей, которые неизменно приписываются неграмотным в вопросах безопасности разработчикам, а их коллеги-теоретики тем временем объясняют, что всех этих проблем можно было бы избежать, если бы применялась самая модная в этом году методология безопасности.

Рядом процветает коммерческая отрасль, предлагающая разнообразные ни к чему не обязывающие гарантии безопасности всем - от обычных пользователей компьютеров до гигантских международных корпораций.

И все же за несколько десятилетий мы, по сути, полностью провалили задачу создания хотя бы самых элементарных пригодных к применению основ для понимания и оценки безопасности современного программного обеспечения. За исключением нескольких блестящих трактатов и экспериментов ограниченного масштаба, у нас даже нет реальных историй успеха, которыми можно было бы поделиться. Внимание почти исключительно сосредоточено на реактивных, вторичных мерах безопасности, таких как управление уязвимостями, обнаружение вредоносного ПО и атак, песочницы и тому подобное, а также, возможно, на выборочном указании на ошибки в чужом коде. Раздражающий, ревниво охраняемый секрет состоит в том, что, когда речь заходит о помощи другим людям в разработке безопасных систем, мы приносим куда меньше пользы, чем от нас следовало бы ожидать; современный Web не является исключением.

Давайте рассмотрим некоторые из самых привлекательных подходов к обеспечению информационной безопасности и попробуем понять, почему они так и не изменили ситуацию.

Заигрывание с формальными решениями

Пожалуй, самый очевидный инструмент для создания безопасных программ - алгоритмически доказать, что они ведут себя именно так, как нужно. Это простая предпосылка, которая интуитивно должна казаться осуществимой. Так почему же этот подход почти ничего нам не дал?

Что ж, начнем с самого прилагательного "безопасный": что именно оно должно означать? Безопасность кажется интуитивно понятным понятием, но в мире вычислений она ускользает от всех попыток дать ей полезное определение. Конечно, можно переформулировать проблему броскими, но в основном бесполезными способами, однако о наличии проблемы говорит уже то, что одно из определений, наиболее часто цитируемых практиками,^[1] звучит так:

Система безопасна, если она ведет себя строго тем способом, который был предусмотрен, - и не делает ничего сверх этого.

Это аккуратное определение, которое смутно очерчивает абстрактную цель, но оно почти ничего не говорит о том, как этой цели достичь. Это компьютерная наука, однако по степени конкретности она поразительно напоминает стихотворение Виктора Гюго:

Любовь - это часть самой души, и она той же природы, что небесное дыхание атмосферы рая.

Можно возразить, что не у практиков следует просить тонких определений, но задайте тот же вопрос группе ученых - и они предложат вам примерно тот же ответ. Например, следующее распространенное академическое определение восходит к модели безопасности Белла - ЛаПадулы, опубликованной в 1960-х годах. Это была одна примерно из дюжины попыток формализовать требования к безопасным системам, в данном случае в терминах конечного автомата;¹ кроме того, это одна из наиболее заметных попыток.

Система безопасна тогда и только тогда, когда она начинает работу в безопасном состоянии и не может перейти в небезопасное состояние.

Определения такого рода, конечно, фундаментально верны и могут служить основой для диссертаций или даже пары государственных грантов. Но на практике модели, построенные на таких основаниях, почти наверняка оказываются практически бесполезными для обобщенной инженерии реального программного обеспечения по меньшей мере по трем причинам:

- Невозможно определить желательное поведение для достаточно сложной компьютерной системы. Ни один орган не способен определить, каким должен быть "предусмотренный способ" или "безопасные состояния" для операционной системы или веб-браузера. Интересы пользователей, владельцев систем, поставщиков данных, владельцев бизнес-процессов, а также производителей программного и аппаратного обеспечения обычно существенно различаются и быстро меняются - если заинтересованные стороны вообще способны и готовы с самого начала ясно и честно раскрывать свои интересы. В довершение всего социология и теория игр подсказывают, что простое суммирование этих частных интересов может вовсе не привести к полезному результату. Эта дилемма, известная как "трагедия общин", лежит в центре многих споров о будущем Интернета.
- Желания не отображаются автоматически в формальные ограничения. Даже если мы сможем достичь идеального высокоуровневого соглашения о том, как система должна вести себя в некотором подмножестве случаев, такие ожидания почти невозможно формализовать как набор допустимых входных данных, состояний программы и переходов между состояниями, а это предварительное условие почти для любого вида формального анализа. Проще говоря, интуитивные понятия вроде "я не хочу, чтобы мою почту читали другие" плохо переводятся на язык математических моделей. Существует несколько экзотических подходов, позволяющих хотя бы частично формализовать такие расплывчатые требования, но они накладывают серьезные ограничения на процессы разработки ПО и часто приводят к наборам правил и моделям, которые намного сложнее самих проверяемых алгоритмов. А затем, в свою очередь, уже их корректность, вероятно, придется доказывать... *ad infinitum*.
- Поведение программного обеспечения очень трудно проанализировать окончательно. Статический анализ компьютерных программ с целью доказать, что они всегда будут вести себя согласно подробной спецификации, - задача, которую пока никому не удалось убедительно продемонстрировать в сложных реальных сценариях, хотя, как можно ожидать, ограниченный успех в сильно суженных условиях или при очень узких целях возможен. Многие случаи, вероятно, невозможно решить на практике из-за вычислительной сложности, а некоторые могут оказаться полностью неразрешимыми из-за проблемы остановки.^[2]

Возможно, еще больше, чем расплывчатость и бесполезность ранних определений, раздражает то, что за прошедшие десятилетия почти не было достигнуто прогресса на пути к чему-то лучшему. Фактически академическая статья, опубликованная в 2001 году Военно-морской исследовательской лабораторией США, отстает от части ранних работ и приходит к куда более неформальному, перечислительному определению безопасности программного обеспечения - определению, которое прямо признает собственные несовершенство и неполноту.²

Система безопасна, если она надлежащим образом защищает обрабатываемую ею информацию от несанкционированного раскрытия, несанкционированного изменения и несанкционированного удержания, также называемого отказом в обслуживании. Мы говорим "надлежащим образом", потому что ни одна практическая система не может достичь этих целей без оговорок; безопасность по своей природе относительна.

В той же статье дается ретроспективная оценка более ранних усилий и неприятных жертв, принесенных ради сохранения теоретической чистоты этих моделей:

Опыт показал, что, с одной стороны, аксиомы модели Белла - ЛаПадулы чрезмерно ограничительны: они запрещают операции, которые требуются пользователям в практических приложениях. С другой стороны, доверенные субъекты, механизм, предоставленный для обхода некоторых из этих ограничений, ограничены недостаточно строго... В результате разработчикам приходилось создавать специальные спецификации желательного поведения доверенных процессов в каждой отдельной системе.

В конечном счете, независимо от количества предложенных изящных конкурирующих моделей, все попытки понять и оценить безопасность реального программного обеспечения на алгоритмических основаниях, похоже, обречены на провал. Это оставляет разработчиков и экспертов по безопасности без метода, который позволил бы делать авторитетные, ориентированные в будущее заявления о качестве созданного кода. Какие еще варианты остаются?

Вступает управление рисками

При отсутствии формальных гарантий и доказуемых метрик, а также с учетом пугающей распространенности уязвимостей в ключевом программном обеспечении, на которое опираются современные общества, бизнесы устремляются к другой броской концепции: управлению рисками.

Идея управления рисками, успешно применяемая в страховом бизнесе и, пожалуй, с меньшим успехом в финансовом мире, просто утверждает, что владельцы систем должны научиться жить с уязвимостями, которые невозможно устранить экономически оправданным способом, и в целом должны масштабировать усилия по следующей формуле:

$$\text{risk} = \text{probability of an event} \times \text{maximum loss}$$

Например, согласно этой доктрине, если компрометация какой-нибудь неважной рабочей станции раз в год будет стоить компании не больше \$1,000 потерянной производительности, организации следует просто заложить этот убыток в бюджет и жить дальше, а не тратить, скажем, \$100,000 на дополнительные меры безопасности или планы резервирования и мониторинга для предотвращения потери. Согласно доктрине управления рисками, эти деньги лучше потратить на изоляцию, защиту и мониторинг критически важного мейнфрейма, который формирует платежные записи для всех клиентов.

Разумеется, расставлять приоритеты в усилиях по безопасности разумно. Проблема в том, что, когда управление рисками выполняется строго по цифрам, оно почти не помогает понять, сдерживать реальные проблемы и управлять ими. Вместо этого оно вводит опасное заблуждение: будто

структурированная недостаточность почти так же хороша, как достаточность, и будто недофинансированные усилия по безопасности плюс управление рисками примерно равноценны должным образом профинансированной работе по безопасности.

Ага, как же. Ничего подобного.

- Во взаимосвязанных системах убытки не ограничены сверху и не привязаны к активу. Строгое управление рисками зависит от способности оценить типичную и максимальную стоимость, связанную с компрометацией ресурса. К сожалению, сделать это можно только проигнорировав тот факт, что многие из самых впечатляющих инцидентов безопасности, такие как атаки на TJX^[3] или Microsoft,^[4] начинались с относительно неважных и запущенных точек входа. Эти первоначальные вторжения вскоре развивались и в конечном счете приводили к почти полной компрометации критически важной инфраструктуры, по пути обходя любую поверхностную сетевую сегментацию. В типичном управлении рисками "по цифрам" первоначальной точке входа назначается меньший вес, потому что по сравнению с другими узлами она имеет низкую ценность. Аналогично внутренний путь эскалации к более чувствительным ресурсам недооценивается как имеющий низкую вероятность использования. И все же пренебрежение и тем и другим оказывается взрывоопасной смесью.
- Неденежные издержки вторжений трудно компенсировать ценностью, создаваемой исправными системами. Потерю доверия пользователей и непрерывности бизнеса, а также перспективу судебных исков и риск внимания регуляторов трудно осмысленно застраховать. Эти последствия, по крайней мере в принципе, могут погубить компании или даже целые отрасли, а любые поверхностные оценки таких исходов почти полностью спекулятивны.
- Имеющиеся данные, вероятно, не отражают будущие риски. В отличие от участников мелкого ДТП, злоумышленники не выйдут вперед, чтобы любезно сообщить о взломах, и не будут исчерпывающе документировать причиненный ущерб. Если вторжение не становится мучительно очевидным из-за небрежности злоумышленника или его разрушительных намерений, оно часто остается незамеченным. Даже если доступны отраслевые данные, основанные на самоотчетах, просто нет надежного способа понять, насколько они полны и какой дополнительный риск вносит текущая практика конкретного бизнеса.
- Статистическое прогнозирование не является надежным предсказателем отдельных исходов. То, что в среднем людей в городах чаще поражает молния, чем калечит медведь, не означает, что вам следует прикрутить громоотвод к шляпе, а затем искупаться в меде. Вероятность того, что компрометация будет связана с конкретным компонентом, на индивидуальном уровне в основном несущественна: инциденты безопасности почти неизбежны, но из тысяч выставленных наружу нетривиальных ресурсов любой сервис может быть использован как вектор атаки - и ни один отдельно взятый сервис, скорее всего, не увидит такого объема событий, который сделал бы статистическое прогнозирование осмысленным в масштабе одной компании.

Просветление через таксономию

Две школы мысли, рассмотренные выше, имеют нечто общее: обе предполагают, что безопасность можно определить как набор вычислимых целей, а получившаяся единая теория безопасной системы или модель приемлемого риска затем изящно спустится вниз, породив оптимальный набор низкоуровневых действий, необходимых для достижения совершенства в проектировании приложений.

Некоторые практики проповедуют противоположный подход, который меньше обязан философии и больше - естественным наукам. Эти практики утверждают, что, подобно Чарльзу Дарвину информационной эпохи, собирая достаточные объемы низкоуровневых экспериментальных данных, мы сможем наблюдать, реконструировать и документировать все более сложные законы, чтобы прийти к некой единой модели безопасных вычислений.

Это последнее мировоззрение приводит нас к проектам вроде финансируемого Министерством внутренней безопасности США Common Weakness Enumeration (CWE), цель которого, по словам самой организации, - разработать единую "теорию уязвимостей"; "улучшить исследование, моделирование и классификацию дефектов программного обеспечения"; а также "предоставить общий язык для обсуждения, поиска и устранения причин уязвимостей безопасности программного обеспечения". Типичный, восхитительно барочный пример получившейся таксономии может выглядеть так:

```
Improper Enforcement of Message or Data Structure
Failure to Sanitize Data into a Different Plane
Improper Control of Resource Identifiers
Insufficient Filtering of File and Other Resource Names
for Executable Content
```

Сегодня словарь CWE содержит около 800 названий, большинство из которых примерно так же способствуют разговору, как процитированное выше.

Немного иная школа натуралистической мысли проявляется в проектах вроде Common Vulnerability Scoring System (CVSS), поддержанной бизнесом совместной инициативы, которая стремится строго количественно описывать известные проблемы безопасности через набор базовых машинно-читаемых параметров. Реальный пример такого дескриптора уязвимости может быть таким:

```
AV:LN / AC:L / Au:M / C:C / I:N / A:P / E:F / RL:T / RC:UR /
CDP:MH / TD:H / CR:M / IR:L / AR:M
```

Ожидается, что организации и исследователи будут преобразовывать этот 14-мерный вектор тщательно выбранным, зависящим от конкретного применения способом, чтобы получить некий объективный, проверяемый, числовой вывод о значимости лежащей в основе ошибки, скажем "42", устраняя необходимость судить о природе уязвимостей более субъективно.

Да, я слегка подшучиваю над этими проектами, но не хочу принижать их усилия. CWE, CVSS и родственные проекты служат благородным целям, например вносят более управляемую размерность в некоторые процессы безопасности, внедряемые крупными организациями. И все же ни один из них не дал большой теории безопасного программного обеспечения, и я сомневаюсь, что такая основа вообще просматривается на горизонте.

В сторону практических подходов

Все признаки указывают на то, что безопасность пока в значительной мере является неалгоритмической проблемой. Отрасль по понятным причинам неохотно признает это открыто, потому что из этого следует, что нет серебряных пуль, которые можно проповедовать или, что еще лучше, коммерциализировать; но если достаточно сильно надавить, в конце концов все в сфере безопасности возвращаются к набору элементарных эмпирических рецептов. Эти рецепты глубоко несовместимы со многими моделями управления бизнесом, но именно они до сих пор действительно работали. Вот они:

- Учиться на ошибках - желательно чужих. Системы следует проектировать так, чтобы предотвращать известные классы ошибок. При отсутствии автоматических или хотя бы изящных решений эта цель лучше всего достигается постоянными рекомендациями по проектированию, гарантией того, что разработчики знают, что может пойти не так, и предоставлением им инструментов, позволяющих выполнять изначально склонные к ошибкам задачи самым простым способом.
- Разрабатывать инструменты для обнаружения и исправления проблем. Недостатки безопасности обычно не имеют очевидных побочных эффектов, пока их не обнаружит злоумышленник: довольно дорогая петля обратной связи. Чтобы противостоять этой проблеме, мы создаем инструменты контроля качества безопасности (QA), проверяющие реализации, и периодически проводим аудиты, чтобы обнаруживать случайные ошибки или системные инженерные недостатки.
- Планировать так, будто скомпрометировано будет все. История учит, что крупные инциденты будут происходить, несмотря на все наши усилия по их предотвращению. Важно реализовать адекватное разделение компонентов, контроль доступа, резервирование данных, мониторинг и процедуры реагирования, чтобы владельцы сервисов могли отреагировать на инциденты до того, как изначально мелкая заминка станет катастрофой библейских масштабов.

Во всех случаях от всех сотрудников, занимающихся информационной безопасностью, требуется солидная доля терпения, творческого подхода и настоящей технической компетентности.

Разумеется, даже такие простые правила здравого смысла - по сути, базовая инженерная строгость - часто наряжают в модные фразы, щедро посыпают набором аббревиатур, например CIA: confidentiality, integrity, availability, то есть конфиденциальность, целостность, доступность, и затем называют "методологиями". Нередко эти методологии являются плохо замаскированными попытками выдать один из самых раздражающих провалов индустрии безопасности за очередную историю успеха и в итоге продать доверчивым клиентам очередное универсальное средство или сертификат. Но, вопреки утверждениям, такие продукты не заменяют практическую сметку и техническое мастерство - по крайней мере сегодня.

В любом случае в оставшейся части этой книги я буду уклоняться от попыток создать или повторно использовать какие-либо из упомянутых выше больших философских каркасов и вместо этого ограничусь здоровой дозой антиинтеллектуализма. Я рассмотрю открытую поверхность современных браузеров, обсудю, как безопасно пользоваться доступными инструментами, какие части Web чаще всего понимают неправильно и как контролировать сопутствующий ущерб, когда все взлетает на воздух.

И это, пожалуй, лучший взгляд на инженерии безопасности, который я могу предложить.

Краткая история Web

Web страдал от сбивающего с толку количества и поразительного разнообразия проблем безопасности. Конечно, некоторые из этих проблем можно отнести к единичным ошибкам в конкретных клиентских или серверных реализациях, но многие вызваны капризными, часто произвольными проектными решениями, которые определяют, как основные механизмы работают и сочетаются друг с другом на стороне браузера.

Наша империя построена на шатком фундаменте - но почему? Возможно, причина в простой недальновидности: в конце концов, кто в невинные времена мог предсказать опасности современных сетей и экономические стимулы, стоящие за сегодняшними крупномасштабными атаками?

К сожалению, хотя такое объяснение подходит для действительно древних механизмов вроде SMTP или DNS, здесь оно не совсем выдерживает проверку. Web сравнительно молод и принял свою нынешнюю форму в обстановке, не так уж сильно отличающейся от той, что мы видим сегодня. Вместо этого ключ к загадке, вероятно, кроется в бурном и необычном пути развития связанных с ним технологий.

Так что простите мне еще одно короткое отступление: вернемся к истокам. Предыстория Web довольно буднична, но все же заслуживает более внимательного взгляда.

Рассказы каменного века: 1945-1994

Историки вычислительной техники часто называют гипотетическое устройство размером с письменный стол, именовавшееся Metex, одним из самых ранних ископаемых свидетельств; его в 1945 году предложил Вэнивар Буш.³ Metex должен был позволять создавать, аннотировать и проходить междокументные ссылки на микрофильме с помощью техники, смутно напоминавшей современные закладки и гиперссылки. Буш смело предположил, что эта простая возможность перевернет область управления знаниями и поиска данных, что забавно, ведь до начала 1990-х такое утверждение еще иногда высмеивали как необразованное и наивное. Увы, полезная реализация этой конструкции тогда была недостижима, поэтому, если не считать футуристических видений, ничего особенного не происходило, пока на сцену не вышли компьютеры на транзисторах.

Следующей осязаемой вехой в 1960-х стало появление IBM Generalized Markup Language (GML), позволявшего аннотировать документы машиночитаемыми директивами, указывавшими функцию каждого блока текста, фактически говоря "это заголовок", "это нумерованный список элементов" и так далее. В течение следующих примерно 20 лет GML, которым первоначально пользовалась лишь горстка текстовых редакторов IBM на громоздких мейнфреймах, стал основой Standard Generalized Markup Language (SGML) - более универсального и гибкого языка, который заменил неудобный синтаксис на двоеточиях и точках знакомым синтаксисом с угловыми скобками.

Пока GML развивался в SGML, компьютеры становились мощнее и удобнее для пользователя. Несколько исследователей начали экспериментировать с бушевской идеей перекрестных ссылок, применяя ее к компьютерному хранению и извлечению документов, пытаясь понять, можно ли перекрестно ссылаться на большие наборы документов на основе какого-нибудь ключа. Смелые компании и университеты занимались такими пионерскими проектами, как ENQUIRE, NLS и Xanadu, но большинство из них не оказало долговременного влияния. Общие претензии к разным проектам сводились к ограниченной практической пригодности, чрезмерной сложности и плохой масштабируемости.

К концу десятилетия два исследователя, Тим Бернерс-Ли и Дэн Коннолли, начали работать над новым подходом к задаче междокументных ссылок - подходом, ориентированным на простоту. Они

запустили проект с разработки HyperText Markup Language (HTML), минималистичного потомка SGML, созданного специально для разметки документов гиперссылками и базовым форматированием. За работой над HTML последовала разработка HyperText Transfer Protocol (HTTP), чрезвычайно простого специализированного механизма доступа к HTML-ресурсам с использованием уже существующих понятий адресов Internet Protocol (IP), доменных имен и путей к файлам. Кульминацией их работы, где-то между 1991 и 1993 годами, стал World Wide Web Тима Бернерса-Ли (рис. 1-1), рудиментарный браузер, который разбирал HTML, позволял пользователям выводить полученные данные на экран и затем переходить с одной страницы на другую щелчком мыши.

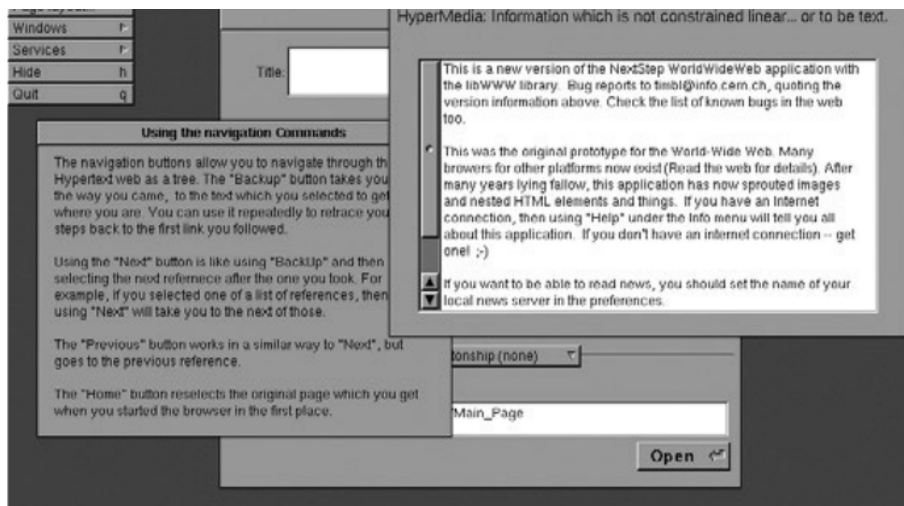


Рис. 1-1. World Wide Web Тима Бернерса-Ли

Многим людям дизайн HTTP и HTML должен был казаться существенным шагом назад по сравнению с более высокими целями конкурирующих проектов. В конце концов, многие ранние инициативы могли похвастаться интеграцией с базами данных, безопасностью и управлением цифровыми правами либо совместным редактированием и публикацией; фактически даже собственный проект Бернерса-Ли ENQUIRE выглядел амбициознее его текущей работы. И все же благодаря низкому порогу входа, немедленной пригодности к использованию и неограниченной масштабируемости, что совпало с появлением мощных и доступных компьютеров и расширением Интернета, скромный проект WWW внезапно оказался хитом.

Ну хорошо, хорошо: он оказался "хитом" по меркам середины 1990-х. Вскоре в Интернете работали уже не меньше чем десятки веб-серверов. К 1993 году HTTP-трафик составлял 0,1 процента всей пропускной способности магистральной сети National Science Foundation. В том же году появился Mosaic, первый достаточно популярный и развитый веб-браузер, созданный в Иллинойском университете. Mosaic расширил первоначальный код World Wide Web, добавив такие возможности, как встраивание изображений в HTML-документы и отправка пользовательских данных через формы, тем самым прокладывая путь к интерактивным мультимедийным приложениям сегодняшнего дня.

Mosaic сделал просмотр страниц приятнее и помог подтолкнуть массовое принятие Web. А в середине 1990-х он послужил основой для двух других браузеров: Mosaic Netscape, позднее переименованного в Netscape Navigator, и Spyglass Mosaic, в конечном счете приобретенного Microsoft и переименованного в Internet Explorer. Появилась и горстка конкурирующих движков, не основанных на Mosaic, включая Opera и несколько текстовых браузеров, таких как Lynx и w3m. Вскоре последовали первые поисковые системы, онлайн-газеты и сайты знакомств.

Первые браузерные войны: 1995-1999

К середине 1990-х стало ясно, что Web пришел надолго и что пользователи готовы отказаться от многих старых технологий в пользу нового претендента. Примерно в это время Microsoft, гигант настольного программного обеспечения, который прежде не спешил принимать Интернет, почувствовала дискомфорт и начала выделять значительные инженерные ресурсы на собственный браузер, в конце концов включив его в операционную систему Windows в 1996 году.^[5] Действия Microsoft запустили период, разговорно известный как "браузерные войны".

Возникшая гонка вооружений между поставщиками браузеров характеризовалась поразительно быстрыми разработкой и внедрением новых возможностей в конкурирующие продукты; эта тенденция часто сопротивлялась всем попыткам стандартизировать или хотя бы должным образом документировать весь добавленный код. Изменения ядра HTML варьировались от глупых, например возможности заставлять текст мигать, изобретения Netscape, ставшего объектом шуток и верным признаком ошибочного веб-дизайна, до заметных, таких как возможность менять гарнитуры шрифтов или встраивать внешние документы в так называемые фреймы. Поставщики выпускали продукты со встроенными языками программирования вроде JavaScript и Visual Basic, подключаемыми модулями для выполнения платформенно-независимых Java- или Flash-апплетов на машине пользователя, а также полезными, но коварными расширениями HTTP вроде cookies. Поддерживалась лишь ограниченная степень поверхностной совместимости, иногда затрудненная патентами и товарными знаками.^[6]

По мере того как Web становился больше и разнообразнее, под видом отказоустойчивости по браузерным движкам расползалась коварная болезнь. Сначала рассуждение выглядело вполне разумно: если браузер А мог показать плохо сделанную, сломанную страницу, а браузер В по какой-либо причине отказывался, пользователи неизбежно воспринимали бы неудачу браузера В как ошибку этого продукта и толпами переходили бы к, казалось бы, более способному клиенту А. Чтобы гарантировать, что их браузеры смогут корректно показать почти любую веб-страницу, инженеры разрабатывали все более сложные и недокументированные эвристики, предназначенные для угадывания намерений небрежных веб-мастеров, часто жертвуя в процессе безопасностью, а иногда даже совместимостью. К сожалению, каждое такое изменение еще больше укрепляло плохие практики веб-дизайна^[7] и вынуждало остальных поставщиков догонять этот хаос, чтобы остаться на плаву. Разумеется, отсутствие достаточно подробных и актуальных стандартов не помогало сдержать распространение этой болезни.

В 1994 году, чтобы смягчить распространение инженерной анархии и управлять развитием HTML, Тим Бернерс-Ли и несколько корпоративных спонсоров создали World Wide Web Consortium (W3C). К несчастью для этой организации, долгое время она могла лишь беспомощно наблюдать, как формат случайным образом расширяют и подкручивают. Первоначальная работа W3C над HTML 2.0 и HTML 3.2 лишь пыталась догнать текущее положение дел, породив недоделанные спецификации, которые в основном устарели уже к моменту публикации. Консорциум также пытался работать над некоторыми новыми и довольно хорошо продуманными проектами, такими как Cascading Style Sheets, но с трудом добивался поддержки поставщиков.

Другие усилия по стандартизации или улучшению уже реализованных механизмов, прежде всего HTTP и JavaScript, велись под эгидой других организаций, таких как European Computer Manufacturers Association (ECMA), International Organization for Standardization (ISO) и Internet Engineering Task Force (IETF). К сожалению, все эти усилия редко были синхронизированы, а некоторые обсуждения и проектные решения контролировались поставщиками или другими заинтересованными сторонами, которых мало заботили долгосрочные перспективы технологии. Результатом стали мертвые стандарты, противоречивые рекомендации и несколько пугающих примеров вредных перекрестных взаимодействий между в остальном аккуратно

спроектированными протоколами - проблема, которая будет особенно очевидна, когда в главе 9 мы обсудим разнообразные механизмы изоляции содержимого.

Скучный период: 2000-2003

Пока попытки обуздать Web буксовали, доминирование Microsoft росло благодаря стратегии включения браузера в операционную систему. К началу нового десятилетия Netscape Navigator уходил со сцены, а Internet Explorer удерживал впечатляющую долю рынка в 80 процентов - число, примерно сопоставимое с тем, которым Netscape обладал всего пятью годами ранее. По обе стороны баррикад безопасность и интероперабельность стали двумя самыми заметными жертвами войны возможностей, но можно было надеяться, что теперь, когда борьба закончилась, разработчики отложат разногласия и вместе начнут исправлять этот беспорядок.

Вместо этого доминирование породило самодовольство: блестяще достигнув своих целей, Microsoft почти не имела стимулов активно инвестировать в браузер. Хотя до версии 5 крупные выпуски Internet Explorer (IE) выходили ежегодно, до появления версии 6 прошло два года, а затем прошло целых пять лет, прежде чем Internet Explorer 6 был обновлен до Internet Explorer 7. Без интереса Microsoft у других поставщиков было очень мало рычагов для внесения радикальных изменений; большинство сайтов не хотело делать улучшения, которые работали бы лишь для небольшой доли посетителей.

Положительной стороной замедления развития браузеров стало то, что W3C смог догнать события и тщательно исследовать несколько новых концепций будущего Web. Среди новых инициатив, завершенных около 2000 года, были HTML 4, очищенный язык, объявивший устаревшими или запретивший многие избыточные или политически некорректные возможности ранних версий, и XHTML 1.1, строгий и хорошо структурированный формат на базе XML, который было проще однозначно разбирать без каких-либо фирменных эвристик. Консорциум также существенно улучшил Document Object Model для JavaScript и Cascading Style Sheets. К сожалению, к концу века Web был уже слишком зрелым, чтобы безболезненно отменить часть старых грехов, но еще слишком молодым, чтобы проблемы безопасности стали достаточно насущными и очевидными для всех. Синтаксис улучшали, теги объявляли устаревшими, валидаторы писали, шезлонги переставляли, но браузеры оставались почти такими же: раздутыми, причудливыми и непредсказуемыми.

Но вскоре произошло кое-что интересное: Microsoft дала миру на первый взгляд неважный, проприетарный API, путано названный XMLHttpRequest. Этот тривиальный механизм должен был иметь небольшое значение, будучи всего лишь попыткой устранить неудобство в веб-версии Microsoft Outlook. Однако XMLHttpRequest оказался гораздо большим: он позволял клиентскому JavaScript вести в основном неограниченные асинхронные HTTP-коммуникации с сервером без необходимости в долгих и разрушительных переходах между страницами. Тем самым API способствовал появлению того, что позднее назовут Web 2.0, - целого ряда сложных, необычайно отзывчивых браузерных приложений, позволявших пользователям работать со сложными наборами данных, сотрудничать, публиковать содержимое и так далее, вторгаясь при этом в священную область "настоящего" устанавливаемого клиентского ПО. Понятно, что это вызвало немалый переполох.

Web 2.0 и вторые браузерные войны: 2004 год и далее

XMLHttpRequest в сочетании с популярностью Интернета и широкой доступностью веб-браузеров подтолкнул Web к новым, захватывающим рубежам - и принес нам шквал ошибок безопасности, затронувших как отдельных пользователей, так и бизнес. Примерно к 2002 году черви и браузерные уязвимости стали темой, к которой часто возвращались СМИ. Microsoft, благодаря своему доминированию на рынке и сравнительно пренебрежительному отношению к безопасности, приняла на себя большую часть последующего репутационного удара. Компания бесечно преуменьшала проблему, но эта тенденция в конце концов создала атмосферу, благоприятную для небольшого бунта.

В 2004 году появился новый претендент в браузерных войнах: Mozilla Firefox, поддерживаемый сообществом потомок Netscape Navigator, перешел в наступление, специально ударив по плохой истории безопасности Internet Explorer и его соответствию стандартам. Получив похвалу и от IT-журналистов, и от экспертов по безопасности, Firefox быстро занял 20 процентов рынка. Хотя новичок вскоре оказался почти столь же поражен ошибками безопасности, как его конкурент из Редмонда, его открытая природа и свобода от необходимости обслуживать упрямых корпоративных пользователей позволяли разработчикам исправлять проблемы намного быстрее.

Примечание. Почему поставщики так яростно конкурировали? Строго говоря, в мире браузеров нельзя напрямую заработать деньги на определенной доле рынка. Тем не менее обозреватели давно предполагают, что дело во власти: встраивая, продвигая или понижая в статусе определенные онлайн-сервисы, даже такие простые, как поисковик по умолчанию, тот, кто контролирует браузер, контролирует значительную часть Интернета.

Помимо Firefox у Microsoft были и другие причины для беспокойства. Ее флагманский продукт, операционная система Windows, все чаще использовался как, возможно, расходуемая стартовая площадка для браузера, тогда как все больше приложений - от редакторов документов до игр - переходило в Web. Это не могло быть хорошей новостью.

Эти факты, вместе с внезапным появлением браузера Apple Safari и, возможно, успехами Opera в мире смартфонов, должно быть, заставили руководителей Microsoft призадуматься. В 1990-х они пропустили ранние признаки важности Интернета; очевидно, они не могли позволить себе повторить ошибку. Microsoft снова придала разработке Internet Explorer импульс, быстро выпустив значительно улучшенные и несколько более безопасные версии 7, 8 и 9.

Конкуренты ответили новыми возможностями и заявлениями о еще лучшем, пусть все еще поверхностном, соответствии стандартам, более безопасном просмотре и улучшенной производительности. Застигнутые врасплох неожиданным успехом XMLHttpRequest и быстро забывшие другие уроки прошлого, поставщики также решили смело экспериментировать с новыми идеями, иногда в одностороннем порядке выкатывая недоделанные или несколько небезопасные конструкции вроде globalStorage в Firefox или httpOnly cookies в Internet Explorer, просто чтобы испытать удачу.

Чтобы еще больше усложнить картину, группа участников, разочарованная творческими разногласиями с W3C, создала совершенно новый орган стандартизации под названием Web Hypertext Application Technology Working Group (WHATWG). WHATWG сыграла ключевую роль в разработке HTML5, первого целостного и учитывающего безопасность пересмотра существующих стандартов, но, как сообщается, Microsoft ее сторонится из-за споров о патентной политике.

На протяжении большей части своей истории Web существовал в уникальной, крайне конкурентной, быстрой, часто чрезмерно политизированной и хаотичной модели разработки без единого видения и без единого набора принципов безопасности. Такое положение дел оставило глубокий след в том, как браузеры работают сегодня и насколько безопасны пользовательские данные, обрабатываемые браузерами.

Скорее всего, эта ситуация в ближайшее время не изменится.

Эволюция угрозы

Очевидно, веб-браузеры, связанные с ними форматы документов и протоколы обмена данными развивались необычным образом. Эта эволюция может объяснять большое количество проблем безопасности, которые мы видим, но сама по себе едва ли доказывает, что эти проблемы уникальны или заслуживают особого внимания. Чтобы завершить эту главу, давайте кратко рассмотрим очень особые свойства, стоящие за наиболее распространенными типами онлайн-угроз безопасности, и поймем, почему у этих угроз не было особенно хороших аналогов в годы до Web.

Пользователь как дефект безопасности

Пожалуй, самое поразительное и полностью нетехническое свойство веб-браузеров состоит в том, что большинство людей, которые ими пользуются, в подавляющем большинстве недостаточно подготовлены.

Конечно, неумелые пользователи были забавной периферийной проблемой с самого начала вычислительной техники. Но популярность Web в сочетании с его удивительно низким порогом входа означает, что мы сталкиваемся с новым противником: большинство пользователей просто не знают достаточно, чтобы оставаться в безопасности.

Долгое время инженеры, работавшие над программным обеспечением общего назначения, делали, казалось бы, произвольные предположения о минимальном уровне компьютерной грамотности, требуемом от пользователей. Большинство таких предположений не имело серьезных последствий; например, неправильное использование текстового редактора обычно почти или совсем не влияло на безопасность системы. Некомпетентные пользователи просто не смогли бы выполнить свою работу - замечательно самокорректирующаяся проблема.

С веб-браузерами все иначе. В отличие от некоторого сложного ПО, ими могут успешно пользоваться люди практически без компьютерной подготовки, люди, которые могут даже не знать, как пользоваться текстовым редактором. Но в то же время безопасно работать с браузерами могут только люди с довольно хорошим пониманием компьютерных технологий и связанного с ними жаргона, включая такие темы, как Public-Key Infrastructure. Само собой, это предварительное условие не выполняется для большинства пользователей некоторых из самых успешных современных веб-приложений.

Браузеры до сих пор выглядят и ощущаются так, будто их проектировали гики для гиков: с периодическими загадочными и непоследовательными сообщениями об ошибках, сложными настройками конфигурации и сбивающим с толку разнообразием предупреждений и запросов безопасности. Примечательное исследование, проведенное исследователями из Berkeley и Harvard в 2006 году, показало, что обычные пользователи почти повсеместно не замечают сигналов, которые разработчику наверняка кажутся совершенно понятными, например наличия или отсутствия значков замка в строке состояния.⁴ В другом исследовании ученые из Stanford и

Microsoft пришли к схожим выводам, изучая влияние современного индикатора безопасности "зеленая строка URL". Механизм, разработанный как более интуитивная альтернатива значкам замка, на самом деле упростил обман пользователей, приучая аудиторию доверять конкретному оттенку зеленого вне зависимости от того, где этот цвет появлялся.⁵

Некоторые эксперты утверждают, что неумелость обычного пользователя не является виной поставщиков ПО и поэтому вообще не является инженерной проблемой. Другие отмечают, что при создании программы, настолько легко доступной и настолько широко распространенной, безответственно заставлять пользователей принимать критически важные для безопасности решения, зависящие от технической компетентности, которая не требуется для работы с программой как таковой.

Впрочем, обвинять одних только поставщиков браузеров тоже несправедливо: у вычислительной отрасли в целом нет надежных ответов в этой области, и доступно очень мало исследований о том, как проектировать сопоставимо сложные пользовательские интерфейсы (UI) действительно надежным образом. В конце концов, даже с банкоматами у нас едва получается все сделать правильно.

Облако, или радости коммунальной жизни

Еще одна своеобразная характеристика Web - драматически недооцененное разделение между несвязанными приложениями и данными, которые они обрабатывают.

В традиционной модели, которой следовали практически все персональные компьютеры последние 15 лет или около того, существуют очень четкие границы между высокоуровневыми объектами данных, то есть документами, пользовательским кодом, то есть приложениями, и ядром операционной системы, которое выступает арбитром всех межприложенческих коммуникаций и аппаратного ввода-вывода (I/O), а также применяет настраиваемые правила безопасности, если приложение выходит из-под контроля. Эти границы хорошо изучены и полезны для построения практических схем безопасности. Файл, открытый в вашем текстовом редакторе, вряд ли сможет украсть вашу электронную почту, если только крайне неудачное сочетание дефектов реализации не подорвет сразу все эти уровни разделения.

В мире браузеров такого разделения практически не существует: документы и код живут как части одних и тех же перемешанных сгустков HTML, изоляция между полностью несвязанными приложениями в лучшем случае частична, причем все сайты номинально делят глобальное окружение JavaScript, а многие типы взаимодействия между сайтами неявно разрешены при наличии очень немногих, если вообще каких-либо, гибких браузерных механизмов арбитража безопасности.

В каком-то смысле эта модель напоминает CP/M, DOS и другие преимущественно немногозадачные операционные системы без надежной защиты памяти, вытеснения CPU или многопользовательских возможностей. Очевидное отличие в том, что мало кто полагался на эти ранние операционные системы, чтобы одновременно запускать несколько недоверенных приложений, поставленных злоумышленником, поэтому особых причин для тревоги не было.

В итоге кажущийся маловероятным сценарий, в котором текстовый файл крадет вашу электронную почту, на самом деле является в Web раздражающе распространенным шаблоном. Почти все веб-приложения вынуждены активно компенсировать нежелательный, вредоносный междоменный доступ и предпринимать громоздкие шаги, чтобы поддерживать хотя бы некоторое разделение кода и отображаемых данных. И рано или поздно почти все веб-приложения терпят неудачу. Проблемы безопасности, связанные с содержимым, такие как cross-site scripting или cross-site request forgery,

чрезвычайно распространены и имеют очень мало аналогов в специализированных, разделенных на отсеки клиентских архитектурах.

Несхождение видений

К счастью, ландшафт безопасности браузеров не совсем безнадежен, и, несмотря на ограниченное разделение между веб-приложениями, несколько избирательных механизмов безопасности обеспечивают элементарную защиту от самых очевидных атак. Но это подводит нас к другой характеристике, которая делает Web настолько интересным предметом: нет общей, целостной модели безопасности, которую можно было бы понять и которой можно было бы следовать. Мы ищем не грандиозное видение мира во всем мире, имейте в виду, а просто общий набор гибких парадигм, применимых к большей части, если не ко всей, соответствующей логике безопасности. В мире Unix, например, модель разрешений пользователя и группы `gwx` является такой сильной объединяющей темой. А в браузерной области?

В браузерной области механизм под названием `same-origin policy` можно было бы считать кандидатом на роль основной парадигмы безопасности, но только до тех пор, пока не выясняется, что он управляет до смешного малым подмножеством междоменных взаимодействий. Если отложить эту деталь в сторону, даже в рамках своей области он имеет не меньше семи различных разновидностей, каждая из которых проводит границы безопасности между приложениями немного в другом месте.^[8] Еще несколько десятков механизмов, не связанных с моделью `same-origin`, контролируют другие ключевые аспекты поведения браузера, по сути реализуя то, что каждый автор в тот день считал лучшим подходом к средствам безопасности.

Как выясняется, сотни маленьких, умных трюков не обязательно складываются в компетентное произведение безопасности. Необычная нехватка целостности очень затрудняет даже решение вопроса о том, где заканчивается одно приложение и начинается другое. С учетом этой реальности как оценивать поверхности атаки, выдавать или отнимать разрешения либо выполнять почти любую другую задачу, связанную с безопасностью? Слишком часто лучший ответ, который мы можем дать, звучит так: "скрестить пальцы".

Любопытно, что многие доброжелательные попытки улучшить безопасность за счет определения новых средств контроля только ухудшают проблему. Многие из этих схем создают новые границы безопасности, которые ради элегантности не идеально совпадают с лохматым наложением уже существующих. Когда новые средства контроля более мелкозернисты, они, вероятно, будут обесценены устаревшими механизмами, предлагая ложное чувство безопасности; когда они более крупнозернисты, они могут устранить некоторые тонкие гарантии, на которые Web прямо сейчас опирается. Адам Барт и Коллин Джексон исследуют тему разрушительной интерференции между политиками безопасности браузеров в своей академической работе.⁶

Межбраузерные взаимодействия: синергия неудач

Общая подверженность экосистемы, состоящей из нескольких разных программных продуктов, могла бы, как кажется, равняться простой сумме дефектов, вносимых каждым приложением. В некоторых случаях итоговая экспозиция может быть меньше, потому что разнообразие повышает устойчивость, но ожидать, что она будет больше, не приходится.

Web снова является исключением из правила. Сообщество безопасности обнаружило значительное количество проблем, которые нельзя приписать какому-то конкретному фрагменту кода, но которые превращаются в реальную угрозу, когда разные браузеры пытаются взаимодействовать друг с другом. Ни один конкретный продукт нельзя легко назначить виноватым:

все они делают свое дело, а проблема лишь в том, что никто не удосужился определить общий этикет, которому все они должны следовать.

Например, один браузер может считать, что в соответствии с его собственной моделью безопасности безопасно передавать определенные URL внешним приложениям или сохранять на диск и затем считывать определенные типы данных. Для каждого такого предположения, скорее всего, существует по крайней мере один браузер, который категорически с ним не согласен и ожидает, что другие стороны будут следовать именно его правилам. Эксплуатируемость этих проблем сильно усугубляется желанием поставщиков просунуть ногу в дверь и попытаться позволить веб-страницам переключаться на их браузер на лету без информированного согласия пользователя. Например, Firefox позволяет открывать страницы в своем браузере, регистрируя протокол `firefoxurl:`; Microsoft устанавливает собственный шлюзовой подключаемый модуль .NET в Firefox; Chrome делает то же самое с Internet Explorer через протокол с именем `cf:`.

Примечание. Особенно в случае таких взаимодействий попытка возложить вину на какую-то конкретную сторону - занятие бесплодное. В недавнем случае ошибки, связанной с `firefoxurl:`, Microsoft и половина сообщества информационной безопасности обвиняли Mozilla, тогда как Mozilla и другая половина экспертов обвиняли Microsoft.⁷ Не имело значения, кто был прав: результат все равно был вполне реальным беспорядком.

Другой набор тесно связанных проблем, практически неслыханный во времена до Web, - несовместимости в внешне похожих механизмах безопасности, реализованных в каждом браузере. Когда модели безопасности различаются, разумная инженерная практика веб-приложения в одном продукте может оказаться недостаточной и ошибочной в другом. Фактически несколько классов элементарных задач, таких как выдача пользователю предоставленного им plaintext-файла, в некоторых браузерах вообще нельзя реализовать безопасно. Однако этот факт не будет очевиден разработчикам, если только они не работают в одном из затронутых браузеров, - и даже тогда им нужно попасть ровно в нужное место.

В конечном счете все характеристики, описанные в этом разделе, вносят вклад в целый новый класс уязвимостей безопасности, который любитель таксономий мог бы назвать неспособностью учесть недокументированное разнообразие. Сегодня этот класс заселен очень плотно.

Распад разделения клиента и сервера

Исследователям информационной безопасности нравится мир статичных, четко назначенных ролей, которые служат привычной точкой отсчета при отображении взаимодействий безопасности в иначе сложном мире. Например, мы говорим об Alice и Bob, двух добропорядочных трудолюбивых пользователях, которые хотят общаться, и о Mallory, коварном злоумышленнике, который охотится за ними. Затем у нас есть клиентское ПО, по сути глупые, иногда недобросовестные терминалы ввода-вывода, легкомысленно запрашивающие сервисы, и скромные серверы, аккуратно выполняющие прихоти клиентов. Разработчики усваивают эти роли и играют по правилам, в процессе создавая довольно понятные и тестируемые сетевые вычислительные среды.

Web начинался как классический пример правильной клиент-серверной архитектуры, но функциональные границы между обязанностями клиента и сервера быстро размылись. Виновник - JavaScript, язык, который дает HTTP-серверам способ делегировать прикладную логику на сторону браузера, то есть "клиента", и дает им две очень убедительные причины так поступать. Во-первых, такой перенос часто приводит к более отзывчивым пользовательским интерфейсам, поскольку серверам не нужно синхронно участвовать в каждом крошечном воображимом изменении состояния UI. Во-вторых, требования к CPU и памяти на стороне сервера, а значит, и стоимость

предоставления сервиса, могут резко снизиться, когда отдельные рабочие станции по всему миру подключаются и помогают выполнить основную часть работы.

Процесс размывания границы клиент-сервер начался достаточно невинно, но было лишь вопросом времени, когда первые механизмы безопасности тоже перейдут на клиентскую сторону вместе со всей прочей будничной функциональностью. Например, какой был смысл тщательно очищать HTML на серверной стороне, если данные все равно только динамически отображались JavaScript на машине клиента?

В некоторых приложениях эта тенденция доводилась до крайности, в конце концов оставляя сервер чуть ли не тупым хранилищем и перенося почти весь разбор, редактирование, отображение и настройку в сам браузер. В таких конструкциях зависимость от сервера могла даже полностью разрываться за счет офлайновых веб-расширений вроде HTML5 persistent storage.

Простой сдвиг места, где происходит вся магия приложения, не обязательно является большой проблемой, но не все обязанности безопасности можно так же легко делегировать клиенту. Например, даже если сервер выступает тупым хранилищем, клиентам нельзя давать неограниченный доступ ко всем данным, сохраненным на сервере для других пользователей, и им нельзя доверять применение правил контроля доступа. В итоге, поскольку держать всю логику безопасности приложения на серверной стороне было нежелательно, а полностью перенести ее на клиентскую было невозможно, большинство приложений оказалось где-то на произвольной середине, без легко различимого и логичного разделения обязанностей между клиентским и серверным компонентами. Получившиеся непривычные конструкции и поведения приложений просто не имели полезных аналогов в элегантном и здоровом мире ролевых игр безопасности.

Ситуация привела не только к беспорядку на уровне проектирования; она привела к несводимой сложности. В традиционной клиент-серверной модели с хорошо специфицированными API можно легко оценивать поведение сервера, не глядя на клиента, и наоборот. Более того, внутри каждого из этих компонентов можно легко изолировать меньшие функциональные блоки и делать предположения об их предполагаемой работе. В новой модели, в сочетании с непрозрачными одноразовыми прикладными API, распространенными в Web, эти аналитические инструменты и возникающая из них легкость рассуждений о безопасности системы были жестоко отняты.

Неожиданный провал стандартизированных протоколов моделирования и тестирования безопасности - еще одна проблема, которая обеспечивает Web совершенно особое и пугающее место во вселенной информационной безопасности.

Доля браузеров на мировом рынке, май 2011 года

Поставщик	Название браузера	Доля рынка	Итого
Microsoft	Internet Explorer 6	10%	52%
Microsoft	Internet Explorer 7	7%	52%
Microsoft	Internet Explorer 8	31%	52%
Microsoft	Internet Explorer 9	4%	52%
Mozilla	Firefox 3	12%	22%
Mozilla	Firefox 4+	10%	22%
Google	Chrome	13%	
Apple	Safari	7%	
Opera Software	Opera	3%	

Источник: данные взяты из публичных отчетов Net Applications.¹

Сноски

[1] [\[назад\]](#) Цитата первоначально приписывается Ивану Арсе, известному охотнику за уязвимостями, примерно 2000 год; с тех пор ее использовали Криспин Коуэн, Майкл Ховард, Антон Чувакин и множество других экспертов по безопасности.

[2] [\[назад\]](#) В 1936 году Алан Тьюринг показал, что, слегка перефразируя, невозможно создать алгоритм, который в общем виде мог бы решать исход других алгоритмов. Разумеется, некоторые алгоритмы вполне разрешимы с помощью доказательств для конкретных случаев, но не все.

[3] [\[назад\]](#) Где-то в 2006 году несколько злоумышленников, предположительно во главе с Альбертом Гонсалесом, атаковали незащищенную беспроводную сеть в розничной точке, а затем пробрались через корпоративные сети розничного гиганта. Они скопировали данные кредитных карт примерно 46 миллионов клиентов, а также номера социального страхования, домашние адреса и прочие сведения еще примерно 450,000 человек. В связи с атакой обвинения были предъявлены одиннадцати людям, один из которых покончил с собой.

[4] [\[назад\]](#) Формально не опубликованная и нейтрально озаглавленная презентация Microsoft *Threats Against and Protection of Microsoft's Internal Network* описывает атаку 2003 года, начавшуюся с компрометации домашней рабочей станции инженера, имевшей долгоживущую VPN-сессию во внутреннюю сеть корпорации. Затем последовали методичные попытки эскалации, кульминацией которых стал доступ злоумышленника к внутренним репозиториям исходного кода и утечка данных из них. По крайней мере для широкой публики преступник остается неизвестным.

[5] [\[назад\]](#) Интересно, что это решение оказалось очень спорным. С одной стороны, можно утверждать, что Microsoft тем самым сильно поспособствовала популяризации Интернета. С другой - она подорвала положение конкурирующих браузеров и могла рассматриваться как антиконкурентная. В конце концов эта стратегия привела к серии затяжных юридических битв вокруг возможного злоупотребления компанией монопольным положением, таких как *United States v. Microsoft*.

[6] [\[назад\]](#) Например, Microsoft не хотела договариваться с Sun о лицензировании товарного знака JavaScript, языка, названного так по рекламным причинам, а не потому, что он имел какое-либо отношение к Java, поэтому выбрала для своей почти, но не совсем идентичной версии название "JScript". Официальная документация Microsoft до сих пор называет это ПО именно так.

[7] [\[назад\]](#) Главные примеры ошибочных и в конечном счете губительных функций браузеров - механизмы распознавания содержимого и кодировки символов; оба будут обсуждаться в главе 13.

[8] [\[назад\]](#) Семь основных разновидностей, обсуждаемых на протяжении части II этой книги, включают политику безопасности для доступа JavaScript к DOM; API XMLHttpRequest; HTTP cookies; API локального хранилища; а также подключаемые модули вроде Flash, Silverlight или Java.

Глава 2. Все начинается с URL

Самый узнаваемый признак Web - простая текстовая строка, известная как Uniform Resource Locator (URL). Каждый корректно сформированный, полностью квалифицированный URL должен однозначно адресовать и уникально идентифицировать один ресурс на удаленном сервере, а заодно выполнять пару связанных вспомогательных функций. Синтаксис URL является краеугольным камнем адресной строки, самого важного индикатора безопасности пользовательского интерфейса (UI) в каждом браузере.

Помимо настоящих URL, используемых для получения содержимого, несколько классов псевдо-URL применяют похожий синтаксис, чтобы предоставить удобный доступ к функциям уровня браузера, включая встроенный движок сценариев, несколько специальных режимов отображения документов и так далее. Возможно, неудивительно, что действия таких псевдо-URL могут существенно влиять на безопасность любого сайта, который решит на них ссылаться.

Способность понять, как конкретный URL будет истолкован браузером и какие побочные эффекты он вызовет, - одна из самых базовых и распространенных задач безопасности, которую пытаются решать и люди, и веб-приложения. Но эта задача может оказаться проблемной. Общий синтаксис URL, созданный Тимом Бернерсом-Ли, в первую очередь кодифицирован в RFC 3986;¹ его практические применения в Web описаны в RFC 1738,² 2616,³ и еще в паре менее значимых стандартов. Эти документы удивительно подробны и в результате задают довольно сложную модель разбора, но они недостаточно точны, чтобы привести к гармоничным, совместимым реализациям во всем клиентском ПО. Кроме того, отдельные поставщики программного обеспечения по собственным причинам решили отклониться от спецификаций.

Давайте подробнее посмотрим, как скромный URL работает на практике.

Структура Uniform Resource Locator

На рис. 2-1 показан формат полностью квалифицированного абсолютного URL, то есть такого, который задает всю информацию, необходимую для доступа к конкретному ресурсу, и никак не зависит от того, откуда начался переход. В отличие от него относительный URL, например `../file.php?text=hello+world`, опускает часть этой информации и должен интерпретироваться в контексте базового URL, связанного с текущим контекстом просмотра.

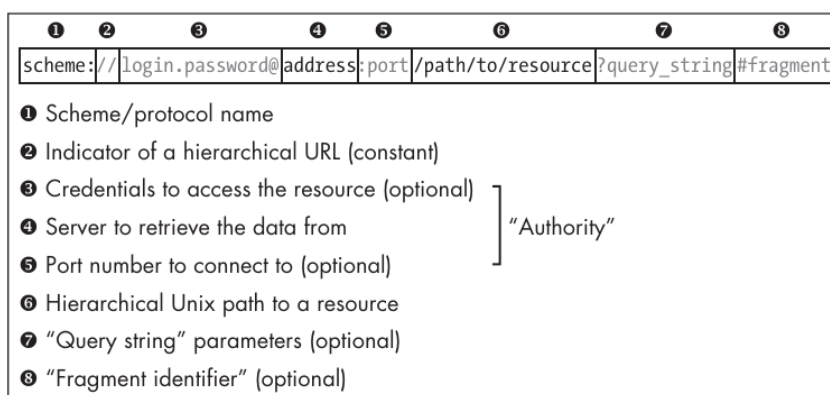


Рис. 2-1. Структура абсолютного URL

Сегменты абсолютного URL выглядят интуитивно понятными, но с каждым связан набор ловушек, поэтому рассмотрим их сейчас.

Имя схемы

Имя схемы - строка без учета регистра, которая заканчивается одним двоеточием и указывает протокол, используемый для получения ресурса. Официальный реестр допустимых схем URL поддерживает Internet Assigned Numbers Authority (IANA), организация, более широко известная управлением пространством IP-адресов.⁴ Текущий список допустимых имен схем IANA включает несколько десятков записей, таких как `http:`, `https:` и `ftp:`; на практике обычные браузеры и сторонние приложения неформально распознают гораздо более широкий набор схем, некоторые из которых имеют особые последствия для безопасности. Особый интерес представляют несколько типов псевдо-URL, таких как `data:` или `javascript:`, обсуждаемые позднее в этой главе и далее в книге.

Прежде чем перейти к дальнейшему разбору, браузерам и веб-приложениям нужно отличить полностью квалифицированные абсолютные URL от относительных. Ключевым различием должно быть наличие допустимой схемы перед адресом, как определено в RFC 1738: в соответствующем стандарту абсолютном URL перед обязательным символом `:` могут появляться только алфавитно-цифровые символы, `+`, `-` и `.` На практике браузеры немного отклоняются от этой рекомендации. Все они игнорируют начальные переводы строк и пробелы. Internet Explorer игнорирует весь диапазон непечатаемых ASCII-символов от `0x01` до `0x1F`. Chrome дополнительно пропускает `0x00`, символ NUL. Большинство реализаций также игнорирует переводы строк и табуляцию в середине имен схем, а Opera принимает в строке символы с установленным старшим битом.

Из-за этих несовместимостей приложения, зависящие от способности различать относительные и абсолютные URL, должны консервативно отвергать любой аномальный синтаксис, но, как мы вскоре увидим, даже этого недостаточно.

Индикатор иерархического URL

Чтобы соответствовать общим правилам синтаксиса, изложенным в RFC 1738, каждый абсолютный иерархический URL обязан содержать фиксированную строку `//` прямо перед секцией `authority`. Если эта строка отсутствует, формат и функция оставшейся части URL не определены для целей данной спецификации и должны рассматриваться как непрозрачное значение, зависящее от схемы.

Примечание. Пример неиерархического URL - протокол `mailto:`, используемый для указания адресов электронной почты и, возможно, строки темы (`mailto:user@example.com?subject=Hello+world`). Такие URL передаются почтовому клиенту по умолчанию без дальнейших попыток их разобрать.

Концепция общего иерархического синтаксиса URL в теории изящна. Она должна позволять приложениям извлекать некоторую информацию об адресе, не зная, как работает конкретная схема. Например, не имея заранее представления о причудливом протоколе `wacky-widget:`, но применив только концепцию общего синтаксиса URL, браузер мог бы решить, что `http://example.com/test1/` и `wacky-widget://example.com/test2/` ссылаются на один и тот же доверенный удаленный хост.

К сожалению, у спецификации есть интересный недостаток: упомянутый RFC ничего не говорит о том, что должен делать разработчик реализации при встрече с URL, в которых схема известна как неиерархическая, но префикс `//` все равно присутствует, или наоборот. Фактически эталонная реализация парсера, приведенная в RFC 1630, содержит непреднамеренную лазейку, придающую

последнему классу URL контринтуитивный смысл. В RFC 3986, опубликованном несколькими годами позже, авторы смущенно признают этот недостаток и разрешают реализациям пытаться разбирать такие URL ради совместимости. В результате многие браузеры интерпретируют следующие примеры неожиданным образом:

- `http:example.com/` В Firefox, Chrome и Safari этот адрес может трактоваться так же, как `http://example.com/`, когда нет полностью квалифицированного контекста базового URL, и как относительная ссылка на каталог с именем `example.com`, когда доступен допустимый базовый URL.
- `javascript://example.com/%0Aalert(1)` Эта строка интерпретируется как допустимый неиерархический псевдо-URL во всех современных браузерах, и код JavaScript `alert(1)` будет выполнен, показав простое диалоговое окно.
- `mailto://user@example.com` Internet Explorer принимает этот URL как допустимую неиерархическую ссылку на адрес электронной почты; часть `//` просто пропускается. Другие браузеры не согласны.

Учетные данные для доступа к ресурсу

Часть URL с учетными данными необязательна. В этом месте можно указать имя пользователя и, возможно, пароль, которые могут требоваться для получения данных с сервера. Метод, с помощью которого эти учетные данные передаются, не задан как часть абстрактного синтаксиса URL и всегда зависит от протокола. Для протоколов, не поддерживающих аутентификацию, поведение URL с учетными данными просто не определено.

Если учетные данные не предоставлены, браузер попытается получить ресурс анонимно. В случае HTTP и некоторых других протоколов это означает, что данные аутентификации не отправляются; для FTP это означает вход в гостевую учетную запись с именем `ftp` и фиктивным паролем.

Большинство браузеров принимает в этой секции почти любые символы, кроме общих разделителей секций URL, с двумя исключениями: Safari по неясным причинам отвергает более широкий набор символов, включая `<`, `>`, `{` и `}`, а Firefox также отвергает переводы строк.^[1]

Адрес сервера

Для всех полностью квалифицированных иерархических URL секция адреса сервера должна задавать DNS-имя без учета регистра, например `example.com`, необработанный IPv4-адрес, например `127.0.0.1`, или IPv6-адрес в квадратных скобках, например `[0:0:0:0:0:0:0:1]`, указывающий расположение сервера, на котором размещен запрошенный ресурс. Firefox также принимает IPv4-адреса и имена хостов в квадратных скобках, но другие реализации сразу их отвергают.

Хотя RFC допускает только каноническую запись IP-адресов, стандартные библиотеки C, используемые большинством приложений, гораздо свободнее: они принимают неканонические IPv4-адреса, смешивающие восьмеричную, десятичную и шестнадцатеричную запись либо объединяющие часть или все октеты в одно целое число. В результате следующие варианты распознаются как эквивалентные:

- `http://127.0.0.1/` Это каноническое представление IPv4-адреса.
- `http://0x7f.1/` Это представление того же адреса, использующее шестнадцатеричное число для первого октета и объединяющее все оставшиеся октеты в одно десятичное значение.

- `http://017700000001/` Тот же адрес обозначен восьмеричным значением с префиксом 0, причем все октеты объединены в одно 32-битное целое.

Похожий расслабленный подход можно увидеть и с DNS-именами. Теоретически DNS-метки должны соответствовать очень узкому набору символов, а именно алфавитно-цифровым символам, `.` и `-`, как определено в RFC 1035,5 но многие браузеры охотно попросят нижележащий резолвер операционной системы найти почти что угодно, и операционная система обычно тоже не станет возражать. Точный набор символов, принимаемых в имени хоста и передаваемых резолверу, различается от клиента к клиенту. Safari наиболее строг, тогда как Internet Explorer наиболее разрешающий. Возможно, стоит отметить, что несколько управляющих символов в диапазонах `0xA0-0xD` и `0xA0-0xAD` игнорируются большинством браузеров в этой части URL.

Примечание. Одно увлекательное поведение URL-парсеров во всех основных браузерах - их готовность трактовать символ `⋮` (идеографическая точка, кодовая точка Unicode U+3002) точно так же, как точку в именах хостов, но не где-либо еще в URL. Сообщается, что причина в некоторых китайских раскладках клавиатуры, где этот символ набрать намного проще, чем ожидаемое 7-битное ASCII-значение.

Порт сервера

Порт сервера - необязательная секция, описывающая нестандартный сетевой порт, к которому нужно подключиться на ранее указанном сервере. Практически все протоколы прикладного уровня, поддерживаемые браузерами и сторонними приложениями, используют TCP или UDP как нижележащий транспортный метод, а и TCP, и UDP опираются на 16-битные номера портов, чтобы разделять трафик между несвязанными сервисами, работающими на одной машине. Каждая схема связана с портом по умолчанию, на котором обычно запускаются серверы этого протокола, например 80 для HTTP, 21 для FTP и так далее, но этот порт по умолчанию можно переопределить на уровне URL.

Примечание. Интересный и непреднамеренный побочный эффект этой возможности состоит в том, что браузеры можно обманом заставить отправлять предоставленные злоумышленником данные случайным сетевым сервисам, которые не говорят на протоколе, ожидаемом браузером. Например, можно направить браузер на `http://mail.example.com:25/`, где 25 - порт, используемый сервисом Simple Mail Transfer Protocol (SMTP), а не HTTP. Этот факт вызвал ряд проблем безопасности и породил несколько несовершенных обходных мер, подробнее обсуждаемых в части II этой книги.

Иерархический путь к файлу

Следующая часть URL, иерархический путь к файлу, задумана как способ идентифицировать конкретный ресурс, который нужно получить с сервера, например `/documents/2009/my_diary.txt`. Спецификация довольно открыто строится поверх семантики каталогов Unix, предписывая разрешение сегментов `./` и `/.` в пути и предоставляя основанный на каталогах метод разбора относительных ссылок в не полностью квалифицированных URL.

Использование файловой модели должно было казаться естественным выбором в 1990-х, когда веб-серверы действовали как простые шлюзы к набору статических файлов и редким исполняемым сценариям. Но с тех пор многие современные фреймворки веб-приложений разорвали оставшиеся связи с файловой системой, напрямую взаимодействуя с объектами базы данных или зарегистрированными местоположениями в резидентном программном коде. Отобразить эти

структуры данных на хорошо ведущие себя URL-пути можно, но это не всегда практикуется или практикуется аккуратно. Все это делает автоматизированное получение содержимого, индексирование и тестирование безопасности сложнее, чем должно быть.

Строка запроса

Строка запроса - необязательная секция, используемая для передачи произвольных неиерархических параметров ресурсу, ранее идентифицированному путем. Один распространенный пример - передача введенных пользователем терминов серверному скрипту, реализующему функцию поиска, например:

```
http://example.com/search.php?query=Hello+world
```

Большинство веб-разработчиков привыкло к определенной структуре строки запроса; этот знакомый формат генерируется браузерами при обработке HTML-форм и следует такому синтаксису:

```
name1=value1&name2=value2...
```

Удивительно, но такая структура не предписана RFC для URL. Вместо этого строка запроса рассматривается как непрозрачный сгусток данных, который конечный получатель может интерпретировать по своему усмотрению, и, в отличие от пути, она не отягощена особыми правилами разбора.

Намеки на широко используемый формат можно найти в информационном RFC 1630,6 в связанном с почтой RFC 2368,7 и в спецификациях HTML, касающихся форм.⁸ Ничто из этого не является обязательным, и поэтому, хотя это может быть невежливо, веб-приложения не совершают ошибки, если используют произвольные форматы для любых данных, которые хотят поместить в эту часть URL.

Идентификатор фрагмента

Идентификатор фрагмента - непрозрачное значение с ролью, похожей на строку запроса, но предоставляющее необязательные инструкции клиентскому приложению, а не серверу. Фактически это значение вообще не должно отправляться на сервер. Ни формат, ни функция идентификатора фрагмента в RFC четко не определены, но есть намек, что он может использоваться для адресации "подресурсов" в полученном документе или предоставления иных специфичных для документа подсказок отображения.

На практике у идентификаторов фрагмента в браузере есть только одно санкционированное применение: указание имени якорного HTML-элемента для навигации внутри документа. Логика проста. Если имя якоря указано в URL и удастся найти соответствующий HTML-тег, документ будет прокручен к этому месту для просмотра; иначе ничего не произойдет. Поскольку информация закодирована в URL, этим конкретным видом длинного документа легко поделиться с другими или сохранить его в закладках. В таком использовании смысл идентификатора фрагмента ограничивается прокруткой существующего документа, поэтому при обновлении только этой части URL в ответ на действия пользователя нет необходимости получать новые данные с сервера.

Это интересное свойство привело к еще одному, более свежему и полностью ad hoc применению этого значения: хранению разнородной информации о состоянии, необходимой клиентским сценариям. Например, рассмотрим приложение для просмотра карт, которое помещает текущие просматриваемые координаты карты в идентификатор фрагмента, чтобы знать, откуда возобновить работу, если ссылку сохраняют в закладках или передадут другим. В отличие от обновления строки

запроса, изменение идентификатора фрагмента на лету не вызывает длительной перезагрузки страницы, что делает этот трюк хранения данных убийственно полезной возможностью.

Собираем все обратно

Каждый из упомянутых выше сегментов URL отделяется определенными зарезервированными символами: слешами, двоеточиями, вопросительными знаками и так далее. Чтобы весь подход был пригодным к использованию, эти разделяющие символы не должны появляться где-либо в URL с какой-либо иной целью. С учетом этого предположения представьте пример алгоритма, который разбивает абсолютные URL на перечисленные функциональные части способом, хотя бы отдаленно согласующимся с тем, как эту задачу выполняют браузеры. Довольно приличный пример такого алгоритма:

Шаг 1. Извлечь имя схемы.

Ищите первый символ `:`. Часть URL слева от него - имя схемы. Завершите разбор с ошибкой, если имя схемы не соответствует ожидаемому набору символов; в таком случае URL может потребоваться трактовать как относительный.

Шаг 2. Поглотить идентификатор иерархического URL.

За именем схемы должна следовать строка `//`. Пропустите ее, если она найдена; завершите разбор с ошибкой, если нет.

Примечание. В некоторых контекстах разбора реализации с тем же удовольствием примут ноль, один или даже три и более слеша вместо двух, руководствуясь удобством. В том же духе Internet Explorer с момента появления принимал обратные слеша (`\`) вместо прямых слешей в любом месте URL, предположительно чтобы помочь неопытным пользователям.^[2] Все браузеры, кроме Firefox, в конце концов последовали этой тенденции и распознают URL вроде `http:\\example.com\\`.

Шаг 3. Забрать секцию authority.

Ищите следующий `/`, `?` или `#`, в зависимости от того, что встретится первым, чтобы извлечь секцию authority из URL. Как упоминалось выше, большинство браузеров также принимает `\` как разделитель вместо прямого слеша, что может потребовать учета. Точка с запятой (`;`) - еще один допустимый разделитель authority в браузерах, отличных от Internet Explorer и Safari; причина такого решения неизвестна.

Шаг 3А. Найти учетные данные, если они есть.

После извлечения секции authority найдите символ at (`@`) в подстроке. Если он найден, начальный фрагмент представляет учетные данные входа, которые следует дополнительно разбить при первом появлении двоеточия, если оно присутствует, чтобы отделить логин от пароля.

Шаг 3В. Извлечь адрес назначения.

Оставшаяся часть секции authority - адрес назначения. Найдите первое двоеточие, чтобы отделить имя хоста от номера порта. Также нужен особый случай для IPv6-адресов, заключенных в скобки.

Шаг 4. Определить путь, если он присутствует.

Если за секцией authority сразу следует прямой слеш, а в некоторых реализациях, как отмечалось ранее, обратный слеш или точка с запятой, ищите следующий `?`, `#` или конец строки, в зависимости от того, что встретится первым. Текст между ними образует секцию пути, которую следует нормализовать согласно семантике путей Unix.

Шаг 5. Извлечь строку запроса, если она присутствует.

Если за последним успешно разобранным сегментом следует вопросительный знак, ищите следующий символ # или конец строки, в зависимости от того, что встретится первым. Текст между ними - строка запроса.

Шаг 6. Извлечь идентификатор фрагмента, если он присутствует.

Если за последним успешно разобранным сегментом следует #, все от этого символа до конца строки является идентификатором фрагмента. В любом случае вы закончили.

Этот алгоритм может показаться будничным, но он раскрывает тонкие детали, о которых обычно не думают даже опытные программисты. Он также показывает, что обычным пользователям крайне трудно понять, как может быть разобран конкретный URL. Начнем с довольно простого случая:

```
http://example.com&gibberish=1234@167772161/
```

Цель этого URL - объединенный IP-адрес, декодирующийся в 10.0.0.1, - неочевидна для неспециалиста, и многие пользователи сочли бы, что посещают example.com.^[3] Но ладно, это был легкий пример. Теперь посмотрим на такой синтаксис:

```
http://example.com\@coredump.cx/
```

В Firefox этот URL приведет пользователя на coredump.cx, потому что example.com\ будет интерпретировано как допустимое значение поля логина. Почти во всех остальных браузерах \ будет истолкован как разделитель пути, и пользователь попадет на example.com.

Для Internet Explorer существует еще более раздражающий пример. Рассмотрим такой:

```
http://example.com;.coredump.cx/
```

Браузер Microsoft разрешает ; в имени хоста и успешно разрешает эту метку благодаря соответствующей настройке домена coredump.cx. Большинство других браузеров автоматически исправят URL на http://example.com/;.coredump.cx и приведут пользователя на example.com, за исключением Safari, где такой синтаксис вызывает ошибку. Если это выглядит хаотично, помните: мы только начинаем разбираться в том, как работают браузеры.

Зарезервированные символы и процентное кодирование

Алгоритм разбора URL, описанный в предыдущем разделе, опирается на предположение, что некоторые зарезервированные, разделяющие синтаксис символы не появятся буквально в URL в каком-либо ином качестве, то есть не станут частью имени пользователя, пути запроса и так далее. Эти общие разделители, нарушающие синтаксис, таковы:

```
: / ? # [ ] @
```

RFC также называет пару разделителей более низкого уровня, не задавая им конкретного назначения, предположительно чтобы позволить реализовывать функции, зависящие от схемы или приложения, внутри любой из секций верхнего уровня:

```
! $ & ' ( ) * + , ; =
```

Все перечисленные выше символы в принципе запрещены, но существуют законные случаи, когда их нужно включить в URL, например чтобы учесть произвольные поисковые термины, введенные пользователем и переданные серверу в строке запроса. Поэтому вместо запрета стандарт предоставляет метод кодирования всех посторонних появлений этих значений. Метод, просто называемый процентным кодированием или URL-кодированием, заменяет символы знаком процента (%), за которым следуют две шестнадцатеричные цифры, представляющие

соответствующее ASCII-значение. Например, / будет закодирован как %2F; верхний регистр привычен, но не обязателен. Из этого следует, что во избежание неоднозначности сам голый знак процента должен кодироваться как %25. Любым посредникам, обрабатывающим существующие URL, включая браузеры и веб-приложения, далее предписывается никогда не пытаться декодировать или кодировать зарезервированные символы в передаваемых URL, потому что смысл такого URL может внезапно измениться.

К сожалению, неизменность зарезервированных символов в существующих URL конфликтует с необходимостью реагировать на любые URL, которые технически незаконны из-за неправильного использования этих символов и встречаются браузеру в реальной сети. Эта тема спецификациями вообще не покрывается, что вынуждает поставщиков браузеров импровизировать и вызывает несовместимости между реализациями. Например, нужно ли URL `http://a@b@c/` преобразовать в `http://a@b%40c/` или, возможно, в `http://a%40b@c/?` Internet Explorer и Safari считают, что первый вариант разумнее; остальные браузеры поддерживают вторую точку зрения.

Оставшиеся символы, не входящие в зарезервированный набор, не должны иметь какого-либо особого значения внутри самого синтаксиса URL. Однако некоторые из них, например непечатаемые управляющие ASCII-символы, явно несовместимы с идеей, что URL должны быть читаемыми человеком и безопасными для передачи. Поэтому RFC описывает путано названное подмножество незарезервированных символов, состоящее из алфавитно-цифровых символов, -, ., _ и ~, и говорит, что только это подмножество и зарезервированные символы в их предусмотренной роли формально могут появляться в URL как есть.

Примечание. Любопытно, что эти незарезервированные символы только разрешено оставлять в неэкранированном виде; делать это не требуется. Пользовательские агенты могут кодировать или декодировать их по своему желанию, и это вообще не меняет смысла URL. Это свойство поднимает еще один способ сбить пользователей с толку: использование неканонических представлений незарезервированных символов. В частности, все следующие варианты эквивалентны:

- `http://example.com/`
- `http://%65xample.%63om/`
- `http://%65%78%61%6d%70%6c%65%2e%63%6f%6d/`^[4]

Ряд в остальном незарезервированных печатаемых символов исключен из так называемого незарезервированного набора. Из-за этого, строго говоря, RFC требуют безусловно кодировать их процентным способом. Однако, поскольку браузерам прямо не поручено обеспечивать соблюдение этого правила, к нему относятся не слишком серьезно. В частности, все браузеры позволяют символам ^, {, | и } появляться в URL без экранирования и отправляют эти символы серверу как есть. Internet Explorer дополнительно пропускает <, > и `; Internet Explorer, Firefox и Chrome все принимают \; Chrome и Internet Explorer разрешают двойную кавычку; а Opera и Internet Explorer передают непечатаемый символ 0x7F (DEL) как есть.

Наконец, вопреки требованиям, изложенным в RFC, большинство браузеров также вообще не кодируют идентификаторы фрагментов. Это создает неожиданную проблему для клиентских сценариев, которые полагаются на эту строку и ожидают, что некоторые потенциально небезопасные символы никогда не появятся буквально. Мы вернемся к этой теме в главе 6.

Обработка не-US-ASCII текста

Многие языки по всему миру опираются на символы за пределами базового 7-битного набора ASCII или стандартной 8-битной кодовой страницы, традиционно используемой всеми PC-совместимыми системами (CP437). Черт возьми, некоторые языки вообще зависят от алфавитов, не основанных на латинице.

Чтобы удовлетворить потребности часто игнорируемой, но огромной неанглоязычной пользовательской базы, задолго до появления Web были придуманы различные 8-битные кодовые страницы с альтернативным набором символов со старшим битом: ISO 8859-1, CP850 и Windows 1252 для западноевропейских языков; ISO 8859-2, CP852 и Windows 1250 для Восточной и Центральной Европы; KOI8-R и Windows 1251 для России. А поскольку некоторые алфавиты невозможно было вместить в пространство из 256 символов, появились сложные кодировки переменной ширины, такие как Shift JIS для катаканы.

Несовместимость этих карт символов затрудняла обмен документами между компьютерами, настроенными на разные кодовые страницы. К началу 1990-х эта растущая проблема привела к созданию Unicode - своего рода универсального набора символов, слишком большого, чтобы поместиться в 8 бит, но предназначенного охватить практически все региональные письменности и специальные пиктограммы, известные человеку. За Unicode последовал UTF-8, относительно простое представление этих символов переменной ширины, теоретически безопасное для всех приложений, способных обрабатывать традиционные 8-битные форматы. К сожалению, для кодирования символов со старшим битом UTF-8 требовал больше байтов, чем большинство конкурентов, и многим пользователям это казалось расточительным и ненужным. Из-за этой критики UTF-8 понадобилось намного больше десяти лет, чтобы получить распространение в Web, и произошло это лишь после того, как все соответствующие протоколы уже устоялись.

Эта неудачная задержка повлияла на обработку URL, содержащих пользовательский ввод. Браузерам очень рано потребовалось поддержать такое использование, но, обратившись к соответствующим стандартам, разработчики не нашли содержательных рекомендаций. Даже годы спустя, в 2005 году, RFC 3986 говорил лишь следующее:

В локальных или региональных контекстах и по мере совершенствования технологий пользователи могли бы выиграть от возможности использовать более широкий диапазон символов; такое использование настоящей спецификацией не определено.

Процентно закодированные октеты... могут использоваться внутри URI для представления символов за пределами диапазона кодированного набора символов US-ASCII, если такое представление разрешено схемой или элементом протокола, в котором URI упоминается. Такое определение должно указывать кодировку символов, используемую для отображения этих символов в октеты перед их процентным кодированием для URI.

Увы, несмотря на эти благие пожелания, ни один из остальных стандартов этой темы не касался. Всегда можно было поместить необработанные символы со старшим битом в URL, но без знания кодовой страницы, в которой их следует интерпретировать, сервер не мог бы понять, должно ли %B1 означать ±, а или какой-нибудь другой извилистый символ, специфичный для родной письменности пользователя.

К сожалению, поставщики браузеров не взяли инициативу на себя и не выработали согласованное решение этой проблемы. Большинство браузеров внутри перекодирует сегменты пути URL в UTF-8 или ISO 8859-1, если этого достаточно, но затем формирует строку запроса в кодовой странице ссылающейся страницы. В некоторых случаях, когда URL вводятся вручную или передаются в определенные специализированные API, символы со старшим битом также могут понижаться до их

7-битных US-ASCII-двойников, заменяться вопросительными знаками или даже полностью искажаться из-за дефектов реализации.

Плохо реализованная или нет, возможность передавать неанглийские символы в строках запроса и путях закрывала очевидную потребность. Традиционный подход процентного кодирования оставил совершенно на холоде только один сегмент URL: ввод со старшим битом нельзя было разрешить как есть при указании имени сервера назначения, потому что по крайней мере в принципе давно устоявшийся стандарт DNS допускал в доменных именах только разделенные точками алфавитно-цифровые символы и дефисы, и хотя никто не придерживался правил, набор исключений различался от одного сервера имен к другому.

Проницательный читатель может удивиться, почему это ограничение вообще важно, то есть почему было важно иметь локализованные доменные имена и в нелатинских алфавитах тоже. Сейчас на этот вопрос, возможно, трудно ответить. Проще говоря, несколько людей считали, что отсутствие таких кодировок мешает бизнесам и частным лицам по всему миру полностью принять Web и пользоваться им, и, правы они были или нет, они твердо решили добиться своего.

Эта цель привела к созданию Internationalized Domain Names in Applications (IDNA). Сначала RFC 3490,⁹ описавший довольно надуманную схему кодирования произвольных строк Unicode с использованием алфавитно-цифровых символов и дефисов, а затем RFC 3492,¹⁰ описавший способ применять это кодирование к DNS-меткам с использованием формата, известного как Punycode. Punycode выглядел примерно так:

```
xn--[US-ASCII part]-[encoded Unicode data]
```

Соответствующий стандарту браузер, получив технически незаконный URL, содержащий буквальный не-US-ASCII символ где-либо в имени хоста, должен был преобразовать имя в Punycode перед выполнением DNS-запроса. Следовательно, встретив Punycode в существующем URL, он должен был поместить в адресную строку декодированную, читаемую человеком форму строки.

Примечание. Объединение всех этих несовместимых стратегий кодирования может дать забавную смесь. Рассмотрим пример URL вымышленного польскоязычного магазина полотенец:

Намерение: http://www.ręcniki.pl/ręcnik?model=Jaś#Złóż_zamówienie

Фактический URL: http://www.xn--rczniki-98a.pl/r%C4%99cnik?model=Ja%B6#Złóż_zamówienie

Метка преобразована в Punycode; путь преобразован в UTF-8; строка запроса преобразована в ISO 8859-2; фрагмент оставлен как буквальный UTF-8.

Из всех подходов к кодированию на основе URL IDNA вскоре оказался самым проблемным. По сути, доменное имя в URL, показанном в адресной строке браузера, - один из важнейших индикаторов безопасности в Web, поскольку он позволяет пользователям быстро отличать сайты, которым они доверяют и с которыми имели дело, от остального Интернета. Когда имя хоста, показанное браузером, состоит из 38 знакомых и различимых символов, только довольно невнимательные жертвы будут обмануты и решат, что их любимый домен `example.com` и сайт-самозванец `examp1e.com` - одно и то же. Но IDNA небрежно и без разбора расширил эти 38 символов примерно до 100,000 глифов, поддерживаемых Unicode, многие из которых выглядят совершенно одинаково и отличаются друг от друга только функциональными различиями.

Насколько все плохо? Рассмотрим, например, кириллицу. В этом алфавите есть ряд омоглифов, которые выглядят практически идентично латинским аналогам, но имеют совершенно другие значения Unicode и разрешаются в совершенно другие DNS-имена Punycode:

Алфавит	a	c	e	i	j	o	p	s	x	y
Latin	U+0061	U+0063	U+0065	U+0069	U+006A	U+006F	U+0070	U+0073	U+0078	U+0079
Cyrillic	U+0430	U+0441	U+0435	U+0456	U+0458	U+043E	U+0440	U+0455	U+0445	U+0443

Когда IDNA был предложен и впервые реализован в браузерах, никто всерьез не рассматривал последствия этой проблемы. Поставщики браузеров, по-видимому, предполагали, что DNS-регистраторы не позволят людям регистрировать похожие имена, а регистраторы считали, что это проблема поставщиков браузеров - иметь однозначные визуальные представления в адресной строке.

В 2002 году значимость проблемы наконец признали все вовлеченные стороны. В том году Евгений Габрилович и Алекс Гонтмахер опубликовали работу "The Homograph Attack",¹¹ подробно исследовавшую эту уязвимость. Они отметили, что любые обходные меры на уровне регистратора, даже если их реализовать, будут иметь роковой изъян. Злоумышленник всегда мог бы купить добропорядочный домен верхнего уровня, а затем на собственном сервере имен настроить запись поддомена, которая после применения преобразования IDNA декодировалась бы в строку, визуально идентичную `example.com/`, где последний символ был бы всего лишь нефункциональным двойником настоящего ASCII-слеши. Результат выглядел бы так:

`http://example.com/.wholesome-domain.com/`

Это только выглядит как настоящий слеш.

Регистратор ничего не может сделать, чтобы предотвратить эту атаку, и мяч оказывается на стороне поставщиков браузеров. Но какие именно варианты у них есть?

Как выясняется, немного. Теперь мы понимаем, что плохо продуманный стандарт IDNA нельзя исправить простым и безболезненным способом. Разработчики браузеров ответили на этот риск возвратом к непонятному Punycode, когда локаль пользователя не соответствует письменности, встреченной в конкретной DNS-метке, что вызывает проблемы при просмотре иностранных сайтов или при использовании импортированных либо просто неправильно настроенных компьютеров; разрешением IDNA только в некоторых национальных доменах верхнего уровня, что исключает использование интернационализированных доменных имен в `.com` и других заметных TLD; а также занесением в черный список некоторых "плохих" символов, напоминающих слеша, точки, пробелы и так далее, что является бесплодным занятием с учетом количества шрифтов, используемых по всему миру.

Эти меры достаточно радикальны, чтобы сильно затруднить принятие интернационализированных доменных имен, вероятно до такой степени, что продолжающееся существование стандарта вызывает больше проблем безопасности, чем приносит реальных преимуществ удобства для неанглоязычных пользователей.

Распространенные схемы URL и их функции

Оставим позади странный мир разбора URL и вернемся к основам. Ранее в этой главе мы подразумевали, что некоторые схемы могут иметь неожиданные последствия для безопасности и что из-за этого любое веб-приложение, обрабатывающее предоставленные пользователем URL, должно быть осторожным. Чтобы объяснить этот момент чуть лучше, полезно рассмотреть все схемы URL, обычно поддерживаемые в типичной браузерной среде. Их можно объединить в четыре базовые группы.

Поддерживаемые браузером протоколы получения документов

Эти схемы, обрабатываемые браузером внутренне, предлагают способ получить произвольное содержимое с использованием конкретного транспортного протокола, а затем отобразить его общей логикой отображения уровня браузера. Это самая элементарная и самая ожидаемая функция URL.

Список обычно поддерживаемых схем в этой категории удивительно короток: `http:` (RFC 2616), основной транспортный режим, используемый в Web и тема следующей главы этой книги; `https:`, зашифрованная версия HTTP (RFC 2818,12); и `ftp:`, более старый протокол передачи файлов (RFC 959,13). Все браузеры также поддерживают `file:` (раньше также известную как `local:`), системно-зависимый метод доступа к локальной файловой системе или общим ресурсам NFS и SMB. Впрочем, эта последняя схема обычно напрямую недоступна со страниц, происходящих из Интернета.

Два дополнительных малоизвестных случая также заслуживают краткого упоминания: встроенная поддержка схемы `gopher:`, одного из неудавшихся предшественников Web (RFC 1436,14), которая все еще присутствует в Firefox, и `shttp:`, альтернативный провалившийся вариант HTTPS (RFC 2660,15), все еще распознаваемый Internet Explorer, но сегодня просто отображаемый на HTTP.

Протоколы, заявленные сторонними приложениями и подключаемыми модулями

Для этих схем совпадающие URL просто передаются внешним специализированным приложениям, реализующим такие функции, как воспроизведение медиа, просмотр документов или IP-телефония. На этом участие браузера в основном заканчивается.

Сегодня существуют десятки внешних обработчиков протоколов, и чтобы покрыть их все, понадобилась бы еще одна толстая книга. Среди самых распространенных примеров - схема `acrobat:`, предсказуемо направляемая в Adobe Acrobat Reader; схемы `callto:` и `sip:`, заявляемые всевозможными мессенджерами и телефонным ПО; схемы `daap:`, `itpc:` и `itms:`, используемые Apple iTunes; протоколы `mailto:`, `news:` и `nntp:`, заявляемые почтовыми и Usenet-клиентами; протоколы `mmst:`, `mmsu:`, `msbd:` и `rtsp:` для потоковых медиаплееров и так далее. Браузеры тоже иногда входят в этот список. Упомянутая ранее схема `firefoxurl:` запускает Firefox из другого браузера, а `cf:` дает доступ к Chrome из Internet Explorer.

По большей части, когда эти схемы появляются в URL, они обычно не влияют на безопасность веб-приложений, которые позволяют им проходить, хотя это не гарантировано, особенно в случае содержимого, поддерживаемого подключаемыми модулями. Стоит отметить, что сторонние обработчики протоколов обычно печально известны ошибками и иногда используются для компрометации операционной системы. Поэтому ограничение возможности перехода к загадочным протоколам - обычная вежливость по отношению к пользователю любого достаточно заслуживающего доверия сайта.

Неинкапсулирующие псевдопротоколы

Целый набор протоколов зарезервирован для предоставления удобного доступа к движку сценариев браузера и другим внутренним функциям без фактического получения удаленного содержимого и, возможно, без создания изолированного контекста документа для отображения результата. Многие из этих псевдопротоколов сильно зависят от браузера и либо недоступны напрямую из Интернета, либо неспособны причинить вред. Однако из этого правила есть несколько важных исключений.

Пожалуй, самое известное исключение - схема `javascript:` (в ранние годы в браузерах Netscape также доступная под псевдонимами вроде `livescript:` или `mocha:`). Эта схема дает доступ к движку программирования JavaScript в контексте текущего просматриваемого сайта. В Internet Explorer `vbscript:` предлагает похожие возможности через проприетарный интерфейс Visual Basic.

Другой важный случай - протокол `data:` (RFC 2397,16), позволяющий создавать короткие встроенные документы без дополнительных сетевых запросов и иногда наследующий значительную часть их операционного контекста от ссылающейся страницы. Пример URL `data::`

```
data:text/plain,Why,%20hello%20there!
```

Эти внешне доступные псевдо-URL имеют острую значимость для безопасности сайта. При переходе к ним их полезная нагрузка может выполняться в контексте исходного домена, возможно похитив чувствительные данные или изменив внешний вид страницы для затронутого пользователя. Конкретные возможности браузерных языков сценариев мы обсудим в главе 6, но, как можно ожидать, они значительны. С другой стороны, правила наследования контекста URL являются темой главы 10.

Инкапсулирующие псевдопротоколы

Этот специальный класс псевдопротоколов может использоваться как префикс к любому другому URL, чтобы принудительно включить специальный режим декодирования или отображения полученного ресурса. Пожалуй, самый известный пример - схема `view-source:`, поддерживаемая Firefox и Chrome и используемая для отображения красиво отформатированного исходного текста HTML-страницы. Эта схема используется так:

```
view-source:http://www.example.com/
```

Другие протоколы, работающие похожим образом, включают `jar:`, который позволяет Firefox на лету извлекать содержимое из ZIP-файлов; `wyciwyg:` и `view-cache:`, дающие доступ к кэшированным страницам в Firefox и Chrome соответственно; странную схему `feed:`, предназначенную для доступа к новостным лентам в Safari;¹⁷ и множество плохо документированных протоколов, связанных с подсистемой справки Windows и другими компонентами Microsoft Windows (`hcp:`, `its:`, `mhtml:`, `mk:`, `ms-help:`, `ms-its:` и `ms-itss:`).

Общее свойство многих инкапсулирующих протоколов состоит в том, что они позволяют злоумышленнику спрятать фактический URL, который в конечном счете будет интерпретирован браузером, от наивных фильтров: `view-source:javascript:` или даже `view-source:view-source:javascript:`, за которым следует вредоносный код, - простой способ добиться этого. Могут существовать некоторые ограничения безопасности, призванные ограничить такие трюки, но полагаться на них не следует. Еще одна существенная проблема, особенно часто повторяющаяся с Microsoft `mhtml:`, состоит в том, что использование протокола может игнорировать некоторые директивы содержимого, предоставленные сервером на уровне HTTP, что, возможно, приведет к массовым неприятностям.¹⁸

Заключительное замечание об обнаружении схем

Само количество псевдопротоколов - главная причина, по которой веб-приложениям нужно тщательно проверять предоставленные пользователем URL. Неровные и зависящие от браузера шаблоны разбора URL в сочетании с открытой природой списка поддерживаемых схем означают, что просто заносить известные плохие схемы в черный список небезопасно. Например, проверку на `javascript:` можно обойти, если вставить в это ключевое слово табуляцию или перевод строки, заменить его на `vbscript:` или добавить перед ним другую инкапсулирующую схему.

Разрешение относительных URL

Относительные URL уже несколько раз упоминались ранее в этой главе, и сейчас они тоже заслуживают дополнительного внимания. Причина их существования в том, что почти на каждой веб-странице в Интернете значительное количество URL ссылается на ресурсы, размещенные на том же сервере, возможно в том же каталоге. Было бы неудобно и расточительно требовать, чтобы каждый раз, когда нужна такая ссылка, в документе появлялся полностью квалифицированный URL, поэтому вместо этого используются короткие относительные URL, например `../other_file.txt`. Недостающие детали выводятся из URL ссылающегося документа.

Поскольку относительные URL разрешено использовать ровно в тех же сценариях, в которых может появляться любой абсолютный URL, внутри браузера нужен метод различения этих двух вариантов. Веб-приложениям способность проводить такое различие тоже полезна, потому что большинство типов URL-фильтров, возможно, захотят тщательно проверять только абсолютные URL, а локальные ссылки пропускать как есть.

Спецификация может создать впечатление, что задача очень проста: если строка URL не начинается с допустимого имени схемы, за которым следуют двоеточие и, желательно, допустимая последовательность `//`, ее следует интерпретировать как относительную ссылку. А если контекста для разбора такого относительного URL нет, его следует отвергнуть. Все остальное - безопасная относительная ссылка, верно?

Предсказуемо, все не так просто. Во-первых, как описывалось в предыдущих разделах, принимаемый набор символов в допустимом имени схемы и шаблоны, принимаемые вместо `//`, различаются от одной реализации к другой. Возможно, еще интереснее то, что существует распространенное заблуждение, будто относительные ссылки могут указывать только на ресурсы на том же сервере; на самом деле существует немало других, менее очевидных вариантов относительных URL.

Давайте быстро посмотрим на известные классы относительных URL, чтобы лучше проиллюстрировать эту возможность.

Схема есть, но authority отсутствует (http:foo.txt)

Эта печально известная лазейка упоминается в RFC 3986 и приписывается недосмотру в одной из ранних спецификаций. Хотя эти спецификации описательно классифицировали такие URL как недопустимые абсолютные ссылки, они также предоставили вольный алгоритм разбора ссылок, охотно интерпретирующий их неправильно.

В последней интерпретации такие URL задавали бы новый протокол и путь, строку запроса или идентификатор фрагмента, но копировали бы секцию authority из ссылающегося местоположения. Этот синтаксис принимается несколькими браузерами, но непоследовательно. Например, в некоторых случаях `http:foo.txt` может трактоваться как относительная ссылка, тогда как `https:example.com` может разбираться как абсолютная.

Схемы нет, но authority присутствует (//example.com)

Это еще одна печально известная, сбивающая с толку, но по крайней мере хорошо документированная особенность. В то время как `/example.com` является ссылкой на локальный ресурс на текущем сервере, стандарт заставляет браузеры трактовать `//example.com` как совсем другой случай: ссылку на другую authority в текущем протоколе. В этом сценарии схема будет скопирована из ссылающегося местоположения, а все остальные детали URL будут получены из относительного URL.

Нет схемы, нет authority, но есть путь (../notes.txt)

Это самый распространенный вариант относительной ссылки. Информация о протоколе и authority копируется из ссылающегося URL. Если относительный URL не начинается со слеша, путь также копируется до самого правого `/`. Например, если базовый URL - `http://www.example.com/files/`, путь остается тем же, но в `http://www.example.com/files/index.html` имя файла отбрасывается. Затем добавляется новый путь, а для объединенного значения выполняется стандартная нормализация пути. Строка запроса и идентификатор фрагмента берутся только из относительного URL.

Нет схемы, нет authority, нет пути, но есть строка запроса (?search=bunnies)

В этом сценарии информация о протоколе, authority и пути дословно копируется из ссылающегося URL. Строка запроса и идентификатор фрагмента берутся из относительного URL.

Присутствует только идентификатор фрагмента (#bunnies)

Вся информация, кроме идентификатора фрагмента, дословно копируется из ссылающегося URL; заменяется только идентификатор фрагмента. Переход по относительному URL такого типа при обычных обстоятельствах не вызывает перезагрузку страницы, как отмечалось ранее.

Из-за риска потенциальных недопониманий между URL-фильтрами уровня приложения и браузером при обработке этих типов относительных ссылок хорошая проектная практика - никогда не выводить предоставленные пользователем относительные URL дословно. Там, где это возможно, их следует явно переписывать в абсолютные ссылки, а все проверки безопасности выполнять уже над полученным полностью квалифицированным адресом.

Памятка по инженерии безопасности

При построении совершенно новых URL на основе пользовательского ввода

- Если вы разрешаете предоставленные пользователем данные в пути, строке запроса или идентификаторе фрагмента: если один из разделителей секций пройдет без надлежащего экранирования, URL может дать эффект, отличный от задуманного вами, например привязать одну из видимых пользователю HTML-кнопок к неправильному серверному действию. Можно ошибаться в сторону осторожности: при вставке значения поля, контролируемого злоумышленником, вы можете просто процентно экранировать все, кроме алфавитно-цифровых символов.
- Если вы разрешаете предоставленное пользователем имя схемы или секцию authority: это серьезный риск внедрения кода и фишинга. Применяйте соответствующие правила проверки ввода, изложенные ниже.

При проектировании фильтров URL-ввода

- Относительные URL: запрещайте их или явно переписывайте в абсолютные ссылки, чтобы избежать проблем. Почти любой другой подход с большой вероятностью небезопасен.
- Имя схемы: разрешайте только известные префиксы, такие как `http://`, `https://` или `ftp://`. Не используйте вместо этого черные списки; это крайне небезопасно.
- Секция authority: имя хоста должно содержать только алфавитно-цифровые символы, `-` и `.`, а за ним могут следовать только `/`, `?`, `#` или конец строки. Разрешение чего-либо еще обернется неприятностями. Если вам нужно исследовать имя хоста, обязательно выполняйте правильное совпадение правой подстроки.

В редких случаях вам может потребоваться учитывать IDNA, скобочную запись IPv6, номера портов или HTTP-учетные данные в URL. Если так, вы должны полностью разобрать URL, проверить все секции и отвергнуть аномальные значения, а затем сериализовать их заново в однозначное, каноническое, хорошо экранированное представление.

При декодировании параметров, полученных через URL

- Не предполагайте, что какой-либо конкретный символ будет экранирован только потому, что так говорит стандарт или так делает ваш браузер. Прежде чем возвращать любые значения, полученные из URL, или помещать их внутрь запросов к базе данных, новых URL и так далее, тщательно очищайте их от опасных символов.

Сноски

[1] [\[назад\]](#) Возможно, это связано с опасениями за FTP, который передает учетные данные пользователя без какого-либо кодирования; в этом протоколе перевод строки, переданный как есть, был бы неправильно интерпретирован сервером как начало новой FTP-команды. Другие браузеры могут передавать FTP-учетные данные в несоответствующей стандарту процентно закодированной форме или просто позже удалять проблемные символы.

[2] [\[назад\]](#) В отличие от операционных систем, произошедших от UNIX, Microsoft Windows использует обратные слеша вместо прямых для разделения путей к файлам, например `c:\windows\system32\calc.exe`. Microsoft, вероятно, пыталась компенсировать возможность того, что пользователи будут сбивы с толку необходимостью вводить в Web другой тип слеша, или

надеялась устранить другие возможные несогласованности с URL file: и похожими механизмами, которые напрямую взаимодействовали бы с локальной файловой системой. Другие особенности файловой системы Windows, например нечувствительность к регистру, однако, не воспроизводятся.

[3] [\[назад\]](#) Этот конкретный трюк на основе @ быстро начали использовать для всевозможного онлайн-мошенничества, нацеленного на обычных пользователей. Попытки смягчить его влияние варьировались от грубых и странно специфичных, например отключения URL-аутентификации в Internet Explorer или ее ограничения предупреждениями в Firefox, до довольно разумных, например подсветки имени хоста в адресной строке нескольких браузеров.

[4] [\[назад\]](#) Похожие неканонические кодировки широко использовались для различных видов атак социальной инженерии, и вследствие этого за годы были внедрены разные контрмеры. Как обычно, некоторые из этих контрмер разрушительны, например Firefox напрямую отвергает процентно закодированный текст в именах хостов, а некоторые довольно хороши, например принудительная "канонизация" адресной строки путем декодирования всего ненужно закодированного текста для целей отображения.

Глава 3. Протокол передачи гипертекста

Следующая важнейшая концепция, которую нам нужно обсудить, - Hypertext Transfer Protocol (HTTP): основной механизм передачи в Web и предпочтительный способ обмена документами, на которые ссылаются URL, между серверами и клиентами. Несмотря на слово *hypertext* в названии, HTTP и собственно гипертекстовое содержимое, язык HTML, часто существуют независимо друг от друга. Тем не менее они переплетены иногда неожиданными способами.

История HTTP дает интересное представление об амбициях его авторов и о растущей значимости Интернета. Самый ранний черновик протокола Тима Бернерса-Ли 1991 года (HTTP/0.91) занимал едва ли полторы страницы и не учитывал даже самые интуитивно ожидаемые будущие потребности, например расширяемость, необходимую для передачи данных, отличных от HTML.

Пять лет и несколько итераций спецификации спустя первый официальный стандарт HTTP/1.0 (RFC 1945,2) попытался исправить многие из этих недостатков примерно на 50 плотно набранных страницах текста. Перенесемся в 1999 год: в HTTP/1.1 (RFC 2616,3) семь указанных авторов попытались предвидеть почти все возможные варианты использования протокола, создав труд объемом более 150 страниц. И это еще не все: на момент написания этой книги текущая работа над HTTPbis,4 по сути заменой спецификации HTTP/1.1, насчитывает около 360 страниц. Хотя значительная часть постепенно накопленного содержимого не имеет отношения к современному Web, эта эволюция ясно показывает, что желание прикручивать новые возможности намного сильнее желания вырезать неудачные.

Сегодня все клиенты и серверы поддерживают не вполне точное надмножество HTTP/1.0, а большинство умеет говорить на достаточно полном диалекте HTTP/1.1 с парой прикрученных расширений. Несмотря на то что практической необходимости в этом нет, несколько веб-серверов и все распространенные браузеры также сохраняют обратную совместимость с HTTP/0.9.

Базовый синтаксис HTTP-трафика

На первый взгляд HTTP - довольно простой текстовый протокол, построенный поверх TCP/IP.^[1] Каждая HTTP-сессия инициируется установлением TCP-соединения с сервером, обычно на порт 80, а затем отправкой запроса, описывающего запрошенный URL. В ответ сервер возвращает запрошенный файл и, в самом простом варианте использования, сразу после этого закрывает TCP-соединение.

Изначальный протокол HTTP/0.9 не предусматривал места для обмена какими-либо дополнительными метаданными между участвующими сторонами. Клиентский запрос всегда состоял из одной строки, начинавшейся с GET, за которой следовали путь URL и строка запроса, а завершалась она одним переводом строки CRLF (ASCII-символы 0x0D 0x0A; серверам также рекомендовалось принимать одиночный LF). Пример запроса HTTP/0.9 мог выглядеть так:

```
GET /fuzzy_bunnies.txt
```

В ответ на это сообщение сервер немедленно возвращал бы соответствующую HTML-нагрузку. Спецификация требовала, чтобы серверы переносили строки возвращаемого документа на 80 символов, но этой рекомендации в действительности не следовали.

У подхода HTTP/0.9 есть несколько существенных недостатков. Например, он не дает браузерам способа сообщать языковые предпочтения пользователей, предоставлять список поддерживаемых типов документов и так далее. Он также не дает серверам способа сообщить клиенту, что запрошенный файл не найден, что он перемещен в другое место или что возвращаемый файл

вообще не является HTML-документом. Наконец, схема неудобна для администраторов серверов: когда передаваемая информация URL ограничена только путем и строками запроса, сервер не может размещать несколько сайтов, различающихся именами хостов, под одним IP-адресом, а IP-адреса, в отличие от DNS-записей, обходятся недешево.

Чтобы исправить эти недостатки и оставить место для будущих изменений, стандарты HTTP/1.0 и HTTP/1.1 принимают немного другой формат разговора: первая строка запроса изменена так, чтобы включать информацию о версии протокола, и за ней следуют ноль или более пар `name: value`, также известных как заголовки, каждая из которых занимает отдельную строку. Распространенные заголовки запроса, включаемые в такие запросы, - `User-Agent` (информация о версии браузера), `Host` (имя хоста URL), `Accept` (поддерживаемые MIME-типы документов^[2]), `Accept-Language` (поддерживаемые языковые коды) и `Referer` (поле с ошибкой в написании, указывающее исходную страницу запроса, если она известна).

Эти заголовки завершаются одной пустой строкой, за которой может следовать любая полезная нагрузка, которую клиент хочет передать серверу; ее длину нужно явно указать дополнительным заголовком `Content-Length`. Содержимое полезной нагрузки непрозрачно с точки зрения самого протокола; в HTML это место обычно используется для отправки данных форм в одном из нескольких возможных форматов, хотя это никоим образом не является требованием.

В целом простой запрос HTTP/1.1 может выглядеть так:

```
POST /fuzzy_bunnies/bunny_dispenser.php HTTP/1.1
Host: www.fuzzybunnies.com
User-Agent: Bunny-Browser/1.7
Content-Type: text/plain
Content-Length: 17
Referer: http://www.fuzzybunnies.com/main.html
```

```
I REQUEST A BUNNY
```

Ожидается, что сервер ответит на этот запрос строкой, указывающей поддерживаемую версию протокола, числовой код состояния, используемый для обозначения ошибок и других особых обстоятельств, и необязательное, читаемое человеком сообщение состояния. Далее идет набор самоочевидных заголовков, завершающийся пустой строкой. Ответ продолжается содержимым запрошенного ресурса:

```
HTTP/1.1 200 OK
Server: Bunny-Server/0.9.2
Content-Type: text/plain
Connection: close
```

```
BUNNY WISH HAS BEEN GRANTED
```

RFC 2616 также разрешает сжимать ответ при передаче одним из трех поддерживаемых методов (`gzip`, `compress`, `deflate`), если только клиент явно не отказался от этого, предоставив подходящий заголовок `Accept-Encoding`.

Последствия поддержки HTTP/0.9

Несмотря на улучшения, внесенные в HTTP/1.0 и HTTP/1.1, нежелательное наследие "глупого" протокола HTTP/0.9 живет до сих пор, даже если обычно скрыто от глаз. Спецификация HTTP/1.0 частично виновата в этом, потому что требовала от всех будущих HTTP-клиентов и серверов поддержки исходного недоделанного черновика. В частности, раздел 3.1 говорит:

Клиенты HTTP/1.0 должны... понимать любой допустимый ответ в формате HTTP/0.9 или HTTP/1.0.

В последующие годы RFC 2616 попытался отступить от этого требования (раздел 19.6: "предписание соответствия предыдущим версиям выходит за рамки спецификации протокола"), но, следуя более ранней рекомендации, все современные браузеры продолжают поддерживать и устаревший протокол.

Чтобы понять, почему этот шаблон опасен, вспомните, что серверы HTTP/0.9 отвечают ничем, кроме запрошенного файла. Нет никакого признака, что отвечающая сторона действительно понимает HTTP и хочет обслуживать HTML-документ. С учетом этого проанализируем, что произойдет, если браузер отправит HTTP/1.1-запрос ничего не подозревающему SMTP-сервису, работающему на порту 25 узла example.com:

```
GET /<html><body><h1>Hi! HTTP/1.1
Host: example.com:25
...
```

Поскольку SMTP-сервер не понимает, что происходит, он, скорее всего, ответит так:

```
220 example.com ESMTP
500 5.5.1 Invalid command: "GET /<html><body><h1>Hi! HTTP/1.1"
500 5.1.1 Invalid command: "Host: example.com:25"
...
421 4.4.1 Timeout
```

Все браузеры, готовые следовать RFC, вынуждены принять эти сообщения как тело допустимого ответа HTTP/0.9 и предположить, что возвращенный документ действительно является HTML. Такие браузеры интерпретируют процитированный контролируемый злоумышленником фрагмент, появляющийся в одном из сообщений об ошибке, так, будто он приходит от владельцев законного сайта на example.com. Это глубоко вмешивается в модель безопасности браузера, обсуждаемую в части II этой книги, и поэтому очень плохо.

Особенности обработки переводов строк

Если оставить в стороне радикальные изменения между HTTP/0.9 и HTTP/1.0, позже в игре было внесено еще несколько настроек базового синтаксиса. Пожалуй, наиболее заметно то, что вопреки букве ранних итераций HTTP/1.1 просит клиентов не только учитывать переводы строк в формате CRLF и LF, но и распознавать одиночный символ CR. Хотя два самых популярных веб-сервера, IIS и Apache, игнорируют эту рекомендацию, на клиентской стороне ей следуют все браузеры, кроме Firefox.

Возникшая несогласованность упрощает разработчикам приложений забывание того, что из любых контролируемых злоумышленником значений, появляющихся где-либо в HTTP-заголовках, нужно удалять не только LF, но и CR. Чтобы проиллюстрировать проблему, рассмотрим следующий ответ сервера, где предоставленное пользователем и недостаточно очищенное значение появляется в одном из заголовков:

```
HTTP/1.1 200 OK[CR][LF]
Set-Cookie: last_search_term=[CR][CR]<html><body><h1>Hi![CR][LF]
[CR][LF]
Action completed.
```

Для Internet Explorer этот ответ может выглядеть так:

```
HTTP/1.1 200 OK
Set-Cookie: last_search_term=
<html><body><h1>Hi!

Action completed.
```

Фактически класс уязвимостей, связанных с протаскиванием переводов строк в HTTP-заголовках, будь то из-за этой несогласованности или просто из-за неспособности фильтровать какой-либо тип перевода строки, достаточно распространен, чтобы иметь собственное имя: header injection или response splitting.

Еще одна малоизвестная и потенциально значимая для безопасности настройка - поддержка многострочных заголовков, изменение, введенное в HTTP/1.1. Согласно стандарту, любая строка заголовка, начинающаяся с пробельного символа, рассматривается как продолжение предыдущей. Например:

```
X-Random-Comment: This is a very long string,
so why not wrap it neatly?
```

Многострочные заголовки распознаются в клиентских запросах IIS и Apache, но не поддерживаются Internet Explorer, Safari или Opera. Поэтому любая реализация, которая полагается на этот синтаксис или просто разрешает его в условиях, где на него может повлиять злоумышленник, может попасть в беду. К счастью, это встречается редко.

Прокси-запросы

Прокси используются многими организациями и интернет-провайдерами, чтобы перехватывать, проверять и пересылать HTTP-запросы от имени своих пользователей. Это может делаться для повышения производительности, например позволяя кэшировать некоторые ответы сервера на близкой системе; для применения политик использования сети, например предотвращения доступа к порнографии; или для предоставления отслеживаемого и аутентифицированного доступа к иначе отделенным сетевым средам.

Обычные HTTP-прокси зависят от явной поддержки браузера: приложение нужно настроить так, чтобы оно отправляло измененный запрос прокси-системе вместо попытки говорить с предполагаемым адресатом. Чтобы запросить HTTP-ресурс через такой прокси, браузер обычно отправляет такой запрос:

```
GET http://www.fuzzybunnies.com/ HTTP/1.1
User-Agent: Bunny-Browser/1.7
Host: www.fuzzybunnies.com
...
```

Ключевое различие между приведенным выше примером и обычным синтаксисом - наличие полностью квалифицированного URL в первой строке запроса (`http://www.fuzzybunnies.com/`), указывающего прокси, куда подключиться от имени пользователя. Эта информация несколько избыточна, учитывая, что заголовок `Host` уже задает имя хоста; единственная причина такого перекрытия в том, что механизмы развивались независимо друг от друга. Чтобы не быть обманутыми сговорившимися клиентами и серверами, прокси должны либо исправлять любые несовпадающие заголовки `Host`, чтобы они соответствовали URL запроса, либо связывать кэшированное содержимое с конкретной парой URL-Host, а не только с одним из этих значений.

Многие HTTP-прокси также позволяют браузерам запрашивать не-HTTP ресурсы, например FTP-файлы или каталоги. В таких случаях прокси оборачивает ответ в HTTP и, возможно, преобразует его в HTML, если это уместно, перед возвратом пользователю.^[3] Тем не менее если прокси не понимает запрошенный протокол или если ему просто неуместно заглядывать в обмениваемые данные, например внутри зашифрованных сессий, нужно использовать другой подход. Для этой цели зарезервирован специальный тип запроса, `CONNECT`, но в RFC HTTP/1.1 он далее не объясняется. Соответствующий синтаксис запроса вместо этого изложен в отдельной черновой спецификации 1998 года.⁵ Он выглядит так:

```
CONNECT www.fuzzybunnies.com:1234 HTTP/1.1
User-Agent: Bunny-Browser/1.7
...
```

Если прокси хочет и может подключиться к запрошенному назначению, он подтверждает этот запрос специальным кодом HTTP-ответа, и роль этого протокола заканчивается. В этот момент браузер начинает отправлять и получать необработанные двоичные данные внутри установленного TCP-потока; прокси, в свою очередь, должен без разбора пересылать трафик между двумя конечными точками.

Примечание. Смешно, но из-за тонкого упущения в черновой спецификации многие браузеры неправильно обрабатывали незашифрованные ответы об ошибках, исходящие от прокси во время попытки установить зашифрованное соединение. Затронутые реализации интерпретировали такие plaintext-ответы так, будто они пришли от сервера назначения по защищенному каналу. Этот сбой фактически устранял все гарантии, связанные с использованием зашифрованных коммуникаций в Web. Чтобы заметить и исправить этот дефект, понадобилось больше десятилетия.⁶

Несколько других классов прокси более низкого уровня не используют HTTP для прямого общения с браузером, но тем не менее проверяют обмениваемые HTTP-сообщения, чтобы кэшировать содержимое или применять определенные правила. Канонический пример - прозрачный прокси, который бесшумно перехватывает трафик на уровне TCP/IP. Подход прозрачных прокси необычайно опасен: такой прокси может посмотреть на IP назначения и заголовок `Host`, отправленные в перехваченном соединении, но у него нет способа сразу понять, действительно ли этот IP назначения связан с указанным именем сервера. Если не выполнить дополнительный поиск и сопоставление, сговорившиеся клиенты и серверы могут устроить с таким поведением настоящий праздник. Без этих дополнительных проверок злоумышленнику нужно просто подключиться к своему домашнему серверу и отправить вводящий в заблуждение заголовок `Host`:

www.google.com, чтобы ответ был заэкширован для всех остальных пользователей так, будто он действительно пришел с www.google.com.

Разрешение дублирующихся или конфликтующих заголовков

Несмотря на относительную многословность, RFC 2616 плохо объясняет, как соответствующий стандарту парсер должен разрешать потенциальные неоднозначности и конфликты в данных запроса или ответа. Раздел 19.2 этого RFC ("Tolerant Applications") рекомендует расслабленный и устойчивый к ошибкам разбор некоторых полей в "однозначных" случаях, но значение самого термина, скажем так, не особенно однозначно.

Например, из-за отсутствия рекомендаций на уровне спецификации примерно половина браузеров предпочтет первое появление конкретного HTTP-заголовка, а остальные - последнее, гарантируя, что почти любая уязвимость внедрения заголовка, какой бы ограниченной она ни была, будет эксплуатироваться по крайней мере для некоторого процента целевых пользователей. На стороне сервера ситуация столь же случайна: Apache учтет первый увиденный заголовок Host, тогда как IIS полностью отвергнет запрос с несколькими экземплярами этого поля.

В связи с этим соответствующие RFC не содержат явного запрета на смешивание потенциально конфликтующих заголовков HTTP/1.0 и HTTP/1.1 и не требуют от серверов или клиентов HTTP/1.0 игнорировать весь синтаксис HTTP/1.1. Из-за такой конструкции трудно предсказать результат косвенных конфликтов между директивами HTTP/1.0 и HTTP/1.1, отвечающими за одно и то же, например Expires и Cache-Control.

Наконец, в некоторых редких случаях разрешение конфликтов заголовков описано в спецификации очень четко, но гораздо труднее понять цель разрешения таких конфликтов в принципе. Например, клиенты HTTP/1.1 обязаны отправлять заголовок Host во всех запросах, но серверы, а не только прокси, также обязаны распознавать абсолютные URL в первой строке запроса вместо традиционного метода, содержащего только путь и строку запроса. Это правило допускает такую диковинку:

```
GET http://www.fuzzybunnies.com/ HTTP/1.1
Host: www.bunnyoutlet.com
```

В этом случае раздел 5.2 RFC 2616 предписывает клиентам игнорировать нефункциональный, но все равно обязательный, заголовок Host, и многие реализации следуют этому совету. Проблема в том, что нижележащие приложения, скорее всего, не знают об этой особенности и вместо этого могут принимать довольно важные решения на основе проверенного значения заголовка.

Примечание. Жалюсь на упущения в RFC для HTTP, важно признавать, что альтернативы могут быть не менее проблемными. В нескольких сценариях, описанных в этом RFC, желание явно предписать обработку некоторых пограничных случаев привело к откровенно абсурдным результатам. Один такой пример - рекомендации по разбору дат в некоторых HTTP-заголовках, по требованию раздела 3.3 RFC 1945. Получившаяся реализация (файл prtime.c в кодовой базе Firefox7) состоит почти из 2,000 строк чрезвычайно запутанного и нечитаемого C-кода только для того, чтобы достаточно отказоустойчиво расшифровать указанную дату, время и часовой пояс, например для решения об истечении срока действия содержимого кэша.

Значения заголовков, разделенные точкой с запятой

Некоторые HTTP-заголовки, такие как `Cache-Control` или `Content-Disposition`, используют синтаксис с разделением точкой с запятой, чтобы вписать несколько отдельных пар `name=value` в одну строку. Причина разрешения такой вложенной нотации неясна, но, вероятно, она связана с верой в то, что это будет более эффективный или более интуитивный подход, чем использование нескольких отдельных заголовков, которые всегда должны были бы идти рука об руку.

Некоторые варианты использования, описанные в RFC 2616, разрешают `quoted-string` как правый параметр в таких парах. `Quoted-string` - синтаксис, в котором последовательность произвольных печатаемых символов окружена двойными кавычками, действующими как разделители. Естественно, сам символ кавычки не может появляться внутри строки, но, что важно, точка с запятой или пробел могут, позволяя отправлять многие в ином случае проблемные значения как есть.

К несчастью для разработчиков, Internet Explorer не особенно хорошо справляется с синтаксисом `quoted-string`, фактически делая эту схему кодирования бесполезной. Браузер разберет следующую строку, которая должна указывать, что ответ является скачиваемым файлом, а не встроенным документом, неожиданным образом:

```
Content-Disposition: attachment; filename="evil_file.exe;.txt"
```

В реализации Microsoft имя файла будет усечено на символе точки с запятой и будет выглядеть как `evil_file.exe`. Такое поведение создает потенциальную опасность для любого приложения, которое полагается на проверку или добавление "безопасного" расширения файла к контролируемому злоумышленником имени файла и в остальном правильно проверяет наличие символа кавычки и переводов строк в этой строке.

Примечание. Дополнительный механизм `quoted-pair` предоставлен, чтобы позволить безопасно использовать кавычки и любые другие символы в строке при префиксе обратным слешем. Однако этот механизм, похоже, специфицирован неправильно и не поддерживается ни одним крупным браузером, кроме Opera. Чтобы `quoted-pair` работал правильно, случайные символы \ пришлось бы запретить в `quoted-string`, чего нет в RFC 2616. `Quoted-pair` также разрешает экранировать любой токен типа `CHAR`, включая переводы строк, что несовместимо с другими правилами разбора HTTP.

Также стоит отметить, что, когда в одном HTTP-заголовке обнаруживаются дублирующиеся поля, разделенные точкой с запятой, их порядок приоритета RFC не определяет. В случае `filename=` в `Content-Disposition` все основные браузеры используют первое появление. Но в других местах последовательности мало. Например, при извлечении значения `URL=` из заголовка `Refresh`, используемого для принудительной перезагрузки страницы через заданное время, Internet Explorer 6 откатится к последнему экземпляру, тогда как все остальные браузеры предпочтут первый. А при обработке `Content-Type` Internet Explorer, Safari и Opera используют первое значение `charset=`, тогда как Firefox и Chrome полагаются на последнее.

Примечание. Пища для размышлений: увлекательный, но в основном не связанный с безопасностью обзор десятков несогласованностей, связанных с обработкой всего одного HTTP-заголовка - `Content-Disposition`, - можно найти на странице, поддерживаемой Юлианом Решке: <http://greenbytes.de/tech/tc2231/>.

Набор символов заголовков и схемы кодирования

Как и документы, заложившие основу обработки URL, все последующие спецификации HTTP в значительной степени избегали темы работы с не-US-ASCII символами внутри значений заголовков. Есть несколько правдоподобных сценариев, где неанглийский текст может законно появиться в этом контексте, например имя файла в Content-Disposition, но когда речь заходит об этом, ожидаемое поведение браузеров по сути не определено.

Первоначально RFC 1945 разрешал токену TEXT, примитиву, широко используемому для определения синтаксиса других полей, содержать 8-битные символы, давая такое определение:

```
OCTET = <any 8-bit sequence of data>
CTL = <any US-ASCII control character
      (octets 0 - 31) and DEL (127)>
TEXT = <any OCTET except CTLs,
       but including LWS>
```

Далее RFC давал загадочный совет: когда в поле TEXT встречаются не-US-ASCII символы, клиенты и серверы могут интерпретировать их как ISO-8859-1, стандартную западноевропейскую кодировку, но не обязаны. Позднее RFC 2616 скопировал ту же спецификацию токенов TEXT, но добавил примечание, что строки не в ISO-8859-1 должны кодироваться с использованием формата, изложенного в RFC 2047,8 первоначально созданного для почтовых коммуникаций. Справедливо; в этой простой схеме закодированная строка открывается префиксом =?, за которым следуют имя набора символов, индикатор типа кодирования ?q? или ?b? (quoted-printable^[4] или base64^[5] соответственно), а затем сама закодированная строка. Последовательность завершается терминатором ?=. Пример может быть таким:

```
Content-Disposition: attachment; filename="?utf-8?q?Hi=21.txt?="
```

Примечание. RFC также следовало бы указать, что любые посторонние шаблоны =?...?= никогда нельзя разрешать как есть в соответствующих заголовках, чтобы избежать непреднамеренного декодирования значений, которые на самом деле изначально не были закодированы.

К сожалению, поддержка этого кодирования RFC 2047 неравномерна. Оно распознается в некоторых заголовках Firefox и Chrome, но другие браузеры менее сговорчивы. Internet Explorer вместо этого предпочитает распознавать URL-подобное процентное кодирование в поле Content-Disposition, привычку, которую перенял и Chrome, и по умолчанию использует в этом случае UTF-8. Firefox и Opera, с другой стороны, предпочитают поддерживать своеобразный процентно закодированный синтаксис, предложенный в RFC 2231,9 разительно отклоняющийся от того, как должен выглядеть синтаксис HTTP:

```
Content-Disposition: attachment; filename*=utf-8'en-us'Hi%21.txt
```

Внимательные читатели могут заметить, что нет единой схемы кодирования, поддерживаемой всеми браузерами одновременно. Эта ситуация побуждает некоторых разработчиков веб-приложений прибегать к использованию необработанных значений со старшим битом в HTTP-заголовках, обычно интерпретируемых как UTF-8, но это несколько небезопасно. Например, в Firefox давний сбой приводит к искажению текста UTF-8 при помещении в заголовок Cookie, позволяя внедренным злоумышленником разделителям cookie материализоваться в неожиданных местах.¹⁰ Иными словами, простых и надежных решений этого хаоса нет.

Говоря о кодировках символов, вероятно, стоит упомянуть проблему обработки символа NUL (0x00). Этот символ, используемый как терминатор строки во многих языках программирования, технически запрещен в HTTP-заголовках, за исключением упомянутого выше неисправного синтаксиса quoted-pair, но, как вы помните, парсерам рекомендуется быть терпимыми. Когда

этому символу разрешают пройти, он, скорее всего, вызывает неожиданные побочные эффекты. Например, заголовки Content-Disposition усекаются на NUL в Internet Explorer, Firefox и Chrome, но не в Opera или Safari.

Поведение заголовка Referer

Как упоминалось ранее в этой главе, HTTP-запросы могут включать заголовок Referer. Этот заголовок содержит URL документа, который каким-то образом вызвал текущий переход. Он должен помогать в некоторых задачах диагностики и способствовать росту Web, подчеркивая перекрестные ссылки между связанными веб-страницами.

К сожалению, этот заголовок также может раскрывать некоторым недружественным сторонам информацию о привычках пользователя при просмотре страниц и может утекать чувствительную информацию, закодированную в параметрах строки запроса URL на ссылающейся странице. Из-за этих опасений и последующих плохих советов о том, как их смягчать, заголовок часто неправильно используют для целей безопасности или применения политики, но он для этой задачи не годится. Главная проблема в том, что невозможно отличить клиента, который не предоставляет заголовок из-за пользовательских настроек приватности, от клиента, который не предоставляет его из-за типа выполняемого перехода, и от клиента, которого вредоносный ссылающийся сайт намеренно заставил скрыть эту информацию.

Обычно этот заголовок включается в большинство HTTP-запросов и сохраняется при HTTP-редиректах, за исключением следующих сценариев:

- После органического ввода нового URL в адресную строку или открытия страницы из закладок.
- Когда переход исходит из документа с псевдо-URL, такого как data: или javascript:.
- Когда запрос является результатом перенаправления, управляемого заголовком Refresh, но не перенаправления на основе Location.
- Всякий раз, когда ссылающийся сайт зашифрован, а запрошенная страница - нет. Согласно разделу 15.1.2 RFC 2616 это делается по соображениям приватности, но в этом не так много смысла. Строка Referer все равно раскрывается третьим сторонам, когда пользователь переходит с одного зашифрованного домена на другой несвязанный зашифрованный домен, и будьте уверены: использование шифрования не является синонимом надежности.
- Если пользователь решает заблокировать или подделать заголовок, изменив настройки браузера или установив ориентированный на приватность подключаемый модуль.

Как должно быть очевидно, четыре из пяти этих условий может целенаправленно вызвать любой недобросовестный сайт.

Типы HTTP-запросов

Первоначальный черновик HTTP/0.9 предоставлял один метод, или "глагол", для запроса документа: GET. Последующие предложения экспериментировали со все более странным набором методов, чтобы разрешить взаимодействия, отличные от получения документа или запуска сценария, включая такие диковинки, как SHOWMETHOD, CHECKOUT или, почему бы и нет, SPACEJUMP.¹¹

Большинство этих мысленных экспериментов было отброшено в HTTP/1.1, который остановился на более управляемом наборе из восьми методов. Для большей части современного Web сколько-нибудь важны только первые два типа запросов - GET и POST.

GET

Метод GET должен обозначать получение информации. На практике он используется почти для всех взаимодействий клиент-сервер в ходе обычной сессии просмотра. Обычные запросы GET не несут предоставляемой браузером полезной нагрузки, хотя строго это и не запрещено.

Ожидание состоит в том, что запросы GET, цитируя RFC, не должны иметь "значимости выполнения действия, отличного от получения", то есть не должны вносить постоянные изменения в состояние приложения. Это требование становится все менее осмысленным в современных веб-приложениях, где состояние приложения часто даже не управляется целиком на серверной стороне; вследствие этого разработчики приложений широко игнорируют совет.^[6]

Примечание. В HTTP/1.1 клиенты могут попросить сервер о любом наборе, возможно несмежных или перекрывающихся, фрагментов целевого документа, указав заголовок Range в GET и, реже, в некоторых других типах запросов. Сервер не обязан соглашаться, но там, где механизм доступен, браузеры могут использовать его для возобновления прерванных загрузок.

POST

Метод POST предназначен для отправки информации, главным образом HTML-форм, на сервер для обработки. Поскольку действия POST могут иметь постоянные побочные эффекты, многие браузеры просят пользователя подтвердить повторную загрузку любого содержимого, полученного с помощью POST, но в основном GET и POST используются почти взаимозаменяемо.

Запросы POST обычно сопровождаются полезной нагрузкой, длина которой указывается заголовком Content-Length. В случае обычного HTML полезная нагрузка может состоять из URL-кодированных или MIME-кодированных данных формы, формата, подробно описанного в главе 4, хотя, опять же, на уровне HTTP синтаксис никак особенно не ограничен.

HEAD

HEAD - редко используемый тип запроса, по сути идентичный GET, но возвращающий только HTTP-заголовки, а не фактическую полезную нагрузку запрошенного содержимого. Браузеры обычно не выпускают запросы HEAD самостоятельно, но метод иногда применяют поисковые роботы и другие автоматизированные инструменты, например чтобы проверить существование файла или время его изменения.

OPTIONS

OPTIONS - метазапрос, который возвращает набор поддерживаемых методов для конкретного URL или для *, то есть для сервера в целом, в заголовке ответа. Метод OPTIONS почти никогда не используется на практике, кроме определения отпечатка сервера; из-за его ограниченной ценности возвращаемая информация может быть не очень точной.

Примечание. Ради полноты нужно отметить, что запросы OPTIONS также являются краеугольным камнем предлагаемой схемы авторизации междоменных запросов и как таковые вскоре могут получить некоторую заметность. Мы вернемся к этой схеме и рассмотрим многие другие будущие возможности безопасности браузеров в главе 16.

PUT

Запрос PUT должен позволять загружать файлы на сервер по указанному целевому URL. Поскольку браузеры не поддерживают PUT, намеренные возможности загрузки файлов почти всегда реализуются через POST к серверному скрипту, а не с помощью этого теоретически более элегантного подхода.

Тем не менее некоторые не-веб HTTP-клиенты и серверы могут использовать PUT для собственных целей. Не менее интересно, что некоторые веб-серверы могут быть неправильно настроены на безразборную обработку запросов PUT, создавая очевидный риск безопасности.

DELETE

DELETE - самоочевидный метод, дополняющий PUT, и столь же редко встречающийся на практике.

TRACE

TRACE - форма запроса "ping", которая возвращает информацию обо всех прокси-переходах, участвовавших в обработке запроса, а также отражает исходный запрос. Запросы TRACE не выпускаются веб-браузерами и редко используются в законных целях. Основное применение TRACE - тестирование безопасности, где он может раскрыть интересные детали внутренней архитектуры HTTP-серверов в удаленной сети. Именно по этой причине администраторы серверов часто отключают этот метод.

CONNECT

Метод CONNECT зарезервирован для установления не-HTTP соединений через HTTP-прокси. Он не предназначен для отправки напрямую серверам. Если поддержка запросов CONNECT случайно включена на конкретном сервере, она может создать риск безопасности, предоставляя злоумышленнику способ туннелировать TCP-трафик в иначе защищенную сеть.

Другие HTTP-методы

Ряд других методов запроса могут применять другие не браузерные приложения или расширения браузеров; самым популярным набором расширений HTTP, вероятно, является WebDAV, протокол авторинга и управления версиями, описанный в RFC 4918.12

Кроме того, API XMLHttpRequest номинально позволяет клиентскому JavaScript выполнять запросы с почти произвольными методами к исходному серверу, хотя эта последняя возможность сильно ограничена в некоторых браузерах; мы рассмотрим это в главе 9.

Коды ответов сервера

Раздел 10 RFC 2616 перечисляет почти 50 кодов состояния, из которых сервер может выбирать при построении ответа. Около 15 из них используются в реальной жизни, а остальные служат для обозначения все более странных или маловероятных состояний, таких как 402 Payment Required или 415 Unsupported Media Type. Большинство состояний, перечисленных в RFC, плохо отображается на поведение современных веб-приложений; единственная причина их существования в том, что кто-то надеялся: когда-нибудь они пригодятся.

Несколько кодов стоит запомнить, потому что они распространены или несут особый смысл, как обсуждается ниже.

200-299: успех

Этот диапазон кодов состояния используется, чтобы указать успешное выполнение запроса:

200 OK. Это обычный ответ на успешный GET или POST. Браузер покажет пользователю возвращенную далее полезную нагрузку или обработает ее каким-либо другим, зависящим от контекста способом.

204 No Content. Этот код иногда используется, чтобы указать успешный запрос, на который не ожидается подробного ответа. Ответ 204 прерывает переход к URL, который его вызвал, и оставляет пользователя на исходной странице.

206 Partial Content. Этот код похож на 200, за исключением того, что возвращается серверами в ответ на диапазонные запросы. У браузера уже должна быть часть документа, иначе он не отправил бы диапазонный запрос, и обычно он изучит заголовок ответа Content-Range, чтобы собрать документ перед дальнейшей обработкой.

300-399: перенаправление и другие сообщения состояния

Эти коды используются для передачи разнообразных состояний, которые не указывают на ошибку, но требуют специальной обработки на стороне браузера:

301 Moved Permanently, 302 Found, 303 See Other. Этот ответ инструктирует браузер повторить запрос в новом местоположении, указанном в заголовке ответа `Location`. Несмотря на различия, проведенные в RFC, при встрече с любым из этих кодов ответа все современные браузеры заменяют POST на GET, удаляют полезную нагрузку, а затем автоматически повторно отправляют запрос.

Примечание. Сообщения перенаправления могут содержать полезную нагрузку, но если она есть, это сообщение не будет показано пользователю, если только перенаправление невозможно, например из-за отсутствующего или неподдерживаемого значения `Location`. Фактически в некоторых браузерах показ сообщения может быть подавлен даже в таком сценарии.

304 Not Modified. Этот не являющийся перенаправлением ответ сообщает клиенту, что запрошенный документ не изменился относительно копии, которая уже есть у клиента. Такой ответ встречается после условных запросов с заголовками вроде `If-Modified-Since`, которые отправляются для повторной проверки кэша документов браузера. Тело ответа пользователю не показывается. Если сервер отвечает так на безусловный запрос, результат будет зависеть от браузера и может быть забавным; например, Opera покажет нефункциональное окно загрузки.

307 Temporary Redirect. Похож на 302, но, в отличие от других режимов перенаправления, браузеры не будут понижать POST до GET при следовании перенаправлению 307. Этот код нечасто используется в веб-приложениях, и некоторые браузеры ведут себя при его обработке не очень последовательно.

400-499: ошибка на стороне клиента

Этот диапазон кодов используется для обозначения условий ошибки, вызванных поведением клиента:

400 Bad Request и связанные сообщения. Сервер не способен или не желает обработать запрос по некоторой неуказанной причине. Полезная нагрузка ответа обычно в какой-то мере объясняет проблему и, как правило, обрабатывается браузером так же, как ответ 200.

Существуют и более конкретные варианты, такие как 411 `Length Required`, 405 `Method Not Allowed` или 414 `Request-URI Too Long`. Почему для отсутствия `Content-Length` там, где он требуется, выделен отдельный код ответа 411, а для отсутствия `Host` полагается только общий 400, остается гадать.

401 Unauthorized. Этот код означает, что пользователю нужно предоставить учетные данные HTTP-аутентификации протокольного уровня, чтобы получить доступ к ресурсу. Обычно браузер затем попросит пользователя ввести данные входа и покажет тело ответа только в том случае, если процесс аутентификации не удался. Этот механизм вскоре будет подробнее объяснен в разделе "HTTP-аутентификация".

403 Forbidden. Запрошенный URL существует, но доступ к нему невозможен по причинам, отличным от неправильной HTTP-аутентификации. Причины могут включать недостаточные разрешения файловой системы, правило конфигурации, препятствующее обработке этого запроса, или какие-либо недостаточные учетные данные, например недействительные cookies или нераспознанный исходный IP-адрес. Ответ обычно будет показан пользователю.

404 Not Found. Запрошенный URL не существует. Тело ответа обычно показывается пользователю.

500-599: ошибка на стороне сервера

Это класс сообщений об ошибках, возвращаемых в ответ на проблемы на стороне сервера:

500 Internal Server Error, 503 Service Unavailable и так далее. Сервер испытывает проблему, мешающую ему выполнить запрос. Это может быть временное состояние, результат неправильной настройки или просто эффект запроса к неожиданному местоположению. Ответ обычно показывается пользователю.

Последовательность сигнализации HTTP-кодов

Поскольку между возвратом большинства кодов 2xx, 4xx и 5xx нет немедленно наблюдаемой разницы, эти значения выбирают без особого рвения. В частности, веб-приложения печально известны тем, что возвращают 200 OK даже тогда, когда произошла ошибка приложения и она сообщается на получившейся странице. Это один из многих факторов, которые делают автоматизированное тестирование веб-приложений гораздо сложнее, чем оно должно быть.

В редких случаях для конкретных применений изобретают новые и не обязательно уместные HTTP-коды. Некоторые из них стандартизованы, например пара сообщений, введенных в RFC WebDAV.13 Другие, такие как статус Microsoft Exchange 449 Retry With, не стандартизованы.

Keepalive-сессии

Первоначально HTTP-сессии задумывались как одноразовые: один запрос на каждое TCP-соединение, промыть и повторить. Накладные расходы на многократное выполнение трехшагового TCP-рукопожатия, а в традиционной модели Unix-сервера еще и на порождение нового процесса, вскоре оказались узким местом, поэтому HTTP/1.1 вместо этого стандартизовал идею keepalive-сессий.

Существующий протокол уже давал серверу понимание того, где заканчивается клиентский запрос: пустая строка, за которой необязательно следует Content-Length байт данных. Но чтобы продолжать использовать существующее соединение, клиенту тоже нужно было знать то же самое о возвращаемом документе; завершение соединения больше не могло служить индикатором. Поэтому keepalive-сессии требуют, чтобы ответ тоже включал заголовок Content-Length, всегда указывающий объем данных, которые последуют дальше. Получив именно столько байтов полезной нагрузки, клиент понимает, что можно отправлять второй запрос и начинать ждать еще один ответ.

Хотя этот механизм очень полезен с точки зрения производительности, его конструкция усиливает влияние ошибок HTTP request splitting и response splitting. Клиенту и серверу обманчиво легко рассинхронизироваться в вопросе о том, какой ответ относится к какому запросу. Для иллюстрации рассмотрим сервер, который считает, что отправляет один HTTP-ответ следующей структуры:

```
HTTP/1.1 200 OK[CR][LF]
Set-Cookie: term=[CR]Content-Length: 0[CR][CR]HTTP/1.1 200 OK[CR]Gotcha: Yup[CR][LF]
Content-Length: 17[CR][LF]
[CR][LF]
Action completed.
```

Клиент же может увидеть два ответа и связать первый со своим самым текущим запросом, а второй - с еще не отправленным запросом^[7], который может даже быть адресован другому имени хоста на том же IP:

```
HTTP/1.1 200 OK
Set-Cookie: term=
Content-Length: 0

HTTP/1.1 200 OK
Gotcha: Yup
Content-Length: 17

Action completed.
```

Если этот ответ увидит кэширующий HTTP-прокси, неправильный результат также может быть глобально закэширован и возвращен другим пользователям, что действительно плохая новость. Гораздо более безопасная конструкция keepralive-сессий включала бы предварительное указание длины и заголовков, и полезной нагрузки либо использование случайно сгенерированной и непредсказуемой границы для разделения каждого ответа. К сожалению, конструкция не делает ни того ни другого.

Keepalive-соединения используются в HTTP/1.1 по умолчанию, если они явно не отключены (Connection: close), и поддерживаются многими серверами HTTP/1.0 при включении заголовком Connection: keep-alive. И серверы, и браузеры могут ограничивать количество одновременных запросов, обслуживаемых на одном соединении, и задавать максимальное время, в течение которого простаивающее соединение сохраняется.

Передача данных блоками

Существенное ограничение keepalive-сессий на основе Content-Length - необходимость для сервера заранее знать точный размер возвращаемого ответа. При работе со статическими файлами это довольно простая задача, поскольку информация уже доступна в файловой системе. При обслуживании динамически генерируемых данных проблема сложнее, потому что вывод нужно целиком закэшировать перед отправкой клиенту. Задача становится непреодолимой, если полезная нагрузка очень велика или производится постепенно, например при потоковой передаче живого видео. В таких случаях предварительное кэширование для вычисления размера полезной нагрузки просто исключено.

В ответ на эту задачу раздел 3.6.1 RFC 2616 дает серверам возможность использовать Transfer-Encoding: chunked, схему, в которой полезная нагрузка отправляется частями по мере готовности. Длина каждой части документа заранее объявляется шестнадцатеричным целым числом, занимающим отдельную строку, но общая длина документа остается неопределенной, пока не встретится финальный блок нулевой длины.

Пример ответа с передачей блоками может выглядеть так:

```
HTTP/1.1 200 OK
Transfer-Encoding: chunked
...

5
Hello
6
world!
0
```

У поддержки блочной передачи данных нет значимых недостатков, кроме возможности того, что патологически большие блоки вызовут переполнения целых чисел в коде браузера, или необходимости разрешать несовпадения между Content-Length и длиной блока. Спецификация отдает приоритет длине блока, хотя любые попытки изящно обрабатывать эту ситуацию выглядят неразумными. Все популярные браузеры правильно справляются с этими условиями, но новым реализациям нужно беречься.

Поведение кэширования

По причинам производительности и экономии пропускной способности HTTP-клиенты и некоторые посредники охотно кэшируют HTTP-ответы для повторного использования. В ранние дни Web это должно было казаться простой задачей, но она становится все более опасной по мере того, как Web охватывает все больше чувствительной, специфичной для пользователя информации, и по мере того, как эта информация обновляется все чаще.

Раздел 13.4 RFC 2616 утверждает, что запросы GET, на которые получен ответ с диапазоном HTTP-кодов, прежде всего 200 OK и 301 Moved Permanently, могут неявно кэшироваться при отсутствии каких-либо других директив, предоставленных сервером. Такой ответ может храниться в кэше неопределенно долго и может повторно использоваться для любых будущих запросов с тем же методом запроса и URL назначения, даже если другие параметры, такие как заголовки Cookie, отличаются. Существует запрет на кэширование запросов, использующих HTTP-аутентификацию, см. раздел "HTTP-аутентификация", но другие методы аутентификации, такие как cookies, в спецификации не распознаются.

Когда ответ закэширован, реализация может решить повторно проверить его перед повторным использованием, но большую часть времени делать это не требуется. Повторная проверка достигается запросом со специальным условным заголовком, например If-Modified-Since, за которым следует дата, записанная в ранее закэшированном ответе, или If-None-Match, за которым следует непрозрачное значение заголовка ETag, возвращенное сервером с более ранней копией. Сервер может ответить кодом 304 Not Modified или вернуть более новую копию ресурса.

Примечание. Пары заголовков Date/If-Modified-Since и ETag/If-None-Match, в сочетании с Cache-Control: private, предлагают сайтам удобный и совершенно непреднамеренный способ хранить в браузере долгоживущие уникальные токены.¹⁴ Того же можно добиться, поместив уникальный токен внутрь кэшируемого JavaScript-файла и возвращая 304 Not Modified на все будущие условные запросы к местоположению, генерирующему токен. В отличие от специально созданных механизмов вроде HTTP cookies, обсуждаемых в следующем разделе, пользователи почти не контролируют, какая информация хранится в кэше браузера, при каких обстоятельствах и как долго.

Неявное кэширование очень проблематично, и поэтому серверы почти всегда должны прибегать к явным директивам HTTP-кэширования. Чтобы помочь с этим, HTTP/1.0 предоставляет заголовок

Expires, указывающий дату, к которой закешированная копия должна быть отброшена; если это значение равно заголовку Date, предоставленному сервером, ответ некешируем. За пределами этого простого правила связь между Expires и Date не определена: не ясно, следует ли сравнивать Expires с системными часами кеширующей системы, что проблематично, если часы клиента и сервера не синхронизированы, или оценивать на основе разницы Expires - Date, что надежнее, но может перестать работать, если Date случайно пропущен. Firefox и Opera используют последнюю интерпретацию, тогда как другие браузеры предпочитают первую. В большинстве браузеров недопустимое значение Expires также подавляет кеширование, но полагаться на это рискованно.

Клиенты HTTP/1.0 также могут включать заголовок запроса Pragma: no-cache, который прокси может интерпретировать как инструкцию получить новую копию запрошенного ресурса вместо возврата существующей. Некоторые прокси HTTP/1.0 также распознают нестандартный заголовок ответа Pragma: no-cache как инструкцию не создавать копию документа.

В отличие от этого, HTTP/1.1 принимает гораздо более содержательный подход к директивам кеширования, вводя новый заголовок Cache-Control. Заголовок принимает значения вроде public (документ можно публично кешировать), private (прокси не разрешено кешировать), no-cache (что немного сбивает с толку: ответ можно кешировать, но не следует повторно использовать для будущих запросов)^[8] и no-store (вообще никакого кеширования). Директивы публичного и приватного кеширования могут сопровождаться квалификатором вроде max-age, указывающим максимальное время хранения старой копии, или must-revalidate, требующим выполнить условный запрос перед повторным использованием содержимого.

К сожалению, серверам обычно необходимо возвращать и директивы кеширования HTTP/1.0, и директивы HTTP/1.1, потому что некоторые типы устаревших коммерческих прокси неправильно понимают Cache-Control. Чтобы надежно предотвратить кеширование по HTTP, может потребоваться использовать следующий набор заголовков ответа:

```
Expires: [current date]
Date: [current date]
Pragma: no-cache
Cache-Control: no-cache, no-store
```

Когда эти директивы кеширования расходятся, поведение трудно предсказать: некоторые браузеры предпочтут директивы HTTP/1.1 и отдадут приоритет no-cache, даже если за ним ошибочно следует public; другие - нет.

Еще один риск HTTP-кеширования связан с небезопасными сетями, такими как публичные Wi-Fi-сети, которые позволяют злоумышленнику перехватывать запросы к определенным URL и возвращать жертве измененное содержимое с длительным сроком кеширования. Если такой отравленный кэш браузера затем повторно используется в доверенной сети, внедренное содержимое неожиданно всплывет снова. Превратно то, что жертве даже не нужно посещать целевое приложение: ссылку на тщательно выбранный чувствительный домен злоумышленник может внедрить в какой-нибудь другой контекст. Хороших решений этой проблемы пока нет; очистка кэша браузера после посещения Starbucks может оказаться очень хорошей идеей.

Семантика HTTP cookies

HTTP cookies не являются частью RFC 2616, но это одно из самых важных расширений протокола, используемых в Web. Механизм cookies позволяет серверам хранить короткие непрозрачные пары `name=value` в браузере, отправляя заголовок ответа `Set-Cookie`, и получать их обратно в будущих запросах через предоставленный клиентом параметр `Cookie`. Cookies с большим отрывом являются самым популярным способом поддерживать сессии и аутентифицировать пользовательские запросы; это одна из четырех канонических форм *ambient authority*^[9] в Web, остальные формы - встроенная HTTP-аутентификация, проверка IP и клиентские сертификаты.

Первоначально реализованный в Netscape Лу Монтulli примерно в 1994 году и описанный в кратком четырехстраничном черновом документе,¹⁵ этот механизм за последние 17 лет так и не был изложен в надлежащем стандарте. В 1997 году RFC 2109,¹⁶ попытался документировать статус-кво, но несколько необъяснимо также предложил ряд масштабных изменений, которые по сей день делают эту спецификацию существенно несовместимой с фактическим поведением любого современного браузера. Другая амбициозная попытка, `Cookie2`, появилась в RFC 2965,¹⁷ но десятилетие спустя она все еще почти не имеет поддержки на уровне браузеров, и эта ситуация вряд ли изменится. Новая попытка написать достаточно точную спецификацию cookies - RFC 6265,¹⁸ - была завершена незадолго до публикации этой книги, наконец положив конец этому связанному со спецификациями мучению.

Из-за длительного отсутствия настоящих стандартов реальные реализации развивались очень интересными и иногда несовместимыми способами. На практике новые cookies можно задавать с помощью заголовков `Set-Cookie`, за которыми следует одна пара `name=value` и ряд необязательных параметров, разделенных точкой с запятой и определяющих область действия и срок жизни cookie.

Expires. Указывает дату истечения срока действия cookie в формате, похожем на используемый для HTTP-заголовков `Date` или `Expires`. Если cookie отправляется без явной даты истечения, он обычно хранится в памяти в течение сессии браузера, которая, особенно на переносных компьютерах с функцией сна, легко может растянуться на несколько недель. Cookies с определенным сроком действия могут регулярно сохраняться на диск и переживать сессии, если пользовательские настройки приватности явно не предотвращают эту возможность.

Max-age. Этот альтернативный, предложенный RFC механизм истечения срока не поддерживается Internet Explorer и поэтому на практике не используется.

Domain. Этот параметр позволяет задать область действия cookie шире, чем имя хоста, вернувшего заголовок `Set-Cookie`. Точные правила и последствия этого механизма области действия для безопасности рассматриваются в главе 9.

Примечание. Вопреки тому, что подразумевает RFC 2109, при использовании этого параметра невозможно ограничить cookies конкретным именем хоста. Например, `domain=example.com` всегда будет соответствовать и `www.example.com`. Не указывать `domain` - единственный способ создать cookies с областью действия только для хоста, но даже этот подход в Internet Explorer работает не так, как ожидается.

Path. Позволяет ограничить область действия cookie конкретным префиксом пути запроса. Это не жизнеспособный механизм безопасности по причинам, объясненным в главе 9, но он может использоваться для удобства, чтобы одноименные cookies, используемые в разных частях приложения, не конфликтовали друг с другом.

Атрибут Secure. Предотвращает отправку получившегося cookie по незашифрованным соединениям.

Атрибут HttpOnly. Убирает возможность читать cookie через `API document.cookie` в JavaScript. Это расширение Microsoft, хотя теперь оно поддерживается всеми основными браузерами.

При будущих запросах к домену, для которого в cookie jar найдены допустимые cookies, браузеры объединяют все применимые пары `name=value` в один разделенный точкой с запятой заголовок `Cookie`, без каких-либо дополнительных метаданных, и вернут их серверу. Если в конкретном запросе нужно отправить слишком много cookies, будут превышены ограничения размера заголовков, применяемые сервером, и запрос может не удался; способа восстановиться из такого состояния нет, кроме ручной очистки cookie jar.

Любопытно, что у HTTP-серверов нет явного метода удаления ненужных cookies. Однако каждый cookie уникально идентифицируется кортежем `имя-домен-путь`, атрибуты `secure` и `httponly` игнорируются, что позволяет просто перезаписать старый cookie известной области действия. Более того, если перезаписывающий cookie имеет дату `expires` в прошлом, он будет немедленно отброшен, фактически давая надуманный способ очистить данные.

Хотя RFC 2109 требует принимать несколько разделенных запятыми cookies внутри одного заголовка `Set-Cookie`, этот подход опасен и больше не поддерживается ни одним браузером. Firefox позволяет задавать несколько cookies за один шаг через JavaScript `API document.cookie`, но необъяснимо требует вместо этого переводы строк как разделители. Ни один браузер не использует запятые как разделители `Cookie`, и распознавание их на стороне сервера следует считать небезопасным.

Еще одно важное различие между спецификацией и реальностью состоит в том, что значения cookies должны использовать формат `quoted-string`, описанный в спецификациях HTTP, см. "Значения заголовков, разделенные точкой с запятой", но на практике этот синтаксис распознают только Firefox и Opera. Поэтому полагаться на значения `quoted-string` небезопасно, как небезопасно и разрешать случайные символы кавычек в контролируемых злоумышленником cookies.

Cookies не гарантированно являются особенно надежными. Пользовательские агенты применяют умеренные ограничения на количество и размер cookies, разрешенных для домена, и как ошибочно задуманную функцию приватности могут также ограничивать их срок жизни. Поскольку столь же надежного отслеживания пользователей можно добиться другими средствами, такими как поведение `ETag/If-None-Match`, описанное в предыдущем разделе, попытки ограничить отслеживание на основе cookies, вероятно, приносят больше вреда, чем пользы.

HTTP-аутентификация

HTTP-аутентификация, как указано в RFC 2617, 19 - первоначальный механизм обработки учетных данных, задуманный для веб-приложений, ныне почти полностью вымерший. Причинами такого исхода могли быть негибкость связанных с ним браузерных UI, сложность приспособления более сложных схем аутентификации не на основе паролей или, возможно, невозможность контролировать, как долго учетные данные кэшируются и с какими другими доменами ими делятся.

В любом случае базовая схема довольно проста. Она начинается с того, что браузер делает неаутентифицированный запрос, на который сервер отвечает кодом 401 `Unauthorized`.^[10] Сервер также должен включить HTTP-заголовок `WWW-Authenticate`, указывающий запрошенный метод аутентификации, строку `realm`, произвольный идентификатор, к которому должны быть привязаны введенные учетные данные, и другие параметры, специфичные для метода, если они применимы.

Ожидается, что клиент тем или иным способом получит учетные данные, закодирует их в заголовке `Authorization` и повторит исходный запрос с включением этого заголовка. Согласно

спецификации, по причинам производительности тот же заголовок `Authorization` также может включаться в последующие запросы к тому же префиксу пути сервера без необходимости второго вызова `WWW-Authenticate`. Также допустимо повторно использовать те же учетные данные в ответ на любые вызовы `WWW-Authenticate` в других местах сервера, если строка `realm` и метод аутентификации совпадают.

На практике этому совету следуют не очень строго: кроме Safari и Chrome, большинство браузеров игнорирует строку `realm` или свободно подходит к сопоставлению путей. С другой стороны, все браузеры ограничивают кэшированные учетные данные не только сервером назначения, но и конкретным протоколом и портом, что дает некоторые преимущества безопасности.

Два метода передачи учетных данных, указанные в исходном RFC, известны как `basic` и `digest`. Первый по сути отправляет пароли в plaintext, закодированном как `base64`. Второй вычисляет одноразовый криптографический хэш, защищающий пароль от просмотра в открытом виде и предотвращающий последующее повторное воспроизведение заголовка `Authorization`. К сожалению, современные браузеры поддерживают оба метода и не различают их каким-либо ясным образом. В результате злоумышленники могут просто заменить слово `digest` на `basic` в исходном запросе, чтобы получить чистый plaintext-пароль, как только пользователь завершит диалог аутентификации. Удивительно, но раздел 4.8 RFC предсказал этот риск и предложил несколько полезных, но в конечном счете проигнорированных советов:

Пользовательским агентам следует рассмотреть такие меры, как показ визуального индикатора в момент запроса учетных данных о том, какая схема аутентификации будет использована, или напоминание самой сильной схемы аутентификации, когда-либо запрошенной сервером, и вывод предупреждения перед использованием более слабой. Также может быть хорошей идеей настроить пользовательский агент так, чтобы он требовал Digest-аутентификацию в целом или от конкретных сайтов.

Помимо этих двух схем аутентификации, указанных в RFC, некоторые браузеры также поддерживают менее распространенные методы, такие как Microsoft NTLM и Negotiate, используемые для бесшовной аутентификации с учетными данными домена Windows.²⁰

Хотя HTTP-аутентификация редко встречается в Интернете, она все еще отбрасывает длинную тень на некоторые типы веб-приложений. Например, когда внешнее изображение, предоставленное злоумышленником, включается в ветку на доске сообщений, а сервер, размещающий это изображение, внезапно решает вернуть 401 Unauthorized на некоторые запросы, пользователи, просматривающие ветку, внезапно увидят несколько загадочный запрос пароля. Дважды проверив адресную строку, многие, вероятно, перепутают этот запрос с просьбой ввести учетные данные форума, и они будут немедленно переданы серверу злоумышленника, размещающему изображение. Упс.

Шифрование на уровне протокола и клиентские сертификаты

Как теперь должно быть очевидно, вся информация в HTTP-сессиях передается по сети в plaintext. В 1990-х это не было бы большой проблемой: да, plaintext раскрывал ваш выбор страниц любопытным интернет-провайдерам и, возможно, другому непослушному пользователю в офисной сети или чрезмерно усердному государственному органу, но это казалось не хуже поведения SMTP, DNS или любого другого распространенного прикладного протокола. Увы, растущая популярность Web как коммерческой платформы усугубила риск, а существенный регресс сетевой безопасности, вызванный появлением по своей природе небезопасных публичных беспроводных сетей, вбил еще один гвоздь в этот гроб.

После нескольких менее успешных ухищрений в RFC 2818 было предложено прямолинейное решение этой проблемы:²¹ почему бы не инкапсулировать обычные HTTP-запросы внутри уже существующего многоцелевого механизма Transport Layer Security (TLS, он же SSL), разработанного несколькими годами ранее? Этот транспортный метод использует криптографию с открытым ключом^[11], чтобы установить конфиденциальный, аутентифицированный канал связи между двумя конечными точками, не требуя никаких изменений на уровне HTTP.

Чтобы веб-серверы могли доказывать свою личность, каждый веб-браузер с поддержкой HTTPS поставляется с внушительным набором открытых ключей, принадлежащих разным центрам сертификации. Центры сертификации - организации, которым поставщики браузеров доверяют криптографически подтверждать, что конкретный открытый ключ принадлежит конкретному сайту, желательно после проверки личности человека, запрашивающего такое подтверждение, и после проверки его заявления о правах на соответствующий домен.

Набор доверенных организаций разнообразен, произволен и не особенно хорошо документирован, что часто вызывает обоснованную критику. Но в конечном счете система обычно справляется со своей задачей достаточно хорошо. Пока задокументирована лишь горстка промахов, включая недавнюю громкую компрометацию компании Comodo,²² и нет зарегистрированных случаев массового злоупотребления привилегиями CA.

Что касается фактической реализации, при установлении нового HTTPS-соединения браузер получает от сервера подписанный открытый ключ, проверяет подпись, которую нельзя подделать без доступа к закрытому ключу CA, проверяет, что подписанные поля `cn` (`common name`) или `subjectAltName` в сертификате указывают, что этот сертификат выдан для сервера, с которым браузер хочет говорить, и подтверждает, что ключ не указан в публичном списке отзыва, например из-за компрометации или мошеннического получения. Если все проверки проходят, браузер может продолжить, шифруя сообщения серверу этим открытым ключом, и быть уверенным, что расшифровать их сможет только эта конкретная сторона.

Обычно клиент остается анонимным: он генерирует временный ключ шифрования, но этот процесс не доказывает личность клиента. Впрочем, такое доказательство можно организовать. Клиентские сертификаты применяются внутри некоторых организаций и приняты на национальном уровне в нескольких странах мира, например для сервисов электронного правительства. Поскольку обычная цель клиентского сертификата - предоставить некоторую информацию о реальной личности пользователя, браузеры по соображениям приватности обычно спрашивают перед отправкой таких сертификатов недавно встреченным сайтам; сверх этого сертификат может действовать как еще одна форма *ambient authority*.

Стоит отметить, что хотя HTTPS как таковой является здоровой схемой, сопротивляющейся и пассивным, и активным злоумышленникам, он почти не скрывает свидетельства доступа к априори публичной информации. Он не маскирует приблизительные размеры HTTP-запросов и ответов, направления трафика и временные шаблоны в типичной сессии просмотра, тем самым позволяя неискушенным пассивным злоумышленникам выяснить, например, какую неловкую страницу Wikipedia жертва просматривает по зашифрованному каналу. Фактически в одном экстремальном случае исследователи Microsoft показали использование такого профилирования пакетов для реконструкции пользовательских нажатий клавиш в онлайн-овом приложении.²³

Сертификаты Extended Validation

В ранние дни HTTPS многие публичные центры сертификации полагались на довольно педантичные и громоздкие проверки личности пользователя и владения доменом перед подписанием сертификата. К сожалению, в погоне за удобством и в интересах снижения цен некоторым теперь требуется немногим больше, чем действительная кредитная карта и возможность поместить файл на сервер назначения для завершения процесса проверки. Такой подход делает большинство полей сертификата, кроме `cn` и `subjectAltName`, недостоверными.

Чтобы решить эту проблему, сегодня по значительно более высокой цене продается новый тип сертификата, помеченный специальным флагом: Extended Validation SSL (EV SSL). Ожидается, что эти сертификаты не только доказывают владение доменом, но и более надежно подтверждают личность запрашивающей стороны после ручной проверки. EV SSL распознается всеми современными браузерами за счет окрашивания части адресной строки в синий или зеленый цвет. Хотя наличие такого уровня сертификатов ценно, идея связать более дорогой сертификат с индикатором, смутно подразумевающим "более высокий уровень безопасности", часто критикуется как хитро замаскированная схема заработка.

Правила обработки ошибок

В идеальном мире HTTPS-соединения, включающие подозрительную ошибку сертификата, такую как грубое несовпадение имени хоста или нераспознанный центр сертификации, должны просто приводить к неудаче установления соединения. Менее подозрительные ошибки, такие как недавно истекший сертификат или несовпадение имени хоста, возможно, могли бы сопровождаться лишь мягким предупреждением.

К сожалению, большинство браузеров без разбора делегировали ответственность за понимание проблемы пользователю, изо всех сил пытаясь и в конечном счете не сумев объяснить криптографию простыми словами, и требуя от пользователя принять бинарное решение: вы действительно хотите увидеть эту страницу или нет? Рис. 3-1 показывает один такой запрос.



Рис. 3-1. Пример диалога предупреждения о сертификате во все еще популярном Internet Explorer 6

Язык и внешний вид предупреждений SSL с годами эволюционировали в сторону все более упрощенных, но все еще проблемных объяснений сути проблемы и более сложных действий, необходимых для обхода предупреждения. Эта тенденция может быть ошибочной: исследования показывают, что пользователи нажимают дальше более чем в 50 процентах случаев даже для самых пугающих и разрушительных предупреждений.²⁴ Легко винить пользователей, но в конечном счете мы, возможно, задаем им неправильные вопросы и предлагаем ровно неправильный выбор. Проще говоря, если считается, что переход через предупреждение в некоторых случаях полезен, более разумным решением было бы предложить открыть страницу в явно помеченном режиме "песочницы", где вред ограничен. А если такого убеждения нет, любые возможности обхода следует полностью устранить; к этой цели стремится Strict Transport Security, экспериментальный механизм, который будет обсуждаться в главе 16.

Памятка по инженерии безопасности

При обработке контролируемых пользователем имен файлов в заголовках Content-Disposition

- Если вам не нужны нелатинские символы: удаляйте или заменяйте любые символы, кроме алфавитно-цифровых, ., - и _. Чтобы защитить пользователей от потенциально вредных или обманчивых имен файлов, вы также можете проверить, что по крайней мере первый символ является алфавитно-цифровым, и заменить все точки, кроме самой правой, чем-нибудь другим, например подчеркиванием. Помните, что разрешение кавычек, точек с запятой, обратных слешей и управляющих символов (0x00-0x1F) внесет уязвимости.
- Если вам нужны нелатинские имена: вы должны использовать RFC 2047, RFC 2231 или URL-подобное процентное кодирование зависящим от браузера способом. Обязательно отфильтровывайте управляющие символы (0x00-0x1F) и экранируйте любые точки с запятой, обратные слеша и кавычки.

При помещении пользовательских данных в HTTP cookies

- Процентно кодируйте все, кроме алфавитно-цифровых символов. Еще лучше - используйте base64. Случайные символы кавычек, управляющие символы (0x00-0x1F), символы со старшим битом (0x80-0xFF), запятые, точки с запятой и обратные слеша могут позволить внедрить новые значения cookies или изменить смысл и область действия существующих cookies.

При отправке контролируемых пользователем заголовков Location

- Обратитесь к памятке из главы 2. Разберите и нормализуйте URL и подтвердите, что схема находится в белом списке допустимых значений и что вам комфортно перенаправлять на указанный хост.

Убедитесь, что любые управляющие символы и символы со старшим битом экранированы правильно. Используйте Punycode для имен хостов и процентное кодирование для остальной части URL.

При отправке контролируемых пользователем заголовков Redirect

- Следуйте советам, приведенным для Location. Учтите, что точки с запятой небезопасны в этом заголовке и не могут быть надежно экранированы, но они также имеют особый смысл в некоторых URL. Ваш выбор - полностью отвергать такие URL или процентно кодировать символ ;, тем самым нарушая синтаксические правила, предписанные RFC.

При построении других типов контролируемых пользователем запросов или ответов

- Изучите синтаксис и потенциальные побочные эффекты рассматриваемого заголовка. В целом внимательно относитесь к управляющим символам и символам со старшим битом, запятым, кавычкам, обратным слешам и точкам с запятой; другие символы или строки могут вызывать беспокойство в зависимости от конкретного случая. Экранируйте или заменяйте эти значения соответствующим образом.
- При создании нового HTTP-клиента, сервера или прокси: не создавайте новую реализацию, если только вам это абсолютно не нужно. Если избежать этого нельзя, внимательно прочитайте эту главу и старайтесь близко имитировать существующую массовую реализацию. Если возможно, игнорируйте советы RFC об отказоустойчивости и прекращайте обработку при встрече с любыми синтаксическими неоднозначностями.

Сноски

[1] [\[назад\]](#) Transmission Control Protocol (TCP) - один из основных коммуникационных протоколов Интернета, предоставляющий транспортный уровень для любых прикладных протоколов, построенных поверх него. TCP предлагает достаточно надежное, подтверждаемое участниками, упорядоченное, сессионное соединение между сетевыми узлами. В большинстве случаев протокол также довольно устойчив к атакам слепой подделки пакетов, предпринимаемым другими, нелокальными узлами в Интернете.

[2] [\[назад\]](#) MIME-тип, он же Internet media type, - простое двухкомпонентное значение, идентифицирующее класс и формат заданного компьютерного файла. Концепция возникла в RFC 2045 и RFC 2046, где служила способом описания почтовых вложений. Реестр официальных значений, таких как text/plain или audio/mpeg, сейчас поддерживает IANA, но ad hoc типы довольно распространены.

[3] [\[назад\]](#) В этом случае некоторые HTTP-заголовки, предоставленные клиентом, могут использоваться прокси внутренне, но они не будут переданы не-HTTP конечной точке, что создает некоторые интересные, хотя и не связанные с безопасностью, протокольные неоднозначности.

[4] [\[назад\]](#) Quoted-printable - простая схема кодирования, заменяющая любые непечатаемые или иначе незаконные символы знаком равенства (=), за которым следует двухзначное шестнадцатеричное представление 8-битного значения кодируемого символа. Любые случайные знаки равенства во входном тексте также должны заменяться на =3D.

[5] [\[назад\]](#) Base64 - нечитаемое человеком кодирование, которое кодирует произвольный 8-битный ввод с помощью 6-битного алфавита из чувствительных к регистру алфавитно-цифровых символов, + и /. Каждые 3 байта ввода отображаются в 4 байта вывода. Если ввод не заканчивается на 3-байтовой границе, это обозначается добавлением одного или двух знаков равенства в конец выходной строки.

[6] [\[назад\]](#) Существует анекдотическая и, возможно, даже правдивая история о неудачливом веб-мастере по имени Джон Брекман. Согласно истории, его сайт был случайно удален роботом поисковой индексации. Робот просто невольно обнаружил неаутентифицированный административный интерфейс на основе GET, который Джон построил для своего сайта, и радостно перешел по каждой ссылке "delete", которую смог найти.

[7] [\[назад\]](#) В принципе, клиенты можно было бы спроектировать так, чтобы они поглощали любые незапрошенные данные ответа сервера перед отправкой любых последующих запросов в keepalive-сессии, ограничивая влияние атаки. Однако это

предложение подрывается практикой HTTP pipelining; ради производительности некоторые клиенты устроены так, что сбрасывают несколько запросов сразу, не дожидаясь полного ответа между ними.

[8] [назад] RFC несколько туманен в этом отношении, но, похоже, намерение состоит в том, чтобы разрешить использовать закешированный документ для таких целей, как работа кнопок навигации "назад" и "вперед" в браузере, но не тогда, когда запрошена нормальная загрузка страницы. Firefox следует этому подходу, тогда как все остальные браузеры считают no-cache и no-store примерно эквивалентными.

[9] [назад] Ambient authority - форма контроля доступа, основанная на глобальном и постоянном свойстве запрашивающей сущности, а не на какой-либо явной форме авторизации, действительной только для конкретного действия. Идентифицирующий пользователя cookie, без разбора включаемый в каждый исходящий запрос к удаленному сайту, без учета того, почему этот запрос выполняется, относится к этой категории.

[10] [назад] Термины authentication и authorization, по-видимому, используются в этом RFC взаимозаменяемо, но в других областях информационной безопасности они имеют различное значение. Authentication обычно используется для обозначения процесса доказательства вашей личности, тогда как authorization - процесс определения, позволяют ли ваши ранее установленные учетные данные выполнить конкретное привилегированное действие.

[11] [назад] Криптография с открытым ключом опирается на асимметричные алгоритмы шифрования для создания пары ключей: закрытого, который владелец держит в секрете и который нужен для расшифровки сообщений, и открытого, распространяемого миру и полезного только для шифрования трафика этому получателю, но не для его расшифровки.

Глава 4. Язык гипертекстовой разметки

Hypertext Markup Language (HTML) - основной способ создания онлайн-документов. Одно из самых ранних письменных описаний этого языка - краткое резюме, опубликованное Тимом Бернерсом-Ли в Интернете в 1991 году.¹ Его предложение описывает синтаксис, производный от SGML, который позволяет аннотировать текстовые документы встроенными гиперссылками и несколькими типами средств компоновки. В последующие годы эта спецификация постепенно развивалась под руководством сэра Бернерса-Ли и Дэна Коннолли, но только в 1995 году, на заре первых браузерных войн, достаточно серьезная и исчерпывающая спецификация языка (HTML 2.0) попала в RFC 1866.²

С этого момента начался полный хаос: в течение следующих нескольких лет конкурирующие поставщики браузеров продолжали вводить всевозможные яркие, ориентированные на представление возможности и подстраивать язык под свой вкус. Было предпринято несколько попыток дополнить исходный RFC, но в конечном счете подход к стандартизации под управлением IETF оказался слишком негибким. Недавно созданный World Wide Web Consortium взял на себя сопровождение языка и в конце концов опубликовал спецификацию HTML 3.2 в 1997 году.³

Новая спецификация пыталась примирить различия в реализациях браузеров и одновременно принять многие красивые украшения, которые нравились публике, такие как настраиваемые цвета текста и переменные гарнитуры. В конечном счете, однако, HTML 3.2 оказался шагом назад для ясности языка и добился лишь ограниченного успеха в попытке догнать фактическое положение дел.

В последующие годы работа над HTML 4 и 4.014 сосредоточилась на очистке HTML от всех накопленных излишеств и на лучшем объяснении того, как элементы документа должны интерпретироваться и отображаться. Она также определила альтернативный, строгий синтаксис XHTML, производный от XML, который было намного проще последовательно разбирать, но гораздо труднее писать без ошибок. Несмотря на всю эту работу, лишь малая доля всех сайтов в Интернете могла по-настоящему заявлять о соответствии каким-либо из этих стандартов, а на клиентской стороне почти или совсем не наблюдалось согласованности в режимах разбора и восстановления после ошибок. Вследствие этого часть работы по улучшению базового языка сошла на нет, и W3C переключил внимание на таблицы стилей, Document Object Model и другие более абстрактные или ориентированные в будущее задачи.

В конце 2000-х часть низкоуровневой работы была возрождена под знаменем HTML5,⁵ амбициозного проекта по нормализации почти каждого аспекта синтаксиса и разбора языка, определению всех связанных API и более строгому контролю поведения браузеров в целом. Время покажет, будет ли он успешен; до тех пор сам язык и каждый из четырех ведущих движков разбора^[1] имеют собственный набор раздражающих особенностей.

Базовые понятия HTML-документов

С чисто теоретической точки зрения HTML опирается на довольно простой синтаксис: иерархическую структуру тегов, параметров тегов вида `name=value` и текстовых узлов между ними, образующих собственно тело документа. Например, простой документ с заголовком страницы, заголовком в теле и гиперссылкой может выглядеть так:

```
<html>
<head>
<title>Hello world</title>
</head>
<body>
<h1>Welcome to our example page</h1>
<a href="http://www.example.com/">Click me!</a>
</body>
</html>
```

Этот синтаксис накладывает некоторые ограничения на то, что может появляться внутри значения параметра или внутри тела документа. Пять символов - угловые скобки, одинарные и двойные кавычки и амперсанд - зарезервированы как строительные блоки HTML-разметки, и их нужно избегать или каким-то образом экранировать, когда они используются вне предназначенной им функции. Самые важные правила таковы:

- Случайные амперсанды (&) никогда не должны появляться в большинстве секций HTML-документа.
- Оба типа угловых скобок очевидно проблематичны внутри тега, если они должным образом не заключены в кавычки.
- Левая угловая скобка (<) опасна внутри текстового узла.
- Символы кавычек, появляющиеся внутри тега, могут иметь нежелательные эффекты в зависимости от точного местоположения, но безвредны в текстовых узлах.

Чтобы позволить этим символам появляться в проблемных местах без побочных эффектов, предусмотрена схема кодирования на основе амперсанда, обсуждаемая в разделе "Кодирование сущностей".

Примечание. Разумеется, наличие такой схемы кодирования не гарантирует ее использования. Неспособность должным образом отфильтровать или экранировать зарезервированные символы при отображении контролируемых пользователем данных является причиной целого ряда чрезвычайно распространенных и смертельно опасных дефектов безопасности веб-приложений. Особенно известный пример - cross-site scripting (XSS), атака, при которой вредоносный JavaScript-код, предоставленный злоумышленником, непреднамеренно отражается где-нибудь в HTML-разметке, фактически давая злоумышленнику полный контроль над внешним видом и работой целевого сайта.

Режимы разбора документа

Для любого HTML-документа директива верхнего уровня `<!DOCTYPE>` может использоваться, чтобы велеть браузеру разбирать файл способом, который хотя бы поверхностно соответствует одному из официально определенных стандартов; в более ограниченной степени тот же сигнал может передаваться и заголовком `Content-Type`. Из всех доступных режимов разбора самое заметное различие существует между XHTML и традиционным HTML. В традиционном режиме парсеры будут пытаться восстановиться после большинства типов синтаксических ошибок, включая несопоставленные открывающие и закрывающие теги. Кроме того, имена тегов и параметров будут считаться нечувствительными к регистру, значения параметров не всегда нужно будет заключать в кавычки, а некоторые типы тегов, такие как `<img>`, будут закрываться неявно. Иными словами, следующий ввод будет нехотя терпим:

```
<hTmL>
<BODY>
<IMG src="/hello_world.jpg">
<a HREF=http://www.example.com/>
Click me!
</oops>
</html>
```

XML-режим, напротив, строг: все теги нужно тщательно балансировать, именовать с правильным регистром и закрывать явно. XML-специфичный синтаксис самозакрывающихся тегов, такой как `<img />`, разрешен. Кроме того, большинство синтаксических ошибок, даже тривиальных, приведет к ошибке и полностью предотвратит отображение документа.

В отличие от обычной разновидности HTML, документы на основе XML также могут изящно включать секции, использующие другие XML-совместимые форматы разметки, такие как MathML, язык разметки математических формул. Это делается путем указания другой настройки пространства имен `xmlns` для конкретного тега, без необходимости в разовых хаках на уровне языка.

Последнее важное различие, которое стоит здесь упомянуть, состоит в том, что традиционные стратегии разбора HTML имеют набор специальных режимов, в которые парсер входит после встречи с определенными тегами и из которых выходит только при появлении конкретной строки-терминатора; все между ними интерпретируется как не-HTML текст. Примеры таких специальных тегов включают `<style>`, `<script>`, `<textarea>` или `<xmp>`. В практических реализациях выход из этих режимов происходит только при буквальном, нечувствительном к регистру совпадении с `</style>`, `</script>` или похожим соответствующим значением; любая другая разметка внутри такого блока не будет интерпретироваться как HTML. Интересно, что существует один официально устаревший тег, `<plaintext>`, из которого вообще нельзя выйти; он действует до конца документа.

Для сравнения, XML-режим более предсказуем. Он в целом запрещает случайные символы `<` и `&` внутри документа, но предоставляет специальный синтаксис, начинающийся с `<![CDATA[` и заканчивающийся `]]>`, как способ инкапсулировать любой необработанный текст внутри произвольного тега. Например:

```
<script>
<![CDATA[
alert('>>> Hello world! <<<');
]]>
</script>
```

Другой заметный специальный режим разбора, доступный и в XHTML, и в обычном HTML, - блок комментария. В XML он довольно просто начинается с `<!--` и заканчивается `-->`. В традиционном HTML-парсере Firefox версий до 4 любое появление `--`, за которым позднее следует `>`, также считается достаточным.

Битва за семантику

Если оставить в стороне низкоуровневый синтаксис языка, HTML также является предметом увлекательной концептуальной борьбы: столкновения идеологии и реальности онлайнового мира. Тим Бернерс-Ли всегда отстаивал видение semantic web, взаимосвязанной системы документов, где каждый функциональный блок, такой как цитата, фрагмент кода, почтовый адрес или заголовок, объясняет свой смысл подходящим машиночитаемым тегом, например `<cite>`, `<code>`, `<address>` или `<h1>`-`<h6>`.

Этот подход, как утверждали он и другие сторонники, упростил бы машинам обход, анализ и индексирование содержимого осмысленным способом, а в ближайшем будущем позволил бы компьютерам рассуждать, используя сумму человеческих знаний. Согласно этой философии, язык разметки должен предоставлять способ стилизовать внешний вид документа, но лишь как побочную возможность.

Сэр Бернерс-Ли никогда не отказался от этой мечты, но именно в этом отношении фактическое использование HTML оказалось совсем не таким, каким он его желал. Веб-разработчики быстро и прагматично свели сущность HTML 3.2 к горстке изменяющих представление, но семантически нейтральных тегов, таких как `<font>`, `<b>` и `<pre>`, и не видели причин дополнительно объяснять браузеру структуру своих документов. W3C пытался бороться с этой тенденцией, но с ограниченным успехом. Хотя такие теги, как `<font>`, были успешно объявлены устаревшими и в значительной степени оставлены в пользу CSS, произошло это лишь потому, что таблицы стилей предложили более мощные и согласованные визуальные средства управления. С помощью CSS разработчики просто начали полагаться на суп из семантически агностических тегов `<span>` и `<div>`, чтобы строить все - от заголовков до кнопок, по которым кликает пользователь, причем способом, полностью непрозрачным для любых автоматизированных инструментов извлечения содержимого.

Несмотря на длительное влияние на дизайн языка, в некотором смысле идея semantic web может устаревать: онлайновое содержимое все реже отображается на концепцию единого просматриваемого документа, а HTML часто сводится к предоставлению удобной поверхности рисования и графических примитивов, с помощью которых JavaScript-приложения строят свои интерфейсы.

Понимание поведения HTML-парсера

Основ HTML-синтаксиса, изложенных в предыдущих разделах, обычно достаточно для понимания смысла корректно сформированных документов HTML и XHTML. Когда используется диалект XHTML, история почти на этом заканчивается: минимальная отказоустойчивость парсера означает, что аномальный синтаксис почти всегда просто приводит к ошибке разбора. Увы, с традиционными, расслабленными HTML-парсерами картина совсем иная: они агрессивно угадывают намерения разработчика страницы даже в очень неоднозначных или потенциально вредных ситуациях.

Поскольку точное понимание предоставленной пользователем разметки необходимо для проектирования многих типов фильтров безопасности, давайте быстро посмотрим на некоторые из этих поведений и особенностей. Для начала рассмотрим следующий эталонный фрагмент:

```
<img src=image.jpg title="Hello world" class=examples>
```

Веб-разработчики обычно удивляются, узнав, что этот синтаксис можно радикально изменить без изменения его смысла для браузера. Например, Internet Explorer позволит вставить символ NUL (0x00) в место, отмеченное в исходной схеме как первое, и такое изменение, скорее всего, собьет с пути все наивные HTML-фильтры. Также не широко известно, что пробелы во втором и четвертом отмеченных местах можно заменить необычными символами вертикальной табуляции (0x0B) или перевода строки (0x0C) во всех браузерах и неразрывным пробелом UTF-8 (0xA0) в Opera.^[2] О, и вот по-настоящему удивительная деталь: в Firefox пробел во втором месте также можно заменить одним обычным слешем, а вот пробел в четвертом - нельзя.

Двигаясь дальше, стоит отметить и место, помеченное как третье. В этой позиции символы NUL игнорируются большинством парсеров, как и многие типы пробельных символов. Не так давно браузеры WebKit принимали в этом месте слеш, но недавние улучшения парсера устранили эту особенность.

Символы кавычек - еще одна интересная тема. Разработчики сайтов знают, что одинарные и двойные кавычки можно использовать, чтобы поместить строку, содержащую пробелы или угловые скобки, в HTML-параметр, но обычно для них становится сюрпризом, что Internet Explorer также признает обратные апострофы (`) вместо настоящих кавычек в месте, помеченном как пятое. Аналогично мало кто понимает, что в любом браузере после параметра в кавычках вставляется неявный пробел и поэтому явный пробел в шестом месте можно пропустить без изменения смысла тега.

Влияние этих шаблонов на безопасность не всегда легко оценить, но рассмотрим HTML-фильтр, которому поручено очищать тег `<img>` с контролируемым злоумышленником параметром `title`. Допустим, во входной разметке этот параметр не заключен в кавычки, если не содержит пробелов и угловых скобок, - конструкция, которую можно увидеть на популярном блоговом сайте. Сначала эта практика может показаться безопасной, но в следующих двух случаях вредоносный внедренный параметр `onerror` материализуется внутри тега:

```
<img ... title=""onerror="alert(1)">
```

и

```
<img ... title=``onerror=`alert(1)`>
```

Еще одна замечательная особенность Internet Explorer, связанная с кавычками, делает эту задачу еще сложнее. В то время как большинство браузеров распознает кавычки только тогда, когда они используются в начале значения параметра, Internet Explorer просто проверяет любое появление знака равенства (=), за которым следует кавычка, и разбирает этот синтаксис довольно неожиданным способом:

```
<img src=test.jpg?value=">Yes, we are still inside a tag!">
```

Взаимодействия между несколькими тегами

Разбор одного тега может быть непростой задачей, но, как можно представить, аномальные сочетания нескольких HTML-тегов будут еще менее предсказуемыми. Рассмотрим следующий тривиальный пример:

```
<i <b>
```

Получив такой синтаксис, большинство браузеров интерпретирует только `<i>` и сочтет строку `<b` недопустимым параметром тега. Однако Firefox версий до 4 при встрече с угловой скобкой сначала автоматически закрывал тег `<i>` и в итоге интерпретировал и `<i>`, и `<b>`. В духе отказоустойчивости WebKit до недавнего времени тоже следовал этой модели.

Похожее поведение можно наблюдать в предыдущих версиях Firefox при работе с именами тегов, содержащими недопустимые символы, в данном случае знак равенства. Вместо того чтобы постараться проигнорировать весь блок, парсер просто сбрасывался и интерпретировал процитированный тег:

```
<i="<b>">
```

Обработка тегов, которые не закрыты до конца файла, не менее увлекательна. Например, следующий фрагмент заставит большинство браузеров интерпретировать тег `<i>` или проигнорировать всю строку, но Internet Explorer и Opera используют другой подход с откатом и увидят вместо него `<b>`:

```
<i foo="<b>" [EOF]
```

Фактически Firefox версий до 4 прибегал к надуманному повторному разбору всякий раз, когда определенные специальные теги, такие как `<title>`, не были закрыты до конца документа:

```
<title>This text will be interpreted as a title
<i>This text will be shown as document body!
[EOF]
```

Последние две особенности разбора имеют интересные последствия для безопасности в любом сценарии, где злоумышленник может преждевременно прервать загрузку страницы. Даже если разметка в остальном довольно хорошо очищена, смысл документа может измениться очень неожиданным образом.

Явные и неявные условия

Чтобы еще больше усложнить работу разбора HTML, некоторые браузеры демонстрируют поведения, которые можно использовать для условного пропуска части разметки в документе. Например, пытаясь помочь начинающим пользователям платформы разработки Microsoft Active Server Pages, Internet Explorer рассматривает блоки `<% ... %>` как полностью нестандартный комментарий, скрывая любую разметку между этими двумя последовательностями символов. Еще одна специфичная для Internet Explorer возможность - явные условные выражения, интерпретируемые парсером и спрятанные внутри стандартных блоков HTML-комментариев:

```
<!--[if IE 6]>
Markup that will be parsed only for Internet Explorer 6
<![endif]-->
```


Многие другие особенности этого типа связаны с идиосинкразиями SGML и XML. Например, из-за поведения обработки комментариев, упомянутого ранее в отступлении, браузеры расходятся в том, как разбирать директивы `!` и `?` (такие как `<!DOCTYPE>` или `<?xml>`), разрешать ли XML-подобные блоки CDATA в режимах не-HTML и какой приоритет отдавать перекрывающимся тегам специальных режимов разбора (таким как `"<style><!-- </style> -->"`).

Советы по выживанию при разборе HTML

Набор поведений разбора, обсуждавшихся в предыдущих разделах, отнюдь не исчерпывающий. Фактически этой теме посвящена целая книга: любознательным читателям рекомендуется взять *Web Application Obfuscation* (Syngress, 2011) Марио Хайдриха, Эдуардо Альберто Вела Навы, Гарета Хейса и Дэвида Линдсея - а затем оплакать судьбу человечества. Суть в том, что строить HTML-фильтры, которые пытаются блокировать известные опасные шаблоны, а остальную разметку пропускать как есть, попросту невозможно.

Единственный разумный подход к очистке тегов - применить реалистичный парсер, который переведет входной документ в иерархическое дерево документа в памяти, а затем вычистить это представление от всех нераспознанных тегов и параметров, а также от любых нежелательных конфигураций тег/параметр/значение. После этого дерево можно аккуратно сериализовать обратно в правильно сформированный, корректно экранированный HTML, который не будет напрягать механизмы исправления ошибок в самом браузере. Многие разработчики думают, что должна быть возможна более простая конструкция, но в конце концов узнают реальность тяжелым путем.

Кодирование сущностями

Снова поговорим о кодировании символов. Как отмечалось на первых страницах этой главы, некоторые зарезервированные символы в общем случае небезопасны внутри текстовых узлов и значений параметров тегов, а в XHTML они часто приводят к прямым синтаксическим ошибкам. Чтобы позволить безопасно использовать такие символы (и дать удобный способ встраивать текст с байтами старшего диапазона), разработчикам доступна простая схема кодирования с амперсандом в начале и точкой с запятой в конце, известная как кодирование сущностями.

Самое знакомое применение этого метода кодирования - включение некоторых заранее определенных именованных сущностей. Для XML определена лишь горстка таких сущностей, но еще несколько сотен разбросаны по спецификациям HTML и поддерживаются всеми современными браузерами. При таком подходе `<` используется для вставки левой угловой скобки; `>` заменяет правую угловую скобку; `&` заменяет сам амперсанд; а, скажем, `→` - это симпатичная стрелка Unicode.

Примечание. В документах XHTML дополнительные именованные сущности можно определить с помощью директивы `<!ENTITY>` и заставить их разворачиваться во внутренне заданные строки или в содержимое URL внешнего файла. Последний вариант, очевидно, небезопасен, если разрешен при обработке недоверенного содержимого; возникающую атаку иногда называют External XML Entity, или сокращенно XXE.

Помимо именованных сущностей, можно также вставить произвольный символ ASCII или Unicode с помощью десятичной записи `&#number;`. В этом случае `<` соответствует левой угловой скобке; `>` заменяет правую; а `😹`, я не шучу, является символом Unicode 6.0 с именем "smiling cat face with tears of joy". Можно использовать и шестнадцатеричную запись, если перед числом стоит `x`. В этом варианте левая угловая скобка становится `<` и так далее.

HTML-парсер распознает кодирование сущностями внутри текстовых узлов и значений параметров и прозрачно декодирует его при построении представления дерева документа в памяти. Поэтому следующие два случая функционально идентичны:

```

```

и

```

```

С другой стороны, следующие два примера не сработают ожидаемым образом, поскольку кодирование вмешивается в структуру самого тега:

```
<img src&#x3d;"http://www.example.com">
```

и

```
<img s&#x72;c="http://www.example.com">
```

Почти прозрачное поведение кодирования сущностями делает важным правильное разрешение таких сущностей до принятия любых решений безопасности о содержимом документа и, если применимо, корректное восстановление кодирования в очищенном выводе позже. Для иллюстрации следующий синтаксис должен распознаваться как абсолютная ссылка на псевдо-URL `javascript:`, а не как загадочный идентификатор фрагмента внутри относительного ресурса с именем `./javascript&`:

```
<a href="javascript&#x3a;alert(1)">
```

К сожалению, даже простая задача распознавания и разбора HTML-сущностей может быть непростой. Например, при традиционном разборе сущности часто могут приниматься даже без завершающей точки с запятой, если следующий символ не является буквенно-цифровым. В Firefox дефисы и точки также принимаются в именах сущностей. Числовые сущности еще проблемнее, поскольку они могут иметь чрезмерно длинную запись с произвольным числом ведущих нулей. Более того, если числовое значение больше 2^{32} , стандартного размера целого числа на многих компьютерных архитектурах, соответствующий символ может быть вычислен неправильно.

Разработчикам, работающим с XHTML, следует помнить и о потенциальной ловушке этого диалекта. Хотя HTML-сущности не распознаются в большинстве специальных режимов разбора, XHTML отличается от традиционного HTML тем, что теги вроде `<script>` и `<style>` сами по себе не включают специальный режим разбора автоматически. Вместо этого для достижения сопоставимого эффекта требуется явный блок `<![CDATA[...]]>` вокруг любых скриптов или таблиц стилей. Поэтому следующий фрагмент со строкой, контролируемой злоумышленником (в остальном очищенной от угловых скобок, кавычек, обратных слешей и переводов строк), совершенно безопасен в HTML, но не в XHTML:

```
<script>
  var tmp = 'I am harmless! &#x27;+alert(1);// Or am I?';
  ...
</script>
```

Семантика интеграции HTTP/HTML

Из главы 3 мы помним, что HTTP-заголовки могут придавать новый смысл всему ответу (Location, Transfer-Encoding и так далее), менять способ представления полезной нагрузки (Content-Type, Content-Disposition) или влиять на клиентскую среду другими, вспомогательными способами (Refresh, Set-Cookie, Cache-Control, Expires и т. п.).

Но что, если HTML-документ доставлен через не-HTTP-протокол или загружен из локального файла? Очевидно, в этом случае нет простого способа выразить или сохранить эту информацию. С частью ее можно легко расстаться, но параметры вроде MIME-типа или набора символов необходимы, и их потеря вынуждает браузеры затем импровизировать. Например, наборы символов вроде UTF-7, UTF-16 и UTF-32 несовместимы с ASCII, а значит, HTML-документы нельзя даже разобрать, не определив, какое из этих преобразований нужно использовать.

Последствия для безопасности эвристик браузерного уровня, используемых для определения наборов символов и типов документов, будут подробно рассмотрены в главе 13. А пока проблема сохранения протокольной информации внутри документа несколько неуклюже решается специальной HTML-директивой `<meta http-equiv=...>`. К тому моменту, когда браузер рассматривает разметку, многие решения об обработке содержимого уже должны быть приняты, но некоторые корректировки еще остаются возможными; например, может быть возможно настроить charset на в целом совместимое значение или указать директивы Refresh, Set-Cookie и кэширования.

В качестве иллюстрации допустимого синтаксиса рассмотрим следующую директиву, которая при появлении в 8-битном ASCII-документе пояснит браузеру, что набор символов документа - UTF-8, а не, скажем, ISO-8859-1:

```
<meta http-equiv="Content-Type" content="text/html; charset=utf-8">
```

С другой стороны, все следующие директивы завершатся неудачей, потому что на этом этапе уже слишком поздно переключаться на несовместимую кодировку UTF-32, менять тип документа на видеоформат или выполнять перенаправление вместо разбора файла:

```
<meta http-equiv="Content-Type" content="text/html; charset=utf-32">
<meta http-equiv="Content-Type" content="video/mpeg">
<meta http-equiv="Location" content="http://www.example.com">
```

Помните, что когда значения `http-equiv` конфликтуют друг с другом или противоречат HTTP-заголовкам, ранее полученным от сервера, их поведение непоследовательно, и полагаться на него нельзя. Например, обычно побеждает первое поддерживаемое значение charset= (и HTTP-заголовки в этом случае имеют приоритет над `<meta>`), но при нескольких конфликтующих значениях Refresh поведение сильно зависит от браузера.

Примечание. Некоторые браузеры пытаются спекулятивно извлекать информацию `<meta http-equiv>` еще до фактического разбора документа, что может приводить к неловким ошибкам. Например, недавно исправленная в Firefox 4 ошибка безопасности заставляла браузер интерпретировать следующую конструкцию как объявление набора символов:

```
<meta http-equiv="Refresh"
content="10;http://www.example.com/charset=utf-7">.[3]
```

Гиперссылки и включение содержимого

Одна из самых важных и значимых для безопасности возможностей HTML - предсказуемо, способность ссылаться на внешнее содержимое и встраивать его. Если оставить в стороне возможности уровня HTTP вроде Location и Refresh, этого можно добиться несколькими простыми способами.

Обычные ссылки

Следующая разметка демонстрирует самый знакомый и самый базовый метод ссылки на внешнее содержимое изнутри документа:

```
<a href="http://www.example.com/">Click me!</a>
```

Эта гиперссылка может указывать на любую из распознаваемых браузером схем, включая псевдо-URL (data:, javascript: и так далее) и протоколы, обрабатываемые внешними приложениями (например, mailto:). Щелчок по тексту (или любым HTML-элементам), вложенному внутрь такого блока `<a href=...>`, обычно заставляет браузер уйти с документа, содержащего ссылку, и перейти в указанное место, если это осмысленно возможно для используемого протокола.

Необязательный параметр target можно использовать, чтобы направить навигацию в другие окна или представления документа. Параметр должен задавать имя целевого представления. Если имя не найдено или доступ запрещен, поведением по умолчанию обычно становится открытие нового окна. Условия, при которых доступ может быть запрещен, являются темой главы 11.

Четыре специальных имени target тоже могут использоваться (как показано слева на рис. 4-1): `_blank` всегда открывает совершенно новое окно, `_parent` выполняет навигацию в представлении более высокого уровня, которое встраивает документ со ссылкой (если такое есть), а `_top` всегда выполняет навигацию в окне браузера верхнего уровня, независимо от того, сколько уровней вложения документов находится между ними. Ах да, четвертая специальная цель, `_self`, идентична отсутствию значения вообще и существует без всякой причины.

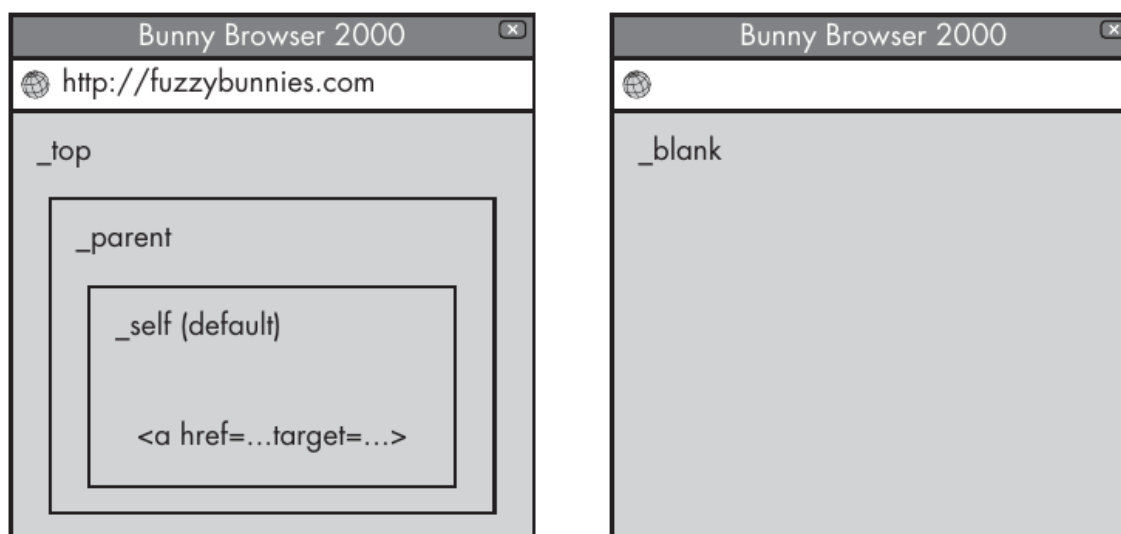


Рис. 4-1. Предопределенные цели для гиперссылок

Формы и запросы, запускаемые формами

HTML-форму можно представить как гиперссылку для сбора информации: когда нажимается кнопка "submit", динамический запрос строится на лету из данных, собранных через любое число полей ввода. Формы позволяют отправлять на сервер пользовательский ввод и файлы, но почти во всех остальных отношениях результат отправки формы похож на переход по обычной ссылке.

Простая разметка формы может выглядеть так:

```
<form method=GET action="/process_form.cgi">
  Given name: <input type=text name=given>
  Family name: <input type=text name=family>
  ...
  <input type=submit value="Click here when done!">
</form>
```

Параметр action работает как значение href, используемое для обычных ссылок, с одним небольшим отличием: если значение отсутствует, форма будет отправлена в местоположение текущего документа, тогда как любые ссылки <a> без назначения просто не будут работать вообще. Также можно указать необязательный параметр target, и он будет вести себя так, как описано в предыдущем разделе.

Примечание. В отличие от тегов <a>, формы необычным образом нельзя вкладывать друг в друга, и в таком случае работоспособным останется только тег <form> верхнего уровня.

Когда значение method установлено в GET или просто отсутствует, все имена вложенных полей и их текущие значения будут экранированы с помощью знакомой схемы процентного кодирования, описанной в главе 2, но с двумя довольно произвольными отличиями. Во-первых, символ пробела (0x20) будет заменен знаком плюса, а не закодирован как %20. Во-вторых, как следствие, любые существующие знаки плюса нужно кодировать как %2B, иначе они будут ошибочно истолкованы как пробелы.

Затем закодированные пары name=value разделяются амперсандами и объединяются в единую строку, например:

```
given=Erwin+Rudolf+Josef+Alexander&family=Schr%C3%B6dinger
```

Полученное значение вставляется в часть запроса целевого URL (заменяя любое существующее содержимое этого раздела) и отправляется на сервер. Полученный ответ затем показывается пользователю в целевой области просмотра.

Ситуация немного сложнее, если параметр method установлен в POST. Для HTTP-запроса этого типа доступны три формата отправки данных. В режиме по умолчанию (называемом application/x-www-form-urlencoded) сообщение строится так же, как для GET, но вместо этого передается в полезной нагрузке запроса, оставляя строку запроса и все остальные части целевого URL нетронутыми.^[4]

Существование второго режима отправки POST, включаемого указанием enctype="text/plain" в теге <form>, трудно оправдать. В этом режиме имена и значения полей вообще не кодируются процентным кодированием (но, в зависимости от браузера, знаки плюса могут использоваться вместо пробелов), а вместо амперсанда используется разделитель новой строки. Получающийся формат по сути бесполезен, поскольку его нельзя разобрать однозначно: переводы строк и знаки равенства, происходящие из формы, неотличимы от вставленных браузером.

Последний режим включается с `enctype="multipart/form-data"` и должен использоваться всякий раз при отправке выбранных пользователем файлов через форму (что возможно со специальным тегом `<input type="file">`). Получающееся тело запроса состоит из серии коротких MIME-сообщений, соответствующих каждому отправленному полю.^[5] Эти сообщения разделяются выбранным клиентом случайным уникальным граничным токеном, который в остальном не должен встречаться в инкапсулированных данных:

```
POST /process_form.cgi HTTP/1.1
...
Content-Type: multipart/form-data; boundary=random1234

--random1234
Content-Disposition: form-data; name="given"

Erwin Rudolf Josef Alexander
--random1234
Content-Disposition: form-data; name="family"

Schrödinger
--random1234
Content-Disposition: form-data; name="file"; filename="cat_names.txt"
Content-Type: text/plain

(File contents follow)
--random1234--
```

Несмотря на кажущийся открытым синтаксис тега, другие методы запроса и форматы отправки не поддерживаются ни одним браузером, и вряд ли это изменится. Некоторое время стандарт HTML5 пытался ввести методы PUT и DELETE в формы, но это предложение быстро отвергли.

Фреймы

Фреймы - это форма разметки, которая позволяет содержимому одного HTML-документа отображаться в прямоугольной области другого, встраивающего, документа. Современные браузеры поддерживают несколько тегов фреймов, но самый распространенный способ достичь этой цели - бесппроблемный и гибкий встроенный фрейм:

```
<iframe src="http://www.example.com/"></iframe>
```

В традиционных HTML-документах этот тег переводит парсер в один из специальных режимов разбора, и весь текст между открывающим и закрывающим тегами просто игнорируется браузерами, понимающими фреймы. Однако в старых браузерах, которые не понимают `<iframe>`, разметка между открывающим и закрывающим тегами обрабатывается нормально, предлагая откровенно бюджетную директиву условного отображения. Такое условное поведение обычно используется, чтобы давать проницательные советы вроде "Эта страница должна просматриваться в браузере, который поддерживает фреймы".

Фрейм - это полностью отдельное представление документа, во многих отношениях идентичное новому окну браузера. У него даже есть собственный контекст выполнения JavaScript. Как и окна браузера, фреймы можно снабдить параметром `name`, а затем направлять в них переходы из тегов `<a>` и `<form>`.

Ограничения на URL `src` для содержимого фрейма примерно похожи на правила, применяемые к обычным ссылкам. Сюда входит возможность указывать фреймы на `javascript:` или загружать протоколы, обрабатываемые внешними приложениями, которые оставляют фрейм пустым и открывают целевое приложение в новом процессе.

Фреймы представляют особый интерес для веб-безопасности, поскольку позволяют почти неограниченно объединять на одной странице типы содержимого, происходящие с не связанных между собой сайтов. Мы еще раз взглянем на проблемы, связанные с таким поведением, в главе 11.

Включение содержимого по типам

Помимо навигации по ссылкам, не зависящей от типа содержимого, и фреймирования документов, HTML также предоставляет несколько способов более легковесного включения нескольких заранее определенных типов внешнего содержимого.

Изображения

Файлы изображений можно получать и отображать на странице с помощью тегов `<img>`, через таблицы стилей и через устаревший параметр `background=` в разметке вроде `<body>` или `<table>`.

Самый популярный тип изображений в интернете - JPEG, формат с потерями, но очень эффективный; за ним следует PNG, формат без потерь и с большим набором возможностей, но более медленный. Все браузеры также поддерживают все более устаревающий формат GIF без потерь и редко встречающийся, обычно несжатый файл растрового изображения Windows (BMP). Все больше движков рендеринга поддерживают и SVG, основанный на XML формат векторной графики и анимации, но включение таких изображений через тег `<img>` подчиняется дополнительным ограничениям.

Список распознаваемых типов изображений можно дополнить всякой всячиной вроде метафайлов Windows (WMF и EMF), Windows Media Photo (WDP и HDP), значков Windows (ICO), анимированного PNG (APNG), изображений TIFF и, в более новое время, WebP. Однако поддержка этих форматов браузерами далеко не универсальна.

Каскадные таблицы стилей

Эти текстовые файлы можно загружать тегом `<link rel=stylesheet href=...>`, хотя `<style src=...>` был бы более интуитивным выбором, и они могут переопределять визуальные аспекты почти любого другого HTML-тега внутри родительского документа (а в некоторых случаях даже включать встроенный JavaScript). Синтаксис и функции CSS являются темой главы 5.

При отсутствии подходящего значения `charset` в заголовке `Content-Type` для загруженной таблицы стилей кодировка, согласно которой этот подресурс будет интерпретироваться, может быть задана включающей стороной через параметр `charset` тега `<link>`.

Скрипты

Скрипты - это текстовые программы, включаемые тегами `<script>` и выполняемые способом, который дает им полный контроль над документом-хозяином. Основной язык сценариев для Web - JavaScript, хотя встроенная версия Visual Basic также поддерживается в Internet Explorer и может использоваться по желанию. В главе 6 подробно рассматриваются клиентские скрипты и их возможности.

Как и в случае с CSS, при отсутствии действительных данных Content-Type набор символов, согласно которому интерпретируется скрипт, может контролироваться включающей стороной.

Содержимое подключаемых модулей

Эта категория охватывает разные двоичные файлы, включаемые тегами `<embed>` или `<object>` либо через устаревший, специфичный для Java тег `<applet>`. Содержимое браузерных подключаемых модулей следует собственным правилам безопасности, которые в некоторой степени рассматриваются в главах 8 и 9. Во многих случаях содержимое, поддерживаемое подключаемым модулем, безопасно считать эквивалентным JavaScript или более мощным, чем JavaScript.

Примечание. Стандарт разрешает загружать через теги `<object>` некоторые типы документов, поддерживаемых браузером, например `text/html` или `text/plain`; в таком случае они образуют близкий эквивалент `<iframe>`. Эта функциональность на практике не используется, и ее обоснование трудно понять.

Другое вспомогательное содержимое

Эта категория включает различные подсказки рендеринга, которые браузер может учитывать или не учитывать; чаще всего они предоставляются через директивы `<link>`. Примеры включают значки сайтов (известные как favicons), альтернативные версии страницы и ссылки навигации по главам.

Несколько других когда-то поддерживаемых методов включения содержимого, таких как тег `<bgsound>` для фоновой музыки, в прошлом были обычным делом, но вышли из моды. С другой стороны, как часть HTML5, новые теги вроде `<video>` и `<audio>` вскоре, как ожидается, наберут популярность.

Существует сравнительно мало согласованности в том, какие URL-схемы принимаются для получения содержимого по типам. Следует ожидать, что протоколы, маршрутизируемые во внешние приложения, будут отвергнуты, поскольку в этом контексте у них нет разумного смысла, но сверх этого не стоит делать много предположений. В качестве меры безопасности большинство браузеров также отвергает схемы, связанные со сценариями, при загрузке изображений и таблиц стилей, хотя Internet Explorer 6 и Opera не следуют этой практике. На момент написания `javascript:` URL также разрешены в тегах `<embed>` и `<applet>` в Firefox, но, например, не в `<img>`.

Почти для всех методов включения содержимого по типам заголовки Content-Type и Content-Disposition, предоставленные сервером, обычно будут игнорироваться (возможно, кроме значения `charset=`), как может игнорироваться и сам код HTTP-ответа. Лучше считать, что всякий раз, когда тело любого предоставленного сервером ресурса хотя бы отдаленно распознается как один из форматов данных, перечисленных в этом разделе, оно может быть интерпретировано именно так.

Примечание о подделке межсайтовых запросов

При всех типах междоменных переходов браузер прозрачно включает любые окружающие учетные данные; следовательно, для сервера запрос, законно исходящий из его собственного клиентского кода, будет выглядеть примерно так же, как запрос, исходящий с вредоносного стороннего сайта, и ему могут быть предоставлены те же привилегии.

Приложения, которые не учитывают эту возможность при обработке любых чувствительных запросов, изменяющих состояние, считаются уязвимыми к подделке межсайтовых запросов (XSRF или CSRF). Эту уязвимость можно смягчить несколькими способами, самый распространенный из которых - включать в такие запросы секретное значение, специфичное для пользователя и сессии (как дополнительный параметр запроса или скрытое поле формы). Злоумышленник не сможет получить это значение, поскольку доступ на чтение к междоменным документам ограничен политикой одного источника (см. главу 9).

Памятка по инженерии безопасности

Хорошая инженерная гигиена для всех HTML-документов

- Всегда выводите согласованную, действительную и поддерживаемую браузером информацию Content-Type и charset, чтобы документ не был интерпретирован вопреки вашему исходному намерению.

При генерации HTML-документов с фрагментами, контролируруемыми злоумышленником

Эту задачу трудно выполнять последовательно во всем веб-приложении, и она является одним из самых значительных источников дефектов безопасности веб-приложений. Рассмотрите возможность использовать фреймворки с контекстно-зависимым автоэкранированием, такие как JSilver или CTemplate, чтобы автоматизировать ее. Если это невозможно, читайте дальше.

- Предоставленное пользователем содержимое в теле текста: всегда кодируйте сущностями <, > и &. Обратите внимание, что некоторые другие шаблоны могут быть опасны в отдельных выходных кодировках, несовместимых с ASCII. Если применимо, обратитесь к главе 13.

Помните, что некоторые метасимволы Unicode (например, U+202E) меняют направление или поток последующего текста. В особенно чувствительных случаях может быть желательно удалять их.

- Специфичные для тегов параметры style и on*: требуются несколько уровней экранирования. Эта практика чрезвычайно подвержена ошибкам, то есть на самом деле не то, за что стоит браться. Если это абсолютно неизбежно, просмотрите шпаргалки в главах 5 и 6.
- Все остальные значения HTML-параметров: всегда используйте кавычки вокруг ввода, контролируемого злоумышленником. Кодируйте сущностями <, >, & и любые посторонние кавычки. Помните, что некоторые параметры требуют дополнительной проверки. Для URL см. шпаргалку в главе 2.

Никогда не пытайтесь заносить в черный список известные плохие значения в URL или любых других параметрах; это обернется против вас и может привести к дефектам выполнения скриптов.

- Специальные режимы разбора (например, блоки `<script>` и `<style>`): для значений, появляющихся внутри строк в кавычках, заменяйте символы кавычек, обратный слеш, `<`, `>` и все непечатаемые символы escape-кодами, подходящими для языка. Для значений, появляющихся вне строк, проявляйте крайнюю осторожность и разрешайте только тщательно проверенные, известные, буквенно-цифровые значения.

В режиме XHTML не забывайте заключать весь раздел скрипта в блок CDATA. Избегайте случаев, требующих нескольких уровней кодирования, таких как построение параметров функции JavaScript `eval(...)` с использованием строк, предоставленных злоумышленником. Никогда не помещайте контролируемые пользователем данные внутрь HTML-комментариев, тегов типа `!` или `?` и других необязательных или необычно разбираемых блоков.

При преобразовании HTML в обычный текст

- Распространенная ошибка - удалять только правильно сформированные теги. Помните, что нужно удалять все левые угловые скобки, даже если соответствующая правая угловая скобка не найдена. Чтобы минимизировать риск ошибок, всегда также экранируйте сущностями угловые скобки и амперсанды в генерируемом выводе.

При написании фильтра разметки для пользовательского содержимого

- Внимательно прочитайте эту главу. Используйте достаточно надежный HTML-парсер, чтобы построить дерево документа в памяти. Обходите дерево, удаляя все нераспознанные или ненужные теги и параметры и очищая любые нежелательные комбинации тег/параметр/значение.

Закончив, заново сериализуйте документ, убедившись, что применяются правильные правила экранирования к значениям параметров и текстовому содержимому. См. первый совет в этой шпаргалке. Учитывайте влияние специальных режимов разбора.

- Из-за несколько противоинтуитивных взаимодействий пространств имен с JavaScript не разрешайте параметры `name` и `id` в пользовательской разметке - по крайней мере, не прочитав сначала главу 6.
- Не пытайтесь очищать существующий сериализованный документ на месте. Это неизбежно ведет к проблемам безопасности.

Сноски

[1] [\[назад\]](#) Для обработки HTML-документов Internet Explorer использует движок Trident, он же MSHTML; Firefox и некоторые производные продукты используют Gecko; Safari, Chrome и несколько других браузеров используют WebKit; а Opera опирается на Presto. За исключением WebKit, совместного open source-проекта, поддерживаемого несколькими поставщиками, эти движки разрабатываются в основном внутри соответствующих браузерных команд.

[2] [\[назад\]](#) Поведение Opera особенно коварно: пробел Unicode не распознается многими функциями стандартных библиотек, используемых в серверных HTML-очистителях, например `isspace(...)` в `libc`. Это повышает риск дефектов реализации.

[3] [\[назад\]](#) Ссылка в оригинале указывает на ошибку Mozilla Bugzilla, связанную с таким разбором charset из Refresh.

[4] [\[назад\]](#) Это может приводить к путанице, поскольку один и тот же параметр может появиться и в строке запроса, и в полезной нагрузке POST. Нет согласованности в том, как разные серверные фреймворки веб-приложений разрешают этот конфликт.

[5] [\[назад\]](#) MIME (Multipurpose Internet Mail Extensions) - формат данных, предназначенный для инкапсуляции и безопасной передачи разных типов документов в сообщениях электронной почты. Этот формат несколько раз неожиданно появляется в мире браузеров. Например, идентификаторы форматов файлов Content-Type также имеют однозначные корни MIME.

Глава 5. Каскадные таблицы стилей

По мере взросления Web в 1990-х разработчикам сайтов все больше требовался согласованный и гибкий способ управлять внешним видом HTML-документов; набора случайных, специфичных для поставщиков параметров тегов, доступных в то время, просто уже не хватало. Рассмотрев несколько конкурирующих предложений, W3C в конце концов остановился на Cascading Style Sheets (CSS), довольно простым текстовым языком описания внешнего вида страниц, предложенном Хоконом Виумом Ли.

Первоначальная спецификация CSS level 1 увидела свет к концу 1996 года,^[1] но дальнейшие редакции этого документа продолжались до 2008 года. Первый черновик CSS level 2 последовал в декабре 1998 года и по состоянию на 2011 год все еще не был окончательно завершен. Работа над самой свежей итерацией, level 3, началась в 2005 году и тоже продолжается до сих пор. Хотя большинство отдельных возможностей, задуманных для CSS2 и CSS3, после лет проб и ошибок были приняты всеми современными браузерами, многие тонкие детали существенно различаются от одной реализации к другой, и отсутствие завершенного стандарта, вероятно, этому способствует.

Несмотря на различия между браузерами, CSS - очень мощный инструмент. С учетом лишь нескольких ограничений таблицы стилей позволяют почти любой HTML-тег почти произвольно масштабировать, позиционировать и украшать, тем самым преодолевая ограничения, изначально наложенные на него базовым языком разметки; в некоторых реализациях в презентационные директивы CSS можно также встраивать программы JavaScript. Поэтому задача размещения контролируемых пользователем значений внутри таблиц стилей или перекодирования любого внешне предоставленного CSS представляет большой интерес для безопасности веб-приложений.

Базовый синтаксис CSS

Таблицы стилей можно поместить в HTML-документ тремя способами: встроить глобально для всего документа блоком `<style>`, получить с внешнего URL через директиву `<link rel=stylesheet>` или прикрепить к конкретному тегу с помощью параметра `style`. Кроме того, документы на основе XML, включая XHTML, могут также использовать малоизвестную директиву `<?xml-stylesheet href=... ?>`, чтобы достичь той же цели.

Первые два метода включения требуют полноценно заданной таблицы стилей, состоящей из любого числа селекторов (директив, описывающих, к каким HTML-тегам будет применяться следующий набор правил), за которыми следуют разделенные точками с запятой правила `name: value` в фигурных скобках. Вот простой пример такого синтаксиса, определяющий внешний вид тегов `<img>`, `<span>` и `<div>`:

```
img {
  border-size: 1px;
  border-style: solid;
}
span, div {
  color: red;
}
```

Селекторы могут ссылаться на конкретный тип тега (например, `img`), на имя класса тегов с точкой в начале (например, `.photos`, что применится ко всем тегам со встроенным параметром `class=photos`) или на сочетание того и другого (`img.company_logo`). Суффиксы селекторов, такие как `:hover` или `:visited`, также могут использоваться, чтобы селектор совпадал только при

определенных обстоятельствах, например когда мышь находится над содержимым или когда конкретная отображаемая гиперссылка уже была посещена ранее.

Так называемые сложные селекторы^[2] - интересная возможность, введенная в CSS2 и расширенная в CSS3. Они позволяют применять заданный набор правил только к тегам, в значениях параметров которых появляются конкретные строки, или к тегам, расположенным в определенном отношении к другой разметке. Один пример такого селектора:

```
a[href^="ftp:"] {  
  /* Styling applicable only to FTP links. */  
}
```

Примечание. О, раз уж мы об этом: как видно из этого примера, в синтаксисе CSS блоки комментариев в стиле `C /* ... */` разрешены везде вне строки в кавычках. С другой стороны, комментарии в стиле `//` вообще не распознаются.

Определения свойств

Внутри блока `{ ... }`, следующего за селектором, а также внутри параметра `style`, прикрепленного к конкретному тегу, можно использовать любое число правил `name: value`, чтобы переопределить почти любой аспект отображения затронутой разметки. Видимость, форма, цвет, позиция на экране, порядок рендеринга, локальный или удаленный шрифт и даже любой дополнительный текст (свойство `content`, поддерживаемое на некоторых псевдоклассах) и форма курсора мыши - все это доступно для изменения.^[3] Простые типы автоматизации, такие как счетчики для нумерованных списков, также доступны через правила CSS.

Значения свойств могут быть оформлены следующим образом:

- Сырой текст. Этот метод используется главным образом для задания числовых значений (с необязательными единицами), RGB-векторов и именованных цветов, а также других заранее определенных ключевых слов (`absolute`, `left`, `center` и т. д.).
- Строки в кавычках. Одинарные или двойные кавычки следует ставить вокруг любых значений, не являющихся ключевыми словами, но в том, как это правило применяется, мало согласованности. Например, кавычки не требуются вокруг имен шрифтов или в некоторых вариантах использования URL, но необходимы для упомянутого выше свойства `content`.
- Функциональная запись. В исходной спецификации CSS упомянуты две псевдофункции, связанные с параметрами: `rgb(...)`, для преобразования отдельных цветовых значений RGB в единый цветовой код, и `url(...)`, требуемая для URL в большинстве, но не во всех контекстах. Вдобавок к этому в последние годы появились еще несколько псевдофункций, включая `scale(...)`, `rotate(...)` или `skew(...)`.

В Internet Explorer также доступна проприетарная функция `expression(...)`; она позволяет вставлять инструкции JavaScript внутрь CSS. Эта функция - одна из важнейших причин, по которым контролируемые злоумышленником таблицы стилей могут представлять серьезный риск безопасности.

Директивы @ и привязки XBL

Помимо селекторов и свойств, в самостоятельных таблицах стилей распознается несколько директив с префиксом @. Все они изменяют смысл таблицы стилей; например, задавая пространство имен или среду отображения, к которой таблица стилей должна применяться. Но две специальные директивы также влияют на поведение процесса разбора. Первая из них - @charset, задающая набор символов текущего CSS-блока; другая - @import, вставляющая внешний файл в таблицу стилей.

Сама директива @import служит хорошим примером идиосинкразий разбора CSS; парсер считает все следующие примеры эквивалентными:

```
@import "foo.css";
@import url('foo.css');
@import 'foo.css';
```

В Firefox внешние директивы содержимого, включая код JavaScript, также могут загружаться из внешнего источника с помощью свойства -moz-binding, специфичного для поставщика способа вpleсти в документ файлы XML Binding Language^[4] (малоизвестный метод предоставления автоматизации для XML-содержимого). Есть разговоры о поддержке XBL и в других браузерах; в этот момент имя свойства изменилось бы, а риск XSS мог бы быть тем или иным образом устранен, а мог бы и нет.

Примечание. Как и можно ожидать, обработка псевдо-URL в @import, url(...) и других схемах включения содержимого на основе CSS является потенциальным риском безопасности. Хотя большинство современных браузеров не принимает в этих контекстах схемы, связанные со сценариями, Internet Explorer 6 разрешает их без оговорок, тем самым создавая вектор внедрения кода, если URL проверяется недостаточно тщательно.

Взаимодействия с HTML

Из обсуждения в предыдущей главе следует, что для любых таблиц стилей, встроенных в HTML-документы, сначала выполняется разбор HTML, и он полностью независим от синтаксических правил CSS. Поэтому небезопасно помещать некоторые синтаксические символы HTML внутрь CSS-свойств, как в следующем примере, даже при правильном использовании кавычек. Распространенная ошибка - разрешать такое:

```
<style>
some_descriptor {
  background: url('http://www.example.com/</style><h1>Gotcha!');
}
</style>
```

Вскоре мы обсудим способ кодировать проблемные символы в таблицах стилей, но сначала быстро взглянем еще на одно очень характерное свойство CSS.

Риски ресинхронизации парсера

Поведение, несомненно вдохновленное HTML и отличающее CSS от большинства других языков, состоит в том, что соответствующие стандарту парсеры, столкнувшись с синтаксической ошибкой, должны продолжать работу и перезапускаться на следующей подходящей фигурной скобке. Спецификация предписывает некоторый поверхностный учет уровня вложенности. В частности, следующий фрагмент таблицы стилей, хотя он очевидно неправильно сформирован, все равно применит указанный стиль рамки ко всем тегам `<img>`:

```
a {
  $$$ This syntax makes absolutely no sense $$$
  !(@*#)!!@ 123
}
img {
  border: 1px solid red;
}
```

Это необычное поведение создает возможность интересно эксплуатировать несовместимости парсеров: если есть способ сбить с пути конкретную реализацию CSS входными данными, которые другим парсерам кажутся действительными, логика ресинхронизации может заставить атакуемый браузер возобновить разбор в неверном месте, например в середине строки, предоставленной злоумышленником.

Наивной иллюстрацией этой проблемы может быть поддержка Internet Explorer многострочных строковых литералов. В этом браузере, казалось бы, безопасно не очищать символы CR и LF в предоставленных пользователем CSS-строках, поэтому некоторые вебмастера могут это разрешать. К сожалению, тот же шаблон заставит любой другой браузер возобновить работу на неожиданном смещении и интерпретировать набор правил `evil_rule`:

```
some_benign_selector {
  content: 'Attacker-controlled text...
           } evil_rule { margin-left: -1000px; }';
}
```

Поддержка многострочных строк - расширение, специфичное для Microsoft, и упомянутая проблема легко исправляется тем, что изначально не следует использовать такой несоответствующий стандарту синтаксис. К сожалению, другие риски десинхронизации вводит сам стандарт. Например, вспомните сложные селекторы: этот синтаксис CSS3 не имеет смысла для парсеров до CSS3. В следующем примере более старая реализация может сдаться, встретив неожиданную угловую скобку, и возобновить разбор с предоставленного злоумышленником `evil_rule`:

```
a[href^='} evil_rule { margin-left: -1000px; }'] {
  /* Harmless, validated rules here. */
}
```

Все еще популярный браузер Internet Explorer 6 был бы уязвим к этому трюку.

Кодирование символов

Чтобы можно было заключать в кавычки зарезервированные или иначе проблемные символы внутри строк, CSS предлагает неортодоксальную схему экранирования: обратный слеш (\), за которым следует от одной до шести шестнадцатеричных цифр. Например, согласно этой схеме буква e может быть закодирована как \65, \065 или \000065. Увы, только последний синтаксис, \000065, будет однозначным, если следующий символ окажется допустимой шестнадцатеричной цифрой; кодирование teak как t\65ak не сработает ожидаемым образом, потому что escape-последовательность будет интерпретирована как \65A, арабский знак в карте символов Unicode.

Чтобы избежать этой проблемы, спецификация принимает неловкий компромисс: после escape-последовательности может идти пробельный символ, который будет интерпретирован как терминатор, а затем удален из строки (например, t\65 ak). К сожалению, вместо этого нельзя использовать более привычные и предсказуемые C-подобные escape-последовательности фиксированной длины, такие как \x65.

Помимо числовой схемы экранирования, можно также поставить обратный слеш перед символом, который не является допустимой шестнадцатеричной цифрой. В таком случае последующий символ будет обработан как литерал. Этот механизм полезен для кодирования символов кавычек и самого обратного слеша, но его не следует использовать для экранирования управляющих символов HTML, таких как угловые скобки. Упомянутый выше приоритет разбора HTML над разбором CSS делает такой подход недостаточным.

В причудливом повороте, из-за несколько неоднозначных указаний в черновиках W3C многие CSS-парсеры распознают произвольные escape-последовательности в местах, отличных от строк в кавычках. Чтобы добавить неприятностей, в Internet Explorer подстановка этих последовательностей, судя по всему, происходит до разбора синтаксиса псевдофункций, фактически делая следующие два примера эквивалентными:

```
color: expression(alert(1))
color: expression\028 alert \028 1 \029 \029
```

Еще больше сбивает с толку то, что в ошибочной попытке сохранить отказоустойчивость реализация Microsoft не распознает escape-коды с обратным слешем внутри значений url(...); это, опять же, сделано, чтобы не ранить чувства пользователей, которые вводят неправильный тип слеша при указании URL.

Эти и похожие особенности делают обнаружение известного опасного синтаксиса CSS чрезвычайно подверженным ошибкам.

Памятка по инженерии безопасности

При загрузке удаленных таблиц стилей

- Вы связываете безопасность своего сайта с доменом происхождения таблицы стилей. Даже в браузерах, которые не поддерживают JavaScript-выражения внутри таблиц стилей, такие возможности, как условные селекторы и ссылки url(...), могут использоваться для эксфильтрации частей вашего сайта.^[5]
- Если сомневаетесь, сделайте локальную копию данных.

- На HTTPS-сайтах требуйте, чтобы таблицы стилей также отдавались по HTTPS.

При помещении контролируемых злоумышленником значений в CSS

- Строки и URL внутри самостоятельных блоков. Всегда используйте кавычки. Экранируйте обратным слешем все управляющие символы (0x00-0x1F), \, <, >, {, } и кавычки с использованием числовых кодов. Также предпочтительно экранировать символы старшего диапазона. Для URL обратитесь к памятке в главе 2, чтобы избежать уязвимостей внедрения кода.
- Строки в параметрах style. Здесь задействованы несколько уровней экранирования. Процесс подвержен ошибкам, поэтому не пытайтесь делать это без абсолютной необходимости. Если это неизбежно, сначала примените приведенные выше правила CSS-экранирования, а затем примените кодирование HTML-параметра к получившейся строке.
- Нестроковые атрибуты. Разрешайте только буквенно-цифровые ключевые слова из белого списка и тщательно проверенные числовые значения. Не пытайтесь вместо этого отвергать известные плохие шаблоны.

При фильтрации CSS, предоставленного пользователем

- Удаляйте все содержимое вне функциональных наборов правил. Не сохраняйте и не генерируйте контролируемые пользователем блоки комментариев, @-директивы и так далее.
- Тщательно проверяйте синтаксис селекторов, разрешая только буквенно-цифровые символы, подчеркивания, пробельные символы и правильно расположенные двоеточия, точки и запятые перед {. Не разрешайте сложные селекторы сопоставления текста; они небезопасны.
- Разбирайте и проверяйте каждое правило в блоке { ... }. Разрешайте только свойства из белого списка с хорошо понятными последствиями и подтверждайте, что они принимают ожидаемые, заведомо безопасные значения. Обратите внимание, что строки, передаваемые некоторым свойствам, иногда могут интерпретироваться как URL даже без обертки url(...).
- Кодировать каждое значение параметра по правилам, изложенным ранее в этом разделе. Отказывайтесь от обработки при любых синтаксических аномалиях.
- Помните, что если специально этому не помешать, CSS может позиционировать пользовательское содержимое вне предполагаемой области рисования или переопределить внешний вид любой части UI вашего приложения. Самый безопасный способ избежать этой проблемы - отображать недоверенное содержимое внутри отдельного фрейма.

При разрешении заданных пользователем значений class в HTML-разметке

- Убедитесь, что предоставленное пользователем содержимое не может повторно использовать имена классов, которые применяются для какой-либо части UI приложения. Если отдельный фрейм не используется, рекомендуется поддерживать отдельные префиксы пространств имен.

Сноски

[1] [\[назад\]](#) Сноска в оригинале указывает на спецификацию CSS level 1 на сайте W3C.

[2] [\[назад\]](#) В оригинале здесь дана ссылка на раздел Selectors Level 3 о селекторах.

[3] [\[назад\]](#) Возможность переопределять курсоры мыши произвольным растровым изображением предсказуемо привела к некоторым ошибкам безопасности. Крупный курсор в сочетании с отслеживанием позиции мыши скриптами мог использоваться, чтобы скрывать или заменять важные элементы UI браузера и обманывать пользователя, заставляя его сделать что-то опасное.

[4] [\[назад\]](#) В оригинале сноски указывает на спецификацию XBL 2.0 W3C.

[5] [\[назад\]](#) Оригинальная сноска указывает на материал о краже истории браузера с помощью CSS.

Глава 6. Браузерные скрипты

Первый браузерный движок сценариев появился в Netscape Navigator примерно в 1995 году благодаря работе Брендана Айка. Интегрированный язык Mocha, как он изначально назывался, дал веб-разработчикам возможность манипулировать HTML-документами, показывать простые системные диалоги, открывать и перемещать окна браузера, а также использовать другие базовые типы клиентской автоматизации без лишних сложностей.

Пока шла серия бета-выпусков, Netscape в итоге переименовала Mocha в LiveScript, а после неловкой брендовой сделки с Sun Microsystems окончательным именем стало JavaScript. Сходств между Mocha Брендана и Java от Sun было мало, но Netscape Corporation рассчитывала, что этот странный маркетинговый брак закрепит господство JavaScript в более прибыльном серверном мире. Компания ясно дала понять это настроение в знаменитом запутанном пресс-релизе 1995 года, который представил язык миру и сразу попытался привязать его к впечатляющему набору случайных коммерческих продуктов.^[1]

Netscape и Sun объявляют JavaScript, открытый, межплатформенный объектный язык сценариев для корпоративных сетей и интернета

[...]

Netscape Navigator Gold 2.0 позволяет разработчикам создавать и редактировать скрипты JavaScript, тогда как Netscape LiveWire позволяет устанавливать, запускать и администрировать программы JavaScript на серверах Netscape, как внутри предприятия, так и через интернет. Netscape LiveWire Pro добавляет поддержку соединения JavaScript с высокопроизводительными реляционными базами данных Illustra, Informix, Microsoft, Oracle и Sybase. Поддержка Java и JavaScript встраивается во все продукты Netscape, чтобы предоставить единую среду клиент/сервер/инструменты, от начала до конца, для построения и развертывания живых онлайн-приложений.

Несмотря на неуместную любовь Netscape к Java, ценность JavaScript для клиентского программирования казалась очевидной, в том числе конкурентам. В 1996 году Microsoft ответила поставкой почти дословной копии JavaScript в Internet Explorer 3.0 вместе с собственным контрпредложением: языком, производным от Visual Basic, под названием VBScript. Возможно, из-за опоздания к началу гонки, а возможно, из-за более неуклюжего синтаксиса VBScript альтернатива Microsoft не получила известности и даже не добилась какой-либо межбраузерной поддержки. В конце концов JavaScript закрепил свое положение на рынке, и отчасти из-за неудачи Microsoft с тех пор в массовых браузерах не предпринимались попытки внедрить новые языки сценариев.

Воодушевленная популярностью JavaScript, Netscape передала часть ответственности за его сопровождение независимому органу, European Computer Manufacturers Association (ECMA). Новые кураторы успешно выпустили ECMAScript, 3rd edition в 1999 году,^[2] но дальнейшее продвижение оказалось значительно труднее. 4-я редакция, амбициозная переработка языка, в итоге была заброшена после нескольких лет споров между поставщиками, а урезанная 5-я редакция,^[3] опубликованная в 2009 году, все еще имеет лишь ограниченную, хотя и стабильно улучшающуюся, поддержку в браузерах. Работа над новой итерацией, названной Harmony, начатая в 2008 году, все еще не завершена. При отсутствии развивающегося и широко принятого стандарта расширения языка, специфичные для поставщиков, распространены, но обычно приносят только боль.

Базовые характеристики JavaScript

JavaScript - довольно простой язык, предназначенный для интерпретации во время выполнения. У него синтаксис, смутно вдохновленный С, за исключением арифметики указателей; прямолинейная бесклассовая объектная модель, как говорят, вдохновленная малоизвестным языком программирования Self; автоматическая сборка мусора; и слабая динамическая типизация.

Сам по себе JavaScript не имеет встроенных механизмов ввода-вывода. В браузере ограниченные возможности взаимодействовать со средой-хозяином предоставляются через набор заранее определенных методов и свойств, сопоставленных с нативным кодом внутри браузера, но, в отличие от того, что можно увидеть во многих других языках программирования, эти интерфейсы довольно ограничены и специально предназначены для своих задач.

Большинство основных возможностей JavaScript довольно непримечательны и должны быть знакомы разработчикам, уже имеющим опыт с С, С++ или, в меньшей степени, Java. Простая программа на JavaScript может выглядеть так:

```
var text = "Hi mom!";
function display_string(str) {
    alert(str);
    return 0;
}
// This will display "Hi mom!".
display_str(text);
```

Поскольку подробный обзор семантики JavaScript выходит за рамки этой книги, позднее в этой главе мы кратко изложим только некоторые его более уникальные и значимые для безопасности свойства. Для читателей, ищущих более систематическое введение в язык, хорошим выбором будет *Eloquent JavaScript* Марейна Хавербеке (No Starch Press, 2011).

Модель обработки скриптов

Каждый HTML-документ, отображаемый в браузере - будь то в отдельном окне или во фрейме, - получает отдельный экземпляр среды выполнения JavaScript вместе с индивидуальным пространством имен для всех глобальных переменных и функций, созданных загруженными скриптами. Все скрипты, выполняющиеся в контексте конкретного документа, разделяют эту общую песочницу и также могут взаимодействовать с другими контекстами через API, предоставленные браузером. Такие междокументные взаимодействия должны выполняться очень явно; случайное вмешательство маловероятно. На поверхности правила изоляции скриптов напоминают модель разделения процессов в современных многозадачных операционных системах, но куда менее всеобъемлющи.

Внутри конкретного контекста выполнения все встреченные блоки JavaScript обрабатываются по отдельности и почти всегда в хорошо определенном порядке. Каждый блок кода должен состоять из любого числа самостоятельных, правильно сформированных синтаксических единиц и будет обработан в три отдельных последовательных шага: разбор, разрешение функций и выполнение кода.

Разбор

Стадия разбора проверяет синтаксис блока скрипта и обычно преобразует его в промежуточное двоичное представление, которое затем может выполняться с более разумной скоростью. Код не имеет глобальных эффектов, пока этот шаг не завершится успешно. В случае синтаксических ошибок весь проблемный блок отбрасывается, а парсер переходит к следующему доступному фрагменту кода.

Чтобы проиллюстрировать поведение соответствующего стандарту JavaScript-парсера, рассмотрим следующий HTML-фрагмент:

```
block #1:
<script>
var my_variable1 = 1;
var my_variable2 =
</script>
block #2:
<script>
2;
</script>
```

Вопреки тому, к чему могут привыкнуть разработчики, воспитанные на C, приведенная выше последовательность не эквивалентна следующему фрагменту:

```
<script>
var my_variable1 = 1;
var my_variable2 = 2;
</script>
```

Причина в том, что блоки `<script>` не конкатенируются перед разбором. Вместо этого первый сегмент скрипта просто вызовет синтаксическую ошибку (присваивание с отсутствующим значением справа), в результате чего весь блок будет проигнорирован и не дойдет до стадии выполнения. Тот факт, что весь сегмент отбрасывается до того, как он сможет произвести какие-либо глобальные побочные эффекты, также означает, что исходный пример не эквивалентен и этому:

```
<script>
var my_variable1 = 1;
</script>
<script>
2;
</script>
```

Это отличает JavaScript от многих других языков сценариев, таких как Bash, где стадия разбора не отделена от выполнения настолько жестко.

В исходном примере, приведенном ранее в этом разделе, произойдет следующее: первый блок будет проигнорирован, но второй (`<script>2;</script>`) будет разобран правильно. Однако при выполнении этот второй блок ничего не сделает, потому что использует чистое числовое выражение как инструкцию кода.

Разрешение функций

После успешного завершения стадии разбора следующий шаг включает регистрацию каждой именованной глобальной функции, которую парсер нашел внутри обрабатываемого в данный момент блока. После этого каждая найденная функция будет доступна последующему выполняемому коду. Благодаря этому дополнительному шагу до выполнения следующий синтаксис сработает безупречно: вопреки тому, к чему могут привыкнуть программисты на С или С++, `hello_world()` будет зарегистрирована до выполнения первой инструкции кода - вызова этой самой функции:

```
<script>
hello_world();
function hello_world() {
    alert('Hi mom!');
}
</script>
```

С другой стороны, измененный пример ниже не даст желаемого эффекта:

```
<script>
hello_world();
</script>
<script>
function hello_world() {
    alert('Hi mom!');
}
</script>
```

Этот измененный случай завершится ошибкой времени выполнения, потому что отдельные блоки кода не обрабатываются одновременно, а рассматриваются в порядке, в котором они становятся доступны движку JavaScript. Блок, определяющий `hello_world()`, еще не будет разобран, когда первый блок уже выполняется.

Чтобы еще больше усложнить картину, слегка неловкая модель разрешения глобальных имен, описанная здесь, применяется только к функциям, но не к объявлениям переменных. Переменные регистрируются последовательно во время выполнения, похожим на другие интерпретируемые языки сценариев способом. Следовательно, следующий пример кода, который просто заменяет нашу глобальную `hello_world()` безымянной функцией, присвоенной глобальной переменной, не сработает как задумано:

```
<script>
hello_world();
var hello_world = function() {
    alert('Hi mom!');
}
</script>
```

В этом случае присваивание переменной `hello_world` еще не будет выполнено к моменту попытки вызова `hello_world()`.

Выполнение кода

После завершения разрешения функций движок JavaScript обычно переходит к упорядоченному выполнению всех инструкций вне блоков функций. На этом этапе выполнение скрипта может завершиться неудачей из-за необработанного исключения или по нескольким другим, более эзотерическим причинам. Однако если такая ошибка встречается, все разрешенные функции внутри провинившегося блока кода останутся вызываемыми, а любые эффекты уже выполненного кода сохранятся в текущем контексте сценариев.

Восстановление после исключений и несколько других характеристик выполнения JavaScript иллюстрируются следующим длинным, но интересным фрагментом кода. Попробуйте самостоятельно проследить этот пример и посмотреть, согласны ли вы с пояснениями.

```
<script>
function not_called() {
    return 42;
}
function hello_world() {
    alert("With this program, anything is possible!");
    do_stuff();
}
alert("Welcome to our demo application.");
hello_world();
alert("Thank you, come again.");
</script>
<script>
alert("Now that you are done, how about a nice game of chess?");
</script>
```

В этой последовательности:

- Функция `not_called()` не выполнится, потому что ее нигде не вызывают.
- Функция `hello_world()` выполнится только при вызове. Она покажет диалог, но затем выбросит исключение из-за неразрешенной ссылки на функцию с именем `do_stuff()`.
- Выполнение программы начнется с инструкции `alert("Welcome...")`.
- Сообщение `With this...` будет показано следующим.
- Код после вызова `hello_world()` не будет достигнут из-за необработанного исключения, вызванного внутри `hello_world()`.
- Предыдущее исключение не мешает следующему независимому блоку выполниться.

Как должно быть видно из этого упражнения, любые неожиданные и необработанные исключения имеют необычное последствие: они могут оставить приложение в несогласованном, но все еще потенциально выполняемом состоянии. Поскольку исключения предназначены для предотвращения распространения ошибок, вызванных непредвиденными проблемами, такая конструкция странна, особенно учитывая, что на многих других фронтах, например в запрете операторов `goto`, JavaScript занимает более фундаменталистскую позицию.

Управление порядком выполнения

Чтобы правильно анализировать свойства безопасности некоторых распространенных шаблонов проектирования веб-приложений, важно понимать модель порядка и времени выполнения движка JavaScript. К счастью, эта модель замечательно здравая.

Почти весь JavaScript, живущий внутри конкретного контекста выполнения, выполняется синхронно. Код не может быть повторно введен из-за внешнего события, пока он все еще выполняется, и нет поддержки потоков, которые могли бы одновременно изменять любую общую память. Пока движок выполнения занят, обработка событий, таймеров, запросов навигации по странице и так далее откладывается; в большинстве случаев весь браузер или, по крайней мере, HTML-рендерер также остается в значительной степени неотзывчивым. Только после остановки выполнения и перехода движка сценариев в состояние простоя обработка поставленных в очередь событий возобновляется. В этот момент код JavaScript может быть введен снова.

Кроме того, JavaScript не предлагает функции `sleep(...)` или `pause(...)`, которая временно освободила бы CPU и позже возобновила выполнение с того же места. Вместо этого, если программист хочет отложить выполнение скрипта, нужно зарегистрировать таймер, который позже инициирует новый поток выполнения. Этот поток должен начаться с начала указанной функции-обработчика или с начала специального, самостоятельного фрагмента кода, предоставленного при настройке таймера. Хотя эти проектные решения могут раздражать, они существенно снижают риск гонок в получающемся коде.

Примечание. В этой синхронной модели выполнения есть несколько, вероятно, непреднамеренных лазеек. Одна из них - возможность выполнения кода, пока выполнение другого фрагмента JavaScript временно приостановлено после вызова `alert(...)` или `showModalDialog(...)`. Такие крайние случаи, однако, встречаются не очень часто.

Разрушительное, блокирующее браузер поведение занятых циклов JavaScript требует реализации некоторых средств смягчения на уровне браузера. Мы подробно рассмотрим эти средства в главе 14. Пока достаточно сказать, что у них есть еще одно крайне необычное последствие: любой бесконечный цикл на самом деле может завершиться способом, похожим на выбрасывание необработанного исключения. После этого движок вернется в состояние простоя, но останется работоспособным, провинившийся код останется вызываемым, а все таймеры и обработчики событий останутся на месте.

Если злоумышленник намеренно вызывает способность неожиданно прервать выполнение кода, интенсивно использующего CPU, это может привести приложение в несогласованное состояние, оборвав операцию, которую автор ожидает всегда успешно завершенной. И это еще не все: другое, тесно связанное следствие этой семантики должно стать очевидным в разделе "JavaScript Object Notation и другие сериализации данных" на странице 104 оригинала.

Возможности инспекции кода и объектов

В языке JavaScript есть примитивная возможность инспектировать декомпилированный исходный код любых ненативных функций, просто вызвав метод `toString()` или `toSource()` на любой функции, которую разработчик хочет изучить. Помимо этой возможности, способы инспектировать ход программ ограничены. Приложения могут использовать доступ к представлению своего документа-хозяина в памяти и находить все встроенные блоки `<script>`, но прямой видимости в любой удаленно загруженный или динамически сгенерированный код нет. Некоторое представление о стеке вызовов также можно получить через нестандартное свойство `caller`, но также нет способа узнать, какая строка кода выполняется в данный момент или какая будет следующей.

Способность динамически создавать новый код JavaScript является более заметной частью языка. Можно приказать движку синхронно интерпретировать строки, переданные во встроенную функцию `eval(...)`. Например, это покажет диалог предупреждения:

```
eval("alert(\"Hi mom!\")")
```

Синтаксические ошибки в любом входном тексте, предоставленном `eval(...)`, заставят эту функцию выбросить исключение. Аналогично, если разбор успешен, любые необработанные исключения, выброшенные интерпретируемым кодом, будут переданы вызывающему коду. Наконец, при отсутствии синтаксических ошибок или проблем времени выполнения значение последней инструкции, вычисленной движком при выполнении предоставленного кода, будет использовано как возвращаемое значение самой `eval(...)`.

Помимо этой функции, можно использовать другие механизмы уровня браузера, чтобы запланировать отложенный разбор и выполнение новых блоков JavaScript после возвращения движка выполнения в состояние простоя. Примеры таких механизмов включают таймеры (`setTimeout`, `setInterval`), обработчики событий (`onclick`, `onload` и так далее) и интерфейсы к самому HTML-парсеру (`innerHTML`, `document.write(...)` и подобные).

В то время как возможность инспектировать код несколько скрыта, возможности интроспекции объектов во время выполнения хорошо развиты в JavaScript. Приложениям разрешено перечислять почти любой метод или свойство объекта с помощью простых итераторов `for ... in` или `for each ... in`, и они могут использовать операторы вроде `typeof`, `instanceof` или "строго равно" (`===`) и свойства вроде `length`, чтобы получить дополнительное представление об идентичности каждого найденного элемента.

Все перечисленные возможности делают практически невозможным для скриптов, выполняющихся в одном контексте, хранить секреты друг от друга. Эта функциональность также затрудняет хранение секретов между контекстами документов - проблема, с которой поставщикам браузеров приходилось бороться очень долго и которая, как вы узнаете в главе 11, все еще не полностью ушла в прошлое.

Изменение среды выполнения

Несмотря на относительную простоту языка JavaScript, выполняемые скрипты имеют множество необычных способов глубоко манипулировать поведением собственной песочницы JavaScript. В некоторых редких случаях эти поведения могут затрагивать и другие документы.

Переопределение встроенных средств

Один из более необычных инструментов в распоряжении вредоносного скрипта - возможность удалять, перезаписывать или затенять большинство встроенных функций JavaScript и практически все предоставленные браузером методы ввода-вывода. Например, рассмотрим поведение следующего кода:

```
// This assignment will not trigger an error.
eval = alert;
// This call will unexpectedly open a dialog prompt.
eval("Hi mom!");
```

И это только начало веселья. В Chrome, Safari и Opera впоследствии можно полностью удалить функцию `eval(...)` с помощью оператора `delete`. Сбивает с толку то, что попытка сделать то же самое в Firefox восстановит исходную встроенную функцию, отменив эффект первоначального переопределения. Наконец, в Internet Explorer попытка удаления породит запоздалое исключение, которое, похоже, в этот момент уже не служит никакой осмысленной цели.

Двигаясь дальше в этом направлении, почти каждый объект, включая встроенные вроде `String` или `Array`, имеет свободно изменяемый прототип. Этот прототип - главный объект, от которого все существующие и будущие экземпляры объектов наследуют свои методы и свойства, образуя грубый эквивалент наследования классов, присутствующего в более полноценных языках программирования. Возможность вмешиваться в прототипы объектов может вызывать довольно противоинтуитивное поведение вновь создаваемых объектов, как показано здесь:

```
Number.prototype.toString = function() {
    return "Gotcha!";
};
// This will display "Gotcha!" instead of "42":
alert(new Number(42));
```

Сеттеры и геттеры

Более интересные возможности объектной модели, доступные в современных диалектах JavaScript, - сеттеры и геттеры: способы предоставить собственный код, который обрабатывает чтение или установку свойств объекта-хозяина. Хотя они не настолько мощны, как перегрузка операторов в C++, их можно использовать, чтобы существующие объекты или прототипы объектов вели себя еще более запутанно. В следующем фрагменте и установка свойства объекта, и последующее чтение этого свойства легко подрываются:

```
var evil_object = {
    set foo() { alert("Gotcha!"); },
    get foo() { return 2; }
};
// This will display "Gotcha!" and have no other effect.
evil_object.foo = 1;
// This comparison will fail.
if (evil_object.foo != 1) alert("What's going on?!");
```

Примечание. Сеттеры и геттеры изначально были разработаны как расширение поставщика, но теперь стандартизованы в ECMAScript edition 5. Эта возможность доступна во всех современных браузерах, но не в Internet Explorer 6 или 7.

Влияние на возможные применения языка

В результате техник, обсуждавшихся в двух предыдущих разделах, скрипт, выполняющийся внутри контекста, однажды загрязненного любым другим недоверенным содержимым, не имеет надежного способа исследовать свою операционную среду или принять корректирующие меры; даже на поведение простых условных выражений или циклов нельзя обязательно полагаться.

Предлагаемые улучшения языка, вероятно, сделают картину еще сложнее. Например, неудавшееся предложение ECMAScript edition 4 содержало полноценную перегрузку операторов, и эта идея может вернуться.

Еще интереснее, что эти проектные решения также затрудняют инспекцию любого контекста выполнения снаружи постраничной песочницы. Например, слепая вера в надежность объекта `location` потенциально враждебного документа привела к немалому числу уязвимостей безопасности в браузерных подключаемых модулях, расширениях на JavaScript и нескольких классах клиентских функций безопасности веб-приложений. Эти уязвимости в конце концов привели к разработке обходных мер на уровне браузера, предназначенных для частичной защиты именно этого объекта от саботажа, но большая часть оставшейся иерархии объектов открыта для вмешательства.

Примечание. Возможность вмешиваться в собственный контекст выполнения ограничена в "strict" mode ECMAScript edition 5. Однако этот режим на момент написания не полностью поддерживается ни одним браузером и задуман как добровольный механизм, включаемый по желанию.

JavaScript Object Notation и другие сериализации данных

Очень важная синтаксическая структура в JavaScript - его очень компактная и удобная встроенная сериализация объектов, известная как JavaScript Object Notation, или JSON (RFC 4627^[4]). Этот формат данных полагается на перегрузку смысла символа фигурной скобки (`{}`). Когда такая скобка используется, чтобы открыть полностью заданную инструкцию, она трактуется привычным образом, как начало вложенного блока кода. Однако в выражении предполагается, что это начало сериализованного объекта. Следующий пример иллюстрирует корректное использование этого синтаксиса и покажет простой диалог:

```
var impromptu_object = {
  "given_name"      : "John",
  "family_name"     : "Smith",
  "lucky_numbers"   : [ 11630, 12067, 12407, 12887 ]
};
// This will display "John".
alert(impromptu_object.given_name);
```

В отличие от однозначных сериализаций чисел, строк или массивов, перегрузка фигурной скобки означает, что блоки JSON не будут правильно распознаны при использовании как самостоятельная инструкция. Это может казаться несущественным, но это преимущество: оно не позволяет любым ответам, предоставленным сервером и соответствующим этому синтаксису, осмысленно включаться между доменами через `<script src=...>`.^[5] Следующий листинг вызовет синтаксическую ошибку, якобы из-за недопустимой кавычки в том, что интерпретатор пытается трактовать как метку кода,^[6] и не будет иметь измеримых побочных эффектов:

```
<script>
{
  "given_name"    : "John",
  "family_name"   : "Smith",
  "lucky_numbers" : [ 11630, 12067, 12407, 12887 ]
};
</script>
```

Примечание. Невозможность включить JSON через `<script src=...>` - интересное свойство, но также хрупкое. В частности, оборачивание ответа в круглые или квадратные скобки либо удаление кавычек вокруг меток сделает синтаксис легко выполняемым в самостоятельном блоке, что может иметь наблюдаемые побочные эффекты. Учитывая быстро развивающийся синтаксис JavaScript, неразумно рассчитывать, что именно такая раскладка кода всегда будет вызывать ошибку разбора в будущие годы. Тем не менее во многих некритичных случаях такого уровня уверенности будет достаточно, чтобы полагаться на него как на простой механизм безопасности.

После получения через канал вроде XMLHttpRequest сериализация JSON может быть быстро и без усилий преобразована в объект в памяти с помощью функции `JSON.parse(...)` во всех распространенных браузерах, кроме Internet Explorer. К сожалению, ради совместимости с Internet Explorer, а иногда просто по привычке, многие разработчики прибегают к столь же быстрому, но куда более опасному трюку:

```
var parsed_object = eval("(" + json_text + ")");
```

Проблема этого синтаксиса в том, что функция `eval(...)`, используемая для вычисления "значения" JSON-выражения, допускает в строке не только чистые JSON-входы, но и любой другой правильно сформированный синтаксис JavaScript. Это может иметь нежелательные глобальные побочные эффекты. Например, вызов функции, встроенный в этот псевдо-JSON-ответ, выполнится:

```
{ "given_name": alert("Hi mom!") }
```

Такое поведение создает дополнительную нагрузку на веб-разработчиков: принимать JSON-полезные нагрузки только из доверенных источников и всегда корректно экранировать потоки, создаваемые их собственным серверным кодом. Предсказуемо, невыполнение этого требования внесло вклад в немалое число ошибок безопасности на уровне приложений.

Примечание. Трудность правильного использования `eval(...)` воплощена в самой спецификации JSON (RFC 4627): якобы безопасная реализация парсера, включенная в этот документ, непреднамеренно разрешает вредоносным JSON-ответам свободно инкрементировать или декрементировать любые переменные программы, состоящие только из букв a, e, f, l, n, r, s, t, u и цифр; этого достаточно, чтобы составить unsafe и около 1 000 других распространенных английских слов. Ошибочное регулярное выражение, узаконенное в этом RFC, встречается по всему интернету и будет встречаться дальше.

Благодаря удобству использования сериализации JSON повсеместны в обмене данными от сервера к клиенту во всех современных веб-приложениях. С форматом конкурируют только другие, менее безопасные сериализации строк или массивов и JSONP.^[7] Однако все эти схемы

несовместимы с `JSON.parse(...)` и должны полагаться на небезопасный `eval(...)`, чтобы преобразоваться в данные в памяти. Другое свойство этих форматов в том, что, в отличие от правильного JSON, они успешно разбираются при загрузке через `<script src=...>` на сторонней странице. Это свойство полезно в некоторых редких случаях, но в основном просто составляет неочевидный риск. Например, учтите, что даже если загрузка сериализации массива через тег `<script>` обычно не имеет измеримых побочных эффектов, злоумышленник мог, по крайней мере до недавних улучшений, изменить сеттеры в прототипе `Array`, чтобы получить предоставленные данные. Распространенная, но часто недостаточная практика добавлять к ответу префикс с циклом `while(1);`, чтобы предотвратить эту атаку, может обернуться интересным образом, если вспомнить возможность завершения бесконечных циклов в JavaScript.

E4X и другие расширения синтаксиса

Как и HTML, JavaScript быстро развивается. Некоторые изменения, внесенные в него за годы, были довольно радикальными и могут в итоге превратить текстовые форматы, которые раньше отвергались парсером, в действительный код JavaScript. Это, в свою очередь, может привести к неожиданному раскрытию данных, особенно в сочетании с широкими возможностями инспекции и изменения кода и объектов, обсуждавшимися ранее в этой главе, а также со способностью использовать `<script src=...>` для загрузки междоменного кода.

Один из более заметных примеров этой тенденции - ECMAScript for XML (E4X),^[8] совершенно ненужный, но элегантный план встроить XML-синтаксис непосредственно в JavaScript как альтернативу сериализациям в стиле JSON. В любом движке, совместимом с E4X, например Firefox, следующие два фрагмента кода были бы примерно эквивалентны:

```
// Normal object serialization
var my_object = { "user": {
    "given_name": "John",
    "family_name": "Smith",
    "id": make_up_value()
} };

// E4X serialization
var my_object = <user>
    <given_name>John</given_name>
    <family_name>Smith</family_name>
    <id>{ make_up_value() }</id>
</user>;
```

Неожиданное следствие E4X состоит в том, что при таком режиме любой правильно сформированный XML-документ внезапно становится действительной целью `<script src=...>`, которая будет разобрана как блок "выражение как инструкция". Более того, если злоумышленник может стратегически разместить символы `{ }` и `&` на включаемой странице или изменить сеттеры правильного прототипа объекта, он может суметь извлечь пользовательский текст, отображаемый в несвязанном документе. Следующий пример иллюстрирует риск:

```
<html xmlns="http://www.w3.org/1999/xhtml">
...
{ steal_stuff(
...
    <span>User-specific secrets here</span>
...
) }
...
</html>
```

Строки, предоставленные злоумышленником, окружают фрагменты с секретами пользователя.

К чести разработчиков Firefox, после нескольких лет жизни с этим дефектом они решили запретить любые E4X-инструкции, охватывающие целиком весь разбираемый скрипт, частично закрыв эту лазейку. Тем не менее текучесть языка очевидна, и она заставляет немного сомневаться в надежности использования JSON-ответов как защиты от междоменного включения скриптов. В тот момент, когда символу { будет придан третий смысл или кавычки-как-метки начнут иметь назначение, безопасность этого формата обмена данными от сервера к клиенту существенно ухудшится. Планируйте заранее.

Стандартная иерархия объектов

Среда выполнения JavaScript структурирована вокруг неявного корневого объекта, который используется как пространство имен по умолчанию для всех глобальных переменных и функций, создаваемых программой. Помимо горстки встроенных средств, предписанных языком, это пространство имен заранее заполнено иерархией функций, реализующих возможности ввода и вывода в среде браузера. Эти возможности включают манипулирование окнами браузера (`open(...)`, `close()`, `moveTo(...)`, `resizeTo(...)`, `focus()`, `blur()` и подобные); настройку таймеров JavaScript (`setTimeout(...)`, `setInterval(...)` и так далее); отображение различных подсказок UI (`alert(...)`, `prompt(...)`, `print(...)`); и выполнение множества других специфичных для поставщика и часто рискованных функций, таких как доступ к системному буферу обмена, создание закладок или изменение домашней страницы.

Объект верхнего уровня также предоставляет JavaScript-ссылки на корневые объекты, принадлежащие связанным контекстам, включая родительский фрейм (`parent`), документ верхнего уровня в текущем окне браузера (`top`), окно, создавшее текущее (`opener`), и все подфреймы текущего документа (`frames[]`). Несколько циклических ссылок на сам текущий корневой объект также включены - например, `window` и `self`. В браузерах, отличных от Firefox, элементы с заданными параметрами `id` или `name` тоже будут автоматически зарегистрированы в этом пространстве имен, позволяя такой синтаксис:

```

...
<script>
  alert(hello.src);
</script>
```

К счастью, при любых конфликтах имен с переменными JavaScript или встроенными средствами данные `id` не получают приоритета, что в значительной степени избегает возможного вмешательства между иначе очищенной, предоставленной пользователем разметкой и внутривидеоментными скриптами.

Остаток иерархии верхнего уровня состоит прежде всего из нескольких выделенных дочерних объектов, которые группируют возможности браузерного API по темам:

`location` object - это набор свойств и методов, позволяющих программе читать URL текущего документа или инициировать навигацию к новому. Последнее действие в большинстве случаев смертельно для вызывающего кода: текущий контекст сценариев вскоре будет уничтожен и заменен новым. Обновление только идентификатора фрагмента (`location.hash`) является исключением из этого правила, как объяснено в главе 2.

Обратите внимание, что при использовании данных `location.*` для построения новых строк, особенно HTML- и JavaScript-кода, небезопасно предполагать, что они экранированы каким-либо конкретным образом. Internet Explorer сохранит угловые скобки как есть в свойстве `location.search`, которое соответствует строке запроса URL. Chrome, с другой стороны,

экранирует их, но пропустит двойные кавычки (") или обратные слешы. Большинство браузеров также не применяет никакого экранирования к ID фрагмента.

`history object` - эта иерархия предоставляет несколько редко используемых методов для перемещения по истории просмотра в пределах окна способом, похожим на нажатие кнопок "назад" и "вперед" в UI браузера. Невозможно напрямую изучить какие-либо из ранее посещенных URL; единственный вариант - вслепую перейти по истории, предоставив числовые смещения, такие как `history.go(-2)`. Некоторые недавние дополнения к этой иерархии будут обсуждаться в главе 17.

`screen object` - базовый API для изучения размеров экрана и окна браузера, DPI монитора, глубины цвета и так далее. Он предлагается, чтобы помочь сайтам оптимизировать представление страницы для конкретного устройства отображения.

`navigator object` - интерфейс для запроса версии браузера, базовой операционной системы и списка установленных подключаемых модулей.

`document object` - безусловно самая сложная из иерархий, это дверь к Document Object Model^[9] текущей страницы; мы взглянем на эту модель в следующем разделе. Несколько функций, не связанных со структурой документа, также появляются в иерархии `document`, обычно из-за произвольных проектных решений. Примеры включают `document.cookie` для манипулирования cookie, `document.write(...)` для добавления HTML к текущей странице и `document.execCommand(...)` для выполнения некоторых задач WYSIWYG-редактирования.

Примечание. Интересно, что информации, доступной через объекты `navigator` и `screen`, достаточно, чтобы с высокой степенью уверенности уникально отпечатать многих пользователей. Это давно известное свойство выразительно демонстрирует Panopticklick, проект Electronic Frontier Foundation: <https://panopticklick.eff.org/>.

Несколько других объектов, предписанных языком, предлагают простые возможности обработки строк или арифметики. Например, `Math.random()` реализует небезопасный, предсказуемый генератор псевдослучайных чисел (безопасная альтернатива PRNG, к сожалению, в большинстве браузеров на данный момент недоступна^[10]), а `String.fromCharCode()` можно использовать для преобразования числовых значений в строки Unicode. В привилегированных контекстах выполнения, недоступных обычным веб-приложениям, также появится немалое число других объектов, специфичных для задач.

Примечание. При доступе к любым объектам, предоставленным браузером, важно помнить, что хотя JavaScript не использует ASCII-строки с завершением NUL, лежащий в основе браузер, написанный на C или C++, иногда использует. Поэтому результаты присваивания строк, содержащих NUL, различным свойствам DOM или передачи их нативным функциям могут быть непредсказуемыми и непоследовательными. Почти все браузеры обрезают присваивания `location.*` на NUL, но только некоторые движки делают то же самое при работе с `DOM *.innerHTML`.

Document Object Model

Document Object Model, доступная через иерархию `document`, предоставляет структурированное представление текущего документа в памяти, как его построил HTML-парсер. Получающееся дерево объектов раскрывает все HTML-элементы на странице, их специфичные для тегов методы и свойства, а также связанные данные CSS. Именно это представление, а не исходный HTML-код, используется браузером для рендеринга и обновления отображаемого сейчас документа.

JavaScript может получать доступ к DOM очень прямолинейно, подобно любым обычным объектам. Например, следующий фрагмент перейдет к пятому тегу внутри блока `<body>` документа, найдет первый вложенный подтег и установит CSS-цвет этого элемента в красный:

```
document.body.children[4].children[0].style.color = "red";
```

Чтобы не продираяться через дерево DOM ради конкретного глубоко вложенного элемента, браузер предоставляет несколько функций поиска по всему документу, таких как `getElementById(...)` и `getElementsByTagName(...)`, а также частично избыточные механизмы группировки вроде `frames[]`, `images[]` или `forms[]`. Эти возможности допускают синтаксис вроде следующих двух строк кода, обе из которых прямо ссылаются на элемент независимо от того, где в иерархии документа он находится:

```
document.getElementsByTagName("input")[2].value = "Hi mom!";  
document.images[7].src = "/example.jpg";
```

По историческим причинам имена некоторых HTML-элементов (`<img>`, `<form>`, `<embed>`, `<object>` и `<applet>`) также напрямую сопоставляются с пространством имен `document`, как показано в следующем фрагменте:

```
  
<script>  
  alert(document.hello.src);  
</script>
```

В отличие от более разумного случая сопоставления `name` и `id` в глобальном пространстве имен (см. предыдущий раздел), такие записи `document` могут затирать встроенные функции и объекты, например `getElementById` или `body`. Поэтому разрешать заданные пользователем имена тегов, например с целью построения форм, может быть небезопасно.

Помимо предоставления доступа к абстрактному представлению документа, многие узлы DOM могут раскрывать свойства вроде `innerHTML` и `outerHTML`, которые позволяют прочесть часть дерева документа обратно как правильно сформированную, сериализованную HTML-строку. Интересно, что в то же свойство можно записать значение, чтобы заменить любую часть дерева DOM результатом разбора фрагмента HTML, предоставленного скриптом. Один пример такого последнего использования:

```
document.getElementById("output").innerHTML = "<b>Hi mom!</b>";
```

Каждое присваивание `innerHTML` должно включать правильно сформированный и самостоятельный блок HTML, который не меняет иерархию документа вне заменяемого фрагмента. Если это условие не выполнено, вход будет принудительно приведен к правильно сформированному синтаксису до замены. Поэтому следующий пример не сработает ожидаемым образом, то есть он не покажет **Hi mom!** жирным и не поместит остаток документа в курсив:

```
some_element.innerHTML = "<b>Hi";
some_element.innerHTML += " mom!</b><i>";
```

Вместо этого каждое из двух присваиваний будет обработано и исправлено по отдельности, что приведет к поведению, эквивалентному этому:

```
some_element.innerHTML = "<b>Hi</b> mom!<i></i>";
```

Важно отметить, что механизм `innerHTML` следует использовать с крайней осторожностью. Помимо того, что он по своей природе склонен к внедрению разметки, если не соблюдается правильное HTML-экранирование, браузерные реализации алгоритмов сериализации DOM-в-HTML часто несовершенны. Недавний, теперь исправленный, пример такой проблемы в WebKit^[11] показан здесь:

```
<textarea>
  &lt;/textarea&gt;&lt;script&gt;alert(1)&lt;/script&gt;
</textarea>
```

Из-за путаницы вокруг семантики `<textarea>` эта, казалось бы, однозначная входная разметка после разбора в дерево DOM и последующего доступа через `innerHTML` неправильно читалась обратно как:

```
<textarea>
  </textarea><script>alert(1)</script>
</textarea>
```

В такой ситуации даже выполнение ничего не меняющего присваивания этой сериализации (например, `some_element.innerHTML += ""`) приводило бы к неожиданному внедрению скрипта. Похожие проблемы имеют обыкновение мучить и другие браузеры. Например, разработчики Internet Explorer, работавшие над кодом `innerHTML`, не знали, что MSHTML распознает обратные апострофы (```) как символы кавычек, и в итоге обрабатывали их неправильно. В их реализации следующая разметка:

```

```

сериализовалась обратно так:

```
<img src=test.jpg alt=`onload=alert(1)>
```

Если оставить отдельные ошибки в стороне, ситуация с `innerHTML` довольно мрачная: раздел 10.3 текущего черновика HTML5 просто признает, что некоторые созданные скриптом структуры DOM совершенно невозможно сериализовать в HTML, и не требует от браузеров вести себя разумно в таком случае. *Caveat emptor!*

Доступ к другим документам

Скрипты могут получить объектные дескрипторы, указывающие на корневую иерархию другого контекста сценариев. Например, по умолчанию каждый контекст может легко ссылаться на `parent`, `top`, `opener` и `frames[]`, все они предоставлены ему в объекте верхнего уровня. Вызов функции `window.open(...)` для создания нового окна также вернет ссылку, как и попытка найти существующее именованное окно с помощью такого синтаксиса:

```
var window_handle = window.open("", "window_name");
```

Когда программа держит дескриптор, указывающий на другой контекст сценариев, она может попытаться взаимодействовать с этим контекстом с учетом проверок безопасности, обсуждаемых в главе 9. Пример простого взаимодействия может быть таким:

```
top.location.path = "/new_path.html";
```

или

```
frames[2].document.getElementById("output").innerHTML = "Hi mom!";
```

При отсутствии действительного дескриптора взаимодействие на уровне JavaScript с несвязанным документом не должно быть возможно. В частности, нет способа найти безымянные окна, открытые в полностью отдельных потоках навигации, по крайней мере до тех пор, пока их имя явно не установит одна из посещенных страниц. Свойство `window.name` это позволяет.

Кодирование символов в скриптах

Движки JavaScript поддерживают несколько знакомых методов строкового кодирования на основе обратного следа, которые можно использовать для экранирования символов кавычек, HTML-разметки и других проблемных фрагментов во встроенном тексте. Эти методы таковы:

- С-подобная краткая запись для некоторых управляющих символов: `\b` для `backspace`, `\t` для горизонтальной табуляции, `\v` для вертикальной табуляции, `\f` для `form feed`, `\r` для `CR` и `\n` для `LF`. Именно этот набор `escape`-кодов распознается и ECMAScript, и RFC JSON.
- Трехзначные, дополненные нулями, 8-битные восьмеричные коды символов без префикса (например, `\145` вместо `e`). Этот вдохновленный C синтаксис не является частью ECMAScript, но на практике поддерживается всеми движками сценариев как в обычном коде, так и в `JSON.parse(...)`.
- Двухзначные, дополненные нулями, 8-битные шестнадцатеричные коды символов с префиксом `x` (`e` становится `\x65`). Эта схема тоже не одобрена ECMAScript или RFC 4627, но, имея корни в языке C, широко поддерживается на практике.
- Четырехзначные, дополненные нулями, 16-битные шестнадцатеричные значения Unicode с префиксом `u` (`e` превращается в `\u0065`). Этот формат санкционирован ECMAScript и RFC 4627 и поддерживается всеми современными браузерами.
- Обратный слеш, за которым следует любой символ, отличный от восьмеричной цифры; символов `b`, `t`, `v`, `f`, `r` или `n`, используемых для других заранее определенных `escape`-последовательностей; и `x` или `u`. В этой схеме последующий символ будет обработан как литерал. ECMAScript разрешает использовать эту схему только для экранирования кавычек и самого символа обратного следа, но на практике принимается и любое другое значение.

Этот подход несколько подвержен ошибкам и, как в случае с CSS, не должен использоваться для экранирования угловых скобок и других синтаксических разделителей HTML. Причина в том, что разбор JavaScript происходит после разбора HTML, и префикс с обратным слешем не будет обработан самим HTML-парсером каким-либо особым образом.

Примечание. Несколько необъяснимо, но Internet Explorer не распознает краткую запись вертикальной табуляции (`\v`), создавая один из более удобных, но очень непослушных способов проверить именно этот браузер:

```
if ("\v" == "\v") alert("Looks like Internet Explorer!");
```

Удивительно, но метод экранирования на основе Unicode, в отличие от остальных, также распознается вне строк. Хотя идея кажется произвольной, поведение немного разумнее, чем в CSS: escape-коды можно использовать только в идентификаторах, и они не будут работать как замена любых синтаксически значимых символов. Поэтому возможно следующее:

```
\u0061alert("This displays a message!");
```

С другой стороны, любая попытка похожим образом заменить круглые скобки или кавычки завершится неудачей.

В отличие от некоторых реализаций C или C++, случайные многострочные строковые литералы не допускаются ни одним движком JavaScript. Тем не менее, несмотря на жесткий запрет в спецификациях ECMAScript, есть одно исключение: одинокий обратный слеш в конце строки можно использовать, чтобы бесшовно соединять многострочные литералы. Это поведение показано ниже:

```
var text = 'This syntax
            is invalid.';
var text = 'This syntax, on the other hand, \
            is OK in all browsers.';
```

Режимы включения кода и риски вложенности

Как должно быть видно из предыдущих обсуждений в этой главе, существует несколько способов выполнить скрипты в контексте текущей страницы. Вероятно, полезно перечислить некоторые из самых распространенных:

- Встроенные блоки `<script>`.
- Удаленные скрипты, загружаемые с помощью `<script src=...>`.^[12]
- URL `javascript:` в различных HTML-параметрах и в CSS.
- Синтаксис CSS `expression(...)` и привязки XBL в некоторых браузерах.
- Обработчики событий (`onload`, `onerror`, `onclick` и т. д.).
- Таймеры (`setTimeout`, `setInterval`).
- Вызовы `eval(...)`.

Сочетание этих методов часто кажется естественным, но оно может создавать очень неожиданные и опасные цепочки разбора. Например, рассмотрим преобразование, которое нужно было бы применить к значению, вставляемому сервером вместо `user_string` в этом коде:

```
<div onclick="setTimeout('do_stuff(\'user_string\')', 1000)">
```

Часто трудно заметить, что значение пройдет не менее трех раундов разбора. Сначала HTML-парсер извлечет параметр `onclick` и поместит его в DOM; затем, когда по кнопке щелкнут, первый раунд разбора JavaScript извлечет синтаксис `setTimeout(...)`; и наконец, через одну секунду после исходного щелчка фактическая последовательность `do_stuff(...)` будет разобрана и выполнена.

Поэтому в приведенном выше примере, чтобы пережить этот процесс, `user_string` нужно дважды закодировать с помощью backslash-последовательностей JavaScript, а затем снова закодировать с помощью HTML-сущностей, именно в таком порядке. Любой иной подход, скорее всего, приведет к внедрению кода.

Еще одна хитрая ситуация с экранированием показана здесь:

```
<script>
var some_value = "user_string";
...
setTimeout("do_stuff('' + some_value + '')", 1000);
</script>
```

Хотя исходное присваивание `some_value` требует экранировать `user_string` только один раз, последующее специальное построение скрипта второго порядка в параметре `setTimeout(...)` вводит уязвимость, если заранее не применить дополнительное экранирование.

Такие шаблоны кодирования часто встречаются в программах JavaScript, и их очень легко пропустить. Гораздо лучше последовательно отговаривать от их использования, чем проводить аудит получившегося кода.

Живой мертвец: Visual Basic

Разобрав большую часть необходимого, связанного с JavaScript, пора почтительно упомянуть давно забытого претендента на трон сценариев. Несмотря на 15 лет пребывания почти в полной неизвестности, браузерный VBScript все еще поддерживается в Internet Explorer. В большинстве аспектов язык Microsoft должен быть функционально эквивалентен JavaScript, и у него есть доступ ровно к тем же API Document Object Model и другим встроенным функциям, что и у JavaScript. Но, как можно ожидать, присутствуют некоторые корректировки и расширения - например, пара специфичных для VB функций вместо встроенных средств JavaScript.

Практически нет исследований свойств безопасности VBScript, надежности парсера или его потенциальных несовместимостей с современным DOM. Отдельные свидетельства позволяют предположить, что на стороне Microsoft язык тоже не получает последовательного внимания. Например, встроенную `MsgBox`^[13] можно использовать для отображения модальных, всегда поверх остальных окон подсказок с гибкостью, совершенно неслыханной в мире JavaScript, оставляя `alert(...)` далеко позади.

Трудно предсказать, как долго VBScript будет продолжать поддерживаться в этом браузере и какие неожиданные последствия для безопасности пользователей и веб-приложений ему еще предстоит иметь. Только время покажет.

Памятка по инженерии безопасности

При загрузке удаленных скриптов

Как и с CSS, вы связываете безопасность своего сайта с доменом происхождения скрипта. Если сомневаетесь, сделайте локальную копию данных. На HTTPS-сайтах требуйте, чтобы все скрипты также отдавались по HTTPS.

При разборе JSON, полученного от сервера

Полагайтесь на `JSON.parse(...)` там, где он поддерживается. Не используйте `eval(...)` или основанную на `eval` реализацию, предоставленную в RFC 4627. Обе небезопасны, особенно при обработке данных от третьих сторон. Более поздняя реализация автора RFC 4627, `json2.js`,^[14] вероятно, подходит.

При помещении предоставленных пользователем данных внутрь блоков JavaScript

- Самостоятельные строки в блоках `<script>`: экранируйте обратным слешем все управляющие символы (`0x00-0x1F`), `\`, `<`, `>` и кавычки с использованием числовых кодов. Также предпочтительно экранировать символы старшего диапазона.

Не полагайтесь на предоставленные пользователем строки для построения динамического HTML. Вместо этого всегда используйте безопасные DOM-возможности, такие как `innerText` или `createTextNode(...)`. Не используйте предоставленные пользователем строки для построения скриптов второго порядка; избегайте `eval(...)`, `setTimeout(...)` и так далее.

- Самостоятельные строки в отдельно отдаваемых скриптах: следуйте тем же правилам, что и для блоков `<script>`. Если ваши скрипты содержат чувствительную, специфичную для пользователя информацию, обязательно учитывайте риски межсайтового включения скриптов; используйте надежные префиксы, ломающие парсер, такие как `}}]'\n`, около начала файла или, как абсолютный минимум, применяйте правильную сериализацию JSON без padding или других корректировок. Дополнительно обратитесь к главе 13 за советами о предотвращении межсайтового скриптинга в не-HTML-содержимом.
- Строки во встроенных обработчиках событий, URL `javascript:` и так далее: здесь задействованы несколько уровней экранирования. Не пытайтесь делать это, потому что процесс подвержен ошибкам. Если это неизбежно, сначала примените приведенные выше правила JS-экранирования, а затем примените кодирование HTML- или URL-параметра, что применимо, к получившейся строке. Никогда не используйте это вместе с `eval(...)`, `setTimeout(...)`, `innerHTML` и подобными.
- Нестроковое содержимое: разрешайте только буквенно-цифровые ключевые слова из белого списка и тщательно проверенные числовые значения. Не пытайтесь вместо этого отвергать известные плохие шаблоны.

При взаимодействии с браузерными объектами на клиентской стороне

- Генерация HTML-содержимого на клиентской стороне: не прибегайте к `innerHTML`, `document.write(...)` и похожим инструментам, потому что они склонны вносить дефекты межсайтового скриптинга, часто неожиданными способами. Вместо этого используйте безопасные методы, такие как `createElement(...)` и `appendChild(...)`, и свойства вроде `innerText` или `textContent`, чтобы строить документ.
- Опора на контролируемые пользователем данные: не делайте предположений о правилах экранирования, применяемых к любым значениям, прочитанным обратно из браузера, и, в частности, к свойствам `location` и другим внешним источникам URL, которые непоследовательны и различаются от одной реализации к другой. Всегда выполняйте собственное экранирование.

Если вы хотите разрешить контролируемые пользователем скрипты на своей странице

Сделать это безопасно практически невозможно. Экспериментальные фреймворки переписывания JavaScript, такие как Caja (<http://code.google.com/p/google-caja/>), являются единственным переносимым вариантом. Также см. главу 16 для информации о sandboxed frames, готовящейся альтернативе для встраивания недоверенных гаджетов на веб-страницы.

Сноски

- [1] [\[назад\]](#) Сноска в оригинале указывает на пресс-релиз Netscape и Sun 1995 года о JavaScript.
- [2] [\[назад\]](#) Сноска в оригинале указывает на спецификацию ECMAScript 3rd edition.
- [3] [\[назад\]](#) Сноска в оригинале указывает на спецификацию ECMAScript 5th edition.
- [4] [\[назад\]](#) Сноска в оригинале указывает на RFC 4627.
- [5] [\[назад\]](#) В отличие от большинства других схем включения содержимого, доступных скриптам, таких как XMLHttpRequest, `<script src=...>` не подчиняется междоменным ограничениям безопасности, описанным в главе 9. Поэтому этот механизм представляет риск безопасности всякий раз, когда сервер использует окружающие полномочия, например cookie, для динамического создания пользовательского JavaScript-кода. Этот класс уязвимостей без особой фантазии называют cross-site script inclusion, или XSSi.
- [6] [\[назад\]](#) Несколько неожиданно JavaScript поддерживает C-подобные помеченные инструкции, такие как `my_label: alert("Hi mom!")`. Это интересно, потому что по философским причинам язык не поддерживает `goto`, а значит, такая метка в большинстве случаев не может быть осмысленно использована.
- [7] [\[назад\]](#) JSONP буквально означает "JSON with padding" и обозначает сериализацию JSON, обернутую в дополнительный код, который для удобства превращает ее в действительную самостоятельную инструкцию JavaScript. Распространенные примеры могут включать вызов функции (например, `callback_function({...JSON data...})`) или присваивание переменной (`var return_value = {...JSON data...}`).
- [8] [\[назад\]](#) Сноска в оригинале указывает на спецификацию ECMAScript for XML (E4X).
- [9] [\[назад\]](#) Сноска в оригинале указывает на спецификацию Document Object Model.
- [10] [\[назад\]](#) В Chrome есть недавно добавленный API `window.crypto.getRandomValues(...)`, а в Firefox - на тот момент неработающий `window.crypto.random(...)`.
- [11] [\[назад\]](#) Сноска в оригинале указывает на исправленную ошибку WebKit, связанную с сериализацией `innerHTML`.
- [12] [\[назад\]](#) На обоих типах блоков `<script>` Microsoft поддерживает псевдодиалект `JScript.Encode`. Этот режим можно выбрать, указав параметр `language` в теге `<script>`, и он просто позволяет закодировать фактический скрипт с помощью тривиального шифра подстановки алфавита, чтобы сделать его нечитаемым для случайных пользователей. С точки зрения безопасности механизм совершенно бесполезен, поскольку "шифрование" легко обратить.
- [13] [\[назад\]](#) Сноска в оригинале указывает на документацию функции `MsgBox`.
- [14] [\[назад\]](#) Сноска в оригинале указывает на реализацию `json2.js` Дугласа Крокфорда.

Глава 7. Не-HTML-типы документов

Помимо HTML-документов, движки рендеринга современных веб-браузеров распознают и отображают около дюжины других файловых форматов; этот список, вероятно, со временем будет расти.

Из-за мощных сценарных возможностей, доступных в некоторых из этих форматов, и из-за выходов браузерной обработки содержимого набор нативно поддерживаемых не-HTML-входов заслуживает на этом этапе более пристального рассмотрения, даже если подробное обсуждение некоторых их менее очевидных последствий для безопасности - таких как content sniffing - придется отложить до части II этой книги.

Plaintext-файлы

Пожалуй, самый прозаический тип не-HTML-документа, распознаваемый каждым браузером без исключения, - это plaintext-файл. В этом режиме рендеринга вход просто отображается как есть, обычно с использованием моношириного шрифта, и, за исключением необязательного перекодирования набора символов, данные никак не изменяются.

Все браузеры распознают plaintext-файлы, отданные с Content-Type: text/plain в HTTP-заголовках. Во всех реализациях, кроме Internet Explorer, plaintext также является запасным методом отображения для ответов HTTP/0.9 без заголовков и данных HTTP/1.x с отсутствующим Content-Type; в обоих случаях plaintext используется, когда все остальные эвристики определения содержимого терпят неудачу. Internet Explorer безусловно откатывается к HTML-рендерингу, верный букве исходных черновиков протокола Тима Бернерса-Ли.

Для удобства разработчиков большинство браузеров также автоматически сопоставляют с plaintext несколько других MIME-типов, включая application/javascript и родственников^[1] или text/css. Интересно, что application/json, значение, предписанное для JSON-ответов в RFC 4627, в этом списке отсутствует, возможно, потому что на практике используется редко.

Plaintext-рендеринг не имеет специфических последствий для безопасности. Тем не менее из-за ряда плохих проектных решений в других компонентах браузеров и стороннем коде даже, казалось бы, безвредные не-HTML-форматы рискуют быть ошибочно распознаны, например, как HTML. Plaintext-документы, контролируемые злоумышленником, вызывают особую тревогу, потому что их компоновка часто довольно свободна и потому особенно благоприятна для ошибочного распознавания. Глава 13 разбирает эти угрозы и дает советы о том, как снизить риск.

Растровые изображения

Браузерные движки рендеринга распознают прямую навигацию к тому же набору форматов растровых изображений, которые обычно поддерживаются в HTML-документах при загрузке через тег , включая JPEG, PNG, GIF, BMP и еще пару других. Когда пользователь напрямую переходит к такому ресурсу, декодированное растровое изображение показывается в окне документа, оставляя пользователю немногим больше возможности, чем прокручивать, увеличивать и уменьшать масштаб и сохранять файл на диск.

При отсутствии информации Content-Type изображения определяются по проверкам заголовка файла. Когда значение Content-Type присутствует, оно сравнивается примерно с дюжиной

заранее определенных типов изображений, и пользователь направляется соответственно. Но если попытка декодировать изображение завершается неудачей, заголовки файла используются для второй догадки. Поэтому возможно, хотя по причинам, рассмотренным в главе 13, часто неразумно, отдавать GIF-файл как `image/jpeg`.

Как и текстовые файлы, растровые изображения являются пассивным ресурсом и не несут необычных рисков безопасности.^[2] Однако всякий раз, когда вы отдаете изображения, предоставленные пользователями, помните, что злоумышленники будут иметь определенную степень контроля над данными, даже если формат тщательно проверяется и масштабируется или перекомпрессируется. Поэтому опасения, что такой формат документа будет неверно интерпретирован браузером или подключаемым модулем, все еще остаются.

Аудио и видео

Очень долго браузеры не имели встроенной поддержки воспроизведения аудио- и видеосодержимого, если не считать малоизвестного и часто высмеиваемого тега `<bgsound>` в Internet Explorer, который до сих пор можно использовать для воспроизведения простых MID- или WAV-файлов. В отсутствие настоящей межбраузерной функциональности воспроизведения мультимедиа аудио и видео почти исключительно были владениями браузерных подключаемых модулей, как специально предназначенных для этого, таких как Windows Media Player или Apple QuickTime, так и универсальных, таких как Adobe Flash, Microsoft Silverlight и так далее.

Продолжающаяся работа над HTML5 стремится изменить это за счет поддержки тегов `<audio>` и `<video>`: удобных, скриптуемых методов взаимодействия со встроенными медиатекодерами. К сожалению, между поставщиками есть существенные разногласия относительно того, какие видеоформаты поддерживать и какие патентные последствия может иметь это решение. Например, хотя многие браузеры уже поддерживают Ogg Theora, свободный, open source, но несколько нишевый кодек, жаркие споры вокруг достоинств поддержки очень популярного, но обремененного патентами и лицензионными отчислениями формата H.264 и перспектив новой альтернативы WebM при поддержке Google, вероятно, будут продолжаться в обозримом будущем.

Как и в случае с другими пассивными медиаформатами, и в отличие от некоторых типов содержимого, рендеримого подключаемыми модулями, ни `<bgsound>`, ни мультимедиа HTML5, как ожидается, не имеют каких-либо необычных последствий для безопасности веб-приложений, пока возможность ошибочного распознавания содержимого надлежащим образом смягчена.^[3]

Документы на основе XML

Читатели, которым обработка обсужденных до сих пор форматов показалась слишком здоровой на их вкус, заслуженно получают угощение. Самое большое и определенно самое интересное семейство поддерживаемых браузером не-HTML-типов документов опирается на общий синтаксис XML и предоставляет более чем достаточную долю интересных сюрпризов.

Несколько форматов, принадлежащих к этой категории, направляются специализированным XML-анализаторам одного назначения, обычно на основе полученного значения Content-Type или других простых эвристик. Но чаще полезная нагрузка направляется в тот же парсер, который используется для рендеринга XHTML-документов, а затем отображается через этот общий конвейер.

В последнем случае фактический смысл документа определяется URL-подобными директивами пространства имен `xmlns`, присутствующими в самой разметке, и параметр пространства имен может не иметь никакого отношения к значению, изначально предоставленному в `Content-Type`. Проще говоря, нет механизма, который помешал бы документу, отданному как `application/mathml+xml`, содержать одну только XHTML-разметку и начинаться с `<html xmlns="http://www.w3.org/1999/xhtml">`.

В наиболее распространенном сценарии пространство имен для всего XML-файла определяется только один раз и прикрепляется к тегу верхнего уровня. Однако в принципе любое число различных директив `xmlns` может появляться в одном файле, придавая разные смыслы каждому разделу документа. Например:

```
<html xmlns="http://www.w3.org/1999/xhtml">
  <u>Hello world!</u>
  <svg xmlns="http://www.w3.org/2000/svg">
    <line x1="0" y1="0" x2="100" y2="100" style="stroke: red" />
  </svg>
</html>
```

Столкнувшись с таким входом, универсальный рендерер обычно сделает все возможное, чтобы осмыслить все распознанные пространства имен и собрать разметку в единый, согласованный документ с нормальным представлением Document Object Model. И если какое-либо из распознанных пространств имен поддерживает сценарии, встроенные скрипты тоже выполняются.

Из-за несколько противоинтуитивного поведения обработки `xmlns` значение `Content-Type` не является подходящим способом контролировать, как будет разобран конкретный XML-документ; присутствие конкретной директивы `xmlns` верхнего уровня также не гарантирует, что позднее не будут учтены другие форматы данных. Поэтому любые контролируемые злоумышленником форматы на основе XML нужно обрабатывать осторожно и очищать очень тщательно.

Обобщенное представление XML

В большинстве браузеров действительный XML-документ, в разметке которого нигде нет пространств имен, распознанных рендерером, будет показан как интерактивное, красиво отформатированное представление дерева документа, как показано на рис. 7-1. Этот режим не особенно полезен конечным пользователям, но может помочь отладке. Тем не менее, когда любое из пространств имен в документе известно браузеру, даже если пространство имен верхнего уровня вообще не распознано, документ будет отрендерен иначе: вся распознанная разметка сработает как задумано, все неподдерживаемые теги просто не будут иметь эффекта, а любой текст между ними будет показан как есть.

Чтобы проиллюстрировать эту стратегию рендеринга, рассмотрим следующий вход:

```
<foo xmlns="http://www.example.com/nonexistent">
  <u>Hello</u>
  <html xmlns="http://www.w3.org/1999/xhtml">
    <u>world!</u>
  </html>
</foo>
```

Приведенный выше пример будет отрендерен как "Hello world!". Первый тег `<u>`, с которым не связано пространство имен, задающее семантику, не будет иметь видимого эффекта. Второй будет понят как XHTML-тег, включающий подчеркивание.

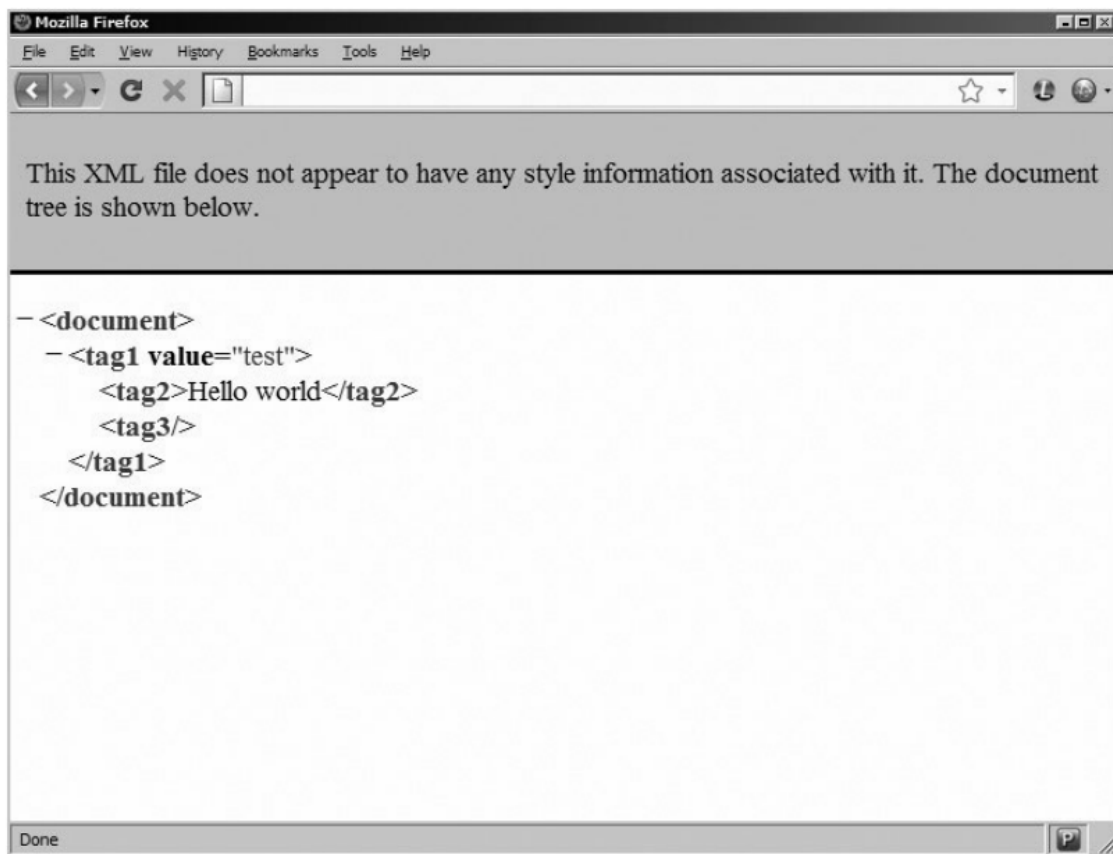


Рис. 7-1. Firefox отображает XML-документ без распознанных пространств имен

Последствия этого отказоустойчивого подхода к рендерингу неизвестных XML-документов и нераспознанных пространств имен тонки, но довольно важны. Например, небезопасно проксировать неочищенную RSS-ленту, хотя этот формат обычно направляется специализированному рендереру и потому не подвержен рискам XSS. Любой браузер без встроенной RSS-читалки может откатиться к обобщенному рендерингу и затем найти HTML, зарытый глубоко внутри ленты.

Scalable Vector Graphics

Scalable Vector Graphics (SVG)^[4] - быстро развивающийся формат векторной графики на основе XML. Впервые опубликованный W3C в 2001 году, он примечателен встроенными возможностями анимации и прямыми возможностями сценариев JavaScript. Следующий пример векторного изображения рисует круг и показывает сообщение при щелчке по этому кругу:

```
<svg xmlns="http://www.w3.org/2000/svg">
  <script><![CDATA[
    function clicked() { alert("Hi mom!"); }
  ]]></script>
  <circle onclick="clicked()" cx="50" cy="50"
    r="50" fill="pink" />
</svg>
```

Формат файлов SVG распознается всеми современными браузерами, кроме Internet Explorer до версии 9, и обрабатывается универсальным XML-рендерером. Изображения SVG можно встраивать в XHTML с соответствующей директивой `xmlns` или включать прямо в не-XML-документы HTML5 с помощью заранее определенного тега `<svg>`.

Интересно, что в нескольких браузерах формат также можно поместить в самостоятельный XML-документ и затем просмотреть напрямую, или его можно загрузить на сторонних страницах через разметку `<img>`. Хотя загружать SVG-изображения через `<img>` безопасно, сценарии в этом сценарии должны быть отключены, размещать SVG-данные, предоставленные пользователями, довольно опасно, потому что в случаях прямой навигации все встроенные скрипты выполняются в контексте размещающего домена. Эта неожиданная проблема означает, что отдача любых SVG-изображений внешнего происхождения потребует очень тщательной синтаксической очистки, чтобы устранить не-SVG-содержимое `xmlns` из XML-контейнера и разрешить только определенные типы разметки в остатке документа.

Примечание. Заголовок `Content-Disposition` в соответствующих HTTP-ответах - потенциальный обходной путь, который разрешает включать SVG через `<img>`, но не обращаться к нему напрямую. Этот подход не совершенен, но ограничивает риск. Использование одноразового домена для размещения таких изображений - еще одна возможность.

Mathematical Markup Language

Mathematical Markup Language (MathML)^[5] - довольно прямолинейное средство для семантического, пусть и немного многословного, представления математических уравнений. Стандарт был изначально предложен W3C в 1998 году и за прошедшие годы был существенно уточнен. Из-за своей несколько нишевой области применения MathML потребовалось более десятилетия, чтобы получить частичную поддержку в браузерах Opera и Firefox, но сегодня он постепенно получает признание. В браузерах, поддерживающих язык, его можно поместить в самостоятельный файл или встроить в XHTML- и HTML5-документы.

В отличие от SVG, MathML не имеет дополнительных соображений безопасности сверх тех, что связаны с обобщенно обрабатываемым XML.

XML User Interface Language

XML User Interface Language (XUL)^[6] - язык презентационной разметки, созданный Mozilla специально для построения браузерных приложений, а не документов. XUL существует потому, что хотя современный HTML часто достаточно мощен для построения базовых графических интерфейсов, он не особенно удобен для некоторых специализированных задач, в которых сильны настольные приложения, таких как реализация обычных диалоговых окон или системных меню.

XUL в настоящее время не поддерживается никаким браузером, кроме Firefox, и, похоже, отключен в недавнем выпуске Firefox 6. В Firefox он обрабатывается универсальным рендерером на основе соответствующего пространства имен `xmlns`. Firefox использует XUL для большей части своего внутреннего UI, но в остальном язык редко встречается в интернете.

С точки зрения безопасности веб-приложений XUL-документы интернет-происхождения можно считать примерно эквивалентными HTML-документам. По сути, у языка есть возможности сценариев JavaScript, и он позволяет широко контролировать внешний вид отрендеренной страницы. Помимо этого свойства у него нет необычных особенностей.

Wireless Markup Language

Wireless Markup Language (WML)^[7] - в основном устаревший "оптимизированный" синтаксис HTML, разработанный в 1990-х консорциумом производителей мобильных телефонов и операторов сотовых сетей. Этот язык на основе XML, часть набора Wireless Application Protocol (WAP), предлагал упрощенный web-подобный опыт просмотра для до-смартфонных устройств с ограниченной пропускной способностью и ресурсами CPU.^[8] Простая WML-страница могла выглядеть так:

```
<wml>
  <card title="Hello world!">
    <a href="elsewhere.wml">Click here!</a>
  </card>
</wml>
```

Поскольку WAP-сервисы нужно было проектировать независимо от обычного HTML-содержимого и работать с закрытыми, недостаточно специфицированными клиентскими архитектурами и другими ограничениями, навязанными операторами, WML так и не стал настолько популярным, как надеялись его сторонники. Почти на всех развитых рынках WML был вытеснен быстрыми смартфонами с доступом в интернет и полнофункциональными HTML-браузерами. Тем не менее наследие языка живет, и он все еще направляется специализированным рендерерам в Opera и Internet Explorer Mobile.

В браузерах, поддерживающих формат, часто можно использовать скрипты на основе WML. Есть два способа добиться этого. Канонический способ - использовать WMLScript (WMLS), среду выполнения, производную от JavaScript, которая зависит от самостоятельных файлов скриптов, в сочетании с крайне невежливым злоупотреблением ID фрагментов для эквивалента, возможно, контролируемых злоумышленником инструкций `eval(...)`:

```
<a href="scriptfile.wmls#some_function()">Click here!</a>
```

Другой способ выполнения скриптов, доступный в более функциональных браузерах, - просто встроить обычные URL `javascript:` или вставить блоки `<script>` в WML-файл.

RSS- и Atom-ленты

Ленты - стандартизированный способ, позволяющий клиентам периодически опрашивать сайты, интересные пользователям, например их любимые блоги, на предмет машиночитаемых обновлений содержимого этих сайтов. Really Simple Syndication (RSS)^[9] и Atom^[10] - два поверхностно похожих, но ожесточенно конкурирующих формата лент на основе XML. Первый, RSS, популярен; второй, Atom, считается хорошим.

Встроенные специализированные рендереры RSS и Atom доступны в Firefox, Safari и Opera. Решение направить XML-документ этим модулям основано на простых, специфичных для браузера эвристиках, например на том, что тег верхнего уровня называется `<rss>` или `<feed>` и не имеет конфликтующих директив `xmlns`. В Firefox разбор RSS может включиться даже если Content-Type равен `image/svg+xml` или `text/html`. Safari с радостью распознает ленты в еще более несвязанных MIME-типах.

Одна интересная особенность обоих форматов лент состоит в том, что они разрешают встраивать в документ подмножество HTML, включая CSS, довольно своеобразным, косвенным способом: как текст, экранированный сущностями. Вот пример такого синтаксиса:

```
<rss>
...
<description type="html">
  &lt;u>& Underlined text! &/u&
</description>
...
</rss>
```

Подмножество HTML, разрешенное в RSS- и Atom-лентах, не определено хорошо, и некоторые рендереры лент ранее разрешали прямые сценарии или навигацию к потенциально опасным псевдо-URL. Однако, возможно, важнее то, что любой браузер без встроенного предварительного просмотра лент может отрендерить файл с помощью обобщенного подхода XML-разбора; если такие ленты не очищены тщательно, последует выполнение скрипта.

Примечание о нерендеримых типах файлов

Ради полноты следует отметить, что все современные браузеры поддерживают ряд специализированных файловых форматов, которые остаются полностью непрозрачными для рендерера или слоя веб-приложения, но тем не менее распознаются различными внутрибраузерными подсистемами.

Подробное исследование этих форматов выходит за рамки этой книги, но среди заметных примеров - манифесты установки подключаемых модулей и расширений, файлы автоматической автоконфигурации HTTP-прокси (PAC), устанавливаемые визуальные темы, Certificate Revocation Lists (CRLs), антивредоносные черные списки сайтов и загружаемые шрифты TrueType и OpenType.

Свойства безопасности этих механизмов следует изучать индивидуально, прежде чем решать, разрешать ли отдавать пользователю какие-либо из этих форматов. За исключением общих соображений размещения содержимого, изложенных в главе 13, они вряд ли навредят размещающему веб-приложению напрямую, но могут создать проблемы для пользователей.

Памятка по инженерии безопасности

При размещении форматов документов на основе XML

Считайте, что полезная нагрузка может быть интерпретирована как XHTML или другой тип документа со скриптами, независимо от Content-Type и директивы xmlns верхнего уровня. Не разрешайте ничем не ограниченную разметку, контролируруемую злоумышленником, где-либо внутри файла. Используйте Content-Disposition: attachment, если данные не предназначены для прямого просмотра; и ленты все равно будут работать.

Для всех не-HTML-типов документов

Используйте корректные, распознаваемые браузером значения Content-Type и charset. Указывайте Content-Disposition: attachment там, где возможно. Проверяйте и ограничивайте синтаксис вывода. Обратитесь к памятке в главе 13, чтобы избежать проблем безопасности, связанных с дефектами content sniffing.

Сноски

- [1] [\[назад\]](#) Официальный MIME-тип для JavaScript - application/javascript, согласно RFC 4329, но в прошлом использовалось около дюжины других значений, например text/javascript, application/x-javascript, application/ecmascript.
- [2] [\[назад\]](#) Естественно, эксплуатируемые ошибки кодирования иногда случаются во всех программах, которые работают со сложными форматами данных, и парсеры изображений не исключение.
- [3] [\[назад\]](#) Но некоторые притянутые за уши взаимодействия между разными технологиями вполне возможны. Например, что если тег <audio> поддерживает сырое несжатое аудио и указывает на чувствительный неаудиодокумент, а затем предложенный HTML5 microphone API используется другим сайтом, чтобы захватить получившуюся волну и восстановить содержимое файла?
- [4] [\[назад\]](#) Сноска в оригинале указывает на спецификацию SVG.
- [5] [\[назад\]](#) Сноска в оригинале указывает на спецификацию MathML.
- [6] [\[назад\]](#) Сноска в оригинале указывает на документацию Mozilla по XUL.
- [7] [\[назад\]](#) Сноска в оригинале указывает на спецификацию WML.
- [8] [\[назад\]](#) Внимательные читатели заметят, что XML - не особенно хороший способ экономить пропускную способность или ресурсы CPU. Для этого набор WAP предоставляет альтернативную, только двоичную сериализацию XML, известную как WBXML.
- [9] [\[назад\]](#) Сноска в оригинале указывает на спецификацию RSS.
- [10] [\[назад\]](#) Сноска в оригинале указывает на спецификацию Atom.

Глава 8. Рендеринг содержимого с помощью браузерных плагинов

Браузерные плагины бывают самых разных форм и видов, но самая распространенная разновидность дает возможность отображать в браузере новые форматы файлов так, как если бы они были HTML. Браузер просто передает полученный файл, предоставляет вспомогательному приложению прямоугольную область рисования в окне документа и по сути отходит в сторону. Такие плагины рендеринга содержимого четко отличаются от браузерных расширений, куда более многочисленной группы, которая обычно полагается на JavaScript-код, чтобы менять способ представления пользователю уже поддерживаемого, внутрибраузерного содержимого.

У браузерных плагинов долгая и пестрая история дефектов безопасности. Фактически, по мнению некоторых аналитиков, 12 из 15 наиболее часто эксплуатировавшихся клиентских уязвимостей в 2010 году можно было отнести на счет качества программного обеспечения плагинов.^[1] Многие из этих проблем связаны с тем, что базовые парсеры изначально не были предназначены для аккуратной обработки вредоносных входов и не получили того пристального внимания, которому подверглась остальная часть Web. Другие проблемы проистекают из необычных моделей безопасности, придуманных разработчиками плагинов, и из столкновения этих разрешений с традиционным устройством веб-браузеров и здравыми ожиданиями разработчиков приложений.

В следующей главе этой книги мы рассмотрим некоторые механизмы безопасности, используемые популярными плагинами. Перед этим погружением имеет смысл посмотреть, как плагины интегрируются с другим онлайн-содержимым и какую общую функциональность они предлагают.

Вызов плагина

Плагины рендеринга содержимого можно активировать несколькими способами. Самый популярный явный метод - использовать разметку `<embed src=...>` или `<object data=...>` в "хостовом" HTML-документе, где параметр `src` или `data` указывает на URL, с которого должен быть получен фактический документ, распознаваемый плагином. Размеры и положение области рисования, выделенной плагину, можно контролировать с помощью CSS или устаревших HTML-параметров.

В этом сценарии каждый тег `<embed>` или `<object>` должен сопровождаться дополнительным параметром `type`. Указанный там MIME-тип будет сравнен со списком MIME-типов, зарегистрированных всеми активными плагинами, и полученный файл будет направлен подходящему обработчику. Если совпадение не найдено, теоретически должно быть показано предупреждение с предложением пользователю скачать плагин, хотя большинство браузеров смотрят на другие сигналы, прежде чем прибегать к такой немыслимой возможности; два распространенных варианта - изучить Content-Type или кажущееся расширение файла, замеченное в URL.

Примечание. Устаревший тег `<applet>`, используемый для загрузки Java-программ и примерно эквивалентный `<object type="application/x-java-applet">`, работает похожим образом, но безусловно игнорирует эти вспомогательные сигналы.

Дополнительный ввод для плагина обычно передается с помощью тегов `<param>`, вложенных внутрь блока `<object>`, или через нестандартные дополнительные параметры, прикрепленные к самой разметке `<embed>`. Первый, более современный подход может выглядеть так:

```
<object data="app.swf" type="application/x-shockwave-flash">
  <param name="some_param1" value="some_value1">
  <param name="some_param2" value="some_value2">
  ...
</object>
```

В этом режиме включения содержимого заголовков Content-Type, возвращенный сервером при получении подресурса, обычно игнорируется, если только параметр `type` не неизвестен браузеру. Это неудачная конструкция по причинам, которые вскоре будут объяснены.

Другой метод отображения содержимого плагина включает прямую навигацию к подходящему файлу. В этом случае, а также в случае `<embed>` или `<object>` с отсутствующим параметром `type`, значение Content-Type, полученное от сервера, учитывается и сравнивается со списком MIME-типов, распознаваемых плагинами. Если совпадение найдено, содержимое направляется соответствующему компоненту. Если поиск по Content-Type не удастся или заголовок отсутствует, некоторые браузеры исследуют тело ответа на известные сигнатуры содержимого; другие просто сдаются.

Примечание. Помимо вышеупомянутых методов, сфокусированных на содержимом, несколько типов плагинов можно загружать напрямую из программ JavaScript или VBScript без необходимости явно создавать какую-либо HTML-разметку или получать внешние данные. Так обстоит дело с ActiveX, печально известным мостом интеграции скрипта с системой, доступным в Internet Explorer. Мы уделим ActiveX некоторое время позже в этой главе, но обо всем по порядку.

Опасности обработки Content-Type в плагинах

Как отмечалось в предыдущем разделе, в некоторых сценариях параметр Content-Type у полученного файла, обрабатываемого плагином, игнорируется, а вместо него используется параметр `type` в соответствующей разметке на встраивающей странице. Хотя это решение отчасти похоже на поведение других тегов включения содержимого по типам, скажем `<img>`, как обсуждалось в разделе "Включение содержимого по типам" на странице 82 оригинала, в мире плагинов оно имеет уникальные и в итоге катастрофические последствия.

Большая проблема в том, что несколько типов плагинов по сути являются полноценными средами выполнения кода и дают выполняемым приложениям, апплетам, ряд специальных привилегий для взаимодействия с доменом происхождения. Например, Flash-файл, полученный с `fuzzybunnies.com`, получил бы доступ к своему домену происхождения, вместе с cookie пользователя, будучи встроенным на явно вредоносном `bunnyoutlet.com`.

В таком сценарии казалось бы важным, чтобы `fuzzybunnies.com` мог ясно сообщить, что конкретный тип документа действительно предназначен для интерпретации плагином, и, следовательно, что некоторые документы не предназначены для такого использования. К сожалению, способа сделать это нет: обработка полученного файла полностью контролируется встраивающим сайтом, в нашем примере злобными задирами, владеющими `bunnyoutlet.com`. Поэтому если домен происхождения размещает какой-либо тип контролируемого пользователями содержимого, даже в номинально безвредном формате, таком как `text/plain` или `image/jpeg`, владельцы `bunnyoutlet.com` могут приказать браузеру не обращать внимания на существующие метаданные и направить этот документ выбранному ими плагину. Простая разметка для достижения этой зловещей цели может быть такой:

```
<object data="http://fuzzybunnies.com/avatars/user11630.jpg"
type="application/x-shockwave-flash">
```

Если такой поворот событий кажется неправильным, то потому, что он неправильный. Исследователи безопасности неоднократно демонстрировали, что довольно легко сконструировать документы, которые, например, одновременно являются действительным изображением и действующим исполняемым файлом, распознаваемым плагином. Хорошо известная уязвимость GIFAR, обнаруженная в 2008 году Билли Риосом,^[2] использовала именно этот трюк: она прятала Java-апплет внутри совершенно коштерного GIF-изображения. В ответ Sun Microsystems, как сообщалось, ужесточила парсер Java JAR-файлов, чтобы снизить риск, но общая угроза таких ошибок все еще очень реальна и, вероятно, снова поднимет свою уродливую голову.

Интересно, что решение некоторых разработчиков полагаться на Content-Type и другие сигналы, если параметр type не распознан, почти так же плохо. Это решение делает невозможным для благонамеренного `fuzzybunnies.com` безопасно встроить безвредное видео от негодяев с `bunnyoutlet.com`, просто указав `type="video/x-ms-wmv"`, потому что если у кого-либо из посетителей нет плагина для этого конкретного медиатипа, `bunnyoutlet.com` внезапно получает право голоса в том, какой тип плагина должен быть загружен на встраивающем сайте вместо него. Некоторые браузеры, такие как Internet Explorer, Chrome или Opera, также могут обратиться к поиску кажущихся расширений файлов, присутствующих в URL, что может привести к интересной ситуации, где ни встраивающая, ни размещающая сторона не имеют настоящего контроля над тем, как отображается документ, и довольно часто командует только злоумышленник.

Гораздо более безопасная конструкция требовала бы, чтобы контролируемый встраивающей стороной параметр type и контролируемый хостом заголовок Content-Type совпадали, по крайней мере внешне. К сожалению, сейчас нет способа добиться этого. Несколько отдельных плагинов стараются вести себя прилично, например после переработки 2008 года Adobe Flash отвергает апплеты, отданные с Content-Disposition: attachment, как и встроенный PDF-ридер в Chrome, но такие улучшения редки и разрозненны.

Помощники рендеринга документов

Значительная часть ландшафта плагинов принадлежит программам, позволяющим показывать прямо в браузере некоторые очень традиционные, "не web" форматы документов. Некоторые из этих программ действительно полезны: Windows Media Player, RealNetworks RealPlayer и Apple QuickTime были основой онлайн-воспроизведения мультимедиа около десятилетия, по крайней мере до их вытеснения Adobe Flash. Достоинства других, однако, более сомнительны. Например, Adobe Reader и Microsoft Office оба устанавливают внутрибраузерные просмотрщики документов, заметно увеличивая поверхность атаки пользователя, хотя неясно, дают ли эти просмотрщики реальную выгоду по сравнению с открытием того же документа в отдельном приложении одним дополнительным щелчком.

Конечно, в идеальном мире размещение или встраивание PDF- или Word-документа не должно иметь прямых последствий для безопасности участвующих сайтов. Но, предсказуемо, реальность возражает. В 2009 году исследователь отметил, что PDF-формы, отправляющие данные на URL `javascript:`, по-видимому, могут приводить к выполнению клиентского кода на встраивающем сайте.^[3] Возможно, еще тревожнее самого этого отчета то, что, по словам исследователя, Adobe сначала отклонила отчет с таким замечанием: "Наша позиция состоит в том, что, подобно HTML-странице, PDF-файл является активным содержимым".

Прискорбно, что размещающая сторона не имеет полного контроля над тем, когда это активное содержимое обнаруживается и выполняется, и что в остальном разумные вебмастера могут думать о PDF- или Word-документах просто как о модном способе представить текст. В действительности, несмотря на свой безвредный вид, многие такие форматы документов, стремясь выглядеть круто, оснащены собственными возможностями гиперссылок или даже языками сценариев. Например, JavaScript-код можно встроить в PDF-документы, а макросы Visual Basic возможны в файлах Microsoft Office. Когда документ со скриптами отображается на HTML-странице, некоторая форма программного моста плагин-браузер обычно допускает степень взаимодействия с встраивающим сайтом, и конструкция таких мостов может варьироваться от смутно сомнительной до откровенно абсурдной.

В одном случае 2007 года Петко Д. Петков заметил, что сайт, размещающий любые PDF-документы, можно атаковать, просто предоставив полностью произвольный JavaScript-код в идентификаторе фрагмента. Эта строка будет выполнена на размещающей странице через мост плагина:^[4]

```
http://example.com/random_document.pdf#foo=javascript:alert(1)
```

Две уязвимости, описанные здесь, теперь исправлены, но урок состоит в том, что при размещении или встраивании любых предоставленных пользователями документов в чувствительных доменах следует проявлять особую осторожность. Последствия таких действий плохо документированы, и их трудно предсказать.

Фреймворки приложений на основе плагинов

Скучная работа рендеринга документов - устоявшаяся роль для браузерных плагинов, но несколько амбициозных поставщиков выходят далеко за пределы этой парадигмы. Цель некоторых плагинов - просто вытеснить HTML и JavaScript, предоставив альтернативные, более функциональные платформы для построения интерактивных веб-приложений. В таком рассуждении есть определенный смысл: браузерам долго не хватало производительности, графических возможностей и мультимедийных кодеков, что сдерживало некоторые потенциальные применения Web. Опора на плагины - разумный краткосрочный способ изменить ситуацию. С другой стороны, когда проприетарные плагины, обремененные патентами и авторскими правами, продвигаются как окончательный способ построить онлайн-экосистему без какого-либо намерения улучшить сами браузеры, открытость Web неизбежно страдает. Некоторые критики, особенно Стив Джобс, считают, что создание жестко контролируемой экосистемы - именно то, к чему стремятся несколько поставщиков плагинов, прежде всего Adobe.^[5]

В ответ на эту воспринимаемую угрозу враждебного захвата Web многие недостатки, которые привели к распространению альтернативных фреймворков приложений, теперь спешно устраняются под расплывчато определенным зонтиком HTML5; теги `<video>` и WebGL^[6] - главные примеры этой работы. Тем не менее некоторые возможности, доступные в плагинах, вероятно, не будут охвачены каким-либо браузерным стандартом в ближайшем будущем. Например, сейчас нет серьезного плана добавить изначально опасные программы с повышенными привилегиями,

поддерживаемые Java, или схемы защиты содержимого через security by obscurity, эвфемистически называемые Digital Rights Management, или DRM.

Поэтому, хотя ландшафт в ближайшие годы резко изменится, можно ожидать, что в той или иной форме проприетарные фреймворки веб-приложений останутся с нами.

Adobe Flash

Adobe Flash - фреймворк веб-приложений, представленный в 1996 году, в разгар Первой войны браузеров. До приобретения Adobe в 2005 году платформа Flash была известна как Macromedia Flash или Shockwave Flash, отсюда расширение файлов .swf, используемое для Flash-файлов, и иногда ее все еще так называют.

Flash - довольно приземленная платформа, построенная поверх основанного на JavaScript языка под названием ActionScript.^[7] Она включает движок рендеринга 2D-векторной и растровой графики и встроенную поддержку нескольких форматов изображений, видео и аудио, таких как популярный и эффективный кодек H.264, используемый для значительной части сегодняшнего онлайн-мультимедиа.

По большинству оценок Flash установлен примерно на 95-99 процентах всех настольных систем.^[8] Эта пользовательская база существенно выше, чем у любого другого плагина медиаплеера. Поддержка плагинов Windows Media Player и QuickTime доступна только примерно на 60 процентах PC, несмотря на агрессивные стратегии поставки в комплекте, тогда как все более непопулярный RealPlayer все еще цепляется за 25 процентов. Рыночная позиция способствует самому значительному и неожиданному использованию продукта: замене всех плагинов воспроизведения мультимедиа, на которые раньше полагались для потокового видео в Web. Хотя плагин также используется для множества других задач, включая реализацию онлайн-игр, интерактивной рекламы и так далее, простое мультимедиа составляет непропорционально большой кусок пирога.

Примечание. Сбивает с толку, что существует и отдельный плагин под названием Adobe Shockwave Player, без слова "Flash", который можно использовать для воспроизведения содержимого, созданного с Adobe Director. Этот плагин иногда ошибочно устанавливают вместо Adobe Flash или рядом с ним, что дает примерно 20-процентную базу установок,^[10] но почти всегда он не нужен. Свойства безопасности этого плагина изучены не особенно хорошо.

Свойства ActionScript

Возможности ActionScript в SWF-файлах в целом аналогичны возможностям JavaScript-кода, встроенного на HTML-страницах, с некоторыми небольшими, но интересными отличиями. Например, программы Flash свободны перечислять все шрифты, установленные в системе, и собирать другие полезные сигналы для fingerprinting системы, недоступные обычным скриптам. Программы Flash также могут использовать полноэкранный рендеринг, облегчая атаки подделки UI, и могут запрашивать доступ к устройствам ввода, таким как камера или микрофон, что требует согласия пользователя. Flash также склонен игнорировать настройки безопасности и приватности браузера и использует собственную конфигурацию для механизмов вроде постоянного хранения данных внутри плагина, хотя в мае 2011 года были объявлены некоторые улучшения в этой области.

Оставшиеся возможности менее удивительны. Мы подробнее обсудим сетевые разрешения и разрешения доступа к DOM у Flash-приложений в следующей главе, но вкратце: по умолчанию каждый Flash-апплет может использовать HTTP-стек браузера и любые управляемые в нем окружающие учетные данные, чтобы обращаться обратно к своему исходному серверу, запрашивать ограниченный набор подресурсов с других сайтов и выполнять навигацию текущего окна браузера или открывать новое. Программы ActionScript также могут договариваться о доступе на уровне браузера к другим сейчас запущенным Flash-приложениям и, в некоторых случаях, получать доступ к DOM встраиваемой страницы. Последняя функциональность реализуется путем внедрения `eval(...)`-подобных инструкций в целевой контекст JavaScript.

ActionScript предоставляет плодородную почву для уязвимостей веб-приложений. Например, функции `getURL(...)` и `navigateToURL(...)`, используемые для навигации браузера или открытия новых окон, иногда вызываются с входами, контролируруемыми злоумышленником. Такое использование опасно. Хотя URL `javascript:` не имеют особого смысла для Flash, функция передаст такие строки браузеру, в некоторых случаях приводя к внедрению скрипта на встраиваемом сайте.

До недавнего времени родственная проблема присутствовала с другими API обработки URL, такими как `loadMovie(...)`. Хотя функция не полагалась на браузер для загрузки документа, она распознавала внутреннюю схему `asfunction:`, которая работает похоже на `eval(...)` и могла тривиально использоваться для выполнения вызова `getURL(...)`:

```
asfunction:getURL,javascript:alert('Hi mom!')
```

Проблема загрузки скриптов из недоверенных источников, обсуждавшаяся в главе 6, имеет эквивалент и в мире плагинов. Во Flash очень небезопасно вызывать некоторые функции, влияющие на состояние среды выполнения ActionScript, например `LoadVars.load(...)`, с URL, контролируемым злоумышленником, даже если схема, из которой загружается ресурс, - `http:` или `https:`.

Еще одна часто упускаемая поверхность атаки - внутренний, упрощенный HTML-парсер, предлагаемый плагином Flash: базовую HTML-разметку можно присваивать свойствам вроде `TextField.htmlText` и `TextArea.htmlText`. Легко забыть, что предоставленное пользователем содержимое должно быть правильно экранировано и в этой среде. Невыполнение этого требования может позволить злоумышленникам изменить внешний вид UI приложения или внедрить потенциально проблемные ссылки, ориентированные на сценарии.

Еще один класс связанных с Flash ошибок безопасности может возникать из-за проектных или реализационных проблем в самом плагине. Например, возьмем API `ExternalInterface.call(...)`. Он предназначен для того, чтобы ActionScript мог вызывать существующие JavaScript-функции на встраиваемой странице, и принимает два параметра: имя вызываемой JavaScript-функции и необязательную строку, передаваемую этой процедуре. Понятно, что первый параметр не должен контролироваться злоумышленником, но кажется безопасным помещать пользовательские данные во второй. Фактически документация приводит следующий фрагмент кода, описывающий именно этот сценарий использования:^[11]

```
ExternalInterface.call("sendToJavaScript", input.text);
```

Этот вызов приведет к внедрению следующей инструкции `eval(...)` на встраиваемой странице:

```
try {
  __flash__toXML(sendToJavaScript, "value of input.text"));
} catch (e) {
  "<undefined/>";
}
```

При написании кода за этим вызовом авторы плагина не забыли использовать backslash-экранирование при выводе второго параметра: `hello"world` становится `hello\"world`. К сожалению, они упустили необходимость экранировать и любые случайные символы обратного слеша. Из-за этого, если значение `input.text` установлено в следующую строку, встроенный скрипт неожиданно выполнится:

```
Hello world!\\"+alert(1)); } catch(e) {} //
```

Я связался с Adobe по поводу этой конкретной проблемы в марте 2010 года. Более чем через год ее ответ был таким: "Мы не внесли никаких изменений в это поведение по причинам обратной совместимости".

Это кажется неудачным.

Microsoft Silverlight

Microsoft Silverlight - универсальная платформа разработки, построенная на Windows Presentation Foundation, GUI-фреймворке, входящем в стек Microsoft .NET. Она дебютировала в 2007 году и сочетает Extensible Application Markup Language (XAML)^[12], альтернативу Microsoft для Mozilla XUL, с кодом, написанным на одном из нескольких управляемых языков .NET,^[13] таких как C# или Visual Basic.

Несмотря на существенные проектные различия и более амбициозную, а также более запутанную, архитектуру, этот плагин прежде всего должен конкурировать с Adobe Flash. Многие возможности, доступные приложениям Silverlight, зеркально повторяют возможности конкурента, включая почти идентичную модель безопасности и похожий мост к встраиваемой странице на основе `eval(...)`. К чести Microsoft, однако, в Silverlight нет эквивалента схемы `asfunction:` или встроенного HTML-рендерера.

Microsoft продвигает Silverlight довольно агрессивно, и он поставляется в комплекте с некоторыми редакциями Internet Explorer. В результате, в зависимости от источника, считается, что его проникновение на настольные системы составляет около 60-75 процентов.^[14] Несмотря на распространенность, Silverlight довольно редко используется для разработки реальных веб-приложений, возможно потому, что обычно не предлагает убедительных преимуществ перед более устоявшимся аналогом или потому, что его архитектура воспринимается как более натянутая и платформенно-специфичная. Netflix, популярный сервис потокового видео и проката, - один из очень немногих высокопрофильных сайтов, которые действительно полагаются на Silverlight для воспроизведения на некоторых устройствах.

Sun Java

Java - язык программирования, связанный с платформонезависимой платформой выполнения управляемого кода. Разработанная в начале-середине 1990-х Джеймсом Гослингом для Sun Microsystems, Java имеет устоявшуюся роль серверного языка программирования и очень сильное присутствие во многих других нишах, включая мобильные устройства. Однако с самого начала Sun надеялась, что Java также займет заметное место на стороне браузера.

Java в браузере предшествовала Flash и большинству похожих плагинов, а ныне устаревший тег `<applet>` свидетельствует о том, насколько важным, уникальным и новым должно было казаться это дополнение в свое время. Но несмотря на эту фору, язык Java почти исчез как внутрибраузерная платформа разработки, и даже в свои лучшие дни он никогда не обладал настоящей известностью. Он сохраняет замечательную 80-процентную базу установок, но этот высокий процент во многом объясняется тем, что Java-плагин поставляется вместе с Java Runtime Environment (JRE), более практически полезным и часто предустановленным компонентом, который нужен для запуска обычных настольных Java-приложений в системе без какого-либо участия браузера.

Причины неудачи Java как браузерной технологии трудно точно определить. Возможно, дело в плохой производительности запуска плагина, неуклюжих UI-библиотеках, из-за которых было трудно разрабатывать отзывчивые и удобные веб-приложения, или в истории ожесточенных судебных споров между Sun и Microsoft, бросивших длинную тень на будущее языка в операционных системах Microsoft.^[15] Каковы бы ни были причины, высокая база установок Java в сочетании с ее маргинальным использованием означает, что создаваемые ею риски намного перевешивают любые потенциальные преимущества для пользователей. У плагина было около 80 уязвимостей безопасности в 2010 году,^[16] а поставщика часто критикуют за очень медленное исправление таких ошибок.

Политики безопасности Java несколько похожи на политики других плагинов, но в некоторых аспектах, таких как понимание политики одного источника или способность ограничивать доступ к встраиваемой странице, она сравнивается неблагоприятно. Следующая глава дает обзор этого. Также стоит отметить, что, в отличие от Flash или Silverlight, некоторые типы криптографически подписанных апплетов могут запрашивать доступ к потенциально опасным возможностям ОС, таким как неограниченная сеть или доступ к файлам, и на пути стоит только легко выманиваемое согласие пользователя.

XML Browser Applications (XBAP)

XML Browser Applications (XBAP)^[17] - тяжеловесная вылазка Microsoft в мир фреймворков веб-приложений, предпринятая в годы, когда битва вокруг Java начала портиться, и до того, как компания выпустила Silverlight.

XBAP напоминает Silverlight тем, что использует ту же Windows Presentation Foundation и архитектуру .NET. Однако вместо того, чтобы быть самодостаточным и быстрым браузерным плагином, он зависит от большого и громоздкого runtime .NET, примерно так же, как Java-плагин зависит от JRE. Он выполняет управляемый код в отдельном процессе `PresentationHost.exe`, часто загружая обширные зависимости во время инициализации. По собственному признанию Microsoft, время загрузки среднего ранее не кэшированного приложения легко могло достигать 10 секунд или больше. Когда технология была представлена в 2002 году, большинство пользователей уже ожидали, что интернет-приложения будут куда отзывчивее.

Модель безопасности приложений XBAP плохо документирована и на данный момент не исследована, возможно из-за ничтожного реального использования XBAP и тупой, многоуровневой архитектуры. Разумно ожидать, что свойства безопасности XBAP будут параллельны модели, которую в итоге приняли для Silverlight, но с более широким доступом к некоторым библиотекам .NET и виджетам UI. И, по-видимому, в результате копирования Sun, программам XBAP также можно дать повышенные привилегии при загрузке из локальной файловой системы или при подписании криптографическим сертификатом.

Microsoft поставляла плагины XBAP вместе со своим .NET Framework до такой степени, что молча устанавливала неудаляемые плагины Windows Presentation Foundation - не только в Internet Explorer, но и в конкурирующие Firefox и Chrome. Этот шаг вызвал заслуженные споры, особенно когда начали поступать первые отчеты об уязвимостях. Mozilla даже временно отключила плагин через автоматическое обновление, чтобы защитить своих пользователей. Тем не менее, несмотря на такие смелые и сомнительные шаги по его популяризации, никто на самом деле не хотел писать XBAP-апплеты, и постепенно технология последовала за Java на свалку истории.

В конце концов Microsoft, похоже, признала эту неудачу и решила сосредоточиться на Silverlight. Начиная с Internet Explorer 9, XBAP по умолчанию отключен для содержимого интернет-происхождения, а сомнительные плагины Firefox и Chrome больше не проталкиваются пользователям автоматически. Тем не менее кажется разумным предположить, что по крайней мере 10 процентов всех пользователей интернета все еще могут просматривать web с установленным на машинах сложным, частично заброшенным и в значительной степени ненужным плагином, и будут продолжать делать это еще пару лет.

Элементы управления ActiveX

По своей сути ActiveX - преемник Object Linking and Embedding (OLE), технологии 1990 года, которая позволяла программам повторно использовать компоненты других приложений стандартизованным, независимым от языка способом. Простым вариантом использования ActiveX было бы приложение электронных таблиц, желающее встроить редактируемое векторное изображение из программы редактирования графики, или простая игра, которая хочет встроить видеоплеер.

Идея не спорная, но к середине 1990-х Microsoft решила, что ActiveX имеет смысл и в браузере. В конце концов, разве сайты не захотят получать выгоду от тех же компонентов Windows, на которые могли полагаться настольные приложения? Подход нарушает идею возвращения открытого, независимого от OS Web, но в остальном впечатляет, как показывает следующий пример JavaScript, который непринужденно создает, редактирует и сохраняет таблицу Excel:

```
var sheet = new ActiveXObject("Excel.Sheet");
sheet.ActiveSheet.Cells(42,42).Value = "Hi mom!";
sheet.SaveAs("c:\\spreadsheet.xls");
sheet.Application.Quit();
```

Если оставить соответствие стандартам в стороне, переход Microsoft к ActiveX оказался катастрофическим с точки зрения безопасности. Многие из открытых компонентов ActiveX были совершенно не готовы вести себя правильно при взаимодействии с недоверенными средами, и за следующие 15 лет исследователи обнаружили несколько сотен значительных уязвимостей безопасности в доступных из web элементах управления ActiveX. Черт, простое наблюдение, что Firefox не поддерживает эту технологию, помогло укрепить его образ безопасности в начале Второй войны браузеров.

Несмотря на это фиаско, Microsoft вызывающе держалась за ActiveX, вкладываясь в постепенное ограничение числа элементов управления, доступных из интернета, и исправление ошибок в тех, которые считала существенными. Только в Internet Explorer 9 Microsoft наконец решила отпустить: Internet Explorer 9 по умолчанию отключает весь доступ ActiveX, требуя нескольких дополнительных щелчков, чтобы использовать его при необходимости.

Примечание. Мудрость делегирования выбора пользователю неясна, особенно поскольку разрешение, предоставленное сайту, распространяется не только на легитимное содержимое этого сайта, но и на любые полезные нагрузки, внедренные из-за ошибок приложения, таких как XSS. Тем не менее Internet Explorer 9 - некоторое улучшение.

Жизнь с другими плагинами

До сих пор мы охватили почти все универсальные браузерные плагины, используемые сегодня. Хотя существует длинный хвост специализированных или экспериментальных плагинов, их использование довольно незначительно и не является тем, что нам нужно учитывать при обзоре общего здоровья онлайн-экосистемы.

Ну, с одним исключением. У неопределенного, но, вероятно, значительного процента онлайн-пользователей можно ожидать набор браузерных плагинов или элементов управления ActiveX, открытых web, которые они никогда сознательно не устанавливали или которые их заставили установить, хотя сомнительно, что они когда-либо получают пользу от введенной функциональности.

Эту непростительную практику иногда принимают в остальном уважаемые и доверенные компании. Например, Adobe заставляет пользователей, желающих скачать Adobe Flash, также устанавливать GetRight, совершенно ненужную стороннюю утилиту загрузки. Microsoft делает то же самое с Akamai Download Manager на своем сайте, ориентированном на разработчиков, в комплекте с забавным обоснованием, выделение мое:^[18]

Что такое Akamai Download Manager и почему я должен его использовать?

Чтобы помочь вам скачивать большие файлы с меньшей вероятностью прерывания, некоторые загрузки требуют использования Akamai Download Manager.

Основная тревога относительно программного обеспечения, установленного таким образом и напрямую открытого вредоносному входу из любой точки интернета, состоит в том, что если оно не спроектировано с крайней осторожностью, в нем, вероятно, будут уязвимости; и действительно, они были и в GetRight, и в Akamai Download Manager. Поэтому риски просмотра web с совершенно ненужным плагином, который послужил конкретной цели один или два раза, намного перевешивают заявленные и обычно нежелательные преимущества.

Памятка по инженерии безопасности

При отдаче файлов, обрабатываемых плагинами

- Данные из доверенных источников: данные из доверенных источников в целом безопасно размещать, но помните, что уязвимости безопасности в апплетах Flash, Java или Silverlight либо в JavaScript-движке Adobe Reader могут влиять на безопасность вашего домена. Избегайте обработки URL, предоставленных пользователями, и генерации или изменения контролируемого пользователем HTML изнутри апплетов, выполняемых плагином. Проявляйте осторожность при использовании моста JavaScript.
- Простое мультимедиа, контролируемое пользователем: контролируемое пользователем мультимедиа относительно безопасно размещать, но обязательно проверяйте и ограничивайте формат, используйте правильный Content-Type и обратитесь к шпаргалке в главе 13, чтобы избежать проблем безопасности, вызванных дефектами content sniffing.
- Форматы документов, контролируемые пользователем: они не являются изначально небезопасными, но имеют повышенный риск способствовать проблемам безопасности из-за дефектов проектирования плагинов. По возможности рассмотрите размещение на выделенном домене. Если нужно аутентифицировать запрос к изолированному домену, делайте это с помощью одноразового токена запроса, а не полагаясь на cookie.
- Активные приложения, контролируемые пользователем: их небезопасно размещать в чувствительных доменах.

При встраивании файлов, обрабатываемых плагинами

Всегда убеждайтесь, что содержимое плагинов на HTTPS-сайтах также загружается по HTTPS,^[19] и всегда явно указывайте параметр type в <object> или <embed>. Обратите внимание, что из-за неавторитетной обработки параметров type нужно проявлять сдержанность при встраивании содержимого плагинов из недоверенных источников, особенно на особо чувствительных сайтах.

- Простое мультимедиа: в целом безопасно загружать простое мультимедиа из сторонних источников, с оговорками, изложенными выше.
- Форматы документов: обычно они безопасны, но несут большой потенциал проблем обработки содержимого плагином и браузером, чем простое мультимедиа. Проявляйте осторожность.
- Flash и Silverlight: в принципе приложения Flash и Silverlight можно безопасно встраивать из внешних источников, если в разметке присутствуют соответствующие флаги безопасности. Если флаги указаны неправильно, вы можете в итоге связать безопасность своего сайта с безопасностью поставщика содержимого. См. советы в шпаргалке главы 9.
- Java: Java всегда связывает безопасность вашего сервиса с безопасностью поставщика содержимого, потому что доступ DOM к встраивающей странице нельзя надежно ограничить. См. главу 9. Не загружайте Java-приложения с недоверенных сайтов.

Если вы хотите написать новый браузерный плагин или компонент ActiveX

Если только вы не решаете важный, распространенный сценарий, который принесет пользу значительной доле интернета, пожалуйста, пересмотрите решение. Если вы закрываете действительно важную потребность, рассмотрите возможность сделать это рецензируемым, стандартизованным способом как часть HTML5.

Сноски

- [1] [\[назад\]](#) Сноска в оригинале указывает на отчет об эксплуатируемых клиентских уязвимостях 2010 года.
- [2] [\[назад\]](#) Сноска в оригинале указывает на описание уязвимости GIFAR.
- [3] [\[назад\]](#) Сноска в оригинале указывает на отчет 2009 года о PDF-формах и URL javascript:.
- [4] [\[назад\]](#) Сноска в оригинале указывает на публикацию Петко Д. Петкова 2007 года о JavaScript в fragment identifier PDF.
- [5] [\[назад\]](#) Сноска в оригинале указывает на эссе Стива Джобса о Flash.
- [6] [\[назад\]](#) WebGL - довольно недавняя попытка принести 3D-графику на основе OpenGL в приложения JavaScript. Первая спецификация стандарта появилась в марте 2011 года, и ожидается широкая поддержка на уровне браузеров.
- [7] [\[назад\]](#) Сноска в оригинале указывает на документацию ActionScript.
- [8] [\[назад\]](#) Сноска в оригинале указывает на статистику распространения Flash.
- [9] [\[назад\]](#) В оригинале приведена еще одна ссылка на оценку установленной базы Flash.
- [10] [\[назад\]](#) Сноска в оригинале указывает на статистику установленной базы Adobe Shockwave Player.
- [11] [\[назад\]](#) Сноска в оригинале указывает на документацию Adobe ExternalInterface.call(...).
- [12] [\[назад\]](#) Сноска в оригинале указывает на документацию XAML.
- [13] [\[назад\]](#) Управляемый код не выполняется напрямую CPU, что было бы изначально небезопасно, поскольку CPU не предназначен для применения правил веб-безопасности. Вместо этого он компилируется в промежуточную двоичную форму и затем интерпретируется во время выполнения специализированной виртуальной машиной. Такой подход быстрее интерпретации скриптов во время выполнения и позволяет применять пользовательскую политику безопасности при выполнении программы.
- [14] [\[назад\]](#) Сноска в оригинале указывает на статистику распространения Silverlight.
- [15] [\[назад\]](#) Судебные битвы начались в 1997 году, когда Microsoft решила выпустить собственную, и в некотором смысле лучшую, версию виртуальной машины Java. Sun Microsystems подала иск, надеясь добиться судебного запрета, который заставил бы Microsoft вместо этого поставлять версию Sun. Сначала компании урегулировали спор в 2001 году, но вскоре после этого снова отправились в суд. В окончательном урегулировании 2004 года Sun ушла с 1,6 миллиарда долларами наличными, но пользователи Windows вообще не получали никакого runtime Java.
- [16] [\[назад\]](#) Сноска в оригинале указывает на статистику уязвимостей Java 2010 года.
- [17] [\[назад\]](#) Сноска в оригинале указывает на документацию XML Browser Applications.
- [18] [\[назад\]](#) Сноска в оригинале указывает на страницу Microsoft об Akamai Download Manager.
- [19] [\[назад\]](#) Если загрузка апплета, доставленного по HTTP, на HTTPS-странице абсолютно неизбежна, безопаснее поместить его внутрь промежуточного HTTP-фрейма, а не непосредственно внутри HTTPS-документа, поскольку это предотвращает использование моста апплет-JavaScript для атак.

Часть II. Функции безопасности браузера

Теперь, когда мы рассмотрели базовые строительные блоки Web, можно спокойно изучить все функции безопасности, которые держат вредоносные веб-приложения на расстоянии. Часть II этой книги рассматривает все - от хорошо известной, но часто неверно понимаемой политики одного источника до малоизвестных и проприетарных настроек зон Internet Explorer. Она объясняет, что эти механизмы могут сделать для вас - и когда они склонны разваливаться.

Глава 9. Логика изоляции содержимого

Большинство гарантий безопасности, предоставляемых веб-браузерами, предназначены для изоляции документов на основе их происхождения. Предпосылка проста: двум страницам из разных источников не должно быть разрешено вмешиваться друг в друга. На практике все может быть сложнее, поскольку нет универсального согласия о том, где начинается и заканчивается отдельный документ или что составляет единое происхождение. Результат - иногда непредсказуемая мозаика противоречивых политик, которые не вполне хорошо работают вместе, но которые нельзя подправить, не затронув глубоко все текущие законные способы использования Web.

Помимо этих проблем, мало ясности и в том, какие действия вообще должны подвергаться проверкам безопасности. Кажется очевидным, что некоторые взаимодействия, такие как переход по ссылке, должны быть разрешены без особых ограничений, поскольку они необходимы для здоровья всей экосистемы, а другие, такие как изменение содержимого страницы, загруженной в отдельном окне, должны требовать проверки безопасности. Но между этими крайностями существует большая серая область, и эта середина часто кажется управляемой скорее броском кубика, чем каким-либо единым планом. В этих мутных водах изобилуют уязвимости вроде подделки межсайтовых запросов (см. главу 4).

Пора начинать исследование. Бросим собственный кубик и начнем путешествие с JavaScript.

Политика одного источника для Document Object Model

Политика одного источника (same-origin policy, SOP) - концепция, введенная Netscape в 1995 году вместе с JavaScript и Document Object Model (DOM), всего через год после создания HTTP-cookie. Базовое правило за этой политикой прямолинейно: для любых двух отдельных контекстов выполнения JavaScript один должен иметь возможность получить доступ к DOM другого только если протоколы, DNS-имена^[1] и номера портов, связанные с их документами-хозяевами, точно совпадают. Любой другой междокументный JavaScript-доступ к DOM должен завершаться неудачей.

Кортеж протокол-хост-порт, введенный этим алгоритмом, обычно называют origin, или источником. Как основа политики безопасности он довольно надежен: SOP реализована во всех современных браузерах с хорошей степенью согласованности и лишь с редкими ошибками.^[2] Фактически выделяется только Internet Explorer, поскольку он игнорирует номер порта при проверках origin. Такая практика несколько менее безопасна, особенно учитывая риск наличия не-HTTP-сервисов, запущенных на удаленном хосте, для HTTP/0.9 веб-серверов (см. главу 3). Но обычно это не имеет заметного значения.

Таблица 9-1 иллюстрирует результат проверок SOP в различных ситуациях.

Таблица 9-1. Результаты проверок SOP

Исходный документ	Доступный документ	Браузер не IE	Internet Explorer
http://example.com/a/	http://example.com/b/	Доступ разрешен	Доступ разрешен
http://example.com/	http://www.example.com/	Несовпадение хоста	Несовпадение хоста
http://example.com/	https://example.com/	Несовпадение протокола	Несовпадение протокола
http://example.com:81/	http://example.com/	Несовпадение порта	Доступ разрешен

Примечание. Изначально эта политика одного источника должна была управлять только доступом к DOM, то есть к методам и свойствам, связанным с содержимым фактически отображаемого документа. Политика постепенно расширялась, чтобы защищать другие очевидно чувствительные области корневого объекта JavaScript, но она не всеобъемлюща. Например, скрипты не из того же источника обычно все еще могут вызвать `location.assign()` или `location.replace(...)` для произвольного окна или фрейма. Масштаб и последствия этих исключений являются темой главы 11.

Простота SOP - одновременно благословение и проклятие. Механизм довольно легко понять и не слишком трудно реализовать правильно, но его негибкость может быть бременем для веб-разработчиков. В некоторых контекстах политика слишком широка, делая невозможной, скажем, изоляцию домашних страниц, принадлежащих отдельным пользователям, если только не выдавать каждому отдельный домен. В других случаях верно противоположное: политика затрудняет бесшовный обмен данными между законно сотрудничающими сайтами, например `login.example.com` и `payments.example.com`.

Попытки исправить первую проблему - сузить понятие `origin` - обычно обречены на провал из-за взаимодействий с другими явными и скрытыми средствами безопасности браузера. Попытки расширить источники или облегчить междоменное взаимодействие встречаются чаще. Два широко поддерживаемых способа достижения этих целей - `document.domain` и `postMessage(...)`, обсуждаемые ниже.

document.domain

Это свойство JavaScript позволяет любым двум сотрудничающим сайтам, разделяющим общий домен верхнего уровня, например `example.com` или даже просто `.com`, договориться, что для будущих проверок одного источника они хотят считаться эквивалентными. Например, и `login.example.com`, и `payments.example.com` могут выполнить следующее присваивание:

```
document.domain = "example.com"
```

Установка этого свойства переопределяет обычную логику сопоставления имен хостов во время проверок политики одного источника. Однако протоколы и номера портов все равно должны совпадать; если это не так, изменение `document.domain` не даст желаемого эффекта.

Обе стороны должны явно согласиться на использование этой возможности. Просто потому, что `login.example.com` установил свой `document.domain` в `example.com`, не означает, что ему будет разрешен доступ к содержимому, происходящему с сайта, размещенного по `http://example.com/`. Этот сайт тоже должен выполнить такое присваивание, даже если здравый смысл подсказал бы, что это по-ор. Этот эффект симметричен. Подобно тому, как страница, установившая `document.domain`, не сможет получить доступ к страницам, которые этого не сделали, действие установки свойства также делает вызывающую сторону в основном, но не полностью,^[3] недоступной для обычных документов, которые ранее считались бы с ней одним источником. Таблица 9-2 показывает эффекты различных значений `document.domain`.

Таблица 9-2. Результаты проверок document.domain

Исходный документ URL	document.domain	Доступный документ URL	document.domain	Результат
http://www.example.com/	example.com	http://payments.example.com/	example.com	Доступ разрешен
http://www.example.com/	example.com	https://payments.example.com/	example.com	Несовпадение протокола
http://payments.example.com/	example.com	http://example.com/	не задан	Доступ запрещен
http://www.example.com/	не задан	http://www.example.com/	example.com	Доступ запрещен

Несмотря на некоторую сложность, намекающую на особую хитрость, `document.domain` не особенно безопасен. Его самая значительная слабость в том, что он приглашает нежеланных гостей. После того как две стороны взаимно установили это свойство в `example.com`, дело не просто в том, что `login.example.com` и `payments.example.com` смогут общаться; `funny-cat-videos.example.com` тоже сможет присоединиться к вечеринке. А из-за степени доступа, разрешенного между страницами, целостность любого из участвующих JavaScript-контекстов просто нельзя гарантировать в сколько-нибудь реалистичной степени. Другими словами, касание `document.domain` неизбежно связывает безопасность вашей страницы с безопасностью самого слабого звена во всем домене. Крайний случай установки значения в `*.com` по сути эквивалентен самоубийству с ассистентом.

postMessage(...)

API `postMessage(...)` - расширение HTML5, которое позволяет немного менее удобные, но заметно более безопасные коммуникации между сайтами не из одного источника без автоматического отказа от целостности любой из участвующих сторон. Сегодня он поддерживается во всех актуальных браузерах, хотя из-за относительной новизны его нет в Internet Explorer 6 или 7.

Механизм позволяет отправить текстовое сообщение любой длины в любое окно, для которого отправитель держит действительный JavaScript-дескриптор (см. главу 6). Хотя у политики одного источника есть ряд пробелов, позволяющих реализовать похожую функциональность другими средствами,^[4] именно этот механизм действительно безопасен в использовании. Он позволяет отправителю указать, каким `origin` разрешено вообще получить сообщение, на случай если URL целевого окна изменился, и предоставляет получателю идентичность отправителя, чтобы целостность канала можно было легко установить. Напротив, унаследованные методы, полагающиеся на лазейки SOP, обычно не дают таких гарантий; если конкретное действие разрешено без надежных проверок безопасности, его обычно может вызвать и вредоносная третья сторона, а не только предполагаемые участники.

Чтобы проиллюстрировать правильное использование `postMessage(...)`, рассмотрим случай, когда документ верхнего уровня, расположенный на `payments.example.com`, должен получить информацию о входе пользователя для отображения. Для этого он загружает фрейм, указывающий на `login.example.com`. Этот фрейм может просто выполнить следующую команду:

```
parent.postMessage("user=bob", "https://payments.example.com");
```

Браузер доставит сообщение только если встраивающий сайт действительно соответствует указанному доверенному `origin`. Чтобы безопасно обработать этот ответ, документ верхнего уровня должен использовать следующий код:

```
// Register the intent to process incoming messages:
addEventListener("message", user_info, false);
// Handle actual data when it arrives:
function user_info(msg) {
  if (msg.origin == "https://login.example.com") {
    // Use msg.data as planned
  }
}
```

`postMessage(...)` - очень надежный механизм, предлагающий значительные преимущества перед `document.domain` и практически всеми прочими партизанскими подходами, предшествовавшими ему; поэтому его следует использовать как можно чаще. Тем не менее им все равно можно злоупотребить. Рассмотрим следующую проверку, ищущую подстроку в доменном имени:

```
if (msg.origin.indexOf(".example.com") != -1) { ... }
```

Как должно быть очевидно, это сравнение совпадет не только с сайтами внутри `example.com`, но и с радостью примет сообщения от `www.example.com.bunnyoutlet.com`. Скорее всего, в своих путешествиях вы наткнетесь на такой код не один раз. Такова жизнь!

Примечание. Недавние изменения HTML5 расширили API `postMessage(...)`, добавив несколько чрезмерно спроектированные "ports" и "channels", предназначенные для облегчения потоковых коммуникаций между сайтами. Поддержка этих возможностей браузерами сейчас очень ограничена, а их практическая полезность неясна, но с точки зрения безопасности они, похоже, не вызывают особых тревог.

Взаимодействия с браузерными учетными данными

Завершая обзор основанной на DOM политики одного источника, важно отметить, что она никак не синхронизирована с окружающими учетными данными, состоянием SSL, сетевым контекстом или многими другими потенциально значимыми для безопасности параметрами, отслеживаемыми браузером. Любые два окна или фрейма, открытые в браузере, останутся друг для друга одним источником, даже если пользователь выйдет из одной учетной записи и войдет в другую, если страница переключится с хорошего HTTPS-сертификата на плохой и так далее.

Это отсутствие синхронизации может способствовать эксплуатируемости других ошибок безопасности. Например, несколько сайтов не защищают свои формы входа от подделки межсайтовых запросов, позволяя любому стороннему сайту просто отправить имя пользователя и пароль и войти за пользователя в учетную запись, контролируемую злоумышленником. Сначала это может казаться безвредным, но когда содержимое, загруженное в браузер до и после этой операции, считается одним источником, влияние обычно игнорируемых "самопричиненных" уязвимостей межсайтового скриптинга (то есть таких, где владелец конкретной учетной записи может атаковать только себя) внезапно становится намного выше, чем казалось раньше. В самом базовом сценарии злоумышленник может сначала открыть и удерживать фрейм, указывающий на чувствительную страницу целевого сайта, например

http://www.fuzzybunnies.com/address_book.php, а затем войти жертвой в контролируемую злоумышленником учетную запись, чтобы выполнить self-XSS в несвязанном компоненте fuzzybunnies.com. Несмотря на изменение HTTP-учетных данных, код, внедренный на этом последнем шаге, получит неограниченный доступ к ранее загруженному фрейму, позволяя украсть данные.

Политика одного источника для XMLHttpRequest

API XMLHttpRequest, упомянутый в этой книге уже несколько раз, дает программам JavaScript возможность выполнять почти неограниченные HTTP-запросы к серверу, с которого произошел документ-хозяин, и читать обратно заголовки ответа и тело документа. Возможность делать это не была бы особенно значимой, если бы механизм не использовал существующий HTTP-стек браузера и его удобства, включая окружающие учетные данные, механизмы кэширования, keep-alive-сессии и так далее.

Простое и довольно самоочевидное использование синхронного XMLHttpRequest может выглядеть так:

```
var x = new XMLHttpRequest();
x.open("POST", "/some_script.cgi", false);
x.setRequestHeader("X-Random-Header", "Hi mom!");
x.send("...POST payload here...");
alert(x.responseText);
```

Асинхронные запросы очень похожи, но выполняются без блокировки движка JavaScript или браузера. Запрос отправляется в фоне, а обработчик события вызывается при завершении.

В изначальном замысле возможность выполнять HTTP-запросы через этот API и читать обратно данные управляется почти дословной копией политики одного источника с двумя небольшими и, казалось бы, случайными изменениями. Во-первых, настройка `document.domain` не влияет на этот механизм, и целевой URL, указанный для `XMLHttpRequest.open(...)`, всегда должен соответствовать истинному `origin` документа. Во-вторых, в этом контексте номер порта учитывается в версиях Internet Explorer до 9, хотя этот браузер игнорирует его в других местах.

Тот факт, что XMLHttpRequest дает пользователю беспрецедентный уровень контроля над HTTP-заголовками в запросе, на самом деле может быть полезен для безопасности. Например, вставка собственного HTTP-заголовка, такого как `X-Coming-From: same-origin`, - очень простой способ проверить, что конкретный запрос не приходит из стороннего домена, поскольку ни один другой сайт не должен иметь возможность вставить собственный заголовок в запрос, выпущенный браузером. Эта гарантия не очень сильна, потому что ни одна спецификация не говорит, что неявное ограничение на междоменные заголовки не может измениться,^[5] тем не менее, когда речь идет о веб-безопасности, такие предположения часто просто то, с чем приходится учиться жить.

Контроль над структурой HTTP-запроса может быть и бременем, потому что вставка некоторых типов заголовков может изменить смысл запроса для целевого сервера или прокси так, что браузер этого не осознает. Например, указание неправильного значения `Content-Length` может позволить злоумышленнику протаскать второй запрос в поддерживаемую браузером keep-alive HTTP-сессию, как показано здесь.

```

var x = new XMLHttpRequest();
x.open("POST", "http://www.example.com/", false);
// This overrides the browser-computed Content-Length header:
x.setRequestHeader("Content-Length", "7");
// The server will assume that this payload ends after the first
// seven characters, and that the remaining part is a separate
// HTTP request.
x.send(
    "Gotcha!\n" +
    "GET /evil_response.html HTTP/1.1\n" +
    "Host: www.bunnyoutlet.com\n\n"
);

```

Если это произойдет, ответ на второй, внедренный запрос может позднее быть неверно интерпретирован браузером, возможно отравляя кэш или внедряя содержимое на другой сайт. Эта проблема особенно выражена, если используется HTTP-прокси и все HTTP-запросы отправляются через общий канал.

Из-за этого риска и после множества проб и ошибок современные браузеры заносят в черный список набор HTTP-заголовков и методов запросов. Это делается с относительно малой согласованностью: хотя Referer, Content-Length и Host запрещены повсеместно, обработка заголовков вроде User-Agent, Cookie, Origin или If-Modified-Since различается от браузера к браузеру. Аналогично, метод TRACE заблокирован везде из-за неожиданного риска, который он создавал для httponly cookie, но метод CONNECT разрешен в Firefox, несмотря на смутный риск вмешательства в HTTP-прокси.

Естественно, реализация этих черных списков сама по себе оказалась занимательным упражнением. Исключительно ради вашего развлечения рассмотрим следующие случаи, которые работали в некоторых браузерах всего три года назад.^[6]

```
XMLHttpRequest.setRequestHeader("X-Harmless", "1\n0wned: Gotcha");
```

или

```
XMLHttpRequest.setRequestHeader("Content-Length: 123 ", "");
```

или просто

```
XMLHttpRequest.open("GET\thttp://evil.com\thttp/1.0\n\n", "/", false);
```

Примечание. Cross-Origin Resource Sharing^[7] (CORS) - предлагаемое расширение XMLHttpRequest, позволяющее выполнять HTTP-запросы между доменами и затем читать их обратно, если в возвращенных данных появляется конкретный заголовок ответа. Механизм меняет семантику API, обсуждаемого здесь, разрешая некоторые "vanilla" междоменные запросы, задуманные как неотличимые от обычной навигации, выполнять через XMLHttpRequest.open(...) без дополнительных проверок; более сложные запросы сначала требуют preflight-запроса на основе OPTIONS. CORS уже доступен в некоторых браузерах, но ему противостоят инженеры Microsoft, которые продвигали конкурирующий подход XDomainRequest в Internet Explorer 8 и 9. Поскольку исход этого конфликта неясен, подробное обсуждение CORS отложено до главы 16, где дается более систематический обзор готовящихся и экспериментальных механизмов.

Политика одного источника для Web Storage

Web storage - простое решение для базы данных, впервые реализованное инженерами Mozilla в Firefox 1.5 и в итоге принятое спецификацией HTML5.^[8] Оно доступно во всех текущих браузерах, но не в Internet Explorer 6 или 7.

После нескольких сомнительных итераций текущая конструкция опирается на два простых JavaScript-объекта: `localStorage` и `sessionStorage`. Оба объекта предлагают идентичный простой API для создания, получения и удаления пар имя-значение в базе данных, управляемой браузером. Например:

```
localStorage.setItem("message", "Hi mom!");  
alert(localStorage.getItem("message"));  
localStorage.removeItem("message");
```

Объект `localStorage` реализует постоянное хранилище, специфичное для `origin`, которое переживает закрытие браузера, тогда как `sessionStorage`, как ожидается, привязано к текущему окну браузера и предоставляет временный механизм кэширования, уничтожаемый в конце сеанса просмотра. Хотя спецификация говорит, что и `localStorage`, и `sessionStorage` должны быть связаны с SOP-подобным `origin`, то есть кортежем протокол-хост-порт, реализации в некоторых браузерах не следуют этому совету, вводя потенциальные ошибки безопасности. Особенно заметно, что в Internet Explorer 8 протокол не учитывается при вычислении `origin`, помещая HTTP- и HTTPS-страницы в общий контекст. Такая конструкция делает для HTTPS-сайтов очень небезопасным хранить или читать чувствительные данные через этот API. Эта проблема исправлена в Internet Explorer 9, но, похоже, переносить исправление назад не планируется.

В Firefox, с другой стороны, `localStorage` ведет себя правильно, а интерфейс `sessionStorage` - нет. HTTP и HTTPS используют общий контекст хранения, и хотя реализована проверка, предотвращающая чтение HTTP-содержимым ключей, созданных HTTPS-скриптами, есть серьезная лазейка: любой ключ, сначала созданный по HTTP, а затем обновленный по HTTPS, останется видимым для незашифрованных страниц. Эта ошибка, впервые сообщенная в 2009 году,^[9] в итоге будет исправлена, но когда - неясно.

Политика безопасности для cookie

Мы обсуждали семантику HTTP-cookie в главе 3, но это обсуждение опустило одну важную деталь: правила безопасности, которые должны быть реализованы, чтобы защитить cookie, принадлежащие одному сайту, от вмешательства несвязанных страниц. Эта тема особенно интересна, потому что выбранный здесь подход предшествует политике одного источника и взаимодействует с ней рядом неожиданных способов.

Cookie должны быть ограничены доменами, и их нельзя легко ограничить только одним значением имени хоста. Параметр `domain`, предоставленный с cookie, может просто совпадать с текущим `hostname`, например `foo.example.com`, но это не мешает cookie отправляться любым возможным поддоменам, таким как `bar.foo.example.com`. Однако можно указать квалифицированный правый фрагмент `hostname`, например `example.com`, чтобы запросить более широкую область.

Забавно, что исходные RFC подразумевают, что инженеры Netscape хотели разрешить cookie, ограниченные точным хостом, но не последовали собственному совету. Синтаксис, придуманный для этой цели, не распознавался потомками Netscape Navigator, да и любой другой реализацией тоже. В ограниченной степени установка `host-scoped` cookie возможна в некоторых браузерах путем полного пропуска параметра `domain`, но этот метод не даст эффекта в Internet Explorer.

Таблица 9-3 иллюстрирует поведение установки cookie в нескольких характерных случаях.

Таблица 9-3. Примеры поведения установки cookie

Cookie установлена на foo.example.com, параметр domain	Область cookie в браузерах не IE	Область cookie в Internet Explorer
значение пропущено	foo.example.com точно	*.foo.example.com
bar.foo.example.com	Cookie не установлена: домен более специфичен, чем origin	Cookie не установлена: домен более специфичен, чем origin
foo.example.com	*.foo.example.com	*.foo.example.com
baz.example.com	Cookie не установлена: несовпадение домена	Cookie не установлена: несовпадение домена
example.com	*.example.com	*.example.com
ample.com	Cookie не установлена: несовпадение домена	Cookie не установлена: несовпадение домена
.com	Cookie не установлена: домен слишком широк, риск безопасности	Cookie не установлена: домен слишком широк, риск безопасности

Единственный другой настоящий параметр ограничения cookie - префикс пути: любую cookie можно установить с указанным значением path. Это приказывает браузеру отправлять cookie обратно только на запросы к совпадающим каталогам; cookie, ограниченная доменом example.com и путем /some/path/, будет включена в запрос к:

`http://foo.example.com/some/path/subdirectory/hello_world.txt`

Этот механизм может обманывать. Пути URL не учитываются при проверках политики одного источника и поэтому не образуют полезной границы безопасности. Независимо от того, как работают cookie, JavaScript-код может просто переходить между любыми URL на одном хосте по своему желанию и внедрять вредоносные полезные нагрузки в такие цели, злоупотребляя любой функциональностью, защищенной cookie, ограниченными путем. Некоторые книги и white papers по безопасности до сих пор рекомендуют path scoring как меру безопасности. В большинстве случаев этот совет мертво неверен.

Помимо настоящих возможностей ограничения, которые вместе с именем cookie составляют кортеж, однозначно идентифицирующий каждую cookie, веб-серверы также могут выводить cookie с двумя специальными, независимо действующими флагами: httponly и secure. Первый, httponly, предотвращает доступ к cookie через API document.cookie в надежде затруднить простое копирование учетных данных пользователя после успешного внедрения вредоносного скрипта на страницу. Второй, secure, прекращает отправку cookie в запросах по незашифрованным протоколам, что позволяет строить HTTPS-сервисы, устойчивые к активным атакам.^[10]

Ловушка этих механизмов в том, что они защищают данные только от чтения, но не от перезаписи. Например, JavaScript-код, доставленный по HTTP, все еще может просто переополнить per-domain cookie jar и затем установить новую cookie без флага secure.^[11] Поскольку заголовок Cookie, отправляемый браузером, не предоставляет метаданных о происхождении конкретной cookie или ее области, такой трюк очень трудно обнаружить. Заметное следствие этого поведения в том, что распространенный "stateless" способ предотвращения уязвимостей CSRF путем одновременного хранения секретного токена в клиентской cookie и в скрытом поле формы, а затем сравнения этих двух значений, не особенно безопасен для HTTPS-сайтов. Посмотрите, сможете ли понять почему!

Примечание. К слову о разрушительном вмешательстве: до 2010 года cookie httponly также конфликтовали с XMLHttpRequest. Авторы этого API просто не подумали специально о том, должна ли функция XMLHttpRequest.getResponseHeader(...) иметь возможность

инспектировать предоставленные сервером значения Set-Cookie, помеченные как httponly, с предсказуемыми результатами.

Влияние cookie на политику одного источника

Политика одного источника имеет некоторое нежелательное влияние на безопасность cookie, конкретно на механизм ограничения путями, но обратное взаимодействие встречается чаще и более проблемно. Трудность в том, что HTTP-cookie часто работают как учетные данные, и в таких случаях способность получить их примерно эквивалентна нахождению способа обойти SOP. Проще говоря, с правильным набором cookie злоумышленник мог бы использовать собственный браузер для взаимодействия с целевым сайтом от имени жертвы; политика одного источника выводится из картины, и все ставки сняты.

Из-за этого свойства любые расхождения между двумя механизмами безопасности могут привести к неприятностям для более ограничительного. Например, относительно распушенные правила domain scoring, используемые HTTP-cookie, означают, что невозможно полностью изолировать чувствительное содержимое, размещенное на `webmail.example.com`, от менее доверенного HTML, присутствующего на `blog.example.com`. Даже если владельцы `webmail`-приложения жестко ограничат свои cookie, обычно ценой усложнения процесса входа, любой злоумышленник, нашедший уязвимость внедрения скрипта на блоговом сайте, может просто переполнить `per-domain cookie jar`, сбросить текущие учетные данные и установить собственные cookie `*.example.com`. Эти внедренные cookie будут отправляться на `webmail.example.com` во всех последующих запросах и будут в значительной степени неотличимы от настоящих.

Этот трюк может казаться безвредным, пока вы не осознаете, что такое действие может фактически войти жертвой в поддельную учетную запись и что в результате некоторые действия, например отправка email, могут быть непреднамеренно записаны внутри этой учетной записи и утечь злоумышленнику до того, как будет замечено что-то неладное. Если `webmail` звучит слишком экзотично, представьте то же самое на Amazon или Netflix: ваши случайные поиски товаров могут быть раскрыты злоумышленнику до того, как вы заметите что-то необычное на сайте. Вдобавок многие сайты просто не готовы обрабатывать вредоносные полезные нагрузки во внедренных cookie, и неожиданные входы могут привести к XSS или похожим ошибкам.

Выходки HTTP-cookie также очень затрудняют защиту зашифрованного трафика от злоумышленников сетевого уровня. Secure-cookie, установленная `https://webmail.example.com/`, все еще может быть затерта и заменена вымышленным значением, установленным поддельной страницей на `http://webmail.example.com/`, даже если на порту 80 целевого хоста вообще нет реального веб-сервиса.

Проблемы с доменными ограничениями

Ошибочная идея разрешить cookie на уровне домена также создает проблемы для поставщиков браузеров и является постоянным источником страданий. Ключевой вопрос - как надежно помешать `example.com` установить cookie для `*.com` и избежать неожиданной отправки этой cookie всем остальным адресатам в интернете.

На ум приходит несколько простых решений, но они разваливаются, когда приходится учитывать TLD уровня стран: `example.com.pl` тоже нужно запретить устанавливать cookie `*.com.pl`. Осознав это, исходная спецификация cookie Netscape дала следующий совет:

Только хосты внутри указанного домена могут устанавливать cookie для домена, и домены должны иметь по крайней мере две или три точки, чтобы предотвратить домены формы ".com", ".edu" и "va.us".

Любой домен, попадающий в один из семи специальных доменов верхнего уровня, перечисленных ниже, требует только двух точек. Любой другой домен требует как минимум трех. Семь специальных доменов верхнего уровня: "COM", "EDU", "NET", "ORG", "GOV", "MIL" и "INT".

Увы, правило трех точек имеет смысл только для регистраторов уровня стран, зеркалирующих иерархию верхнего уровня (example.co.uk), но не для столь же многочисленной группы стран, принимающих прямые регистрации (example.fr). Фактически есть места, где разрешены оба подхода; например, и example.jp, и example.co.jp вполне допустимы.

Из-за оторванности этого совета от реальности большинство браузеров проигнорировали его и вместо этого реализовали мозаику условных выражений, которые только привели к новым неприятностям. В одном случае более десяти лет можно было фактически устанавливать cookie для *.com.pl. Комплексные исправления обработки country-code top-level domains были поставлены во все современные браузеры за последние четыре года, но на момент написания они не были перенесены в Internet Explorer 6 и 7 и, вероятно, никогда не будут.

Примечание. Чтобы добавить неприятностей, Internet Assigned Numbers Authority за последние годы добавила немалое число доменов верхнего уровня, например .int и .biz, и рассматривает предложение разрешить произвольные регистрации generic top-level domain. Если до этого дойдет, cookie, вероятно, придется проектировать заново с нуля.

Необычная опасность localhost

Одно немедленно очевидное следствие существования domain-level scoping у cookie состоит в том, что довольно небезопасно делегировать любые hostname внутри чувствительного домена любой недоверенной или просто уязвимой стороне; это может затронуть конфиденциальность и неизменно целостность любых учетных данных, хранящихся в cookie, а следовательно, и любой другой информации, обрабатываемой целевым приложением.

Это очевидно, но в 2008 году Тэвис Орманди заметил нечто куда менее интуитивное и куда более смешное:^[12] из-за port-agnostic поведения HTTP-cookie дополнительная опасность кроется в довольно популярной и удобной административной практике добавления записи localhost к домену и направления ее на 127.0.0.1.^[13] Когда Орманди впервые опубликовал свое уведомление, он утверждал, что такая практика широко распространена, что не было спорным утверждением, и включил следующий вывод resolver tool, чтобы проиллюстрировать свою мысль:

```
localhost.microsoft.com has address 127.0.0.1
localhost.ebay.com has address 127.0.0.1
localhost.yahoo.com has address 127.0.0.1
localhost.fbi.gov has address 127.0.0.1
localhost.citibank.com has address 127.0.0.1
localhost.cisco.com has address 127.0.0.1
```

Почему это было бы риском безопасности? Проще говоря, это помещает HTTP-сервисы на собственной машине пользователя в тот же домен, что и остаток сайта, и, что важнее, помещает все сервисы, которые только выглядят как HTTP, в то же самое ведро. Такие сервисы обычно не открыты в интернет, поэтому нет воспринимаемой необходимости проектировать их осторожно или поддерживать в актуальном состоянии. Конкретный пример Тэвиса - сервис управления принтерами, предоставляемый CUPS (Common UNIX Printing System), который выполнил бы JavaScript, предоставленный злоумышленником, в контексте `example.com`, если вызвать его следующим образом:

```
http://localhost.example.com:631/jobs/?[...]  
&job_printer_uri=javascript:alert("Hi mom!")
```

Уязвимость в CUPS можно исправить, но на всех операционных системах, вероятно, есть множество других сомнительных локальных сервисов, от инструментов управления дисками до панелей статуса антивируса. Введение записей, указывающих обратно на `127.0.0.1` или любые другие адреса, которые вы не контролируете, связывает безопасность cookie внутри вашего домена с безопасностью случайного стороннего ПО. Этого хорошо бы избегать.

Cookie и "легитимный" DNS hijacking

Опасности domain-scoping политики для cookie не заканчиваются на localhost. Другое непреднамеренное взаимодействие связано с распространенной и широко критикуемой практикой некоторых ISP и других поставщиков DNS-сервисов перехватывать доменные lookup для несуществующих, обычно ошибочно набранных, хостов. В этой схеме вместо возвращения предписанного стандартом ответа NXDOMAIN от вышестоящего name server, что затем вызвало бы сообщение об ошибке в браузере или другом сетевом приложении, поставщик подделывает запись, подразумевая, что это имя разрешается в его сайт. Его сайт, в свою очередь, изучает заголовок Host, предоставленный браузером, и показывает пользователю нежелательную платную контекстную рекламу, которая выглядит смутно связанной с его интересами просмотра. Обычное оправдание такой практики - предложение более удобного опыта просмотра; настоящий стимул, конечно, заработать больше денег.

Среди интернет-провайдеров, полагавшихся на эту практику, были Cablevision, Charter, Earthlink, Time Warner, Verizon и многие другие. К сожалению, их подход не только морально сомнителен, но и создает существенный риск безопасности. Если рекламный сайт содержит любые уязвимости внедрения скрипта, злоумышленник может эксплуатировать их в контексте любого другого домена, просто обратившись к уязвимой функциональности через адрес вроде `nonexistent.example.com`. В сочетании с устройством HTTP-cookie эта практика подрывает безопасность любых произвольно выбранных сервисов в интернете.

Предсказуемо, уязвимости внедрения скрипта можно найти в таких поспешно спроектированных рекламных ловушках без особых усилий. Например, в 2008 году Дэн Камински обнаружил и опубликовал уязвимость межсайтового скриптинга на страницах, управляемых Earthlink.^[14]

Ладно, ладно: пора перестать заикливаться на cookie и двигаться дальше.

Правила безопасности плагинов

Браузеры не предоставляют разработчикам плагинов единый и расширяемый API для применения политик безопасности; вместо этого каждый плагин решает, какие правила должны применяться к выполняемому содержимому и как приводить их в действие. Следовательно, хотя модели безопасности плагинов в некоторой степени вдохновлены политикой одного источника, они расходятся с ней рядом способов.

Этот разрыв может быть опасен. В главе 6 мы обсуждали склонность плагинов полагаться на инспекцию JavaScript-объекта `location`, чтобы определить `origin` своей размещающей страницы. Эта ошибочная практика заставила разработчиков браузеров ограничить способность программ JavaScript вмешиваться в некоторые части своей среды выполнения, чтобы спасти положение. Другой связанный, распространенный источник несовместимостей - интерпретация URL. Например, в середине 2010 года один исследователь обнаружил, что Adobe Flash испытывал трудности со следующим URL:^[15]

`http://example.com:80@bunnyoutlet.com/`

Плагин решил, что `origin` любого кода, полученного через этот URL, должен быть установлен в `example.com`, но браузер, получив такой URL, естественно, получил бы данные с `bunnyoutlet.com`, а затем передал бы их сбиту с толку плагину для выполнения.

Хотя эта конкретная ошибка теперь исправлена, другие уязвимости такого типа, вероятно, можно ожидать в будущем. Воспроизведение некоторых особенностей разбора URL, обсуждавшихся в главах 2 и 3, может быть глупой затеей и в идеале вообще не должно предприниматься.

Было бы невежливо заканчивать эту главу на такой мрачной ноте. Оставив системные проблемы в стороне, посмотрим, как некоторые из самых популярных плагинов подходят к задаче применения политики безопасности.

Adobe Flash

Модель безопасности Flash прошла крупную переработку в 2008 году,^[16] и с тех пор она стала достаточно надежной. Каждому загруженному Flash-апплету теперь назначается SOP-подобный `origin`, выведенный из его исходного URL,^[17] и предоставляются номинальные разрешения, связанные с `origin`, примерно сопоставимые с разрешениями JavaScript. В частности, каждый апплет может загружать аутентифицированное cookie содержимое со своего исходного сайта, загружать некоторые ограниченные типы данных из других источников и выполнять `same-origin XMLHttpRequest`-подобные HTTP-вызовы через API `URLRequest`. Набор допустимых методов и заголовков запросов для этого последнего API управляется довольно разумно и, на момент написания, более ограничен, чем большинство браузерных черных списков для самого `XMLHttpRequest`.^[18]

Поверх этой разумной базы три гибких, но легко неверно используемых механизма позволяют в некоторой степени изменить такое поведение, как обсуждается далее.

Средства безопасности уровня разметки

Встраивающая страница может указать три специальных параметра, предоставленных через теги `<embed>` или `<object>`, чтобы контролировать, как апплет будет взаимодействовать со своей страницей-хозяином и самим браузером:

- Параметр `AllowScriptAccess`. Эта настройка контролирует способность апплета использовать мост `JavaScript ExternalInterface.call(...)` (см. главу 8), чтобы выполнять инструкции `JavaScript` в контексте встраивающего сайта. Возможные значения - `always`, `never` и `sameorigin`; последняя настройка дает доступ к странице только если страница является одним источником с самим апплетом. До переработки безопасности 2008 года плагин по умолчанию использовал `always`; текущий `default` - намного более безопасный `sameorigin`.
- Параметр `AllowNetworking`. Эта плохо названная настройка ограничивает разрешение апплета открывать или направлять окна браузера и выполнять HTTP-запросы к своему исходному серверу. Когда установлено `all`, значение по умолчанию, апплет может вмешиваться в браузер; когда установлено `internal`, он может выполнять только неразрушающие внутренние коммуникации через плагин `Flash`. Установка этого параметра в `none` отключает большинство сетевых API целиком.^[19] До недавних улучшений безопасности `allowNetworking=all` открывал несколько способов обойти `allowScriptAccess=none`, например вызовом `getURL(...)` на `URL javascript:.` Однако на момент написания все scripting URLs в этом сценарии должны быть в черном списке.
- Параметр `AllowFullScreen`. Этот параметр контролирует, следует ли разрешить апплету переходить в полноэкранный режим рендеринга. Возможные значения - `true` и `false`, по умолчанию `false`. Как отмечалось в главе 8, решение дать `Flash`-апплетам эту возможность проблематично из-за рисков подделки UI; ее не следует включать без настоящей необходимости.

Security.allowDomain(...)

Метод `Security.allowDomain(...)`^[20] позволяет `Flash`-апплетам предоставлять доступ к своим переменным и функциям любому `JavaScript`-коду или другим апплетам, приходящим из другого `origin`. Покупатель, будь осторожен: как только такой доступ предоставлен, надежного способа сохранить целостность исходного контекста выполнения `Flash` уже нет. Решение предоставлять такие разрешения не следует принимать легкомысленно, а практику вызова `allowDomain("*")` обычно следует сурово наказывать.

Обратите внимание, что доступен и странно названный метод `allowInsecureDomain(...)`. Существование этого метода не означает, что `allowDomain(...)` особенно безопасен; скорее, "insecure" вариант предоставлен для совместимости с древней, до-2003 семантикой, полностью игнорировавшей разделение HTTP/HTTPS.

Файлы междоменной политики

С помощью `loadPolicyFile(...)` любой Flash-апплет может приказать своей runtime-среде получить файл политики безопасности почти с произвольного URL. Этот XML-документ, обычно называемый `crossdomain.xml`, будет интерпретирован как выражение согласия на междоменный доступ уровня сервера к `origin`, определенному путем изучения URL политики.^[21] Синтаксис файла политики довольно самоочевиден и может выглядеть так:

```
<cross-domain-policy>
  <allow-access-from domain="foo.example.com"/>
  <allow-http-request-headers-from domain="*.example.com"
    headers="X-Some-Header" />
</cross-domain-policy>
```

Политика может разрешать действия вроде загрузки `cross-origin` ресурсов или выполнения произвольных `URLRequest`-вызовов с заголовками из белого списка через HTTP-стек браузера. Разработчики Flash пытаются применять некоторую степень разделения путей: политика, загруженная из конкретного подкаталога, в принципе может разрешить доступ только к файлам внутри этого пути. На практике, однако, взаимодействия с SOP и с различными семантиками сопоставления путей современных браузеров и фреймворков веб-приложений делают опору на эту границу неразумной.

Примечание. Создание сырых TCP-соединений через `XMLSocket` тоже возможно и контролируется XML-политикой, но после переработки Flash 2008 года `XMLSocket` требует, чтобы отдельный файл политики был доставлен на TCP-порту 843 целевого сервера. Это довольно безопасно, потому что на этом порту не работают другие распространенные сервисы, а во многих операционных системах только привилегированные пользователи могут запускать сервисы на портах ниже 1024. Из-за взаимодействий с некоторыми механизмами уровня `firewall`, такими как `FTP protocol helpers`, эта конструкция все еще может вызывать некоторое вмешательство на сетевом уровне,^[22] но эта тема твердо выходит за рамки этой книги.

Как и ожидалось, плохо настроенные политики `crossdomain.xml` - ощутимый риск безопасности. В частности, очень плохая идея указывать правила `allow-access-from`, направленные на любой домен, в котором вы не полностью уверены. Более того, указание `*` как значения этого параметра примерно эквивалентно выполнению `document.domain = "com"`. То есть это желание смерти.

Риски подделки файлов политики

Помимо возможности ошибок конфигурации, другой риск безопасности в основанной на политиках модели Adobe состоит в том, что случайные контролируемые пользователем документы могут быть интерпретированы как междоменные политики вопреки намерению владельца сайта.

До 2008 года Flash использовал печально известный слабый парсер политик, который при обработке файлов `loadPolicyFile(...)` пропускал произвольный мусор в начале в поисках открывающего тега `<cross-domain-policy>`. Он также просто игнорировал MIME-тип, возвращенный сервером при скачивании ресурса. В результате простое размещение действительного JPEG-изображения, предоставленного пользователем, могло стать серьезным риском безопасности. Плагин также перескакивал через любые HTTP-redirects, делая опасным даже такое простое действие, как выдача HTTP-redirect в место, которое вы не контролируете, иначе безвредный акт.

После столь нужной переработки поведения `loadPolicyFile` многие грубые ошибки были исправлены, но значения по умолчанию все еще не идеальны. С одной стороны, `redirects` теперь

работают интуитивно, а файл должен быть правильно сформированным XML-документом. С другой, допустимые MIME-типы включают `text/*`, `application/xml` и `application/xhtml+xml`, что кажется немного слишком широким. `text/plain` или `text/csv` могут быть ошибочно интерпретированы как файл политики, а так быть не должно.

К счастью, чтобы снизить проблему, инженеры Adobe решили внедрить `meta-policies`, политики, размещенные в заранее определенном местоположении верхнего уровня (`/crossdomain.xml`), которое злоумышленник не может переопределить. `Meta-policy` может задавать общесайтовые ограничения для всех остальных политик, загруженных с URL, предоставленных злоумышленником. Самое важное из этих ограничений - `<site-control permitted-cross-domain-policies="...">`. Этот параметр, установленный в `master-only`, просто приказывает плагину игнорировать `subpolicies` целиком. Другое, менее радикальное значение, `by-content-type`, разрешает загружать дополнительные политики, но требует, чтобы у них был недвусмысленный заголовок `Content-Type`, установленный в `text/x-cross-domain-policy`.

Нет нужды говорить, что очень рекомендуется использовать `meta-policy`, задающую одну из этих двух директив.

Microsoft Silverlight

Если переход от Flash к Silverlight кажется резким, то потому, что их легко спутать. Плагин Silverlight заимствует у Flash с замечательным рвением; фактически можно сказать, что большинство различий между их моделями безопасности обусловлено исключительно номенклатурой.

Платформа Microsoft использует тот же подход к определению `origin`, заменяет `allowScriptAccess` на `enableHtmlAccess`, заменяет `crossdomain.xml` слегка отличающимся синтаксисом `clientaccesspolicy.xml`, предоставляет `API System.Net.Sockets` вместо `XMLSocket`, использует `HttpRequest` вместо `URLRequest`, переставляет цветы и меняет шторы в гостиной.

Сходства поразительны, вплоть до списка заблокированных заголовков запроса для `API HttpRequest`, который даже включает `X-Flash-Version` из спецификации Adobe.^[23] Такая согласованность не проблема: фактически она предпочтительнее необходимости учитывать совершенно новую модель безопасности. Плюс, к чести Microsoft, компания внесла пару желанных улучшений, включая отказ от небезопасной логики `allowDomain` в пользу `RegisterScriptableObject`, подхода, который позволяет открывать сторонним доменам только явно указанные обратные вызовы.

Java

Java от Sun, теперь официально принадлежащая Oracle, - очень любопытный случай. Java - плагин, вышедший из употребления, а его архитектура безопасности за последнее десятилетие или около того не получала большого внимания. Тем не менее из-за большой установленной базы трудно просто игнорировать его и двигаться дальше.

К сожалению, чем пристальнее смотришь, тем очевиднее, что идеи, принятые Java, имеют тенденцию быть несовместимыми с современным Web. Например, класс `java.net.HttpURLConnection`^[24] позволяет выполнять HTTP-запросы с учетными данными к исходному сайту апплета, но "исходный сайт" понимается как любой сайт, размещенный на конкретном IP-адресе, как санкционировано проверкой `java.net.URL.equals(...)`. Эта модель по сути отменяет любую изоляцию между виртуальными хостами HTTP/1.1 - изоляцию, строго применяемую политикой одного источника, HTTP-cookie и практически всеми другими механизмами браузерной безопасности, используемыми сегодня.

Двигаясь дальше в этом направлении, класс `java.net.URLConnection`^[25] позволяет апплету устанавливать произвольные заголовки запроса, включая `Host`, а другой класс, `Socket`,^[26] разрешает неограниченные TCP-соединения к произвольным портам исходного сервера. Все эти поведения не одобряются в браузере и в любом другом современном плагине.

Origin-agnostic доступ от апплета к встраиваемой странице предоставляется через механизм `JSObject` и, как ожидается, контролируется встраиваемой стороной через атрибут `mayscript`, указанный в тегах `<applet>`, `<embed>` или `<object>`.^[27] Документация предполагает, что это возможность безопасности:

По соображениям безопасности поддержка `JSObject` не включена в Java Plug-in по умолчанию. Чтобы включить поддержку `JSObject` в Java Plug-in, в теге `EMBED/OBJECT` должен присутствовать новый атрибут `MAYSCRIPT`.

К сожалению, документация забывает упомянуть, что другой тесно связанный механизм, `DOMService`,^[28] игнорирует эту настройку и дает апплетам в значительной степени неограниченный доступ к встраиваемой странице. Хотя `DOMService` не поддерживается в Firefox и Opera, он доступен в других браузерах, что делает любую попытку загрузить стороннее Java-содержимое эквивалентной предоставлению полного доступа к встраиваемому сайту.

Упс.

Примечание. Интересный факт: недавние версии Java пытаются копировать поддержку `crossdomain.xml`, доступную во Flash.

Как жить с неоднозначными или неожиданными источниками

На этом завершается наш обзор базовых политик безопасности и механизмов изоляции согласия. Если есть одно наблюдение, которое стоит сделать, то оно такое: большинство этих механизмов зависит от наличия правильно сформированного, канонического `hostname`, из которого можно вывести контекст для всех последующих операций. Но что если эта информация недоступна или представлена не в ожидаемой форме?

Вот тогда начинается веселье. Давайте посмотрим на некоторые распространенные крайние случаи, пусть даже только для мимолетного развлечения.

IP-адреса

Из-за того, что IP-адреса не были учтены при проектировании HTTP-cookie и политики одного источника, почти все браузеры исторически разрешали документам, загруженным, скажем, с `http://1.2.3.4/`, устанавливать cookie для "домена" с именем `*.3.4`. Настройка `document.domain` похожим образом тоже работала бы. Фактически некоторые из этих поведений все еще присутствуют в старых версиях Internet Explorer.

Такое поведение вряд ли повлияет на массовые веб-приложения, потому что такие приложения не предназначены для доступа через IP-based URL и часто просто не будут правильно работать. Но горстка систем, используемых прежде всего техническим персоналом, предназначена для доступа по IP-адресам; у этих систем может вообще не быть настроенных DNS-записей. В таких случаях способность `http://1.2.3.4/` внедрять cookie для `http://123.234.3.4/` может быть проблемой. Административные интерфейсы домашних роутеров, доступные по IP, тоже представляют некоторый интерес.

Hostname с лишними точками

В своей основе алгоритмы установки cookie все еще зависят от подсчета числа точек в URL, чтобы определить, приемлем ли конкретный параметр domain. Чтобы принять решение, счет обычно сопоставляется со списком из нескольких сотен записей в поддерживаемом поставщиком Public Suffix List (<http://publicsuffix.org/>).

К несчастью для этого алгоритма, часто можно поставить лишние точки в hostname и все равно заставить его разрешаться правильно. Неканонические представления hostname с лишними точками обычно учитываются резолверами уровня OS и, если учитываются, запутают браузер. Хотя такой браузер не будет автоматически считать домен вроде `www.example.com.pl` с лишней точкой в конце тем же самым, что настоящий `www.example.com.pl`, тонкое и, казалось бы, безвредное отличие в URL может ускользнуть даже от самых внимательных пользователей.

В таком случае взаимодействие с URL с завершающей точкой может быть небезопасным, поскольку другие документы, разделяющие домен `*.com.pl`, могут с относительной легкостью внедрять междоменные cookie.

Эта проблема подсчета точек была впервые замечена примерно в 1998 году.^[29] Примерно через десятилетие многие поставщики браузеров решили внедрить базовые смягчения, добавив еще один специальный случай в соответствующий код; на момент написания Орега все еще подвержена этому трюку.

Не полностью квалифицированные hostname

Многие пользователи просматривают Web с DNS-резолверами, настроенными добавлять локальные суффиксы ко всем найденным hostname, часто не зная об этом. Такие настройки обычно санкционируются ISP или работодателями через данные автоматической сетевой конфигурации (Dynamic Host Configuration Protocol, DHCP).

Для любого пользователя, просматривающего с такой настройкой, разрешение DNS labels неоднозначно. Например, если путь поиска DNS включает `coredump.cx`, то `www.example.com` может разрешиться в настоящий сайт `www.example.com` или в `www.example.com.coredump.cx`, если такая запись существует. Результаты частично контролируются настройками конфигурации и в некоторой степени могут быть подвержены влиянию злоумышленника.

Для браузера оба местоположения выглядят одинаково, что может иметь интересные побочные эффекты. Рассмотрим один особенно извращенный случай: должен ли `http://com`, который на самом деле разрешается в `http://com.coredump.cx/`, иметь возможность установить cookie `*.com`, просто пропустив параметр domain?

Локальные файлы

Поскольку локальные ресурсы, загруженные через протокол `file:`, не имеют связанного с ними явного `hostname`, браузер не может вычислить нормальный `origin`. Очень долго поставщики просто решили, что лучший образ действий в таком случае - отказаться от политики одного источника. Таким образом, любой HTML-документ, сохраненный на диск, автоматически получал доступ к любым другим локальным файлам через XMLHttpRequest или DOM и, еще более необъяснимо, мог таким же способом получать доступ к любому содержимому из интернета.

Это оказалось ужасным проектным решением. Никто не ожидал, что простой акт скачивания HTML-документа поставит под угрозу все локальные файлы пользователя и его онлайн-учетные данные. В конце концов, доступ к тому же документу через Web был бы совершенно безопасен.

Многие браузеры в последние годы пытались закрыть эту лазейку с разной степенью успеха:

Chrome и, как следствие, другие WebKit-браузеры полностью запрещают любой междокументный доступ DOM или XMLHttpRequest из `file: origins` и игнорируют вызовы `document.cookie` или директивы `<meta http-equiv="Set-Cookie" ...>` в этой среде. Доступ к контейнеру `localStorage`, общему для всех `file: документов`, разрешен, но вскоре это может измениться.

Firefox - браузер Mozilla разрешает доступ только к файлам внутри каталога исходного документа, а также близлежащим подкаталогам. Эта политика довольно хороша, но все равно создает некоторый риск для документов, сохраненных или ранее скачанных в это место. Доступ к `cookie` через `document.cookie` или `<meta http-equiv="Set-Cookie" ...>` возможен, и все `file: cookie` видимы любому другому локальному JavaScript-коду.^[30] То же верно для доступа к механизмам хранения.

Internet Explorer 7 и выше - неограниченный доступ к локальному и интернет-содержимому из `file: origins` разрешен, но сначала требует от пользователя щелкнуть по неспецифичному предупреждению, чтобы выполнить JavaScript. Последствия этого действия не объясняются ясно: подсистема помощи загадочно утверждает, что "Internet Explorer restricts this content because occasionally these programs can malfunction or give you content you don't want", и многих пользователей вполне можно обманом заставить пройти через подсказку.

Семантика `cookie` в Internet Explorer похожа на Firefox. Однако Web storage в этом `origin` не поддерживается.

Opera и Internet Explorer 6 разрешают неограниченный доступ DOM или XMLHttpRequest без дальнейших проверок. Некомпартиментализированные `file: cookie` тоже разрешены.

Примечание. Плагины живут по собственным правилам в стране `file:`. Flash использует модель песочницы `local-with-filesystem`,^[31] которая дает в значительной степени неограниченный доступ к локальной файловой системе независимо от политики, применяемой самим браузером, тогда как выполнение Java- или Windows Presentation Framework-апплетов из локальной файловой системы в некоторых случаях может быть примерно эквивалентно запуску недоверенного бинарного файла.

Псевдо-URL

Поведение псевдо-URL, таких как `about:`, `data:` или `javascript:`, изначально составляло значительную лазейку в реализациях политики одного источника. Все такие URL считались бы одним источником и разрешали бы неограниченный междоменный доступ из любого другого ресурса, загруженного по той же схеме. Текущее поведение, которое очень отличается, будет темой следующей главы этой книги; вкратце, статус-кво отражает несколько раундов поспешно реализованных улучшений и является сложной смесью специфичных для браузеров специальных случаев и правил наследования `origin`.

Браузерные расширения и UI

Несколько браузеров позволяют UI-элементам на JavaScript или некоторым установленным пользователем браузерным расширениям работать с повышенными привилегиями. Эти привилегии могут включать обход конкретных проверок SOP или вызов обычно недоступных API, чтобы писать файлы, изменять настройки конфигурации и так далее.

Привилегированный JavaScript - заметная возможность Firefox, где он используется с XUL для построения больших частей пользовательского интерфейса браузера. Chrome тоже полагается на привилегированный JavaScript в меньшей, но все еще заметной степени.

Политика одного источника не поддерживает привилегированные контексты каким-либо специальным образом. Фактический механизм, с помощью которого предоставляются дополнительные привилегии, может включать загрузку документа через специальную и обычно недоступную схему URL, такую как `chrome:` или `res:`, а затем добавление специальных случаев для этой схемы в других частях кода браузера. Другой вариант - просто переключить бинарный флаг для JavaScript-контекста независимо от его фактического `origin` и позднее проверять этот флаг. Во всех случаях поведение стандартных API, таких как `localStorage`, `document.domain` или `document.cookie`, может быть трудно предсказать, и полагаться на него не следует: некоторые браузеры пытаются поддерживать изоляцию между контекстами, принадлежащими разным расширениям, но большинство - нет.

Примечание. При написании браузерных расширений любое взаимодействие с непривилегированными контекстами должно выполняться с крайней осторожностью. Исследовать недоверенные контексты может быть трудно, а использование механизмов вроде `eval(...)` или `innerHTML` может открыть пути повышения привилегий.

Другие применения источников

Что ж, пока это все, что нужно сказать о браузерной логике изоляции содержимого. Возможно, стоит отметить, что концепция `origins` и `security mechanisms` на основе хоста или домена не ограничивается этой конкретной задачей и много раз появляется в мире браузеров. Другие квази-`origin-based` возможности приватности или безопасности включают настройки и кэшированную информацию, связанные с обработкой cookie по сайтам, блокировкой всплывающих окон, предоставлением геолокации, управлением паролями, доступом к камере и микрофону во Flash и многим, многим другим. Эти возможности имеют тенденцию хотя бы в некоторой степени взаимодействовать с функциями безопасности, описанными в этой главе; вскоре мы исследуем эту тему подробнее.

Памятка по инженерии безопасности

Хорошая гигиена политики безопасности для всех сайтов

Чтобы защитить пользователей, включайте файл `crossdomain.xml` верхнего уровня с параметром `permitted-cross-domain-policies`, установленным в `master-only` или `by-content-type`, даже если вы нигде на своем сайте не используете Flash. Это предотвратит ошибочную интерпретацию несвязанного содержимого, контролируемого злоумышленником, как вторичного файла `crossdomain.xml`, что фактически подрвало бы гарантии политики одного источника в браузерах с Flash.

При опоре на HTTP-cookie для аутентификации

- Используйте флаг `httponly`; проектируйте приложение так, чтобы JavaScript не нужно было обращаться к аутентификационным cookie напрямую. Чувствительные cookie следует ограничивать как можно жестче, предпочтительно вообще не указывая `domain`.
- Если приложение должно быть только HTTPS, cookie должны быть помечены как `secure`, и вы должны быть готовы корректно обрабатывать внедрение cookie. HTTP-контексты могут перезаписывать `secure-cookie`, хотя не могут их читать. Криптографическое подписание cookie может помочь защититься от неограниченного изменения, но не защищает от замены cookie жертвы другим набором законно полученных учетных данных.

При организации междоменных коммуникаций в JavaScript

- Не используйте `document.domain`. Полагайтесь на `postMessage(...)` там, где возможно, и обязательно правильно указывайте `destination origin`; затем проверяйте `origin` отправителя при получении данных на другом конце. Остерегайтесь наивных `substring matches` для доменных имен: `msg.origin.indexOf(".example.com")` очень небезопасно.
- Обратите внимание, что разные `pre-postMessage` трюки обхода SOP, такие как опора на `window.name`, не защищены от вмешательства и не должны использоваться для обмена чувствительными данными.

При встраивании активного содержимого третьих сторон, обрабатываемого плагинами

Сначала обратитесь к памятке в главе 8 за общими советами.

- Flash: не указывайте `allowScriptAccess=always`, если полностью не доверяете владельцу исходного домена и безопасности его сайта. Не используйте эту настройку при встраивании HTTP-апплетов на HTTPS-страницах. Также рассмотрите ограничение `allowFullScreen` и `allowNetworking` по необходимости.
- Silverlight: не указывайте `enableHtmlAccess=true`, если не доверяете исходному домену, как выше.

- Java: Java-апплеты нельзя безопасно встраивать из недоверенных источников. Пропуск `mayscript` не полностью предотвращает доступ к встраиваемой странице, поэтому не пытайтесь так делать.

При размещении собственного выполняемого плагинами содержимого

- Обратите внимание, что многие механизмы междоменных коммуникаций, предоставляемые браузерными плагинами, могут иметь непреднамеренные последствия. В частности, избегайте правил `crossdomain.xml`, `clientaccesspolicy.xml` или `allowDomain(...)`, указывающих на домены, которым вы не полностью доверяете.

При написании браузерных расширений

- Избегайте опоры на `innerHTML`, `document.write(...)`, `eval(...)` и другие подверженные ошибкам шаблоны кодирования, которые могут вызвать внедрение кода на сторонних страницах или в привилегированном JavaScript-контексте.
- Не принимайте критичные для безопасности решения путем инспекции недоверенных JavaScript security contexts, поскольку их поведение может быть обманчивым.

Сноски

[1] [\[назад\]](#) Этот и большинство других механизмов браузерной безопасности основаны на DNS-метках, а не на изучении нижележащих IP-адресов. У этого есть любопытное следствие: если IP конкретного хоста меняется, злоумышленник может суметь общаться с новым адресатом через браузер пользователя, возможно совершая злоупотребления и скрывая истинное происхождение атаки, или взаимодействуя с внутренней сетью жертвы, обычно недоступной из-за firewall. Намеренное изменение IP для этой цели известно как DNS rebinding. Браузеры пытаются в некоторой степени смягчать DNS rebinding, например кэшируя результаты DNS lookup на некоторое время (DNS pinning), но эти защиты несовершенны.

[2] [\[назад\]](#) Один значительный источник ошибок политики одного источника - наличие нескольких отдельных процедур разбора URL в коде браузера. Если подход к разбору, используемый в HTTP-стеке, отличается от используемого для определения JavaScript origins, могут возникать проблемы. Safari, в частности, боролся со значительным числом обходов SOP, вызванных патологическими URL, включая многие входы, обсуждавшиеся в главе 2.

[3] [\[назад\]](#) Например, в Internet Explorer одна страница все еще сможет направлять любые другие документы, которые номинально были тем же источником, но стали "изолированы" после установки `document.domain`, на URL `javascript:..`. Это позволяет любому JavaScript выполняться в контексте такого псевдоизолированного домена. Вдобавок ничто, очевидно, не мешает исходной странице просто установить собственный `document.domain` в значение, идентичное целевому, чтобы устранить границу. Другими словами, на способность сделать документ не-одним-источником с другими страницами через `document.domain` не следует полагаться ни для чего даже отдаленно серьезного или значимого для безопасности.

[4] [\[назад\]](#) Подробнее об этом в главе 11, но самый заметный пример - кодирование данных в идентификаторах фрагментов URL. Это возможно потому, что навигация фреймов к новому URL в большинстве случаев не подчиняется ограничениям безопасности, а навигация к URL, где меняется только идентификатор фрагмента, фактически не вызывает перезагрузки страницы. JavaScript во фрейме может просто опрашивать `location.hash` и так обнаруживать входящие сообщения.

[5] [\[назад\]](#) Фактически в прошлом у многих плагинов были проблемы в этой области. Особенно заметно, что Adobe Flash разрешал произвольные междоменные HTTP-заголовки до 2008 года, когда его модель безопасности прошла существенную переработку. До 2011 года тот же плагин страдал от долгоживущей ошибки реализации, из-за которой он повторно отправлял любые пользовательские заголовки на несвязанный сервер после предоставленного злоумышленником HTTP-кода `redirect 307`. Обе проблемы сейчас исправлены, но время от обнаружения до исправления оказалось тревожным.

[6] [\[назад\]](#) Сноска в оригинале указывает на примеры исторических ошибок обработки заголовков в XMLHttpRequest.

[7] [\[назад\]](#) Сноска в оригинале указывает на спецификацию Cross-Origin Resource Sharing.

[8] [\[назад\]](#) Сноска в оригинале указывает на спецификацию Web Storage.

[9] [\[назад\]](#) Сноска в оригинале указывает на отчет об ошибке Firefox sessionStorage.

[10] [\[назад\]](#) Неважно, что `https://webmail.example.com/` предлагается только по HTTPS. Если он использует cookie, не привязанную к зашифрованному протоколу, злоумышленник может просто дождаться, пока жертва перейдет на `http://www.fuzzybunnies.com/`, молча внедрить на эту страницу фрейм, указывающий на `http://webmail.example.com/`, а затем перехватить получившееся TCP-рукопожатие. Браузер затем отправит все cookie `webmail.example.com` по незашифрованному каналу, и на этом игра по сути окончена.

[11] [\[назад\]](#) Даже если эту возможность предотвратить разделением хранилищ для `httponly` и обычных cookie, множественные одноименные, но по-разному ограниченные cookie должны сосуществовать, и они будут отправляться вместе в любых совпадающих запросах. Они не будут сопровождаться полезными метаданными, а их порядок будет неопределенным и специфичным для браузера.

[12] [\[назад\]](#) Сноска в оригинале указывает на публикацию Тэвиса Орманди о localhost в доменах.

[13] [\[назад\]](#) Этот IP-адрес зарезервирован для loopback-интерфейсов; любая попытка подключиться к нему направит вас обратно к сервисам, запущенным на вашей собственной машине.

[14] [\[назад\]](#) Сноска в оригинале указывает на публикацию Дэна Камински об XSS в страницах Earthlink.

- [15] [\[назад\]](#) Сноска в оригинале указывает на отчет 2010 года о разборе URL в Adobe Flash.
- [16] [\[назад\]](#) Сноска в оригинале указывает на переработку модели безопасности Flash 2008 года.
- [17] [\[назад\]](#) В некоторых контекстах Flash может неявно разрешать доступ из HTTPS origins к HTTP origins, но не наоборот. Обычно это безвредно, и поэтому этому не уделяется специального внимания далее в этом разделе.
- [18] [\[назад\]](#) Сноска в оригинале указывает на документацию Flash по ограничениям заголовков URLRequest.
- [19] [\[назад\]](#) Не следует предполагать, что эта настройка предотвращает передачу третьим сторонам любых чувствительных данных, доступных вредоносному апплету. Есть много side channels, которые любой Flash-апплет мог бы использовать для утечки информации сотрудничающей стороне без прямых сетевых запросов. В самом простом и универсальном случае нагрузкой CPU можно манипулировать, чтобы отправлять отдельные биты информации любому одновременно загруженному апплету, который непрерывно измеряет отзывчивость своей runtime-среды.
- [20] [\[назад\]](#) Сноска в оригинале указывает на документацию `Security.allowDomain(...)`.
- [21] [\[назад\]](#) Сноска в оригинале указывает на документацию Adobe по policy files.
- [22] [\[назад\]](#) Сноска в оригинале указывает на взаимодействия XMLHttpRequest с сетевыми механизмами.
- [23] [\[назад\]](#) Сноска в оригинале указывает на список заблокированных заголовков Silverlight XMLHttpRequest.
- [24] [\[назад\]](#) Сноска в оригинале указывает на документацию `java.net.HttpURLConnection`.
- [25] [\[назад\]](#) Сноска в оригинале указывает на документацию `java.net.URLConnection`.
- [26] [\[назад\]](#) Сноска в оригинале указывает на документацию Java Socket.
- [27] [\[назад\]](#) Сноска в оригинале указывает на документацию Java JavaScript и mayscript.
- [28] [\[назад\]](#) Сноска в оригинале указывает на документацию Java DOMService.
- [29] [\[назад\]](#) Сноска в оригинале указывает на раннее обсуждение проблемы hostname с лишними точками.
- [30] [\[назад\]](#) Поскольку между file: cookie нет компартиментализации, небезопасно полагаться на них в законных целях. Некоторые локально установленные HTML-приложения игнорируют этот совет, и, следовательно, их cookie могут быть легко изменены любым скачанным, возможно вредоносным, HTML-документом, просмотренным пользователем.
- [31] [\[назад\]](#) Сноска в оригинале указывает на модель Flash local-with-filesystem.

Глава 10. Наследование источника

Некоторые веб-приложения полагаются на псевдо-URL, такие как `about:`, `javascript:` или `data:`, чтобы создавать HTML-документы, которые не содержат никакого предоставленного сервером содержимого, а вместо этого заполняются данными, полностью построенными на клиентской стороне. Такой подход устраняет задержку, связанную с обычными HTTP-запросами к серверу, и приводит к куда более отзывчивым пользовательским интерфейсам.

К сожалению, исходное видение политики одного источника не учитывало такой сценарий использования. В частности, буквальное применение правил сопоставления протокола, хоста и порта, обсуждавшихся в главе 9, заставило бы каждый документ `about:blank`, созданный на клиентской стороне, иметь источник, отличный от родительской страницы, что помешало бы осмысленно им манипулировать. Более того, все окна `about:blank`, созданные совершенно несвязанными сайтами, принадлежали бы одному источнику и при подходящих обстоятельствах могли бы вмешиваться друг в друга без какого-либо надзора.

Чтобы устранить эту несовместимость клиентских документов с политикой одного источника, браузеры постепенно разработали несовместимые и иногда противоинтуитивные подходы к вычислению синтетического источника и разрешений доступа для псевдо-URL. Понимание этих правил важно само по себе, а также подготовит почву для обсуждения некоторых других исключений SOP в главе 11.

Наследование источника для `about:blank`

Схема `about:` используется в современных браузерах для множества целей, большинство из которых не видны напрямую обычным веб-страницам. Однако документ `about:blank` - интересный специальный случай: этот URL можно использовать для создания минимальной DOM-иерархии, по сути действительного, но пустого документа, в который родительский документ позднее может записать произвольные данные.

Вот пример типичного использования этой схемы:

```
<iframe src="about:blank" name="test"></iframe>
<script>
...
  frames["test"].document.body.innerHTML = "<h1>Hi mom!</h1>";
...
</script>
```

Примечание. В HTML-разметке, приведенной в этом примере, и при создании новых окон или фреймов вообще, `about:blank` можно опустить. Это значение используется по умолчанию, когда создатель родительского документа не указал никакого другого URL.

Во всех браузерах большинство типов навигации к `about:blank` приводит к созданию нового документа, который наследует свой SOP-источник от страницы, инициировавшей навигацию. Наследованный источник отражается в свойстве `document.domain` нового контекста выполнения JavaScript, а DOM-доступ к любым другим источникам или из них не разрешен.

Эта простая формула верна для таких действий навигации, как щелчок по ссылке, отправка формы, создание нового фрейма или окна из скрипта или программная навигация существующего документа. Тем не менее есть исключения, самые заметные из которых - несколько специальных, контролируемых пользователем методов навигации. К ним относятся ручной ввод `about:blank` в адресной строке, переход по закладке или выполнение жеста, зарезервированного для открытия

ссылки в новом окне или вкладке.^[1] Эти действия приведут к документу, занимающему уникальный синтетический источник, к которому не может получить доступ никакая другая страница.

Другой специальный случай - загрузка обычного предоставленного сервером документа, который затем перенаправляет на `about:blank` с помощью `Location` или `Refresh`. В Firefox и браузерах на основе WebKit такое перенаправление приводит к уникальному, недоступному источнику, похожему на сценарий, описанный в предыдущем абзаце. В Internet Explorer, с другой стороны, получающийся документ будет доступен родительской странице, если перенаправление происходит внутри `<iframe>`, но не если оно произошло в отдельном окне. Поведение Opera понять труднее всего: `Refresh` приводит к документу, доступному родительской странице, но `redirect Location` даст получающейся странице источник сайта, выполнившего `redirect`.

Кроме того, родительский документ может направить существующий фрейм документа на URL `about:blank`, даже если существующий документ, показанный в этом контейнере, имеет источник, отличный от вызывающего.^[2] Вновь созданный пустой документ унаследует источник от вызывающего во всех браузерах, кроме Internet Explorer. В случае Internet Explorer такая навигация завершится успешно, но приведет к недоступному документу. Скорее всего, это поведение не было намеренным.

Если от этого описания у вас кружится голова, обработка документов `about:blank` сведена в таблицу 10-1.

Таблица 10-1. Наследование источника для URL `about:blank`

Браузер	Новая страница	Существующая страница не из того же источника	Redirect Location	Redirect Refresh	Ввод URL или жест
Internet Explorer	Наследует от вызывающего	Уникальный источник, доступ запрещен	Фрейм: наследует от родителя. Окно: уникальный источник	Уникальный источник	Уникальный источник
Firefox	Наследует от вызывающего	Уникальный источник	Уникальный источник	Уникальный источник	Уникальный источник
Все WebKit	Наследует от вызывающего	Доступ запрещен	Уникальный источник	Уникальный источник	Уникальный источник
Opera	Наследует от вызывающего	Наследует от вызывающего	Наследует от перенаправляющей стороны	Наследует от родителя	Уникальный источник

Наследование для `data: URL`

Схема `data:`,^[3] впервые описанная в главе 2, была разработана, чтобы позволить небольшим документам, таким как значки, удобно кодироваться и затем напрямую встраиваться в HTML-документ, экономя время на HTTP round trips. Например:

```

```

Когда схема `data:` используется вместе с подресурсами конкретного типа, единственное необычное соображение безопасности состоит в том, что она создает проблему для плагинов, которые хотят вывести разрешения апплета из его исходного URL. Источник нельзя вычислить, глядя только на URL, а поведение несколько непредсказуемо и сильно зависит от конкретного плагина. Например, Adobe Flash сейчас отвергает любые попытки использовать документы `data:`.

Важнее случая содержимого конкретного типа использование `data`: как назначения для окон и фреймов. Во всех браузерах, кроме Internet Explorer, схему можно использовать как улучшенный вариант `about:blank`, как в этом примере:

```
<iframe src="data:text/html;charset=utf-8,<h1>Hi mom!</h1>">
</iframe>
```

В этом сценарии нет убедительной причины для URL `data`: вести себя иначе, чем `about:blank`. В реальности, однако, в некоторых браузерах он будет вести себя иначе, и поэтому использовать его нужно осторожно.

- Браузеры WebKit. В Chrome и Safari всем документам `data`: назначается уникальный, недоступный источник, и они вообще не наследуют от родителя.
- Firefox. В Firefox источник для документов `data`: наследуется от навигационного контекста, похожим на `about:blank` образом. Однако, в отличие от `about:blank`, ручной ввод URL `data`: или открытие таких URL из закладок приводит к тому, что новый документ наследует источник от страницы, на которой произошла навигация.
- Opera. На момент написания для всех URL `data`: используется общий "пустой" источник, доступный родительскому документу. Этот подход небезопасен, поскольку может разрешить междоменный доступ к фреймам, созданным несвязанными страницами, как показано на рис. 10-1. Я сообщил об этом поведении Opera, и, вероятно, оно скоро будет изменено.
- Internet Explorer. URL `data`: не поддерживаются в версиях Internet Explorer до 8. Схема поддерживается только для выбранных типов подресурсов в Internet Explorer 8 и 9 и не может использоваться для навигации.

Таблица 10-2 обобщает текущее поведение URL `data`:

Таблица 10-2. Наследование источника для URL `data`:

Браузер	Новая страница	Существующая страница не из того же источника	Redirect Location	Redirect Refresh	Ввод URL или жест
Internet Explorer 6/7	Не поддерживается	Не поддерживается	Не поддерживается	Не поддерживается	Не поддерживается
Internet Explorer 8/9	Не поддерживается для навигации	Не поддерживается для навигации	Не поддерживается для навигации	Не поддерживается для навигации	Не поддерживается для навигации
Firefox	Наследует от вызывающего	Уникальный источник	Наследует от предыдущей страницы	Наследует от предыдущей страницы	Наследует от предыдущей страницы
Все WebKit	Уникальный источник	Доступ запрещен	Уникальный источник	Уникальный источник	Уникальный источник
Opera	Общий источник, это ошибка	Доступ запрещен	Наследует от родителя	Наследует от родителя	Наследует от родителя

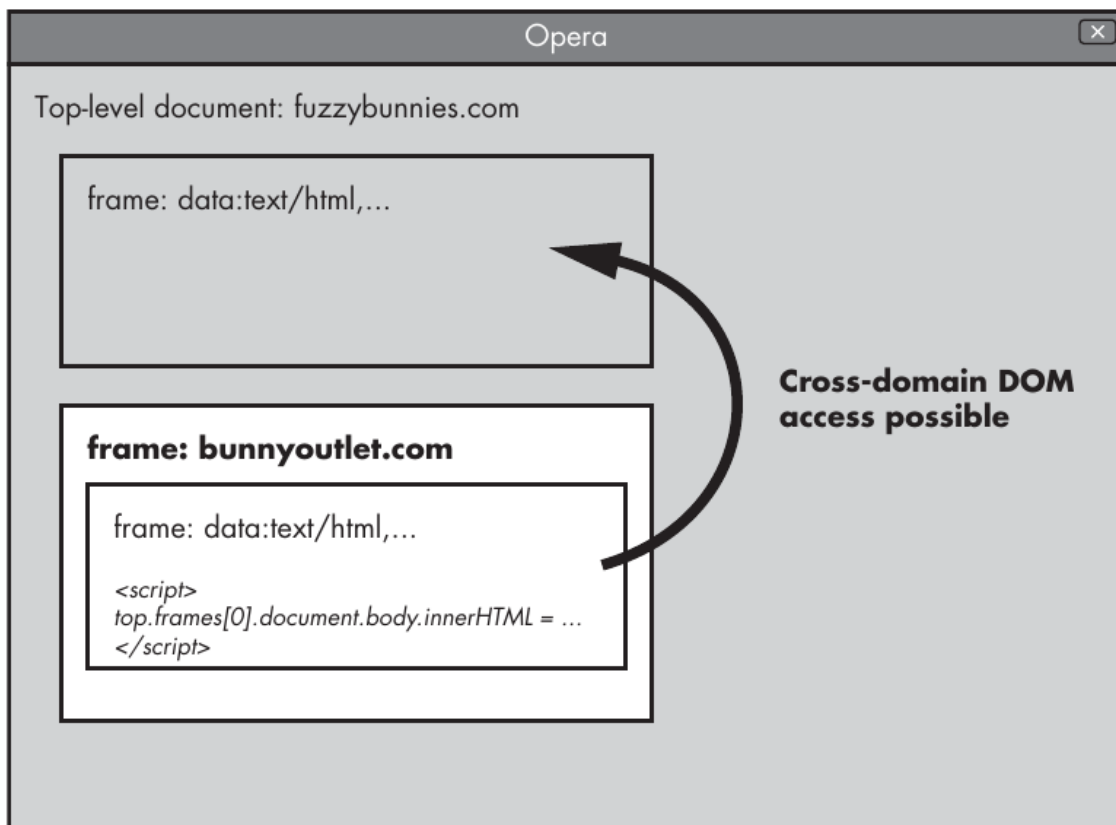


Рис. 10-1. Доступ между URL data: в Opera

Наследование для javascript: и vbscript: URL

Псевдо-URL, связанные со сценариями, такие как `javascript:`, - очень любопытный механизм. Использование их для загрузки некоторых типов подресурсов приведет к выполнению кода в контексте документа, который пытается загрузить такую операцию, с учетом некоторых непоследовательных ограничений, обсуждавшихся в главе 4. Примером может быть:

```
<iframe src="javascript:alert('Hi mom!')"></iframe>
```

Более интересно, и куда менее очевидно, чем создание новых подресурсов, что навигация существующих окон или фреймов к URL `javascript:` заставит встроенный JavaScript-код выполняться в контексте страницы, на которую выполняется навигация, а не навигирующего документа, даже если URL введен вручную или загружен из закладки.

Учитывая такое поведение, очевидно, что очень небезопасно разрешать одному документу направлять любой другой контекст не из того же источника на URL `javascript:`, поскольку это позволило бы обойти все остальные механизмы изоляции содержимого: просто загрузите `fuzzybunnies.com` во фрейм, затем направьте этот фрейм на `javascript:do_evil_stuff()` - и дело сделано. Следовательно, такая навигация запрещена во всех браузерах, кроме Firefox. Firefox по какой-то причине, похоже, разрешает ее, но скрытно меняет семантику. Когда источник вызывающего и цель навигации не совпадают, он выполняет полезную нагрузку `javascript:` в специальном `null-origin`, где нет собственного DOM и не зарегистрированы никакие предоставленные браузером функции ввода-вывода, что разрешает только чисто алгоритмические операции.

Случай cross-origin опасен, но его same-origin аналог - нет: внутри одного источника любое содержимое свободно направляет само себя или своих соседей к URL javascript: по собственной воле. В этом случае схема javascript: учитывается при переходе по ссылкам, отправке форм, вызове location.assign(...) и так далее. В WebKit и Opera redirect Refresh на javascript: тоже сработает; другие браузеры отвергают такую навигацию из-за смутных и, вероятно, неуместных опасений внедрения скрипта.

Обработка scripting URLs изложена в таблице 10-3.

Таблица 10-3. Наследование источника для scripting URLs

Браузер	Новая страница	Существующая same-origin страница	Существующая non-same-origin страница	Redirect Location	Redirect Refresh	Ввод URL или жест
Internet Explorer	Наследует от вызывающего	Наследует от страницы, на которую выполняется навигация	Доступ запрещен	Доступ запрещен	Доступ запрещен	Наследует от страницы, на которую выполняется навигация
Firefox	Null context	Наследует от страницы, на которую выполняется навигация	Null context	Доступ запрещен	Доступ запрещен	Наследует от страницы, на которую выполняется навигация
Все WebKit	Доступ запрещен	Наследует от страницы, на которую выполняется навигация	Доступ запрещен	Доступ запрещен	Наследует от страницы, на которую выполняется навигация	Наследует от страницы, на которую выполняется навигация
Opera	Доступ запрещен	Наследует от страницы, на которую выполняется навигация	Доступ запрещен	Доступ запрещен	Наследует от страницы, на которую выполняется навигация	Наследует от страницы, на которую выполняется навигация

Поверх этой увлекательной семантики есть еще один поворот, уникальный для схемы javascript:: в некоторых случаях обработка таких URL, содержащих скрипт, включает второй шаг. В частности, если предоставленный код корректно вычисляется, а значение последней инструкции не является void и может быть преобразовано в строку, эта строка будет интерпретирована как HTML-документ и заменит страницу, на которую выполнялась навигация, унаследовав источник от вызывающего. Логика, управляющая этим любопытным поведением, очень похожа на ту, что влияет на поведение URL data:. Пример такого выражения, заменяющего документ:

```
javascript:"<b>2 + 2 = " + (2+2) + "</b>"
```

Примечание об ограниченных псевдо-URL

Несколько причудливое поведение трех упомянутых выше классов URL - about:blank, javascript: и data: - это все, о чем большинству сайтов нужно беспокоиться. Тем не менее браузеры используют ряд других документов без внутреннего, четко определенного источника, например about:config в Firefox, привилегированную JavaScript-страницу, которую можно использовать для настройки различных внутренних параметров браузера, или chrome://downloads в Chrome, где перечислены недавно загруженные документы со ссылками для открытия любого из них. Эти документы остаются постоянным источником проблем безопасности, даже если они не доступны напрямую из интернета.

Из-за несовместимости этих URL с границами, контролируемые политикой одного источника, нужно проявлять особую осторожность, чтобы такие URL были достаточно изолированы от другого содержимого всякий раз, когда они загружаются в браузере в результате действия пользователя или какого-либо другого косвенного процесса уровня браузера. Интересный случай, иллюстрирующий риск, - ошибка 2010 года в том, как Firefox обрабатывал `about:neterror`.^[4] Всякий раз, когда Firefox не может корректно получить документ с удаленного сервера, состояние, которое обычно легко вызвать тщательно сконструированной ссылкой, он помещает целевой URL в адресную строку, но загружает `about:neterror` вместо тела документа. К сожалению, из-за небольшой оплошности эта специальная страница ошибки была бы `same-origin` с любым документом `about:blank`, открытым любым содержимым интернет-происхождения, тем самым позволяя злоумышленнику внедрить произвольное содержимое в окно `about:neterror`, сохраняя отображаемый целевой URL.

Мораль этой истории? Избегайте соблазна играть в азартные игры с политикой одного источника; вместо этого работайте вместе с ней. Обратите внимание, что если бы `about:neterror` был сделан иерархическим URL вместо попытки отслеживать синтетические источники, это предотвратило бы ошибку.

Памятка по инженерии безопасности

Из-за их несовместимости с политикой одного источника URL `data:`, `javascript:` и неявные или явные URL `about:blank` следует использовать осторожно. Когда производительность не критична, предпочтительнее сначала заселять новые фреймы и окна, указывая им на предоставленный сервером пустой документ с определенным `origin`.

Помните, что URL `data:` и `javascript:` не являются полноценной заменой `about:blank`, и использовать их следует только тогда, когда это абсолютно необходимо. В частности, сейчас небезопасно предполагать, что окна `data:` недоступны между доменами.

Сноски

[1] [\[назад\]](#) Обычно это делается удержанием CTRL или SHIFT при щелчке по ссылке либо щелчком правой кнопкой мыши для доступа к контекстному меню с последующим выбором подходящего варианта.

[2] [\[назад\]](#) Точные обстоятельства, делающие это возможным, будут фокусом главы 11. Пока достаточно сказать, что этого можно добиться во многих условиях специфичным для браузера способом. Например, в Firefox вызывают `window.open(..., 'target')`, а в Internet Explorer нужный способ - вызов `target.location.assign(...)`.

[3] [\[назад\]](#) Сноска в оригинале указывает на описание схемы `data:`.

[4] [\[назад\]](#) Сноска в оригинале указывает на ошибку Firefox 2010 года, связанную с `about:neterror`.

Глава 11. Жизнь вне правил одного источника

Политика одного источника - самый важный механизм, который у нас есть, чтобы держать враждебные веб-приложения на расстоянии, но она также несовершенна. Хотя она должна давать надежную степень разделения между любыми двумя различными и четко идентифицируемыми источниками содержимого, она часто не справляется с этой задачей.

Чтобы понять этот разрыв, вспомните, что, вопреки здравому смыслу, политика одного источника никогда не была задумана как всеобъемлющая. Ее первоначальный фокус, иерархия DOM, то есть просто объект `document`, открытый JavaScript-коду, оставлял многие периферийные возможности JavaScript полностью открытыми для междоменного вмешательства, что потребовало специальных исправлений. Например, через несколько лет после появления SOP поставщики поняли, что разрешать сторонним документам изменять свойство `location.host` несвязанного окна - плохая идея и что такая операция может отправить потенциально чувствительные данные, присутствующие в других сегментах URL, на сайт, указанный злоумышленником. С тех пор политика была расширена, чтобы хотя бы частично защищать этот и еще пару чувствительных объектов, но в некоторых менее очевидных случаях остаются неловкие лазейки.

Другая проблема в том, что многие междоменные взаимодействия происходят полностью вне JavaScript и его объектной иерархии. Действия вроде загрузки сторонних изображений или таблиц стилей глубоко укоренены в устройстве HTML и никак существенно не зависят от сценариев. В принципе, их можно было бы дооснастить средствами безопасности на основе `origin`, но это нарушило бы работу существующих сайтов. Кроме того, некоторые считают, что такое решение противоречило бы принципам проектирования, сделавшим Web тем, чем он является; они полагают, что способность свободно перекрестно ссылаться на содержимое не должна ущемляться.

В свете этого разумно исследовать границы политики одного источника и узнать о богатой жизни, которую веб-приложения могут вести вне ее пределов. Мы начнем с навигации документов - механизма, который сначала кажется поразительно простым, но на самом деле совсем не таков.

Взаимодействия окон и фреймов

В Web способность направлять браузер с одного сайта на другой считается само собой разумеющейся. Некоторые распространенные методы достижения такой навигации обсуждаются по всей части I этой книги; самые заметные из них - HTML-ссылки, формы и фреймы, HTTP-redirects, а также вызовы JavaScript `window.open(...)` и `location.*`.

Действия вроде направления вновь открытого окна на URL вне домена или указания параметра `src` фрейма интуитивны и не требуют дальнейшего обзора. Но когда мы смотрим на способность одной страницы направлять другой, уже существующий документ, царство интуиции внезапно заканчивается.

Изменение местоположения существующих документов

В простые дни до появления HTML-фреймов только один документ мог занимать данное окно браузера, и только это одно окно находилось под контролем документа. Однако фреймы изменили эту парадигму, позволив нескольким различным и полностью отдельным документам быть соединенными в единое логическое представление, сосуществуя в общей области экрана. Введение механизма также потребовало еще одного шага: чтобы разумно реализовывать некоторые сайты на основе фреймов, любому из компонентных документов, отображаемых в окне, нужна была способность направлять соседние фреймы или, возможно, сам документ верхнего уровня. Например, представьте страницу из двух фреймов с оглавлением слева и фактической главой справа. Щелчок по названию главы в левой панели должен направить главу в правой панели, и ничего больше.

Механизм, придуманный для этой последней цели, довольно прост: можно указать параметр `target` у ссылок `<a href=...>` или форм либо предоставить имя окна JavaScript-методу `window.open(...)`, чтобы направить любое другое, ранее именованное представление документа.

В середине 1990-х, когда эта функциональность впервые появилась, казалось, что нет нужды включать в эту логику какие-либо особые проверки безопасности; любая страница могла по своему желанию направлять любое другое именованное окно или фрейм, отображаемый браузером, в новое место.

Чтобы понять последствия такой конструкции, важно на мгновение остановиться и изучить обстоятельства, при которых конкретный документ вообще может получить имя. Для фреймов история проста: чтобы легко ссылаться на фрейм на встраивающей странице, практически все фреймы имеют атрибут `name`, а некоторые браузеры, такие как Chrome, также смотрят на `id`. Окна браузера, с другой стороны, обычно анонимны, то есть их свойство `window.name` является пустой строкой, если только они не созданы программно; в последнем случае имя указывает тот, кто создает представление. Однако анонимные окна не обязательно остаются анонимными. Если вредоносное приложение отображается в таком окне даже ненадолго, оно может установить свойство `window.name` в любое значение, и этот эффект сохранится.

Вышеупомянутая способность нацеливать окна и фреймы по имени - не единственный способ их направлять; JavaScript-программы, держащие оконные дескрипторы, указывающие на другие документы, могут напрямую вызывать некоторые методы DOM, вообще не зная имени своей цели. Код, предоставленный злоумышленником, обычно не будет держать дескрипторы полностью несвязанных окон, но он может обходить свойства вроде `opener`, `top`, `parent` или `frames[]`, чтобы находить даже дальних родственников в пределах одного потока навигации. Пример такого далеко идущего поиска и последующей навигации:

```
opener.opener.frames[2].location.assign("http://www.bunnyoutlet.com/");
```

Эти две техники поиска не взаимоисключающие: JavaScript-программы могут сначала получить дескриптор несвязанного, но именованного окна через `window.open(...)`, а затем пройти по свойствам `opener` или `frames[]` этого контекста, чтобы добраться до его интересных родственников поблизости.

Как только подходящий дескриптор найден любым способом, исходный контекст может использовать один из нескольких методов и свойств DOM, чтобы изменить адрес документа, отображаемого в этом представлении. В каждом современном браузере вызов метода `<handle>.location.replace(...)` или присваивание значения свойствам `<handle>.location` или `<handle>.location.href` должны сделать свое дело. Забавно, что из-за случайных особенностей реализации другие теоретически эквивалентные подходы, такие как вызов `<handle>.location.assign(...)` или `<handle>.window.open(..., "_self")`, могут работать не всегда.

Итак, возможно направлять несвязанные документы в новые места - но посмотрим, что может пойти не так.

Риски захвата фреймов

Способность одного домена направлять окна, созданные другими сайтами, или окна, которые просто больше не являются одним источником со своим создателем, обычно не вызывает серьезной тревоги. Такая расслабленная конструкция может раздражать и создавать некоторый небольшой, умозрительный риск фишинга,^[1] но в общей картине это не очень выраженная проблема и не особенно отличительная. Возможно, именно поэтому первоначальные авторы соответствующих API не слишком задумывались обо всем механизме.

Увы, концепция HTML-фреймов глубоко меняет картину: любое приложение, полагающееся на фреймы для построения доверенного пользовательского интерфейса, находится в очевидной опасности, если несвязанному сайту разрешено захватить такие UI-элементы, не оставив никаких следов атаки в адресной строке. На рис. 11-1 показан один такой правдоподобный сценарий атаки.

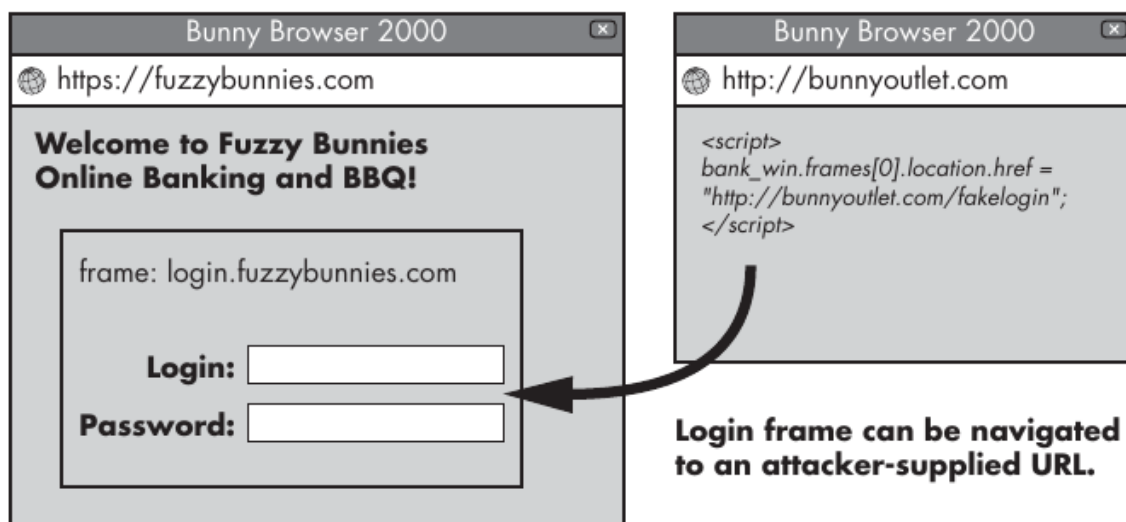


Рис. 11-1. Исторически разрешенный опасный сценарий навигации фреймов

Георги Гунински, один из пионеров исследований безопасности браузеров, уже в 1999 году понял, что, разрешая неограниченную навигацию фреймов, мы движемся к серьезным неприятностям. После его отчетов поставщики попытались внедрить ограничения навигации фреймов в середине 2000 года.^[2] Их реализация ограничила всю межфреймовую навигацию областью одного окна, предотвращая вмешательство вредоносных веб-страниц в любые другие одновременно открытые браузерные сеансы.

Удивительно, но даже такую простую политику оказалось трудно реализовать правильно. Лишь в 2008 году Firefox устранил этот класс проблем,^[3] тогда как Microsoft по сути игнорировала проблему до 2006 года. Но, оставив эти неудачи в стороне, теперь все должно быть хорошо, верно?

Политика потомков фреймов и междоменные коммуникации

Простого ограничения безопасности, обсужденного в предыдущем разделе, на самом деле было недостаточно. Причиной был новый класс веб-приложений, иногда известных как mashups, которые объединяли данные из разных источников, позволяя пользователям персонализировать рабочую среду и обрабатывать данные инновационными способами. К несчастью для поставщиков браузеров, такие веб-приложения часто полагались на сторонние гаджеты, загружаемые через теги `<iframe>`, а их разработчики не могли разумно ожидать, что загрузка одного фрейма из вредоносного источника поставит под угрозу все остальные фреймы на странице. Однако простая и элегантная политика навигации уровня окна означала разрешение именно этого.

Примерно в 2006 году Microsoft согласилась, что текущий подход неустойчив, и разработала более безопасную политику потомков для навигации фреймов в Internet Explorer 7. Согласно этой политике, навигация фреймов не из того же источника разрешена только если сторона, запрашивающая навигацию, разделяет origin с одним из предков целевого представления. На рис. 11-2 показан сценарий навигации, разрешенный этой новой политикой.

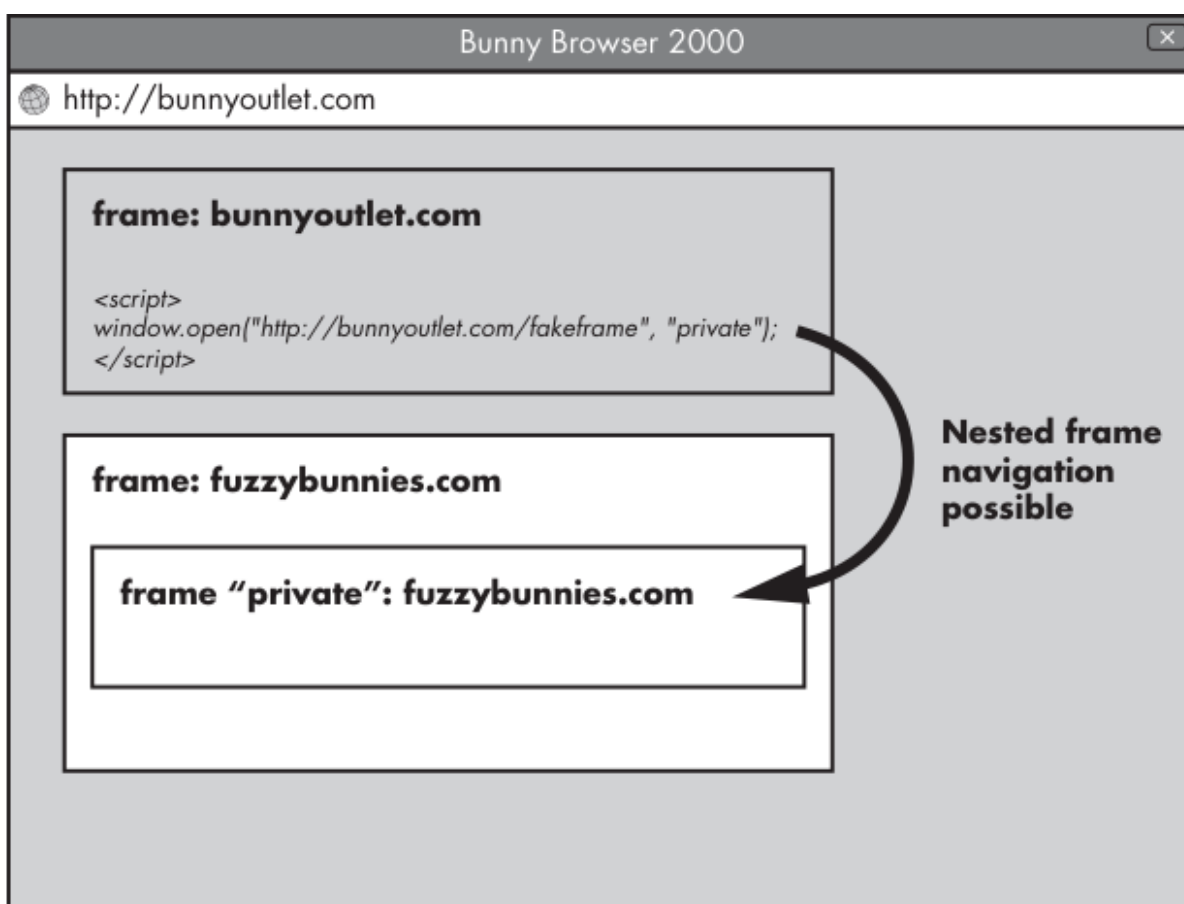


Рис. 11-2. Сложная, но допустимая навигация между фреймами не из одного источника

Как и со многими другими улучшениями безопасности, Microsoft так и не перенесла эту политику назад в все еще популярный Internet Explorer 6 и никогда убедительно не давила на пользователей, чтобы они отказались от старой и все более небезопасной, но все еще поверхностно поддерживаемой версии браузера. С более позитивной стороны, к 2009 году трое исследователей безопасности (Адам Барт, Коллин Джексон и Джон К. Митчелл) убедили Mozilla, Opera и WebKit внедрить похожую политику в своих браузерах,^[4] наконец закрыв лазейку mashup для значительного большинства пользователей интернета.

Ну, почти закрыв. Даже у новой надежной политики есть тонкий дефект. Обратите внимание на рис. 11-2, что вредоносный сайт `http://bunnyoutlet.com/` может вмешаться в приватный фрейм, который `http://fuzzybunnies.com/` создал для собственного использования. На первый взгляд вреда нет: домен злоумышленника показан в адресной строке, поэтому жертва теоретически не должна быть обманом вовлечена в осмысленное взаимодействие с подмененным UI `http://fuzzybunnies.com/`. Увы, есть подвох: некоторые веб-приложения научились использовать фреймы не для создания пользовательских интерфейсов, а для передачи программных сообщений между источниками. Для приложений, которым нужно поддерживать Internet Explorer 6 и 7, где `postMessage(...)` недоступен, трюки, похожие на подход, показанный на рис. 11-3, являются обычным делом.

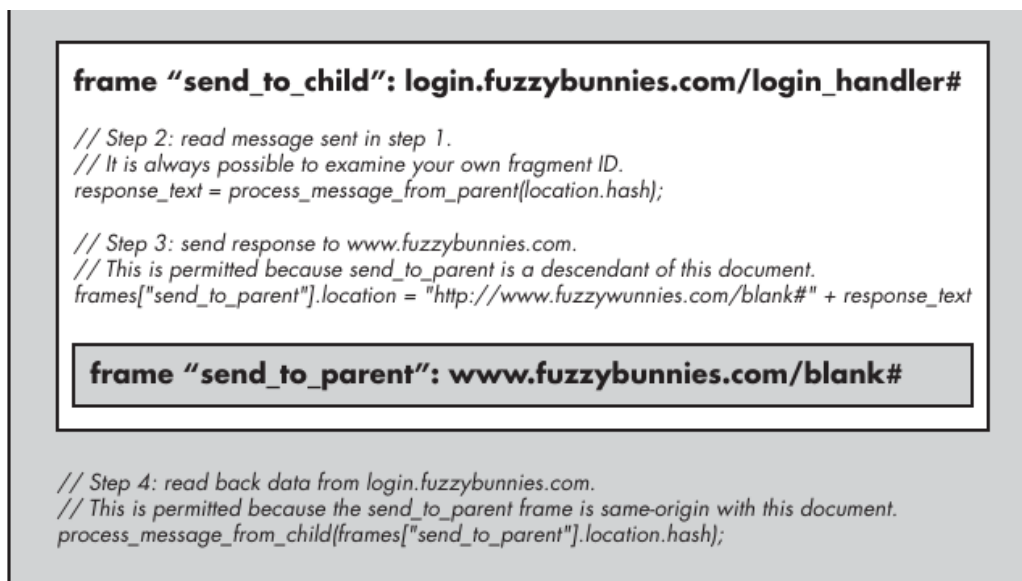


Рис. 11-3. Потенциальная схема междоменных коммуникаций через *fragment identifier*

Если приложение, полагающееся на похожий трюк, встроено вредоносным сайтом, целостность коммуникационных фреймов может быть нарушена, и злоумышленник сможет внедрять сообщения в поток. Даже использования `postMessage(...)` могут быть под угрозой: если сторона, отправляющая сообщение, не указывает целевой `origin` или если получатель не проверяет исходное местоположение, захват фрейма принесет злоумышленнику ровно такую же выгоду.

Незапрошенное фреймирование

Предыдущее обсуждение cross-frame navigation подсвечивает одну из более интересных слабостей модели безопасности браузера, а также разрыв между целями проектирования HTML и целью политики одного источника. Но это еще не все: сама концепция междоменного фреймирования довольно рискованна. Почему? Что ж, любая вредоносная страница может встроить стороннее приложение без ведома пользователя, не говоря уже о согласии. Более того, она может скрыть этот факт, наложив другие визуальные элементы поверх фрейма и оставив видимым лишь небольшой кусок исходного сайта, например кнопку, выполняющую действие, изменяющее состояние.

В такой среде любого пользователя, вошедшего в целевое приложение с окружающими учетными данными, можно легко обманом заставить взаимодействовать с замаскированным элементом UI и выполнить нежелательное и непреднамеренное действие, например изменить настройки `sharing` для профиля социальной сети или удалить данные.

Эту атаку можно усилить, если вредоносный сайт использует свойство CSS2 `opacity`, чтобы сделать целевой фрейм полностью невидимым, не затрагивая его фактическое поведение. Любой щелчок в области, занятой таким прозрачным фреймом, будет доставлен элементам UI внутри него (см. рис. 11-4). Кроме того, сочетая CSS `opacity` с JavaScript-кодом, заставляющим фрейм следовать за указателем мыши, можно провести атаку довольно надежно почти в любой ситуации: убедить пользователя щелкнуть где угодно в окне документа не особенно трудно.

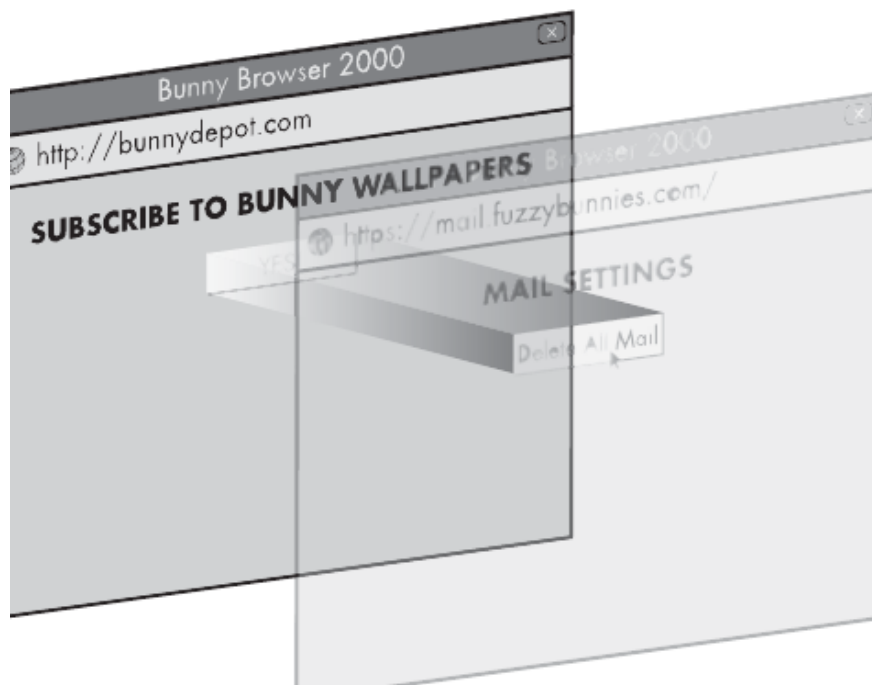


Рис. 11-4. Упрощенный пример атаки с совмещением UI через CSS `opacity`

Исследователи в некоторой степени признавали возможность такого обмана с начала 2000-х, но достаточно убедительная атака не была продемонстрирована до 2008 года, когда Роберт Хансен и Джеремайя Гроссман широко обнародовали проблему.^[5] Так родился термин *clickjacking*.

Высокий профиль отчета Хансена и Гроссмана и их интересный *proof-of-concept* пример пробудили интерес поставщиков. Однако этот интерес оказался недолгим, и, похоже, нет простого способа решить эту проблему, не принимая серьезных рисков. Единственный хотя бы отдаленно правдоподобный способ снизить влияние - добавить эвристики уровня рендерера, запрещающие доставку событий междоменным фреймам, которые частично закрыты или отображались недостаточно долго. Но это решение достаточно сложное и мохнатое, чтобы быть непопулярным.^[6] Вместо этого проблему заклеили пластырем. Новый HTTP-заголовок `X-Frame-Options` позволяет обеспокоенным сайтам отказаться от фреймирования целиком (`X-Frame-Options: deny`) или согласиться только на фреймирование внутри одного источника (`X-Frame-Options: same-origin`).^[7] Этот заголовок поддерживается во всех современных браузерах, в Internet Explorer начиная с версии 8,^[8] но на самом деле мало что делает для устранения уязвимости.

Во-первых, opt-in природа защиты означает, что большинство сайтов не примут ее или не примут достаточно быстро; фактически исследование 2011 года по топ-10 000 направлений в интернете обнаружило, что едва 0,5 процента использовали эту возможность.^[9]

Чтобы добавить неприятностей, предложенный механизм бесполезен для приложений, которые хотят встраиваться на сторонние сайты, но желают сохранять целостность своих UI. Различные mashups и гаджеты, распространяемые социальными сетями кнопки "like" и управляемые онлайн-интерфейсы обсуждений - все они под угрозой.

За пределами угрозы одного щелчка

Как подразумевает название, атака clickjacking, описанная Гроссманом и Хансенom, нацелена на простые UI-действия в один щелчок. В реальности, однако, проблема с обманчивым фреймированием сложнее, чем предполагали ранние сообщения. Один пример более сложного взаимодействия - действие выбора, перетаскивания и отпускания фрагмента текста. В 2010 году Пол Стоун предложил ряд способов, которыми такое действие можно замаскировать под правдоподобное взаимодействие с сайтом злоумышленника,^[10] самый заметный из которых - сходство между drag-and-drop и использованием скромной полосы прокрутки уровня документа. Одно и то же действие click-drag-release может использоваться для взаимодействия с легитимным элементом UI или для того, чтобы невольно перетащить часть заранее выделенного текста из чувствительного документа и бросить ее во фрейм, контролируемый злоумышленником. Междоменный drag-and-drop больше не разрешен в WebKit, но на момент написания другие поставщики браузеров все еще обсуждают правильный способ устранить этот риск.

Еще более сложная проблема - перенаправление нажатий клавиш. Где-то в 2010 году я заметил, что можно выборочно перенаправлять нажатия клавиш между доменами, изучая код нажатой клавиши с помощью события onkeydown в JavaScript. Если нажатая клавиша совпадала с тем, что вредоносный сайт хотел ввести в целевое приложение, фокус HTML-элемента можно было на мгновение изменить на скрытый <iframe>, тем самым обеспечив доставку фактических нажатий клавиш целевому веб-приложению, а не безвредному текстовому полю, с которым пользователь, как кажется, взаимодействует.^[11] Используя этот метод, злоумышленник может синтезировать произвольно сложный текст в другом домене от имени пользователя, например пригласить злоумышленника администратором блога жертвы.

Поставщики браузеров устранили проблему выборочного перенаправления нажатий клавиш, запретив изменения фокуса элемента в середине keypress, но это не закрыло лазейку полностью. В конце концов, в некоторых случаях злоумышленник может предсказать, какая клавиша будет нажата следующей и примерно когда, что позволяет выполнить превентивное, слепое переключение фокуса. Два самых очевидных случая - веб-игра в жанре action или тест скорости набора, поскольку оба обычно включают быстрое нажатие клавиш, на которые влияет злоумышленник.

Фактически все становится еще лучше: даже если вредоносное приложение полагается только на ввод свободного текста, например предлагая пользователю форму отправки комментария, часто можно угадать, какой символ будет нажат следующим, исходя только из нескольких предыдущих нажатий. Английский текст, как и текст в большинстве других человеческих языков, сильно избыточен, и во многих случаях значительный объем ввода можно предсказать заранее: можно держать пари, что за a-a-r-d-v последует a-r-k, и почти всегда вы будете правы.

Междоменное включение содержимого

Фреймирование и навигация - отдельный источник неприятностей, но помимо этих механизмов HTML поддерживает ряд других способов взаимодействовать с данными не из того же источника. Обычный шаблон проектирования для этих возможностей прост и кажется безопасным: ограниченный формат данных, который повлияет на внешний вид документа, извлекается и разбирается без прямого показа источнику, который на него сослался. Примеры механизмов, следующих этому правилу, включают разметку вроде <script src=...>, <link rel=stylesheet href=...>, и несколько связанных случаев, обсуждавшихся по всей части I этой книги.

К сожалению, дьявол в деталях. Когда эти механизмы впервые предлагались, никто не задал несколько чрезвычайно насущных вопросов:

- Должны ли эти подресурсы запрашиваться с окружающими учетными данными, связанными с их origin? Если да, есть опасность, что ответ может содержать чувствительные данные, не предназначенные для запрашивающей стороны. Вероятно, лучше было бы требовать какой-либо явной формы аутентификации или уведомлять сервер об origin запрашивающей страницы.
- Должны ли соответствующие парсеры проектироваться так, чтобы минимизировать риск перепутать один тип документа с другим? И должны ли серверы контролировать, как интерпретируются их ответы, например через заголовок Content-Type? Если нет, каковы последствия, скажем, интерпретации приватного JPEG-изображения пользователя как скрипта?
- Должна ли запрашивающая страница не иметь никакого способа вывести что-либо о содержимом полученных полезных нагрузок? Если да, то эту цель нужно учитывать с предельной осторожностью при проектировании всех связанных API. Если такое разделение не является целью, важность предыдущих вопросов еще более выражена.

Разработчики действовали с противоречивыми предположениями на эти темы или, возможно, вообще о них не думали, что привело к ряду глубоких рисков безопасности. Например, в большинстве браузеров раньше было возможно читать произвольный, аутентифицированный cookie текст, зарегистрировав обработчик onerror на междоменных загрузках `<script>`: подробное сообщение "syntax error", сгенерированное браузером, включало фрагмент полученного файла. И все же ни одна проблема в этой категории не интереснее сбоя, обнаруженного Крисом Эвансом в 2009 году.^[12] Он заметил, что фирменная отказоустойчивость CSS-парсеров, которые, как вы помните, восстанавливаются после синтаксических ошибок, пытаясь ресинхронизироваться на ближайшей фигурной скобке, также является фатальным дефектом безопасности.

Чтобы понять проблему, рассмотрим следующий простой HTML-документ. Этот документ содержит два появления строки, контролируемой злоумышленником, и между ними - чувствительное, специфичное для пользователя значение, в данном случае имя пользователя:

```
<head>
  <title>Page not found: ');} gotcha { background-image: url('/</title>
</head>
<body>
  ...
  <span class="header">You are logged in as: John Doe</span>
  ...
  <div class="error_message">
    Page not found: ');} gotcha { background-image: url('/
  </div>
  ...
</body>
```

Предположим, что злоумышленник заманил жертву на свою страницу и на этой странице использовал `<link rel=stylesheet>`, чтобы загрузить вышеупомянутый междоменный HTML-документ вместо таблицы стилей. Браузер жертвы с радостью подчинится: он запросит документ с cookie жертвы, проигнорирует Content-Type в последующем ответе и передаст полученное содержимое CSS-парсеру. Парсер весело проигнорирует все синтаксические ошибки до того, что выглядит как CSS-правило с именем gotcha. Затем он обработает псевдофункцию `url(' ...`, потребляя весь последующий HTML, включая секретное имя пользователя, пока не достигнет совпадающей кавычки и закрывающей скобки. Когда эта поддельная таблица стилей позднее применяется к элементу `class=gotcha` на сайте злоумышленника, браузер попытается загрузить получившийся URL и в процессе утечет секретное значение на сервер злоумышленника.

Внимательные читатели могут отметить, что стандарт CSS не поддерживает многострочные строковые литералы, и поэтому этот трюк не сработал бы как описано. Это отчасти правда: в

большинстве браузеров попытка будет успешной только если критический сегмент страницы не содержит случайных переводов строк. Некоторые веб-приложения оптимизированы так, чтобы избегать ненужных пробельных символов, и поэтому будут уязвимы, но большинство веб-разработчиков щедро используют переводы строк, срывая атаку. Увы, как отмечалось в главе 5, один браузер ведет себя иначе: Internet Explorer принимает многострочные строки в таблицах стилей и многие другие вопиющие синтаксические нарушения, случайно усиливая влияние этого дефекта.

Примечание. После выявления этой проблемы Крис Эванс добивался исправлений во всех основных браузерах, и на момент написания большинство реализаций отвергает междоменные таблицы стилей, которые не начинаются сразу с действительного CSS-правила или отдаются с несовместимым заголовком Content-Type; same-origin таблицы стилей обрабатываются менее ограничительно. Единственным поставщиком, сопротивлявшимся этому, была Microsoft, которая изменила мнение только после демонстрации успешной proof-of-concept атаки против Twitter.^[13] После этого раскрытия Microsoft согласилась не только устранить проблему в Internet Explorer 8, но и, что для нее нехарактерно, перенести именно это исправление назад в Internet Explorer 6 и 7.

Благодаря усилиям Криса таблицы стилей - решенная проблема, но похожие проблемы неизбежно будут повторяться для других типов междоменных подресурсов. В таких случаях не все проступки можно списать на грехи прошлого. Например, когда поставщики браузеров внедрили <canvas>, простой механизм HTML5, позволяющий JavaScript создавать векторную и растровую графику,^[14] многие реализации не ставили ограничений на загрузку междоменных изображений на canvas и затем чтение их обратно пиксель за пикселем. На момент написания эта проблема тоже решена: canvas, которого коснулось междоменное изображение, становится "tainted" и в него можно только писать, но не читать. Но когда нам приходится исправлять каждый такой случай индивидуально, что-то очень не так.

Примечание о cross-origin подресурсах

До сих пор мы сосредоточивались на рисках того, что вредоносные сайты направляют или включают содержимое, принадлежащее доверенным сторонам. Тем не менее способность загружать некоторые типы подресурсов из других origins имеет значительные последствия, даже если ею активно не злоупотребляет сторонний сайт.

В части I книги мы намекали, что загрузка скрипта или таблицы стилей из другого origin фактически приравняет безопасность документа, выполняющего загрузку, к безопасности origin загруженного подресурса; в частности, загрузка HTTP-скрипта на HTTPS-странице отменяет большинство преимуществ шифрования. Аналогично, загрузка скрипта от поставщика, инфраструктура которого уязвима к атаке, может быть почти такой же проблемной, как ненадлежащее обслуживание собственных серверов.

Помимо скриптов и таблиц стилей, к другим типам содержимого, которые могут привести к серьезным неприятностям, относятся удаленные шрифты, недавнее дополнение к CSS, и плагины с доступом к встраиваемой странице, например allowScriptAccess=always для Flash. Также несколько опасно загружать изображения, значки, курсоры или HTML-фреймы из недоверенных источников, хотя влияние этого в некоторой степени ограничено и зависит от конкретного использования.

Современные браузеры пытаются обнаруживать случаи, когда HTTPS-документы загружают HTTP-ресурсы, состояние, известное как mixed content. Однако делают они это довольно

непоследовательно: Internet Explorer - единственный браузер, который по умолчанию блокирует большинство типов mixed content, и ожидается, что Chrome последует его примеру, но ни Internet Explorer, ни Firefox, ни Opera последовательно не обнаруживают mixed content на тегах `<embed>`, `<object>` или `<applet>`. В браузерах, отличных от Internet Explorer, действие по умолчанию - тонкое предупреждение, например восклицательный знак рядом со значком замка, или загадочный диалог, что очень мало защищает пользователя, но может насторожить достаточно внимательного веб-разработчика.

Что касается другой разновидности mixed content - загрузки подресурсов между доменами, предлагающими разные уровни доверия, - у браузеров нет способа это обнаружить. Решение включить содержимое из сомнительных источников часто принимается слишком легкомысленно, и такие ошибки трудно заметить, пока не станет поздно.

Примечание. Еще одна интересная проблема с междоменными подресурсами состоит в том, что они могут запрашивать у браузера определенные дополнительные разрешения или учетные данные. Связанные с этим подсказки безопасности браузера обычно не спроектированы с учетом таких сценариев и не всегда достаточно ясно показывают, какой origin запрашивает разрешение и на основе каких отношений с сайтом верхнего уровня. Мы обсуждали одну такую проблему в главе 3: подсказку аутентификации, показываемую в ответ на HTTP-код 401. Несколько других связанных случаев появятся в главе 15.

Побочные каналы, связанные с приватностью

Еще одно неудачное и заслуживающее внимания следствие пробелов в политике одного источника - способность собирать информацию о взаимодействии пользователя с несвязанными сайтами. Некоторые из самых простых примеров, большинство из которых известны уже более десяти лет,^[15] включают следующее:

- Использование обработчиков `onload` для измерения времени загрузки определенных документов как признака того, были ли они ранее посещены и закешированы браузером.^[16]
- Использование `onload` и `onerror` на тегах `<img>`, чтобы увидеть, может ли быть загружено требующее аутентификации изображение на стороннем сайте, тем самым раскрывая, вошел пользователь на этот сайт или нет. Бонус: иногда сообщение об ошибке, раскрытое обработчику `onerror`, тоже будет включать фрагменты целевой страницы.
- Загрузка несвязанного веб-приложения в скрытый фрейм и изучение свойств вроде числа и имен подфреймов, созданных на этой странице, доступных через массив `<handle>.frames[]`, или набора глобальных переменных, иногда утекающих через семантику оператора `delete`, чтобы обнаружить то же самое. Естественно, набор сайтов, которые пользователь посещает или где он вошел, может быть довольно чувствительным.

Помимо этих трюков, особенно пугающий класс проблем приватности связан с двумя API, созданными несколько лет назад, чтобы помочь сайтам понимать стиль, примененный к любому элементу документа: сумму браузерных значений по умолчанию, CSS-правил и любых runtime-изменений, сделанных автоматически браузером или выполненных через JavaScript. Эти два API - `getComputedStyle`, предписанный CSS Level 2,^[17] и `currentStyle`, проприетарный для Internet Explorer.^[18] Их функциональность, вместе со способностью назначать отличительное оформление посещенным ссылкам с помощью псевдокласса `:visited`, означает, что любой вредоносный JavaScript может быстро отобразить и исследовать тысячи URL, чтобы увидеть, какие из них окрашены иначе из-за присутствия в истории просмотра пользователя, тем самым строя

надежный, обширный и, возможно, компрометирующий обзор онлайн-привычек пользователя с беспрецедентной эффективностью и надежностью.

Эта проблема известна по меньшей мере с 2002 года, когда Эндрю Кlover опубликовал краткую заметку о ней в популярной рассылке BUGTRAQ.^[19] В последующие годы проблема получала мало внимания, пока примерно в 2006 году не появилась серия демонстраций, нацеленных на неспециалистов, и последующий общественный резонанс. Несколько лет спустя Firefox и браузеры WebKit внедрили улучшения безопасности, чтобы ограничить степень оформления, возможную в селекторах `:visited`, и ограничить способность инспектировать получающиеся составные CSS-данные.

Тем не менее такие исправления никогда не будут идеальными. Даже если они делают автоматизированный сбор данных невозможным, меньшие объемы данных можно получить с помощью пользователя. Конкретный пример: Коллин Джексон и несколько других исследователей предложили простую схему с fake CAPTCHA^[20], состоящей из семисегментных, похожих на LCD цифр.^[21] Вместо реальной рабочей задачи число, которое видел бы пользователь, зависело от основанного на `:visited` оформления, примененного к наложенным ссылкам (см. рис. 11-5); вводя это число обратно на страницу, пользователь невольно сообщал автору сайта, какое именно оформление было применено и, следовательно, какие сайты появились в истории просмотра жертвы.

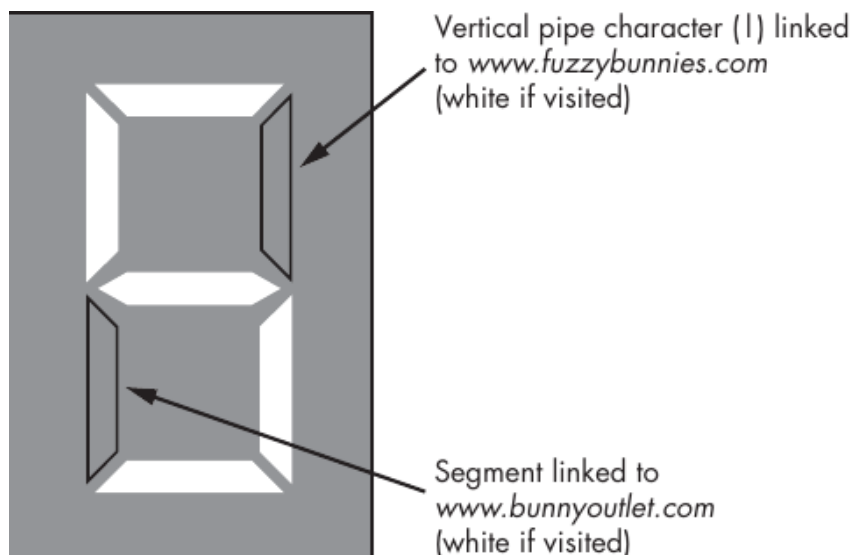


Рис. 11-5. Поддельный семисегментный дисплей для чтения оформления ссылок

Другие лазейки SOP и их применения

Хотя эта глава сосредоточилась на областях, где ограничения политики одного источника имеют ясное негативное влияние на безопасность или приватность онлайн-просмотра, в схеме есть несколько случайных пробелов, которые в большинстве случаев не имеют особых последствий. Например, во многих версиях Internet Explorer было возможно манипулировать значением `window.opener` или `window.name` несвязанного окна. Тем временем в Firefox сейчас нет ограничений на установку `location.hash` между доменами, хотя все остальные частичные свойства `location` ограничены.

Главное значение этих механизмов в том, что их часто перепрофилируют для построения междоменных каналов коммуникации в браузерах, не поддерживающих `API.postMessage(...)`. Такие механизмы часто построены на шаткой почве: отсутствие применения SOP обычно

единообразно и означает, что любой сайт, а не только "авторизованные" стороны, сможет вмешиваться в данные. Способность вредоносных сторон направлять вложенные фреймы, как обсуждалось в разделе "Риски захвата фреймов" на странице 175 оригинала, дополнительно усложняет картину.

Памятка по инженерии безопасности

Хорошая гигиена безопасности для всех сайтов

- Отдавайте все содержимое своего сайта с X-Frame-Options: sameorigin. Делайте точечные исключения только для конкретных, хорошо понятых местоположений, которым требуется междоменное встраивание. Старайтесь не зависеть от JavaScript-кода "framebusting" для предотвращения фреймирования, потому что такой код очень трудно написать правильно.
- Возвращайте специфичные для пользователя, чувствительные данные, не предназначенные для междоменной загрузки, в хорошо ограниченных форматах, которые вряд ли будут ошибочно интерпретированы как самостоятельные скрипты, таблицы стилей и так далее. Всегда используйте правильный Content-Type.

При включении междоменных ресурсов

- Во многих сценариях, особенно при работе со скриптами, таблицами стилей, шрифтами и некоторыми типами содержимого, обрабатываемого плагинами, вы связываете безопасность своего сайта с исходным доменом подресурса. Если сомневаетесь, сделайте локальную копию данных. На HTTPS-сайтах требуйте, чтобы все подресурсы отдавались по HTTPS.

При организации междоменных коммуникаций в JavaScript

- Обратитесь к памятке в главе 9. Не используйте схемы cross-frame communication на основе location.hash, window.name, frameElements и похожих эфемерных трюков, если не готовы иметь дело с внедренным содержимым.
- Не ожидайте, что подфреймы на вашей странице будут сидеть спокойно, особенно если вы не используете X-Frame-Options, чтобы ограничить способность других сайтов фреймировать ваше приложение. В некоторых случаях злоумышленник может направить такие фреймы в другое место без вашего ведома или согласия.

Сноски

[1] [\[назад\]](#) Одна потенциальная атака такова: открыть легитимный сайт, скажем <http://trusted-bank.com/>, в новом окне, дождаться, пока пользователь осматривает адресную строку, а затем быстро изменить местоположение на контролируемый злоумышленником, но похожий по имени сайт, например <http://trustea-bank.com/>. Вероятность успешно обмануть жертву может быть выше, чем когда пользователь сразу переходит на плохой URL.

[2] [\[назад\]](#) Сноска в оригинале указывает на ранние ограничения навигации фреймов.

[3] [\[назад\]](#) Сноска в оригинале указывает на исправления Firefox 2008 года для навигации фреймов.

[4] [\[назад\]](#) Сноска в оригинале указывает на работу Adam Barth, Collin Jackson и John C. Mitchell о политике навигации фреймов.

[5] [\[назад\]](#) Сноска в оригинале указывает на публикацию Robert Hansen и Jeremiah Grossman о clickjacking.

[6] [\[назад\]](#) Сноска в оригинале указывает на обсуждение сложности renderer-level защит.

[7] [\[назад\]](#) Сноска в оригинале указывает на документацию X-Frame-Options.

[8] [\[назад\]](#) В старых версиях Internet Explorer разработчики веб-приложений иногда прибегали к JavaScript, пытаясь определить, совпадает ли объект window с parent, условие, которое должно выполняться, если фрейма более высокого уровня нет. К

сожалению, из-за гибкости JavaScript DOM такие проверки, как и многие типы возможных корректирующих действий, печально ненадежны.

[9] [\[назад\]](#) Сноска в оригинале указывает на исследование 2011 года по использованию X-Frame-Options на популярных сайтах.

[10] [\[назад\]](#) Сноска в оригинале указывает на исследования Paul Stone по drag-and-drop атакам.

[11] [\[назад\]](#) Сноска в оригинале указывает на описание selective keystroke redirection.

[12] [\[назад\]](#) Сноска в оригинале указывает на исследование Chris Evans 2009 года о CSS-парсерах.

[13] [\[назад\]](#) Сноска в оригинале указывает на proof-of-concept атаку против Twitter и последующие исправления Microsoft.

[14] [\[назад\]](#) Сноска в оригинале указывает на спецификацию HTML5 canvas.

[15] [\[назад\]](#) Сноска в оригинале указывает на ранние описания side channels через браузер.

[16] [\[назад\]](#) Сноска в оригинале указывает на использование времени загрузки для определения кэша.

[17] [\[назад\]](#) Сноска в оригинале указывает на CSS Level 2 getComputedStyle.

[18] [\[назад\]](#) Сноска в оригинале указывает на документацию Internet Explorer currentStyle.

[19] [\[назад\]](#) Сноска в оригинале указывает на заметку Andrew Clover в BUGTRAQ.

[20] [\[назад\]](#) CAPTCHA, иногда раскрываемая как Completely Automated Public Turing test to tell Computers and Humans Apart, - термин для задачи безопасности, которая считается трудной для решения компьютерными алгоритмами, но должна быть легкой для человека. Обычно реализуется показом изображения нескольких случайно выбранных, сильно искаженных символов с просьбой ввести их обратно. CAPTCHA может использоваться, чтобы препятствовать автоматизации некоторых задач, таких как открытие новых учетных записей или отправка значительных объемов email. Излишне говорить, что из-за прогресса компьютерной обработки изображений надежные CAPTCHA становятся все труднее и для людей.

[21] [\[назад\]](#) Сноска в оригинале указывает на работу Collin Jackson и соавторов о чтении истории через оформление ссылок.

Глава 12. Другие границы безопасности

Все описанные ранее политики изоляции содержимого на уровне источника, а также сопутствующая логика наследования контекста и навигации документов вместе образуют основную часть модели безопасности браузера. Непроницаемая и хрупкая, эта модель к тому же неполна: несколько интересных пограничных случаев полностью выпадают из любых схем, основанных на origin.

Риски безопасности, связанные с такими пограничными случаями, нельзя устранить простой тонкой настройкой механизмов, обсуждавшихся ранее в этой книге. Вместо этого приходится с нуля создавать дополнительные, иногда безнадежно несовершенные границы безопасности. Эти новые границы могут, например, дополнительно ограничивать способность мошеннических веб-страниц переходить к определенным URL.

В этой главе мы быстро рассмотрим некоторые из наиболее значимых примеров лазеек в модели, основанной на origin, и способы, которыми поставщики браузеров с ними справлялись.

Навигация к чувствительным схемам

В прошлом поставщики браузеров рассуждали так: нет никакого вреда в том, чтобы разрешить любой странице в интернете перейти к документу, хранящемуся на жестком диске пользователя, с помощью протокола `file:` или открыть новое окно, указывающее на привилегированный ресурс, например страницу `about:config` в Firefox. В конце концов, думали они, исходный документ и место назначения не будут иметь один origin, а значит, любой прямой доступ к чувствительным данным будет предотвращен.

Многие годы, исходя из этой логики, браузеры разрешали такую навигацию. Увы, это решение оказалось не только крайне сбивающим с толку,^[1] но и опасным. Опасность связана с тем, что многие программы, включая браузеры, склонны хранить разные типы содержимого, пришедшего из интернета, в файловой системе; временные файлы и кэшированные документы - обычный пример. Во многих случаях злоумышленник мог в некоторой степени контролировать создание и содержимое таких файлов, и, если ресурсы создавались в предсказуемом месте, последующая навигация к правильному URL `file:` могла позволить злоумышленнику выполнить собственную полезную нагрузку в этом желанном origin, с доступом к любому другому файлу на диске и, возможно, к любому другому сайту в интернете.

Сравнимо катастрофические последствия наблюдались и с различными привилегированными URL, обрабатываемыми внутри браузера. Способность напрямую переходить к адресам вроде `about:config` в Firefox не только позволяла эксплуатировать потенциальные уязвимости в привилегированных скриптах - проступок, от которого поставщики браузеров не застрахованы, - но и приводила к компрометации системы, если из-за буквального применения политики одного источника браузер наивно считал, что `about:config` и `about:blank` происходят из одного origin.

Усвоив уроки болезненных происшествий, современные браузеры обычно контролируют навигацию на основе трех уровней схем URL:

- **Неограниченные.** Эта категория включает практически все настоящие сетевые протоколы, такие как HTTP, HTTPS, FTP; большинство инкапсулирующих псевдопротоколов, таких как `mhtml:` или `jar:`; а также все схемы, зарегистрированные для плагинов и внешних приложений. Навигация к таким URL никак специально не ограничивается.

- **Частично ограниченные.** Эта категория включает несколько чувствительных к безопасности схем, таких как `file:`, и специальные псевдо-URL, такие как `javascript:` или `vbscript:`. Навигация к ним не запрещена полностью, но подчиняется дополнительным проверкам безопасности, специфичным для схемы. Например, доступ к `file:` обычно разрешается только из других документов `file:`, что требует открыть первый документ вручную. Правила навигации к URL `javascript:` обсуждались в главе 10.
- **Полностью ограниченные.** Эта категория включает привилегированные страницы в пространствах имен `about:`, `res:`, `chrome:` и похожих, специфичных для браузера. Обычным непривилегированным HTML-документам не разрешается переходить к ним ни при каких обстоятельствах.

Доступ к внутренним сетям

Проблемы с доступом к чувствительным протоколам - всего лишь прелюдия к куда более серьезному вопросу, который каким-то образом ускользнул от создателей политики одного источника. Проблема в том, что `origin`, полученные из DNS, могут не иметь никакого отношения к фактическим границам сетевого уровня или к тому, как эти границы меняются со временем. Вредоносному скрипту может быть предоставлен доступ в рамках одного `origin` к сайтам интрасети в локальной сети жертвы, даже если межсетевой экран не позволяет злоумышленнику взаимодействовать с этими адресатами напрямую.

Есть как минимум три характерные площадки для таких атак.

Инфильтрация `origin`

Когда пользователь посещает мошенническую сеть, например открытую беспроводную сеть в аэропорту или кафе, злоумышленник в этой сети может обманом заставить браузер жертвы открыть URL вроде `http://us-payroll/`. Когда это происходит, злоумышленник может предоставить собственное, подмененное содержимое для этого сайта. Пугающе то, что если затем пользователь принесет тот же браузер в корпоративную сеть, ранее внедренное содержимое получит доступ в рамках одного `origin` к настоящей версии `http://us-payroll/`, вместе с окружающими учетными данными пользователя.

Постоянство внедренного содержимого может достигаться несколькими способами. Самый простой метод для злоумышленника - просто внедрять скрытый фрейм `http://us-payroll/` на каждую посещаемую страницу в надежде, что пользователь переведет переносной компьютер в спящий режим с все еще запущенным браузером, а затем перенесет его в другую сеть. Другая техника - отравление кэша: создание долгоживущих кэшированных объектов, которые браузер будет использовать вместо получения свежей копии с целевого сайта. Существует и несколько других, более темных подходов.

DNS rebinding

Эта, пожалуй, менее серьезная, но более легко эксплуатируемая проблема упоминалась в сноске 1 в главе 9. Коротко говоря, поскольку политика одного источника смотрит только на DNS-имя хоста, а не на IP-адрес, злоумышленник, владеющий bunnyoutlet.com, волен сначала ответить на DNS-запрос пользователя публичным IP, например 213.134.128.25, а затем переключиться на адрес, зарезервированный для частных сетей, например 10.0.0.1. Документы, загруженные из обоих источников, будут считаться принадлежащими одному origin, что даст злоумышленнику способность взаимодействовать с внутренней сетью жертвы.

Смягчающий фактор в том, что такое взаимодействие не будет включать корректные окружающие учетные данные, которые жертва обычно имеет для целевого сайта: с точки зрения браузера он все еще разговаривает с bunnyoutlet.com, а не, скажем, с упомянутым выше сайтом us-payroll. И все же перспектива того, что злоумышленник сможет изучать внутреннюю сеть и, возможно, попытаться подобрать подходящие учетные данные перебором или выявлять уязвимости, тревожна.

Простая эксплуатация изъянов XSS или XSRF

Даже за пределами политики одного источника сама возможность переходить к URL интрасети означает, что злоумышленник может попытаться вслепую нацелиться на известные или предполагаемые уязвимости в локально работающем ПО. Поскольку внутренние приложения считаются защищенными от вредоносных пользователей, их часто не проектируют и не поддерживают по тем же стандартам, что и код, обращенный наружу.

Один яркий пример этой проблемы - десятки уязвимостей, обнаруженных за годы во внутренних веб-интерфейсах управления домашними сетевыми маршрутизаторами, выпущенными компаниями вроде Linksys (Cisco), Netgear, D-Link, Motorola и Siemens. Уязвимости cross-site request forgery в этих приложениях в крайних случаях могут позволить злоумышленникам получить доступ к устройству и перехватывать или изменять весь сетевой трафик, идущий к нему или через него.

Пока что разрыв между механизмами безопасности браузера и сетевой сегментацией остается нерешенной проблемой в инженерии браузеров. Несколько браузеров пытаются ограничить влияние DNS rebinding, кэшируя DNS-ответы на заранее заданное время - практика, известная как DNS pinning, - но защита несовершенна, а оставшиеся векторы атак по-прежнему существуют.

Примечание. Необычно, но в этом вопросе Internet Explorer оказывается впереди, предлагая необязательный способ снизить риск. Пользователи Microsoft в некоторой степени защищены, если в параметрах конфигурации локальной интрасети переключат загадочную настройку зоны с названием "websites in less privileged web content zone can navigate into this zone" в состояние "disable". К сожалению, модель зон в Internet Explorer несет с собой неожиданные ловушки, как мы обсудим в главе 15.

Запрещенные порты

Исследователи безопасности предупреждали, что способность браузеров отправлять в значительной степени неограниченные тела междоменных запросов, например с помощью `<form method="POST" enctype="text/plain">`, может мешать некоторым другим отказоустойчивым, но не HTTP-сетевым службам. Например, рассмотрим SMTP, доминирующий протокол передачи почты: при взаимодействии с ничего не подозревающим браузером большинство серверов, говорящих на SMTP, терпеливо проигнорируют первые несколько непонятных строк, связанных с HTTP-заголовками, а затем выполнят любые SMTP-команды, появившиеся в теле запроса. По сути, браузер мог бы использоваться как прокси для пересылки спама.

Связанная, но менее хорошо исследованная тревога, обсуждавшаяся в главе 3, - риск того, что злоумышленник будет разговаривать с не HTTP-службами, работающими в том же домене, что и целевое веб-приложение, и обманом заставит браузер неверно интерпретировать возвращенные, возможно частично контролируемые злоумышленником данные как HTML, доставленный по HTTP/0.9. Такое поведение могло бы раскрыть cookies или другие учетные данные, связанные с целевым сайтом.

Устройство HTTP делает невозможным особенно надежное решение этих проблем. Вместо этого поставщики браузеров ответили довольно неубедительно: поставкой списка запрещенных TCP-портов, к которым нельзя отправлять запросы. Для Internet Explorer версий 6 и 7 список состоит из следующих номеров портов:

Порт	Служба
19	chargen
21	ftp
25	smtp
110	pop3
119	nntp
143	imap2

Версии 8 и 9 Internet Explorer дополнительно запрещают порты 220 (imap3) и 993 (ssl imap3).

Все остальные браузеры, обсуждаемые в этой книге, используют другой, общий список:

Порт	Служба	Порт	Служба
1	tcpmux	115	sftp
7	echo	117	uucp-path
9	discard	119	nntp
11	systat	123	ntp
13	daytime	135	loc-srv
15	netstat	139	netbios
17	qotd	143	imap2
19	chargen	179	bgp
20	ftp-data	389	ldap
21	ftp	465	ssl smtp
22	ssh	512	exec
23	telnet	513	login

Порт	Служба	Порт	Служба
25	smtp	514	shell
37	time	515	printer
42	name	526	tempo
43	nickname	530	courier
53	domain	531	chat
77	priv-rjs	532	netnews
79	finger	540	uucp
87	ttylink	556	remotefs
95	supdup	563	ssl nntp
101	hostname	587	smtp submission
102	iso-tsap	601	syslog
103	gppitnp	636	ssl ldap
104	acr-nema	993	ssl imap
109	pop2	995	ssl pop3
110	pop3	2049	nfs
111	sunrpc	4045	lockd
113	auth	6000	X11

Конечно, у этих правил есть разные исключения, специфичные для протоколов. Например, URL ftp: очевидно разрешено обращаться к порту 21, обычно связанному с этим протоколом.

Текущее решение ущербно по нескольким причинам; самая важная из них, возможно, в том, что в обоих списках есть многочисленные вопиющие пропуски, и, учитывая число сетевых протоколов, придуманных к настоящему времени, у них просто нет шанса когда-либо стать исчерпывающими. Например, никакое правило не мешает браузеру разговаривать с серверами Internet Relay Chat (IRC), которые используют отказоустойчивый текстовый протокол, не совсем непохожий на SMTP.

Списки также не обновляются регулярно, чтобы отражать исчезновение почти вымерших сетевых протоколов или появление новых. Наконец, они могут несправедливо и неожиданно наказывать системных администраторов за выбор нестандартных портов для некоторых служб, которые они хотят скрыть от публичного доступа: такой выбор означает отказ от этого механизма защиты на уровне браузера.

Ограничения для сторонних cookies

С момента своего появления HTTP cookies ошибочно понимались как инструмент, позволивший онлайн-рекламодателям нарушать приватность пользователей в беспрецедентной и ранее недостижимой степени. В последующие годы это настроение повторяла массовая пресса. Например, в 2001 году New York Times опубликовала пространное разоблачение о якобы уникальных рисках HTTP cookies и даже процитировала Лоуренса Лессига, известного специалиста по праву и политического активиста:^[2]

До cookies Web был по существу приватным. После cookies Web становится пространством, способным к чрезвычайному наблюдению.

Широкая атака на один HTTP-заголовок продолжалась в течение десятилетия, постепенно смещая фокус именно к сторонним cookies. Сторонние cookies - это cookies, устанавливаемые доменами, отличными от домена документа верхнего уровня; обычно они связаны с процессом загрузки изображений, фреймов или апплетов со сторонних сайтов. Причина, по которой они привлекли внимание, в том, что операторы рекламных сетей приняли такие cookies как удобный способ пометить пользователя, который видит их рекламу, встроенную на fuzzybunnies.com, а затем узнать этого пользователя через похожую встроенную рекламу, обслуживаемую на playboy.com.

Поскольку явно нежелательная возможность выполнять междоменное отслеживание такого типа была ошибочно слита воедино с самим существованием сторонних cookies, давление на поставщиков браузеров продолжало расти. В одном случае Wall Street Journal прямо обвинил Microsoft в сговоре с рекламодателями за то, что компания не устранила сторонние cookies из своего продукта.^[3]

Разумеется, читатели этой книги поймут, что заикленность на HTTP cookies глубоко ошибочна. Нет сомнений, что некоторые стороны используют этот механизм для несколько зловещих целей, но ничто не делает его уникально подходящим для такой задачи; существует много других эквивалентных способов хранить уникальные идентификаторы на компьютерах посетителей, например теги на основе кэша, ранее обсуждавшиеся в главе 3. Кроме того, просто невозможно запретить сотрудничающим сайтам использовать существующие уникальные отпечатки каждого браузера, раскрываемые через объектную модель JavaScript или плагины вроде Flash, чтобы по своему желанию сопоставлять и анализировать шаблоны междоменного просмотра. Сайты, встраивающие рекламу ради прибыли, вполне готовы сотрудничать со сторонами, оплачивающими их счета.

На самом деле распространенная зависимость от HTTP cookies дает пользователям особое преимущество: в отличие от многих легко принимаемых альтернатив, этот механизм специально создан для такой цели и связан с достаточно хорошо спроектированными и детализированными средствами управления приватностью. Полномка cookies не мешает отслеживанию, но уберет у конечного пользователя всякую видимость прозрачности. Другой известный активист в области приватности и безопасности, Эд Фелтен, однажды сказал: "Если вы собираетесь меня отслеживать, пожалуйста, используйте cookies".^[4]

Недобросовестное онлайн-отслеживание - значимая социальная проблема, и могут понадобиться новые технические механизмы, чтобы пользователи могли сообщать свои предпочтения приватности хорошо ведущим себя сайтам, например недавно добавленный заголовок запроса DNT,^[5] развернутый в Firefox 4. Чтобы справиться с теми, кто ведет себя плохо, может потребоваться и регуляторная рамка. В отсутствие такой рамки в Internet Explorer 9 Microsoft экспериментирует с управляемым черным списком известных плохих источников отслеживающих cookies, но шансы, что это отвадит мутные деловые практики, невелики.

В любом случае, несмотря на слабую или отсутствующую содержательную ценность, продолжающийся общественный шум против сторонних cookies в конце концов привел к тому, что несколько поставщиков браузеров поставили недоделанные и легко обходимые решения, позволявшие им заявить, что они что-то сделали.

- В Internet Explorer установка и чтение сторонних cookies по умолчанию блокируются, кроме сеансовых cookies, сопровождаемых удовлетворительным заголовком P3P. P3P (Platform for Privacy Preferences)^[6] - это метод построения машиночитаемых, юридически обязывающих кратких изложений политики приватности сайта, будь то XML-файл или компактная политика в HTTP-заголовке. Например, ключевое слово TEL в HTTP-заголовке означает, что сайт использует собранную информацию для целей телемаркетинга. Никакая техническая мера не мешает сайту солгать в заголовке P3P, но предполагается, что возможные юридические последствия будут от этого удерживать.

Примечание. Невероятно амбициозная 111-страничная спецификация РЗР заставила решение рухнуть под собственным весом. Крупные компании обычно очень неохотно принимают РЗР как решение технических проблем из-за юридического следа спецификации, тогда как малый бизнес и отдельные владельцы сайтов копируют рецепты заголовков РЗР почти или совсем без понимания того, что они должны сообщать.

- В Safari задача установки сторонних cookies по умолчанию блокируется, но ранее выданные cookies можно свободно читать. Однако это поведение можно переопределить, если пользователь сначала взаимодействует с документом, устанавливающим cookie. Такое взаимодействие может быть намеренным, но вполне может быть и не таким: трюки, связанные с clickjacking и описанные в главе 11, применимы и к этому сценарию.
- В других браузерах сторонние cookies разрешены по умолчанию, но предоставляется параметр конфигурации для изменения поведения. Включение этого параметра ограничивает возможность устанавливать сторонние cookies, но чтение существующих никак не ограничивается.

Для целей этих проверок cookie считается пришедшим от третьей стороны, если оно загружено с полностью несвязанного домена. Например, фрейм, указывающий на `bunnyoutlet.com` и загруженный на `fuzzybunnies.com`, соответствует этому критерию, но `www1.fuzzybunnies.com` и `www2.fuzzybunnies.com` считаются находящимися в отношениях первой стороны. Логика, используемая для такого определения, хрупка и страдает от тех же проблем, что и область действия cookie-доменов. Например, в Internet Explorer 6 и 7 сравнения в некоторых доменах странового уровня выполняются неверно.

Примечание. Крестовый поход против сторонних cookies можно было бы считать безвредным упражнением, но у него были и отрицательные последствия. Браузеры, отвергающие сторонние cookies, очень усложняют построение аутентификации на основе cookies для встраиваемых гаджетов и других типов mashup-приложений, а также затрудняют использование "sandbox"-доменов для изоляции недоверенного, но приватного содержимого от основного приложения, чтобы ограничить влияние изъянов внедрения скриптов.

Памятка по инженерии безопасности

При создании веб-приложений во внутренних сетях

- Предполагайте, что решительные злоумышленники смогут взаимодействовать с такими приложениями через браузер жертвы, независимо от любых средств безопасности сетевого уровня. Убедитесь, что соблюдаются надлежащие инженерные стандарты, и требуйте HTTPS с защищенными cookies для всех чувствительных приложений, чтобы минимизировать риск атак инфильтрации origin.

При запуске не HTTP-служб, особенно на нестандартных портах

- Оцените влияние того, что браузеры могут непреднамеренно отправлять HTTP-запросы к службе, и влияние интерпретации ответа как HTTP/0.9. Для уязвимых протоколов рассмотрите немедленное разрывание соединения, если полученные данные начинаются с GET или POST, как одну из возможных мер предосторожности.

При использовании сторонних cookies для гаджетов или изолированного содержимого

- Если вам нужно поддерживать Internet Explorer, будьте готовы использовать политики P3P и оценить их юридическое значение. Если вам нужно поддерживать Safari, возможно, придется прибегнуть к альтернативному механизму хранения учетных данных, например HTML5 localStorage.

Сноски

[1] [\[назад\]](#) Например, в системах Windows распространенной шуткой было незаметно встроить <iframe>, указывающий на file:///c:/, чтобы показать содержимое жесткого диска жертвы; из-за этого некоторые пользователи верили, что страница каким-то образом получила доступ к их файлам.

[2] [\[назад\]](#) John Schwartz, "Giving Web a Memory Cost Its Users Privacy", New York Times, 4 сентября 2001 г.

[3] [\[назад\]](#) Julia Angwin и Geoffrey A. Fowler, "Microsoft Quashed Effort to Boost Online Privacy", Wall Street Journal, 2 августа 2010 г.

[4] [\[назад\]](#) Ed Felten, "If You're Going to Track Me, Please Use Cookies", Freedom to Tinker, 13 сентября 2008 г.

[5] [\[назад\]](#) Заголовок Do Not Track описан Mozilla как способ передать предпочтение пользователя не отслеживаться.

[6] [\[назад\]](#) Platform for Privacy Preferences Project, спецификация W3C.

Глава 13. Механизмы распознавания содержимого

До сих пор мы рассмотрели немало браузерных возможностей, задуманных с добрыми намерениями, которые по мере взросления технологии оказались неадекватными и прямо опасными. Но теперь приготовьтесь к чему-то особому: в истории Web ничто не оказалось настолько ошибочным, как content sniffing.

Изначальная предпосылка распознавания содержимого была простой: поставщики браузеров предполагали, что в некоторых случаях будет уместно, даже желательно, игнорировать обычно авторитетные метаданные, полученные от сервера, например заголовок Content-Type. Вместо того чтобы уважать заявленное разработчиком намерение, реализации, поддерживающие content sniffing, могут попытаться угадать правильный образ действий, применяя к возвращенной полезной нагрузке собственные эвристики, чтобы компенсировать возможные ошибки. Напомним из главы 1, что во время Первой войны браузеров поставщики превратили отказоустойчивую совместимость в плохо продуманное конкурентное преимущество.

Совсем скоро функции content sniffing стали существенной и вредной частью общей картины безопасности браузеров. К своему ужасу и недоверию веб-разработчики вскоре заметили, что они не могут безопасно размещать некоторые номинально безвредные типы документов, такие как text/plain или text/csv, от имени своих пользователей; любая попытка сделать это неизбежно создавала риск, что такое содержимое будет неверно истолковано как HTML.

Возможно, отчасти в ответ на эти опасения в 1999 году практика незапрошенного content sniffing была явно запрещена в HTTP/1.1:

Если и только если media type не задан полем Content-Type, получатель может попытаться угадать media type путем проверки его содержимого и/или расширений имени URI, использованного для идентификации ресурса.

Увы, это необычно ясное требование появилось немного поздно. Большинство браузеров уже в той или иной степени нарушали это правило, и, не имея удобного способа оценить возможные последствия, их авторы колебались и не спешили просто выбросить нарушающий код. Хотя несколько самых вопиющих ошибок за последнее десятилетие осторожно откатили, две компании - Microsoft и Apple - в значительной степени сопротивлялись этим усилиям. Они решили, что совместимость со сломанными веб-приложениями должна перевешивать очевидные проблемы безопасности. Чтобы успокоить критиков, они реализовали пару несовершенных вторичных механизмов безопасности, призванных снизить риск.

Сегодня лоскутное одеяло политик обработки содержимого и затем развернутых ограничений отбрасывает длинную тень на онлайн-мир, делая почти невозможным создание некоторых типов веб-сервисов без надуманных и иногда дорогих трюков. Чтобы понять эти ограничения, начнем с описания нескольких сценариев, в которых номинально пассивный документ может быть ошибочно определен как HTML или что-то похожее.

Логика определения типа документа

Самый простой и наименее спорный тип эвристик определения документа, реализованный во всех современных браузерах, - это логика обработки отсутствующего заголовка Content-Type. Такая ситуация, встречающаяся очень редко, может быть вызвана тем, что разработчик случайно пропустил или неправильно набрал имя заголовка, либо тем, что документ загружается через не HTTP-транспорт, например ftp: или file:.

Для HTTP исходные RFC явно разрешают браузеру искать подсказки в полезной нагрузке, когда значение Content-Type недоступно. Для других протоколов обычно используется тот же подход, часто как естественное следствие устройства нижележащего кода.

Эвристики, используемые для определения типа документа, обычно сводятся к проверке статических сигнатур, связанных с несколькими десятками известных форматов файлов, таких как изображения и распространенные файлы, обрабатываемые плагинами. Ответ также будет просканирован на известные подстроки, чтобы обнаружить форматы без сигнатур, например HTML; в этом случае браузер будет искать знакомые теги, такие как <body>, и т. д. Во многих браузерах также учитываются сигналы, не относящиеся к содержимому, например конечные строки .html или .swf в сегменте пути URL.

Конкретика логики content sniffing резко различается от одного браузера к другому и плохо документирована или стандартизована. Для примера рассмотрим обработку файлов Adobe Flash (SWF), отданных без Content-Type: в Opera они безусловно распознаются по проверке сигнатуры содержимого; в Firefox и Safari требуется явный суффикс .swf в URL; а Internet Explorer и Chrome вообще не распознают SWF автоматически.

Будьте уверены, формат SWF не исключительный случай. Например, при работе с HTML-файлами Chrome и Firefox автоматически определяют документ только если один из нескольких заранее заданных HTML-тегов появится в самом начале файла; Firefox при этом охотно "обнаружит" HTML только по наличию расширения .html в URL, даже если узнаваемой разметки не видно. Internet Explorer, напротив, просто всегда по умолчанию выберет HTML при отсутствии Content-Type, а Opera просканирует первые 1000 байтов возвращенной полезной нагрузки на известные HTML-теги.

Предположение, стоящее за всем этим безумием, состоит в том, что отсутствие Content-Type выражает намеренное желание издателя страницы. Но это предположение не всегда верно и стало причиной немалого числа ошибок безопасности. При этом большинство веб-серверов активно принуждают наличие заголовка Content-Type и вставляют значение по умолчанию, если оно явно не сгенерировано серверными скриптами, обрабатывающими запросы пользователей. Так что, возможно, беспокоиться не о чем? К сожалению, на этом история content sniffing не заканчивается.

Некорректные MIME-типы

HTTP RFC разрешает content sniffing только при отсутствии данных Content-Type; браузеру открыто запрещено переугадывать намерение вебмастера, если заголовок присутствует в любой форме. На практике, однако, к этому совету не относятся серьезно. Следующий маленький шаг с обрыва состоял в решении включать эвристики, если MIME-тип, возвращенный сервером, сочтен хоть как-то недействительным.

Согласно RFC, заголовок Content-Type должен состоять из двух буквенно-цифровых токенов, разделенных косой чертой (type/subtype), потенциально с последующими параметрами, разделенными точкой с запятой. Эти токены могут содержать любые непробельные семибитные

ASCII-символы, кроме нескольких специальных "разделителей", общего набора, включающего символы вроде @, ? и самой кривой черты. Большинство браузеров пытается принудить этот синтаксис, но делает это непоследовательно; отсутствие кривой черты почти повсеместно воспринимается как приглашение к content sniffing, как и включение пробелов и некоторых, но не всех, управляющих символов в первой части идентификатора, то есть в токене типа. С другой стороны, технически незаконное использование символов с установленным старшим битом или разделителей влияет на действительность этого поля только в Opera.

Причины такого устройства трудно понять, но справедливости ради влияние на безопасность пока все еще довольно ограничено. С точки зрения разработчиков веб-приложений, нужно проявлять осторожность, не делать опечаток в значениях Content-Type и не позволять пользователям задавать произвольные, контролируемые ими MIME-типы, просто проверенные по черному списку известных плохих вариантов. Эти требования могут быть неожиданными, но обычно они не имеют большого значения. Так к чему мы в итоге подходим?

Специальные значения Content-Type

Первым ясным сигналом, что content sniffing становится по-настоящему опасным, стала обработка на вид ничем не примечательного MIME-типа, известного как application/octet-stream. Это конкретное значение вообще не упоминается в спецификации HTTP, но получает особую, хотя и расплывчатую, роль глубоко внутри RFC 2046.^[1]

Рекомендуемое действие для реализации, получающей сущность application/octet-stream, - просто предложить поместить данные в файл, отменив любой Content-Transfer-Encoding, или, возможно, использовать их как вход для процесса, заданного пользователем.

Первоначальный смысл этого MIME-типа может быть не кристально ясен из одной только цитаты, но обычно он понимается как способ для веб-серверов указать, что возвращенный файл не имеет специального значения для сервера и не должен иметь его для клиента. Следовательно, большинство веб-серверов по умолчанию используют application/octet-stream для всех типов непрозрачных, не веб-файлов, таких как загружаемые исполняемые файлы или архивы, если лучшего соответствия Content-Type не найдено. Однако в редких случаях ошибок администратора, например из-за удаления важных директив AddType в конфигурационных файлах Apache, веб-серверы могут также откатываться к этому MIME-типу для документов, предназначенных для потребления внутри браузера. Эту ошибку конфигурации, конечно, очень легко обнаружить и исправить, но Microsoft, Opera и Apple все равно решили ее компенсировать. Браузеры этих поставщиков охотно включают content sniffing всякий раз, когда видят application/octet-stream.^[2]

Это конкретное проектное решение внезапно усложнило для веб-приложений размещение бинарных файлов от имени пользователя. Например, любая платформа хостинга кода должна проявлять осторожность, возвращая исполняемые файлы или архивы исходного кода как application/octet-stream, потому что есть риск, что они будут неверно истолкованы как HTML и показаны inline. Это большая проблема для любой системы хостинга ПО или веб-почты и для многих других типов веб-приложений. Им немного безопаснее использовать любой другой обобщенно звучащий MIME-тип, например application/binary, потому что для него в коде браузера нет специального случая.

Помимо особой обработки application/octet-stream, существует второе, куда более разрушительное исключение для text/plain. Это решение, уникальное для Internet Explorer и

Safari, восходит к RFC 2046. В этом документе `text/plain` получает двойную функцию: во-первых, передавать простые текстовые документы, то есть такие, которые "не предусматривают и не допускают команд форматирования, спецификаций атрибутов шрифта, инструкций обработки, директив интерпретации или разметки содержимого"; во-вторых, предоставлять fallback-значение для любых текстовых документов, иначе не распознанных отправителем.

Различие между fallback для `application/octet-stream` и `text/plain` имело полный смысл для почтовых сообщений, теме, которой изначально был посвящен этот RFC, но оказалось куда менее релевантным для Web. Тем не менее некоторые веб-серверы приняли `text/plain` как fallback-значение для определенных типов ответов, наиболее заметно для вывода CGI-скриптов.

Логика `text/plain`, впоследствии реализованная в Internet Explorer и Safari для обнаружения HTML в таком случае, - очень плохая новость: она лишает веб-разработчиков способности безопасно использовать этот MIME-тип для генерации пользовательских простых текстовых документов и не предлагает альтернатив. Это привело к существенному числу уязвимостей веб-приложений, но по сей день разработчики Internet Explorer, похоже, не раскаиваются и не изменили поведение своего кода по умолчанию.

Разработчики Safari, с другой стороны, осознали риск и попытались снизить его, сохранив функциональность на месте, но не оценили сложность Web. Решение, реализованное в их браузере, состоит в том, чтобы полагаться на вторичный сигнал в дополнение к наличию правдоподобно выглядящей HTML-разметки в теле документа. Наличие расширения вроде `.html` или `.xml` в конце пути URL интерпретируется их реализацией как признак того, что `content sniffing` можно выполнять безопасно. В конце концов, владелец сайта иначе не назвал бы файл таким образом, верно?

Увы, принятый ими сигнал почти ничего не стоит. Как выясняется, почти все веб-фреймворки поддерживают как минимум один из нескольких методов кодирования параметров в сегменте пути URL вместо более традиционно используемой `query`-части. Например, в Apache один такой механизм известен как `PATH_INFO`, и он включен по умолчанию. Используя такую схему передачи параметров, злоумышленник обычно может добавить нефункциональный мусор к пути, тем самым сбив браузер с толку, не влияя на то, как сервер ответит на сам отправленный запрос.

Для иллюстрации следующие два URL, вероятно, дадут один и тот же эффект для сайтов, работающих на Apache или IIS:

```
http://www.fuzzybunnies.com/get_file.php?id=1234
```

и

```
http://www.fuzzybunnies.com/get_file.php/evil.html?id=1234
```

В некоторых менее распространенных веб-фреймворках может сработать и следующий подход:

```
http://www.fuzzybunnies.com/get_file.php;evil.html?id=1234
```

Нераспознанный тип содержимого

Несмотря на очевидные проблемы с `text/plain`, инженеры, работавшие над Internet Explorer, решили завести эвристики браузера еще дальше. Internet Explorer применяет и `content sniffing`, и сопоставление расширений^[3] не только к горстке обобщенных MIME-типов, но и к любому типу документа, который браузер немедленно не распознает. Эта широкая категория может включать все, от JSON (`application/json`) до мультимедийных форматов вроде Ogg Vorbis (`audio/ogg`).

Такое устройство, естественно, проблематично и вызывает серьезные проблемы при размещении любых контролируемых пользователем форматов документов, кроме небольшого списка универсально поддерживаемых MIME-типов, зарегистрированных внутри браузера или направляемых в несколько часто устанавливаемых внешних приложений.

На этом привычки Internet Explorer к `content sniffing` тоже окончательно не заканчиваются: браузер также прибегает к проверке полезной нагрузки при работе с внутренне распознанными форматами документов, которые по какой-либо причине не удастся корректно разобрать. В версиях Internet Explorer до 8 отдача предоставленного пользователем, но не проверенного файла, заявляющего себя как JPEG-изображение, может привести к обработке ответа как HTML. И становится еще веселее: даже тонкая ошибка, например отдача допустимого GIF-файла с `Content-Type: image/jpeg`, запускает тот же путь кода. Черт возьми, несколько лет назад Internet Explorer даже обнаруживал HTML в любом допустимом, корректно отданном PNG-файле. К счастью, эта логика с тех пор отключена, но оставшиеся странности все еще похожи на минное поле.

Примечание. Чтобы полностью оценить риск `content sniffing` на допустимых изображениях, заметьте, что не особенно трудно построить изображения, которые корректно проходят проверку, но несут выбранные злоумышленником ASCII-строки, например HTML-разметку, в сырых данных изображения. На самом деле сравнительно легко построить изображения, у которых после очистки, масштабирования и повторного сжатия известным детерминированным алгоритмом в результирующем бинарном потоке почти из ниоткуда появится почти произвольная строка.

К чести Microsoft, в Internet Explorer 8 и новее компания решила запретить большинство типов `gratuitous content sniffing` для известных MIME-типов в категории `image/*`. Она также запретила обнаружение HTML, но не XML, для форматов изображений, не распознаваемых браузером, таких как `image/jp2` (JPEG2000).

Если не считать этой одной настройки, Microsoft показала довольно слабую готовность вносить значимые изменения в свою логику `content sniffing`, и ее инженеры публично защищали необходимость поддерживать совместимость со сломанными сайтами.^[4] Вероятно, Microsoft хочет избежать гнева крупных институциональных клиентов, многие из которых полагаются на древние и плохо спроектированные интрасетевые приложения и зависят от особенностей монокультуры на клиентской стороне, основанной на Internet Explorer.

В любом случае из-за негативной реакции, с которой Internet Explorer столкнулся из-за логики обработки `text/plain`, более новые версии предлагают частичный обходной путь: необязательный HTTP-заголовок `X-Content-Type-Options: nosniff`, который позволяет владельцам сайтов отказаться от большинства спорных эвристик содержимого. Использование этого заголовка настоятельно рекомендуется; к сожалению, его поддержку не перенесли назад в версии 6 и 7 браузера, а в других браузерах она ограничена. Иными словами, на него нельзя полагаться как на единственную защиту от `content sniffing`.

Примечание. Пища для размышления: согласно данным, собранным в исследовании SHODAN и Криса Джона Райли в 2011 году,^[5] только около 0,6 процента из 10 000 самых популярных сайтов в интернете использовали этот заголовок на уровне всего сайта.

Защитное использование Content-Disposition

Заголовок Content-Disposition, несколько раз упоминавшийся в части I этой книги, в некоторых случаях можно считать защитой от content sniffing. Функция этого заголовка в спецификации HTTP/1.1 удовлетворительно не объясняется. Вместо этого он документирован только в RFC 2183,^[6] где его роль объясняется лишь применительно к почтовым приложениям:

Части тела могут быть обозначены как attachment, чтобы указать, что они отделены от основного тела почтового сообщения и что их отображение не должно быть автоматическим, а должно зависеть от некоторого дальнейшего действия пользователя. MUA^[7] может вместо этого представить пользователю bitmap-терминала иконическое представление вложений или, на символьных терминалах, список вложений, из которого пользователь может выбрать для просмотра или сохранения.

HTTP RFC признает использование Content-Disposition: attachment в веб-домене, но не раскрывает его предполагаемую функцию. На практике, увидев этот заголовок при обычной загрузке документа, большинство браузеров покажут диалог загрузки файла, обычно с тремя кнопками: "open", "save" и "cancel". Браузер не будет пытаться интерпретировать документ дальше, если только не выбран вариант "open" или документ не сохранен на диск и затем открыт вручную. Для варианта "save" необязательный параметр filename, включенный в заголовок, также используется, чтобы предложить имя загрузки. Если это поле отсутствует, имя файла будет выведено из печально ненадежных данных пути URL.

Поскольку заголовок не дает большинству браузеров немедленно интерпретировать и отображать возвращенную полезную нагрузку, он особенно хорошо подходит для безопасного размещения непрозрачных загружаемых файлов, таких как упомянутые выше архивы или исполняемые файлы. Более того, поскольку он игнорируется при загрузках типоспецифичных подресурсов, таких как `<img>` или `<script>`, его также можно применять для защиты контролируемых пользователем JSON-ответов, изображений и т. д. от рисков content sniffing. Причина, по которой все реализации игнорируют Content-Disposition для этих типов навигации, не особенно ясна, но, учитывая преимущества, сейчас лучше не ставить эту логику под вопрос.

Один пример достаточно надежного использования Content-Disposition и других HTTP-заголовков для сдерживания content sniffing на JSON-ответе может выглядеть так:

```
Content-Type: application/json; charset=utf-8
X-Content-Type-Options: nosniff
Content-Disposition: attachment; filename="json_response.txt"

{ "search_term": "<html><script>alert('Hi mom!')</script>", ... }
```

Защитное использование Content-Disposition настоятельно рекомендуется там, где возможно, но важно понимать, что механизм не является ни обязательным для всех user agents, ни хорошо документированным. В менее популярных браузерах, таких как Safari Mobile, заголовок может не иметь эффекта; в основных браузерах, таких как Internet Explorer 6, Opera и Safari, серия багов Content-Disposition в тот или иной момент делала заголовок неэффективным в случаях, контролируемых злоумышленником.

Другая проблема зависимости от Content-Disposition в том, что пользователь все еще может быть склонен нажать "open". От обычных пользователей нельзя ожидать настороженности к просмотру Flash-апплетов или HTML-документов только потому, что на пути появляется prompt загрузки. В большинстве браузеров выбор "open" помещает документ в origin file:, что само по

себе может быть проблематичным; недавние улучшения в Chrome, конечно, помогают. В Opera документ будет показан в контексте исходного домена. Возможно, Internet Explorer делает лучший выбор: HTML-документы помещаются в специальную песочницу с помощью механизма mark-of-the-web, подробнее описанного в главе 15, но даже в этом браузере Java- или Flash-апплеты не получают пользы от этой возможности.

Директивы содержимого для подресурсов

Большинство HTTP-заголовков, связанных с содержимым, таких как Content-Type, Content-Disposition и X-Content-Type-Options, в значительной степени не влияют на загрузки типоспецифичных подресурсов, таких как `<img>`, `<script>` или `<embed>`. В этих случаях встраивающая сторона почти полностью контролирует, как ответ будет интерпретирован браузером.

Content-Type и Content-Disposition также могут не получать большого внимания при обработке запросов, инициированных из кода, выполняемого плагином. Например, вспомните из главы 9, что любые документы text/plain или text/csv могут быть интерпретированы Adobe Flash как чувствительные к безопасности политики crossdomain.xml, если в корневом каталоге целевого сервера нет подходящей метаполитики на уровне сайта. Хотите ли вы назвать это "content sniffing" или просто "слепотой к content-type", проблема все равно вполне реальна.

Следовательно, даже когда все ранее обсужденные HTTP-заголовки используются религиозно, важно всегда учитывать возможность того, что сторонняя страница может обманом заставить браузер интерпретировать эту страницу как один из нескольких проблемных типов документов; апплеты и связанное с апплетами содержимое, PDF, таблицы стилей и скрипты обычно вызывают особую тревогу. Чтобы минимизировать риск происшествий, следует тщательно ограничивать структуру и набор символов любой отдаваемой полезной нагрузки или использовать "sandbox"-домены для изоляции любых типов документов, которые нельзя особенно хорошо ограничить.

Загруженные файлы и другое не HTTP-содержимое

Поведение HTTP-заголовков вроде Content-Type, Content-Disposition и X-Content-Type-Options может быть запутанным и полным исключений, но по крайней мере они складываются в достаточно последовательное целое. И все же легко забыть, что во многих реальных случаях метаданные, содержащиеся в этих заголовках, просто недоступны, и тогда все ставки сняты. Например, обработка документов, полученных через ftp: или сохраненных на диск и открытых через протокол file:, сильно зависит от браузера и протокола и часто удивляет даже самых опытных экспертов по безопасности.

При открытии локальных файлов браузеры обычно отдают приоритет данным расширения файла, и если расширение является одним из зашифрованных значений, известных браузеру, например .txt или .html, большинство браузеров принимает эту информацию за чистую монету. Chrome - исключение; он попытается автоматически определить некоторые "пассивные" типы документов, такие как JPEG, даже внутри документов .txt. HTML, однако, строго вне пределов.

Когда дело касается других расширений, зарегистрированных для внешних программ, поведение немного менее предсказуемо. Internet Explorer обычно вызовет внешнее приложение, но большинство других браузеров прибегнет к content sniffing, ведя себя так, будто загрузили документ по HTTP без установленного Content-Type. Все браузеры также откатятся к content sniffing, если расширение неизвестно, скажем .foo.

Сильная зависимость от данных расширения файла и content sniffing для документов file: создает интересный контраст с нормальной обработкой ресурсов, происходящих из интернета. В Web Content-Type в целом является авторитетным описателем типа документа. Информация о расширении файла большую часть времени игнорируется, и совершенно законно разместить функциональный JPEG-файл по адресу вроде <http://fuzzybunnies.com/gotcha.txt>. Но что происходит, когда этот документ скачивают на диск? В таком случае эффективное значение ресурса неожиданно изменится: при доступе к нему через протокол file: браузер может настаивать на отображении его как текстового файла, строго на основе данных расширения.

Пример выше довольно безвреден, но другие векторы продвижения содержимого, например превращение изображения в исполняемый файл, могут быть более тревожными. В этом отношении Opera и Internet Explorer попытаются изменить расширение, чтобы оно соответствовало MIME-типу для нескольких известных значений Content-Type. Другие браузеры, однако, не предлагают такой степени защиты и могут даже быть полностью сбиты с толку ситуацией, в которой оказались. Рис. 13-1 запечатлел Firefox в один такой неловкий момент.

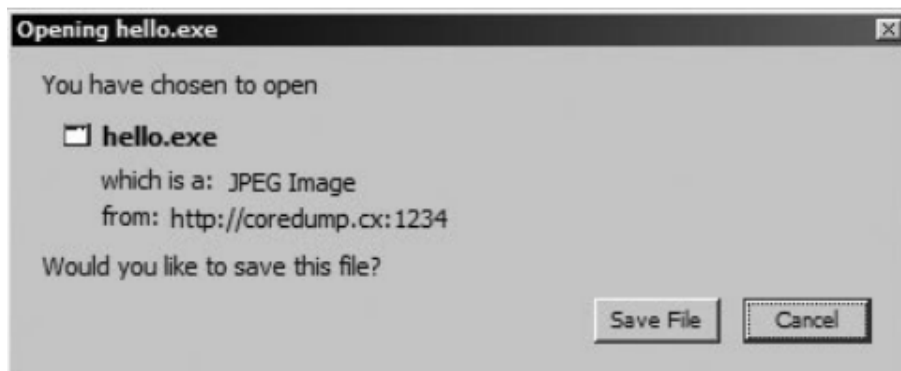


Рис. 13-1. Prompt, показанный Firefox при сохранении документа Content-Type: image/jpeg, отданного с Content-Disposition: attachment. Имя файла hello.exe выводится браузером из нефункционального суффикса PATH_INFO, добавленного злоумышленником в конец URL. Prompt ошибочно утверждает, что файл .exe является JPEG Image; фактически после сохранения на диск это будет исполняемый файл.

Эта проблема подчеркивает важность возврата явного, безвредного значения имени файла всякий раз, когда используется Content-Disposition: attachment, чтобы жертву нельзя было обманом заставить скачать формат документа, который владелец сайта никогда не намеревался размещать.

С учетом сложной логики, используемой для URL file:, простота обработки ftp: может стать шоком. При доступе к документам по FTP большинство браузеров не уделяет специального внимания расширениям файлов и просто передается необузданному content sniffing. Одно исключение - Opera, где данные расширения все еще имеют приоритет. С инженерной точки зрения преобладающий подход к FTP может казаться логичным: протокол можно считать примерно эквивалентным HTTP/0.9. Тем не менее это устройство также нарушает принцип наименьшего удивления. Владелец серверов не ожидают, что, разрешая пользователям загружать документы .txt на FTP-сайт, они автоматически соглашаются размещать активное HTML-содержимое внутри своего домена.

Обработка наборов символов

Определение типа документа - одна из более важных частей головоломки обработки содержимого, но точно не единственная. Для всех типов текстовых файлов, отображаемых в браузере, нужно принять еще одно решение: необходимо определить и применить к входному потоку подходящее преобразование набора символов. Выходная кодировка, которую ищет браузер, обычно UTF-8 или UTF-16; входная, напротив, остается на усмотрение автора страницы.

В простейшем сценарии подходящий метод кодирования будет предоставлен сервером в параметре `charset` заголовка `Content-Type`. В случае HTML-документов та же информация в некоторой степени может передаваться и через директиву `<meta>`. Браузер попытается спекулятивно извлечь и интерпретировать эту директиву до фактического разбора документа.

К сожалению, опасные свойства некоторых кодировок символов, а также действия браузера, когда параметр `charset` отсутствует или не распознан, снова делают жизнь намного интереснее, чем подразумевало бы упомянутое простое правило. Чтобы понять, что может пойти не так, сначала нужно признать три особых класса наборов символов, которые могут изменить семантику HTML- или XML-документов:

- Наборы символов, допускающие неканонические представления стандартных 7-битных ASCII-кодов. Такие неканонические последовательности могут использоваться, чтобы хитро закодировать элементы HTML-синтаксиса, например угловые скобки или кавычки, способом, переживающим простую серверную проверку. Например, печально проблемная кодировка UTF-7 позволяет закодировать символ `<` как пятисимвольную последовательность `+ADw-`, строку, которую большинство серверных фильтров радостно пропустит как есть. В похожем духе спецификация UTF-8 формально запрещает, но технически допускает представление `<` излишне многословными 2-5-байтовыми последовательностями, от `0xC0 0xBC` до `0xFC 0x80 0x80 0x80 0xBC`.^[8]
- Кодировки переменной длины, которые придают особое значение одному или нескольким байтам, следующим за специальным префиксом. Такая логика может привести к тому, что легитимные элементы HTML-синтаксиса будут "поглощены" как часть непреднамеренного многобайтового литерала. Например, префиксный код Shift JIS `0xE0` может привести к поглощению последующей угловой скобки или кавычки в Internet Explorer, Firefox и Opera, но не в Chrome, возможно серьезно меняя смысл inline-разметки.
- Кодировки, полностью несовместимые с 8-битным ASCII. Эти случаи просто приведут к очень разному взгляду клиента и сервера на структуру документа. Распространенные примеры включают UTF-16 или UTF-32.

Может возникнуть и противоположная проблема: сервер может быть убежден, что выводит многобайтовый литерал, но этот литерал может быть отвергнут браузером и интерпретирован как несколько отдельных символов. В EUC-KR префикс `0x8E` учитывается только если последующий символ имеет ASCII-код `0x41` или выше. Если он меньше, ожидаемого эффекта не будет, но не все серверные реализации могут это заметить.

Итог в том, что если сервер не владеет безупречно набором символов, который он генерирует, и не уверен, что клиент не применит неожиданное преобразование к полезной нагрузке, могут возникнуть серьезные осложнения. Например, рассмотрим веб-приложение, которое удаляет угловые скобки из выделенной контролируемой пользователем строки в следующем фрагменте HTML:

```
You are currently viewing:
<span class="blog_title">
  +ADw-script+AD4-alert("Hi mom!")+ADw-/script+AD4-
</span>
```

Если этот документ интерпретируется принимающей стороной как UTF-7, фактически разобранный разметка будет выглядеть так:

```
You are currently viewing:
<span class="blog_title">
  <script>alert("Hi mom!")</script>
</span>
```

Похожая проблема, на этот раз связанная с поглощением байта в кодировке Shift JIS, показана ниже. Многобайтовому префиксу разрешено поглотить закрывающую кавычку, и в результате соответствующий HTML-тег не завершается ожидаемым образом, позволяя злоумышленнику внедрить в разметку дополнительный обработчик onerror:

```

...this is still a part of the markup...
...but the server doesn't know...
" onerror="alert('This will execute!')"
<div>
...page content continues...
</div>
```

Просто необходимо предотвращать автоматическое определение набора символов для всех текстовых документов, содержащих любые контролируемые пользователем данные. Большинство браузеров включает определение набора символов, если параметр charset не найден в заголовке Content-Type или в теге <meta>. Между реализациями есть заметные различия, например только Internet Explorer охотно обнаруживает UTF-7, но никогда не следует предполагать, что результат sniffing набора символов будет безопасным.

Автоматическое определение набора символов также будет предпринято, если набор символов не распознан или набран с ошибкой; эта проблема усугубляется тем, что именование charset может быть неоднозначным, а веб-браузеры непоследовательны в том, насколько терпимо относятся к распространенным вариациям имен. Как один пример, Internet Explorer распознает и ISO-8859-2, и ISO8859-2 без дефиса после части ISO как допустимые идентификаторы набора символов в заголовке Content-Type, но не распознает UTF8 как alias для UTF-8. Неверный выбор может причинить серьезную боль.

Примечание. Забавный факт: заголовок X-Content-Type-Options не влияет на логику sniffing наборов символов.

Byte Order Marks

С определением набора символов мы еще не закончили! Internet Explorer нужно отдельно выделить за еще одну драматически ошибочную практику обработки содержимого: склонность отдавать приоритет так называемой byte order mark (BOM), последовательности байтов, которую можно поместить в начало файла для идентификации его кодировки, перед явно предоставленными данными charset. Когда такой маркер обнаруживается во входном файле, заявленный набор символов игнорируется.

В таблице 13-1 показаны несколько распространенных маркеров. Из них печатаемая BOM UTF-7 особенно коварна.

Примечание. Инженеры Microsoft признают проблему такого устройства и на момент написания говорят, что логика может быть пересмотрена в зависимости от результатов тестов совместимости. Если проблема будет решена к моменту выхода этой книги, честь им и хвала. До тех пор позволять злоумышленнику контролировать первые несколько байтов HTTP-ответа, который иначе не защищен Content-Disposition, может быть плохой идеей, и, кроме добавления padding к ответу, нет способа обойти этот сбой.

Таблица 13-1. Распространенные byte order markers (BOM)

Имя кодировки	Последовательность byte order mark
UTF-7	+/, за которым следует 8, 9, + или /
UTF-8	0xEF 0xBB 0xBF
UTF-16 little endian	0xFF 0xFE
UTF-16 big endian	0xFE 0xFF
UTF-32 little endian	0xFF 0xFE 0x00 0x00
UTF-32 big endian	0x00 0x00 0xFE 0xFF
GB-18030	0x84 0x31 0x95 0x33

Наследование и переопределение набора символов

При оценке потенциального влияния стратегий обработки наборов символов в современных веб-браузерах следует учитывать еще два малоизвестных механизма. Обе функции могут позволить злоумышленнику навязать нежелательную кодировку символов другой странице, не полагаясь на sniffing символов.

Первый рассматриваемый аппарат, поддерживаемый всеми, кроме Internet Explorer, известен как наследование набора символов. Согласно этой политике любая кодировка, определенная для фрейма верхнего уровня, может быть автоматически применена к любым вложенным документам, у которых нет собственного действительного значения charset. Изначально такое наследование распространяется на все сценарии фрейминга, даже между полностью несвязанными сайтами. Однако когда Штефан Эссер, Абхишек Арья и несколько других исследователей продемонстрировали ряд правдоподобных атак, использующих эту возможность для принудительного разбора UTF-7 на ничего не подозревающих целях, разработчики Firefox и WebKit решили ограничить поведение фреймами одного origin. Opera все еще разрешает междоменное наследование. Хотя она не поддерживает UTF-7, другие проблемные кодировки, такие как Shift JIS, вполне доступны.

Другой механизм, заслуживающий упоминания, - способность вручную переопределить текущий набор символов. Эта функция доступна через меню View > Encoding или похожее в большинстве браузеров. Использование этого меню для изменения набора символов приводит к повторному разбору страницы и всех ее подфреймов, включая междоменные, с выбранной кодировкой, независимо от любых директив charset, встреченных ранее для этого содержимого.

Поскольку пользователей легко можно обманом заставить выбрать альтернативную кодировку для страницы, контролируемой злоумышленником, просто чтобы просмотреть ее корректно, такое устройство должно заставить вас немного напрячься. От обычных пользователей нельзя ожидать, что они поймут: их выбор также применится к скрытым тегам <iframe> и что такое вроде бы безобидное действие может включить cross-site scripting атаки против несвязанных веб-ресурсов. На самом деле давайте говорить прямо: большинство из них не знает и не должно знать, что такое <iframe>.

Charset подресурсов, управляемый разметкой

Мы приближаемся к концу эпического путешествия по паутине странностей обработки содержимого, но еще не совсем закончили. Внимательные читатели могут вспомнить, что в разделе "Type-Specific Content Inclusion" на странице 82 я упоминал: на некоторых типах подресурсов, а именно таблицах стилей и скриптах, встраивающая страница может указать собственное значение charset, чтобы применить конкретное преобразование к полученному документу, например:

```
<script src="http://fuzzybunnies.com/get_js_data.php" charset="EUC-JP">
```

Этот параметр учитывается всеми браузерами, кроме Opera. Там, где он поддерживается, он обычно не имеет приоритета над charset в Content-Type, если только этот второй параметр не отсутствует или не распознан. Но у каждого правила есть исключение, и слишком часто имя этого исключения - Internet Explorer 6. В этом все еще популярном браузере кодировка, указанная разметкой, переопределяет HTTP-данные.

Имеет ли это поведение практическое значение? Чтобы полностью понять последствия, быстро вернемся к главе 6, где мы обсуждали тему защиты сгенерированного сервером, пользовательского, JSON-подобного кода от междоменного включения. Один пример приложения, которому нужна такая защита, - searchable address book в веб-почтовом приложении: поисковый термин предоставляется в URL, а JavaScript-сериализация совпадающих контактов возвращается браузеру, но должна быть защищена от включения на несвязанных сайтах.

Теперь предположим, что разработчик придумал простой трюк, чтобы помешать сторонним веб-страницам загружать эти данные через <script src=...>: одиночный префикс // используется, чтобы превратить весь ответ в комментарий. Вызывающие стороны из того же origin, использующие API XMLHttpRequest, могут просто изучить ответ, удалить префикс и передать данные в eval(...), но удаленные вызывающие стороны, пытающиеся злоупотребить синтаксисом <script src=...>, останутся ни с чем.

В этой схеме запрос к `/contact_search.php?q=smith` может дать следующий ответ:

```
// var result = { "q": "smith", "r": [ "j.smith@example.com" ] };
```

Пока поисковый термин корректно экранируется или фильтруется, схема кажется безопасной. Но когда мы понимаем, что злоумышленник может заставить ответ интерпретироваться как UTF-7, картина резко меняется. На вид безобидный поисковый термин, который с точки зрения сервера не содержит незаконных символов, все же может неожиданно декодироваться в:

```
// var result = { "q": "smith[CR][LF]
var gotcha
= { "
", "r": [ "j.smith@example.com" ] };
```

Этот ответ, загруженный через `<script src=... charset=utf-7>` внутри браузера жертвы, дает злоумышленнику доступ к части адресной книги пользователя.

Это не просто мысленный эксперимент: подход // довольно распространен в Web, и Масато Кинугава, известный исследователь, нашел несколько популярных веб-приложений, затронутых этим багом. Более искусственный вариант той же атаки возможен и против других префиксов, предотвращающих выполнение, таких как `while (1);`. В конечном счете проблемы междоменного переопределения `charset` на тегах `<script>` - одна из причин, по которым в главе 6 мы настоятельно рекомендуем использовать надежный `parser-stopping prefix`, чтобы не дать интерпретатору когда-либо посмотреть на какие-либо биты, контролируемые злоумышленником. О, и если учесть поддержку E4X, картина становится еще интереснее,^[9] но оставим это здесь.

Определение для не HTTP-файлов

Чтобы завершить эту главу, посмотрим на последнюю недостающую деталь: определение кодировки набора символов для документов, доставленных по не HTTP-протоколам. Как и следовало ожидать, документы, сохраненные на диск и затем открытые через протокол `file:`, или загруженные иными способами, когда обычные метаданные `Content-Type` отсутствуют, обычно подвергаются логике определения набора символов. Однако, в отличие от эвристик определения документа, между всеми возможными способами доставки нет существенной разницы: во всех случаях поведение `sniffing` примерно одинаково.

Нет чистого и переносимого способа решить эту проблему для всех текстовых документов, но конкретно для HTML влияние `sniffing` набора символов можно смягчить, встроив директиву `<meta>` в тело документа:

```
<meta http-equiv="Content-Type" content="text/html; charset=...">
```

Не следует отказываться от `Content-Type` в пользу этого индикатора. В отличие от `<meta>`, заголовок работает для не HTML-содержимого, и его проще принудить и аудитировать на уровне всего сайта. Тем не менее документы, которые, вероятно, будут сохранены на диск и содержат контролируемые злоумышленником фрагменты, выиграют от избыточного тега `<meta>`. Просто убедитесь, что это значение действительно совпадает с `Content-Type`.

Памятка по инженерии безопасности

Хорошие практики безопасности для всех сайтов

- Настройте веб-сервер так, чтобы он добавлял заголовок `X-Content-Options: nosniff` ко всем HTTP-ответам.
- Обратитесь к памятке в главе 9, чтобы настроить подходящую метаполитику `/crossdomain.xml`.
- Настройте сервер так, чтобы он добавлял значения `charset` и `Content-Type` по умолчанию ко всем ответам, где иначе их не было бы.
- Если вы не используете передачу параметров на основе пути, такую как `PATH_INFO`, рассмотрите отключение этой функции.

При генерации документов с частично контролируемым злоумышленником содержимым

- Всегда возвращайте явное, действительное, хорошо известное значение `Content-Type`. Не используйте `text/plain` или `application/octet-stream`.
- Для любых текстовых документов возвращайте явное, действительное, хорошо известное значение `charset` в заголовке `Content-Type`; UTF-8 предпочтительнее любых других кодировок переменной ширины. Не предполагайте, что `application/xml+svg`, `text/csv` и другим не HTML-документам не нужен заданный набор символов. Для HTML рассмотрите избыточную директиву `<meta>`, если мыслимо, что файл может быть скачан пользователем. Остерегайтесь опечаток: UTF8 не является действительным alias для UTF-8.
- Используйте `Content-Disposition: attachment` и подходящее явное значение `filename` для ответов, которые не нужно просматривать напрямую, включая JSON-данные.
- Не позволяйте пользователю контролировать первые несколько байтов файла. Ограничивайте ответ настолько, насколько возможно. Не пропускайте NUL, управляющие символы или значения с установленным старшим битом, если это не абсолютно необходимо.
- Выполняя серверные преобразования кодировок, убедитесь, что ваши конвертеры отвергают все неожиданные или недействительные входные данные, например `overlong UTF-8`.

При размещении файлов, созданных пользователями

Рассмотрите использование `sandbox`-домена, если возможно. Если вы намерены размещать неограниченные или неизвестные форматы файлов, `sandbox`-домен необходим. В противном случае как минимум сделайте следующее:

- Используйте `Content-Disposition: attachment` и подходящее явное значение `filename`, совпадающее с параметром `Content-Type`.
- Тщательно проверяйте входные данные и всегда используйте подходящий, широко распознаваемый MIME-тип. Отдача JPEG как `image/gif` может привести к неприятностям. Воздержитесь от размещения MIME-типов, которые вряд ли поддерживаются популярными браузерами.
- Воздержитесь от использования `Content-Type: application/octet-stream` и вместо этого используйте `application/binary`, особенно для неизвестных типов документов. Воздержитесь от возврата `Content-Type: text/plain`. Не допускайте заданных пользователем заголовков `Content-Type`.

Сноски

- [1] [\[назад\]](#) RFC 2046, раздел о application/octet-stream.
- [2] [\[назад\]](#) В Internet Explorer реализованная логика тонко отличается от сценария, где Content-Type отсутствует. Вместо того чтобы всегда предполагать HTML, браузер сканирует первые 256 байтов на популярные HTML-теги и другие заранее заданные сигнатуры содержимого. С точки зрения безопасности, однако, это не очень существенное различие.
- [3] [\[назад\]](#) Естественно, сопоставление расширений на основе пути по сути бесполезно по причинам, обсуждавшимся в предыдущем разделе; но в случае Internet Explorer 6 все становится еще хуже. В этом браузере расширение может появиться в query-части URL. Ничто не мешает злоумышленнику просто добавить ?foo=bar.html к запрошенному URL, фактически гарантируя, что эта проверка всегда будет удовлетворена.
- [4] [\[назад\]](#) Публичная позиция Microsoft по сохранению compatibility behavior в логике MIME sniffing.
- [5] [\[назад\]](#) Исследование SHODAN и Chris John Riley, 2011 г.
- [6] [\[назад\]](#) RFC 2183, описание Content-Disposition.
- [7] [\[назад\]](#) MUA означает "mail user agent", то есть клиентское приложение, используемое для получения, отображения и составления почтовых сообщений.
- [8] [\[назад\]](#) Сегодня эта проблема смягчается большинством браузеров: их парсеры теперь имеют дополнительные проверки, чтобы принципиально отвергать overlong UTF-8-кодировки. Однако того же нельзя сказать обо всех возможных серверных UTF-8-библиотеках.
- [9] [\[назад\]](#) E4X - ECMAScript for XML, расширение JavaScript для работы с XML-литералами.

Глава 14. Работа с вредоносными скриптами

В предыдущих пяти главах мы рассмотрели довольно широкий набор механизмов безопасности браузера, и, оглядываясь назад, справедливо сказать, что почти все они имеют общую цель: не дать вредоносному содержимому неправомерно вмешиваться в любые другие, легитимные веб-страницы, отображаемые в браузере. Это важное направление, но также довольно узкое; подрыв границ между несвязанными сайтами - большая часть репертуара любого злоумышленника, но точно не единственный трюк в книге.

Другая значимая задача безопасности на уровне проектирования, с которой приходится сталкиваться всем браузерам, состоит в том, что злоумышленники могут злоупотреблять хорошо задуманными возможностями сценариев, чтобы нарушать работу сторонних сайтов или выдавать себя за них, фактически не взаимодействуя с целевым содержимым. Например, если JavaScript-коду, контролируемому злоумышленником, разрешено создавать на экране произвольные окна без оформления, злоумышленник может обнаружить, что вместо поиска способа внедрить вредоносную полезную нагрузку в содержимое, отдаваемое на `fuzzybunnies.com`, проще открыть окно с правдоподобной копией адресной строки, тем самым убедив пользователя, что показанное содержимое пришло с доверенного сайта.

К несчастью для жертв, в ранние дни Web восприимчивости JavaScript API к атакам, призванным нарушить работу пользователей или сбить их с толку, не уделяли настоящего внимания, и, в отличие от вопросов междоменной изоляции содержимого, этот класс проблем до сих пор не воспринимается очень серьезно. Ситуация вряд ли скоро изменится: ресурсы поставщиков растянуты между исправлением сравнительно более серьезных изъянов уровня реализации в печально багованных кодовых базах браузеров и выпуском новых блестящих функций безопасности, которые одновременно умиротворяют разработчиков веб-приложений, пользователей и массовую прессу.

Атаки отказа в обслуживании

Возможность злоумышленника обрушить браузер или иным образом сделать его неработоспособным - одна из самых распространенных, очевидных и при этом наименее оцененных проблем современного Web. В эпоху гаджетов и mashups она также может иметь неожиданно неприятные последствия.

Самая заметная причина, по которой большинство браузеров восприимчиво к атакам denial-of-service (DoS), проста и связана с недостатком планирования: ни нижележащие форматы документов, ни возможности, выставленные через языки сценариев, не проектировались так, чтобы иметь разумный, ограниченный худший случай по CPU или памяти. Иными словами, любой достаточно сложный HTML-файл или бесконечный цикл JavaScript мог поставить нижележащую операционную систему на колени. Хуже того, попытки ввести обязательные ограничения ресурсов или дать пользователям способ вернуть контроль над убежавшим браузером после посещения мошеннической страницы встречают сопротивление. Например, авторы многих недавно предложенных HTML5 API не дают советов по предотвращению атак исчерпания ресурсов и даже не признают такую необходимость, потому что считают: любые ограничения, наложенные сегодня, вероятно, будут мешать росту Web через 5 или 10 лет. Разработчики браузеров, в свою очередь, отказываются что-либо делать без руководства на уровне стандартов.

Распространенный утилитарный аргумент против любых предлагаемых DoS-защит состоит в том, что они бессмысленны: браузер безнадежно легко обрушить множеством способов, так зачем

сегодня принимать специальные меры против конкретного вектора? С этой точкой зрения трудно спорить, но важно заметить, что она действует как самоисполняющееся пророчество: устойчивый рост числа DoS-векторов делает все менее вероятным, что ситуация когда-либо скоро будет комплексно исправлена.

Примечание. Справедливости ради, вычислительная сложность некоторых операций - не единственная причина, по которой браузеры легко обрушить. Поставщики также ограничены необходимостью поддерживать значительную степень синхронности во время шагов рендеринга страницы и выполнения скриптов (см. главу 6). Такое устройство избавляет разработчиков сайтов от необходимости писать реентерабельный и thread-safe код и дает существенные преимущества по сложности кода и безопасности. К сожалению, оно также сильно упрощает одному документу блокировку всего браузера или по крайней мере значительной его части.

Независимо от всех этих соображений, и даже если поставщики браузеров отказываются признавать DoS-риски как конкретный изъян, влияние таких атак трудно игнорировать. Во-первых, всякий раз, когда браузер падает, возникает существенный риск потери данных в самом браузере или в любых приложениях, косвенно затронутых атакой. Кроме того, на некоторых сайтах социальных сетей злоумышленник может просто заблокировать жертве доступ к сайту, поделившись с ней вредоносным гаджетом или, возможно, даже удачно выбранным изображением, и тем самым навсегда помешать этому человеку пользоваться сервисом.

Некоторые распространенные трюки, используемые для вывода браузера из строя, включают загрузку сложных XHTML- или SVG-документов, открытие очень большого числа окон, запуск бесконечного JavaScript-цикла, выделяющего память, постановку в очередь значительного числа вызовов `postMessage(...)` и т. д. Хотя эти примеры зависят от реализации, каждый браузер предлагает немало способов добиться такой цели. Даже в Chrome, который использует отдельные процессы рендеринга для изоляции несвязанных страниц, несложно уронить весь браузер: процесс верхнего уровня посредничает при выполнении множества доступных скриптам и иногда затратных по памяти или CPU задач.

Учитывая сказанное, неудивительно, что, несмотря на в целом пренебрежительное отношение, основные браузеры все же реализуют несколько DoS-контрмер. Они не складываются в согласованную стратегию и были развернуты только в ответ на массовое злоупотребление конкретными API или для смягчения неопасных, но распространенных ошибок программирования. Тем не менее кратко рассмотрим их.

Ограничения времени выполнения и использования памяти

Из-за уже упомянутой необходимости принуждать некоторую степень синхронности для многих типов JavaScript-операций большинство поставщиков браузеров предпочитают осторожность и выполняют скрипты синхронно с большей частью остального кода браузера. У этого устройства есть очевидный недостаток: значительная часть браузера может полностью перестать отвечать, пока JavaScript-движок, скажем, пытается вычислить фальшивый цикл `while` (1). В Opera и Chrome пользовательский интерфейс верхнего уровня все еще будет в основном отзывчивым, хотя и вялым, но в большинстве других браузеров даже закрыть окно браузера через обычный UI будет невозможно.

Поскольку бесконечные циклы довольно легко создать случайно, Internet Explorer, Firefox, Chrome и Safari, чтобы помочь разработчикам, принуждают умеренное ограничение времени для любых непрерывно или почти непрерывно выполняющихся скриптов. Если скрипт делает браузер

неотзывчивым дольше нескольких секунд, пользователю будет показан диалог и дана возможность прервать выполнение. Выбор этой возможности даст результат, похожий на встречу с необработанным исключением, то есть отказ от текущего потока выполнения.

К сожалению, такой лимит не является особенно надежной защитой от вредоносных скриптов. Например, независимо от выбора пользователя, все еще можно возобновить выполнение через таймеры или обработчики событий, и легко вообще избежать появления диалога, периодически ненадолго возвращая CPU в idle-состояние, чтобы сбросить счетчик. Кроме того, как отмечалось ранее, есть способы захватить CPU-ресурсы, не прибегая к busy loops: рендеринг сложных XHTML-, SVG- или XSLT-документов может быть столь же разрушительным и не подчиняется никаким проверкам.

Если оставить в стороне время выполнения, предпринимались попытки контролировать объем памяти, используемый выполняемыми скриптами. Размер стека вызовов ограничен значением, зависящим от браузера, между 500 и 65535, и попытка более глубокой рекурсии приведет к безусловной остановке. Размер script heap, с другой стороны, обычно никак существенно не ограничен; страницы могут выделять и использовать гигабайты памяти. Фактически большинство ранее реализованных ограничений, например лимит 16 МБ в Internet Explorer 6, было удалено в более новых выпусках.

Ограничения соединений

Во многих веб-приложениях каждая веб-страница состоит не только из собственно HTML-документа, полученного с URL, видимого в адресной строке, но и из нескольких десятков других, отдельно загружаемых подресурсов, таких как изображения, таблицы стилей и скрипты. Поскольку запрос всех этих элементов через индивидуально установленные HTTP-соединения может быть медленным, читатель может помнить из главы 3, что протокол был расширен, чтобы предлагать keep-alive-сеансы и request pipelining. Но даже с этими улучшениями остается одна упрямая проблема. Внутреннее ограничение протокола состоит в том, что сервер всегда должен отправлять ответы в том же порядке, в котором он получил запросы, поэтому если любой из подресурсов, каким бы незначительным он ни был, генерируется чуть дольше, загрузка всех последующих будет задержана.

Чтобы обойти эту проблему и оптимизировать производительность, когда keep-alive-запросы или pipelining нельзя использовать, все браузеры разрешают открывать несколько одновременных HTTP-соединений к целевому серверу. Так браузер может выдавать несколько запросов параллельно.

К сожалению, схема параллельных соединений может быть дорогой для целевого сайта, особенно если сервер полагается на традиционную архитектуру обработки соединений на основе `fork()`.^[1] Поэтому, чтобы ограничить риск случайного или намеренного запуска распределенной DoS-атаки, число параллельных соединений нужно ограничивать умеренным значением на хост, обычно между 4 и 16. Более того, чтобы злоумышленники не перегружали сам браузер или не влияли на производительность близлежащего сетевого оборудования, общее число одновременных соединений ко всем адресатам также ограничивается небольшим кратным лимита на хост.

Примечание. Во многих реализациях лимит соединений на хост принуждается по DNS-меткам, а не по IP-адресам. Поэтому злоумышленник все еще может указать несколько фиктивных DNS-записей в собственных доменах на любой несвязанный целевой IP и обойти первое ограничение. Глобальный лимит соединений, впрочем, все равно будет действовать.

Хотя число одновременных HTTP-сеансов ограничено, практических ограничений на то, как долго активный сеанс может поддерживаться живым, нет, пока не встретятся таймауты TCP/IP на уровне ядра. Такое устройство может позволить злоумышленникам просто исчерпать глобальный лимит соединений, разговаривая с парой намеренно медленных адресатов и не давая пользователю делать что-либо полезное тем временем.

Фильтрация pop-up

API `window.open(...)` и `window.showModalDialog(...)`^[2] позволяют веб-страницам создавать новые окна браузера, указывая их на любые иначе разрешенные URL. В обоих случаях браузеру можно дать указание не показывать некоторые элементы оформления окна для нового загруженного документа или расположить окно на экране определенным образом. Простое использование `window.open(...)` может выглядеть так:

```
window.open("/hello.html", "_blank", "menubar=no,left=50,top=50");
```

Помимо этих двух JavaScript-методов, новые окна также могут открываться косвенно при программном взаимодействии с некоторыми HTML-элементами. Например, можно вызвать метод `click()` у HTML-ссылки или метод `submit()` у формы. Если соответствующая разметка включает параметр `target`, результирующая навигация произойдет в новом окне указанного имени.

Как и следовало ожидать, способность случайных веб-страниц открывать новые окна браузера вскоре оказалась проблематичной. В конце 1990-х многие игроки тогда еще молодой индустрии онлайн-рекламы решили, что им нужно привлечь внимание к своим объявлениям любой ценой, даже ценой глубокого раздражения и отчуждения аудитории. Автоматическое порождение окон исключительно для показа яркой рекламы казалось отличным способом вести бизнес и заводить новых друзей.

Реклама pop-up и pop-under^[3] быстро стала одним из самых известных и ненавистных аспектов Web. И по веской причине: особенно с pop-under было вполне обычно накопить их дюжину после двух-трех часов обычного просмотра.

Из-за массовых жалоб поставщики браузеров вмешались и реализовали простое ограничение: ложные попытки страниц, не находящихся в белом списке, создавать новые окна тихо игнорировались.^[4] Исключения делались для попыток, предпринятых сразу после щелчка мышью или похожего действия пользователя. Например, в случае JavaScript способность вызвать `window.open(...)` предоставлялась коду, выполнявшемуся в ответ на событие `onclick`, и вскоре после этого отзывалась. В Internet Explorer и WebKit это разрешение истекает в момент выхода из обработчика события. Другие браузеры могут признавать короткий grace period примерно в одну секунду.

Функция блокировки pop-up сначала сократила pop-up-рекламу, но в конечном счете оказалась довольно неэффективной: многие сайты просто ждали, пока пользователь щелкнет где-нибудь на странице, чтобы перейти по ссылке или даже прокрутить документ, и в ответ порождали новые окна. Другие просто перешли к еще более разрушительным практикам, таким как interstitials - полноэкранный рекламе, через которую нужно щелкнуть, чтобы попасть к содержимому, которое вы на самом деле хотите читать.

Если оставить в стороне рекламную гонку вооружений, война с `window.open(...)` интересна и с точки зрения DoS. Создание сотен тысяч окон, исчерпывающее принуждаемые ОС лимиты на число UI handles, - верный способ обрушить браузер и нарушить работу других приложений. Любой механизм, ограничивающий эту возможность, был бы, по крайней мере в теории, ценной защитой. Но не тут-то было: невероятно, но только Internet Explorer и Chrome разумно ограничивают

фактическое число вызовов `window.open(...)` в ответ на один щелчок. В других браузерах, как только временное разрешение открывать окна предоставлено, злоумышленник может полностью сорваться с цепи и открыть столько окон, сколько пожелает.

Ограничения использования диалогов

Если оставить в стороне неприятности, связанные с окнами, все скрипты веб-происхождения могут открывать некоторые диалоги, обрабатываемые браузером или ОС. Полезность этих диалогов для современных веб-приложений минимальна, но они все еще составляют еще одну интересную часть ландшафта безопасности браузеров. API, иницирующие диалоги, включают `window.alert(...)`, используемый для отображения простых текстовых сообщений; `window.prompt(...)` и `window.confirm(...)`, используемые для запроса базового пользовательского ввода; и `window.print(...)`, который вызывает диалог печати уровня ОС. Несколько малоизвестных расширений поставщиков, таких как Mozilla `window.sidebar.addPanel(...)` и `window.sidebar.addSearchEngine(...)`, для создания закладок и регистрации новых поисковых провайдеров соответственно, тоже входят в этот список.

Помимо вышеупомянутых JavaScript-методов, несколько типов диалогов можно породить косвенно. Например, можно вызвать метод `click()` на кнопке загрузки файла или перейти к загружаемому файлу, что обычно вызывает предоставленный ОС диалог выбора файла. Навигация к URL, требующему HTTP-аутентификации, также обычно вызывает `prompt` уровня браузера.

Итак, что делает диалоги такими интересными? Проблема с такими запросами весьма отличается от проблемы программно созданных окон. В отличие от в основном асинхронного API `window.open(...)`, диалоги приостанавливают выполнение JavaScript и откладывают многие другие действия, такие как навигация или доставка событий, фактически не позволяя создавать диалоги в больших количествах для исчерпания ресурсов и обрушения приложения. Но их модальное поведение также является их проклятием: они не дают взаимодействовать с некоторой частью браузера, пока пользователь не закроет сам диалог.

Это создает интересную лазейку. Если новый диалог открывается немедленно после закрытия старого, жертва может быть заблокирована от жизненно важной части UI браузера, часто даже теряя способность закрыть окно или уйти с нарушающей страницы. Авторы malware иногда злоупотребляют этой особенностью, чтобы заставить обычных, запаниковавших пользователей выполнить опасное действие, например скачать и запустить недоверенный исполняемый файл, просто чтобы им позволили продолжить работу: любой другой выбор в инициированном скриптом диалоге безопасности только заставит тот же диалог появляться снова и снова.

Вероятно, из-за этой связанной с malware темы поставщики браузеров начали экспериментировать с менее разрушительными методами запроса. В Chrome, например, некоторые из самых распространенных модальных диалогов имеют флажок, позволяющий пользователю подавить будущие попытки страницы использовать нарушающий API, то есть до следующей перезагрузки. В Opera можно остановить выполнение скриптов на странице. И в Opera, и в последних версиях Firefox многие распространенные диалоги модальны только относительно контролируемой документом области окна, все еще позволяя закрыть вкладку или ввести другой URL в адресной строке. Тем не менее покрытие таких улучшений ограничено.^[5]

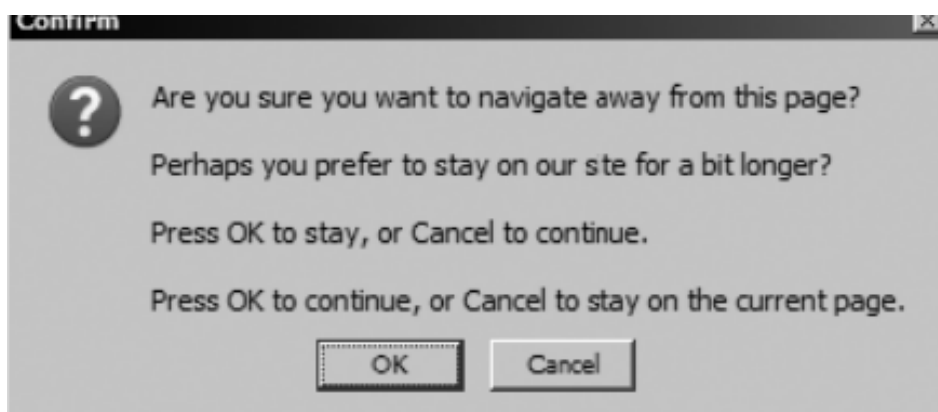


Рис. 14-1. Firefox сгенерировал глубоко сбивающий с толку и расплывчатый диалог после выполнения обработчика `onbeforeunload` на веб-странице. Обработчик дает авторам страницы шанс объяснить последствия ухода со страницы, например потерю несохраненных данных, и запросить у пользователя окончательное решение. На этом снимке первая и последняя строки исходят от самого браузера; две средние строки - "объяснение", предоставленное вместо этого безымянным вредоносным сайтом. Влияние этого конкретного диалога на безопасность минимально, но это замечательный пример плохого UI-дизайна. К сожалению, почти идентичный диалог также используется Internet Explorer, а большинство других браузерных диалогов ненамного лучше.

Примечание. Многие диалоги уровня браузера плохо объясняют, откуда возник запрос и каково его назначение. В некоторых случаях, например с диалогом Firefox, показанным на рис. 14-1, результат может быть комичным, но у такой нелепости есть и более зловещая сторона. Порождение авторитетно звучащих диалогов, якобы исходящих от самой операционной системы, - распространенный трюк авторов *malware*, чтобы сбить с толку менее опытных пользователей. Нетрудно представить, почему это работает.

Проблемы позиционирования и внешнего вида окон

Ладно, ладно, перейдем дальше, за пределы, пожалуй, не вдохновляющей и непопулярной темы DoS-изъянов. В различных API, связанных с UI, есть намного больше интересного, и `window.open(...)` - особенно любопытный случай. Вспомните из обсуждения ранее в этой главе, что эта скромная функция позволяет веб-приложениям не только создавать новые окна, но и располагать их в конкретном месте на экране. Несколько других методов, таких как `window.moveTo(...)`, `window.resizeTo(...)`, `window.focus()` или `window.blur()`, дополнительно позволяют перемещать такое окно по экрану, масштабировать его или укладывать в стек определенным образом. Наконец, `window.close()` позволяет незаметно избавиться от него, когда скрипту оно больше не нужно.

Как и большинство других возможностей манипуляции UI, эти API вскоре оказались источником боли. После серии забавных трюков, включавших создание "скрытых" окон путем размещения их частично или полностью за пределами экрана либо превращения их в совсем крошечные, эти функции теперь требуют, чтобы вновь созданные окна имели определенные минимальные размеры и полностью оставались в видимой области рабочего стола. Все еще можно создать окно, которое постоянно скачет по экрану и избегает всех попыток закрыть его мышью, но, учитывая прочитанное до сих пор, это заслуживает только тяжелого вздоха.

Ограничения размера окна, однако, не означают, что все содержимое адресной строки должно быть видно пользователю. Слишком маленькое окно можно использовать, чтобы ввести

пользователя в заблуждение относительно origin документа, просто аккуратно обрезав hostname, как показано на рис. 14-2. Поставщики браузеров знают об этой проблеме как минимум с моего отчета в 2010 году,^[6] но на момент написания только Internet Explorer использует более-менее убедительное, хотя и тонкое смягчение: он добавляет . . . в конец любых сокращенных hostnames в адресной строке.



Рис. 14-2. Окно, аккуратно подобранное скриптом по размеру так, что настоящий origin отображаемого содержимого сокращен сбивающим с толку образом. Фактический URL этой страницы с кошачьей темой - <http://www.example.com.coredump.cx/>, а не <http://www.example.com/>.

Еще одна интересная проблема с управляемым скриптом позиционированием окон - перспектива создания нескольких хитро выровненных, перекрывающихся окон, которые образуют то, что кажется одним окном документа с адресной строкой, не соответствующей частям отображаемого документа. Эта атака, которую я люблю называть window splicing, пожалуй, лучше всего показана на рис. 14-3.

Позиционирование окон предлагает несколько интересных, хотя и притянутых сценариев атак, но манипуляция содержимым программно созданного окна тоже имеет некоторое отношение к безопасности браузеров. Мы уже упоминали, что одна из возможностей API `window.open(...)` - способность скрывать некоторые элементы chrome браузера, например полосы прокрутки и меню, во вновь открытом окне. Пример такого ограничивающего UI вызова:

```
window.open("http://example.com/", "_blank", "location=no,menubar=no");
```

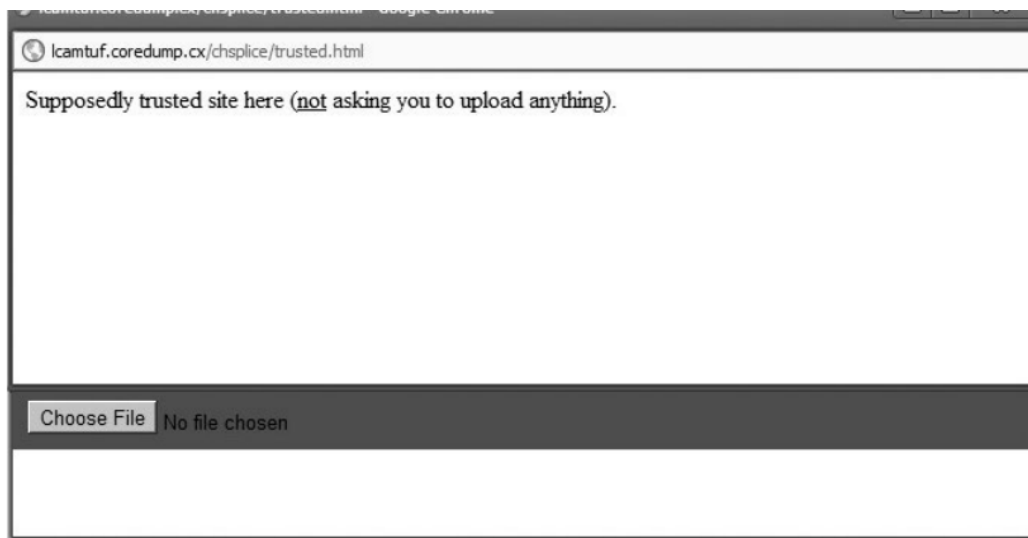


Рис. 14-3. Атака *window-splicing* в Chrome. То, что может казаться одним документом, фактически является композицией из двух перекрывающихся, выровненных окон. Пользователя подводят к мысли, что кнопка загрузки файла принадлежит домену, показанному в адресной строке верхнего окна, но это не так. Некоторые визуальные признаки выдают подвох, например часть рамки окна имеет немного другой оттенок, но они слишком тонки, чтобы пользователь легко их заметил.

Одна из этих настроек, `location=no`, предназначалась для скрытия адресной строки. Это, конечно, ужасная идея: она позволяет злоумышленнику не только скрыть настоящую адресную строку, но и загрузить страницу, которая просто показывает пиксельно точное изображение адресной строки с совершенно несвязанным URL. Черт, при минимальных усилиях эта фальшивая адресная строка может даже быть полностью интерактивной.

Осознав опасности такого устройства, большинство браузеров со временем начали отображать минималистичную, *read-only* адресную строку в любых окнах, открытых с `location=no`; Apple, однако, не видит вреда в том, чтобы эта настройка работала так, как изначально задумывалось в 1990-х. Жаль: рис. 14-4 показывает простую атаку на ее UI. Я связался с Apple по поводу этой атаки где-то в 2010 году, но ответа пока не получил.



Рис. 14-4. Разрешать сайтам скрывать адресную строку в Safari - плохая идея. Отображаемый документ получен не с `http://www.example.com/`; вместо этого страница просто показывает снимок настоящей адресной строки в окне, созданном вызовом `window.open("http://coredump.cx/...", "location=no")`.

Microsoft справилась не намного лучше: хотя они подлатали `window.open(...)`, они забыли о `window.createPopup(...)`, древнем и малоизвестном API, все еще не подчиненном необходимым проверкам.

Timing-атаки на пользовательские интерфейсы

Проблемы, которые мы до сих пор обсуждали в этой главе, могут быть трудны для исправления, но по крайней мере в принципе решения не находятся вне досягаемости. И все же вот нелепый вопрос: может ли текущая модель веб-сценариев быть фундаментально несовместима с тем, как работают люди? Под этим я имею в виду не просто опасности социальной инженерии, доставляемой через Web и нацеленной на невнимательных и легко сбиваемых с толку; скорее, я спрашиваю, возможно ли, что скрипты могут последовательно переигрывать бдительных и знающих жертв просто из-за внутренних ограничений человеческого познания.

Вопрос достаточно странный, чтобы его задавали нечасто, но ответ может быть положительным. Учтите, что у типичного внимательного человека обычная задержка между визуальным стимулом и произвольной моторной реакцией составляет от 100 до 300 миллисекунд.^[7] Люди не останавливаются на такой срок, чтобы оценить ситуацию после каждого мельчайшего мышечного движения; вместо этого мы подсознательно планируем серию выученных моторных действий заранее и обрабатываем сенсорную обратную связь по мере ее последующего поступления. На долю секунды мы не можем прервать заранее обдуманное действие, даже если что-то пошло ужасно не так.

Увы, на современных персональных компьютерах многое может произойти всего за одну десятую этого интервала. В частности, скрипты могут открывать новые окна, перемещать их или закрывать существующие; они также могут инициировать или прерывать запросы системного уровня. В такой среде проектирование UI, чувствительных к безопасности, далеко не так просто, как кажется, и от некоторых типов атак может быть просто невозможно защититься без серьезной смены парадигмы в проектировании ПО.

Чтобы проиллюстрировать проблему, рассмотрим страницу, которая пытается начать незапрошенную загрузку опасного типа файла. Загрузка обычно инициирует диалог уровня браузера с тремя вариантами: "open", "save" и "cancel". Разумные пользователи выберут последний вариант, но не если злоумышленник лишит их шанса сделать это.

Предположим, что через несколько миллисекунд после открытия диалога, возможно еще до того, как пользователь вообще зарегистрирует его наличие, сверху создается новое окно, скрывающее его из вида. В этом окне злоумышленник размещает аккуратно позиционированную кнопку или ссылку, по которой пользователь, вероятно, щелкнет, например кнопку закрытия раздражающей interstitial-рекламы. Когда пользователь пытается выполнить это совершенно разумное действие, вредоносная страница может использовать события `onmousemove`, чтобы отслеживать положение и скорость указателя мыши и довольно точно предсказать момент предстоящего щелчка. Закрытие `overlay`-окна за несколько миллисекунд до этого щелчка, только чтобы открыть кнопку "open" в той же позиции, неизбежно приведет пользователя к этому выбору в диалоге безопасности. Пользователь просто ничего не может сделать. Я продемонстрировал практическую атаку на Firefox в этом духе в 2007 году.^[8]

В ответ на атаки на диалоги безопасности за последние несколько лет были реализованы различные задержки безопасности, требующие от 500 миллисекунд до 5 секунд между попаданием диалога в фокус и включением опасных кнопок для пользовательского ввода. Но такие задержки плохо воспринимаются дизайнерами браузерного UI: они их ненавидят, считая, что продукт должен быть максимально отзывчивым и что раздражение пользователя некликабельными кнопками или обратными отсчетами - значимая проблема usability. Некоторые даже добивались удаления существующих таймаутов из устаревших UI.^[9] HTML5-запросы на предоставление геолокации затронуты этим взглядом. Многие браузеры не защищены от атаки на этот UI сколько-нибудь существенно.^[10]

Чтобы еще больше усложнить картину, пользовательские интерфейсы уровня браузера - не единственная забота для timing-атак на UI. Чувствительная к безопасности или приватности функциональность многих доверенных сайтов также может быть атакована, а исправить эту проблему намного труднее, чем добавить таймеры задержки к горстке известных опасных UI системного уровня.

Примечание. Помимо перехвата щелчков или нажатий клавиш на уровне миллисекунд, неоднократно демонстрировалось, что при минимальном и вроде бы безобидном обусловливании здоровых и сосредоточенных испытуемых можно надежно заставить игнорировать даже очень заметные и необычные визуальные стимулы. Печально известный эксперимент Invisible Gorilla,^[11] показанный на рис. 14-5, - особенно известный пример. Почти все зрители, смотрящие подготовленный исследователями ролик, не замечают ясно видимого человека в костюме гориллы в толпе. Следствие в том, что даже опытных пользователей можно приучить игнорировать сигналы вроде изменений адресной строки или SSL-индикаторов в браузере, и это очень тревожная мысль. Единственная причина, по которой мы не пытаемся решить эту проблему сегодня, в том, что немногие авторы эксплойтов являются поведенческими учеными. Но если вы высокопрофильная цель, это кажется рискованной ставкой.



Рис. 14-5. Один кадр из эксперимента *Invisible Gorilla*, любезно предоставленный Дэниелом Саймонсом (<http://dansimons.com/>). Когда зрителей просят посмотреть это видео и посчитать, сколько раз игроки передают баскетбольный мяч, большинство не замечает человека в костюме гориллы, который небрежно проходит через комнату примерно на середине ролика. Правда! Перейдите на <http://theinvisiblegorilla.com/videos.html> и попробуйте на друге.

Памятка по инженерии безопасности

Когда вы разрешаете созданные пользователями `<iframe>`-гаджеты на своем сайте

- Не делайте этого, если не готовы жить с последствиями. Вы не можете надежно помешать вредоносному гаджету запускать DoS-атаки на ваших пользователей. Любой такой гаджет также сможет вызывать различные малоизвестные диалоги, которые, как правило, не будут различать вашу страницу верхнего уровня и домен, где размещен гаджет.

При создании UI, чувствительных к безопасности

- Из-за риска состояний гонки в UI избегайте ситуаций, где жизненно важную настройку можно изменить одним нажатием клавиши или одним щелчком. Требуйте как минимум две операции, например выбрать флажок и затем нажать Save. Если действий в один щелчок не избежать, рассмотрите проверку других сигналов. Например, находился ли указатель мыши в текущем окне 500 миллисекунд назад?

Сноски

[1] [назад] Традиционное устройство большинства Unix-служб состоит в наличии главного процесса-"слушателя" и создании нового процесса для обработки каждого принятого соединения. Для разработчика эта модель замечательна своей простотой, но она несет много существенных скрытых издержек для операционной системы, которой иногда трудно обрабатывать больше нескольких сотен одновременных соединений.

[2] [\[назад\]](#) Малоизвестный метод `showModalDialog(...)` назван несколько неверно. По сути он эквивалентен `window.open(...)`, но должен смутно имитировать поведение модального диалога, блокируя скрипты в вызывающем контексте, пока такое "диалоговое" окно не будет закрыто. Точное поведение этого API случайным образом различается между браузерами. Например, иногда другие страницы могут навигировать нижележащее окно или выполнять новые скрипты, пока исходный JS-код, вызвавший `showModalDialog(...)`, еще выполняется.

[3] [\[назад\]](#) "Pop-under" - это pop-up-окно, которое сразу после создания перемещается в конец стека окон с помощью `opener.window.focus()` или `window.blur()`. Pop-under, пожалуй, чуть меньше отвлекают, чем pop-up, потому что пользователю не нужно немедленно действовать, чтобы вернуться к исходному документу. Но ненавидят их не меньше.

[4] [\[назад\]](#) Например, вызов `window.open(...)` не генерировал бы исключение. Возвращаемое значение в таком случае, однако, не стандартизировано, что затрудняет надежное обнаружение заблокированного pop-up. В Internet Explorer и Firefox функция вернет `null`; в Safari она вернет другое специальное значение, `undefined`; в Opera будет предоставлен фиктивный дескриптор окна; а в Chrome возвращенный дескриптор окна даже будет иметь квазифункциональный DOM.

[5] [\[назад\]](#) По причинам usability случайным страницам в интернете больше не разрешается прерывать ожидающую навигацию средствами, отличными от этого конкретного диалога `onbeforeunload`. Удивительно, но заложенная в дизайн способность навсегда удерживать пользователя на вредоносной странице и отменять любые попытки навигации была встречена не очень хорошо.

[6] [\[назад\]](#) Отчет автора о сокращении адресной строки окна, 2010 г.

[7] [\[назад\]](#) Типичные значения задержки зрительно-моторной реакции.

[8] [\[назад\]](#) Практическая демонстрация атаки на Firefox, 2007 г.

[9] [\[назад\]](#) См., например, Mozilla bug 561177, где один из инженеров UI Firefox предложил удалить задержку безопасности из prompt установки plug-in.

[10] [\[назад\]](#) Проблемы prompt совместного доступа к геолокации HTML5 в браузерах.

[11] [\[назад\]](#) Daniel Simons, эксперимент Invisible Gorilla.

Глава 15. Внешние привилегии сайтов

Чтобы завершить обсуждение всех заслуживающих внимания функций безопасности браузера, мы рассмотрим несколько механизмов, которые предоставляют специальные привилегии сайтам, вручную выбранным пользователем или жестко зашитым авторами самого браузера. Подход, используемый в этих случаях, резко контрастирует со схемами, которые мы обсуждали раньше: все они полагаются на довольно разумное изучение внутренних свойств отображаемого содержимого. Обычно реализация заставляла бы нас смотреть на источник документа, контекст, в котором он отображается, или природу операции, которую документ пытается выполнить; но, если не считать результатов этих проверок, браузер никогда не отдавал бы предпочтение одному иначе ничем не примечательному origin.

Привилегии на уровне сайта довольно грубо нарушают этот принцип беспристрастности по причинам от сомнительных до, чаще, просто утилитарных. Есть убедительные причины usability для переноса некоторых по природе опасных возможностей в мир браузеров, но нет хорошего способа программно решить, каким веб-приложениям можно доверить доступ к ним. Делегирование этой задачи человеку, возможно, лучшее, что мы можем сделать.^[1]

Естественно, создание касты привилегированных приложений может быть весьма проблематичным, потому что границы между любыми двумя веб-приложениями с самого начала не особенно хорошо определены, из-за чего трудно точно удерживать разрешения. А поскольку уже несовершенные границы применяются только к некоторым межсайтовым взаимодействиям, уязвимости вроде XSS или XSRF могут дополнительно усугубить беду. В итоге между намерением разрешения на уровне сайта и фактическими последствиями такого предоставления доступа может возникнуть значительный разрыв.

Разрешения сайтов, управляемые браузером и плагинами

Балансируя безопасность, приватность и удобство, поставщики браузеров иногда оказываются между молотом и наковальней. Некоторые предлагаемые возможности кажутся необходимыми для дальнейшего роста Web, но просто слишком опасны, чтобы сделать их доступными каждому сайту в интернете. Примеры таких проблемных механизмов включают предоставление доступа к потокам видеокamеры или микрофона,^[2] разрешение сайтам запрашивать данные геолокации пользователя,^[3] установку расширений или тем браузера, а также открытие уведомлений рабочего стола.

В качестве обхода этой проблемы поставщики требуют, чтобы пользователь одобрил запрос приложения, прежде чем ему будет разрешен доступ к привилегированному API. При первой попытке использовать ограниченную функциональность пользователю обычно предоставляется визуальный сигнал, от иконки до модального диалога, и дается три варианта: проигнорировать запрос, разрешить его один раз или постоянно авторизовать запрашивающий сайт для доступа к API. Из этих вариантов последний наиболее интересен: если он выбран, весь будущий доступ от совпадающего хоста будет автоматически одобряться, иногда без какого-либо дальнейшего визуального указания.

Примечание. Большинство белых списков смотрят только на имя хоста, а не на протокол или порт. Поэтому любая запись в таких списках будет соответствовать более чем одному SOP origin. В частности, авторизация `https://fuzzybunnies.com/` для доступа к вашей камере может также авторизовать незашифрованный сайт `http://fuzzybunnies.com/` делать то же самое.

Предоставлять сайтам доступ к функциям, чувствительным к приватности или безопасности, нужно осторожно, потому что, как отмечалось ранее, последствия выходят за рамки простого доверия авторам приложения из белого списка. Разрешение предоставляется любому содержимому, выполняемому в совпадающем origin, независимо от того, как полезная нагрузка туда попала, что сильно усиливает влияние простых и в долгосрочной перспективе неизбежных ошибок реализации. Уязвимость внедрения скрипта в привилегированном origin больше не просто раскрывает данные, хранящиеся внутри приложения, но может также раскрыть чувствительные потоки данных, исходящие от клиента.

Жестко заданные домены

В дополнение к списку привилегированных доменов, авторизованных пользователем, некоторые браузеры или браузерные плагины поставляются со списком выбранных поставщиком сайтов или SOP origins, которым даны существенные привилегии для перенастройки или обновления частей браузера либо операционной системы. Среди наиболее заметных примеров этой тенденции: `update.microsoft.com`, который распознается ActiveX controls, поставляемыми с Microsoft Windows, и которому разрешено устанавливать обновления ПО; `addons.mozilla.org` и `chrome.google.com`, распознаваемые соответствующими браузерами и получающие специальные привилегии на установку расширений или тем; или `www.macromedia.com`, которому разрешено перенастраивать Adobe Flash.

Устройства этих механизмов различаются и, как правило, не документированы удовлетворительно. Некоторые функции требуют проверки второго уровня, например криптографической подписи или согласия пользователя, но другие не требуют. В целом распространение таких привилегированных доменов тревожно, потому что ясно: они не будут иммунны к обычным проблемам безопасности, от которых страдает остальной современный Web. Показательный пример: `http://xssed.com/` перечисляет шесть публично заявленных XSS-уязвимостей в `addons.mozilla.org`.^[4]

Менеджеры паролей на основе форм

Удивлены? Не стоит. Упоминание менеджеров паролей может казаться неуместным, но эту технологию очень полезно рассматривать как косвенную форму привилегии, привязанной к сайту. Прежде чем объяснять, кратко рассмотрим, почему управление паролями вообще реализовано в современных браузерах и как оно фактически работает.

Ответ на первый вопрос довольно прост: сегодня почти каждый крупный сайт требует или по крайней мере настойчиво предлагает всем посетителям открыть учетную запись. Вход обычно нужен, чтобы настраивать внешний вид сайта, и является предварительным условием для взаимодействия с другими зарегистрированными пользователями. К сожалению, эти специфичные для сайтов системы аутентификации не синхронизированы, если не считать нескольких экспериментов с "federated login" ограниченного масштаба, таких как OpenID,^[5] и фактически вынуждают широкую публику создавать и запоминать несколько десятков надежных паролей, по одному для каждого часто посещаемого места. Такой подход трудно поддерживать, и он ведет к массовому и опасному повторному использованию паролей; именно здесь поставщики браузеров решили вмешаться.

Менеджеры паролей на основе форм - незлегантное, но прагматичное решение проблемы разрастания учетных данных на уровне сайтов. Они применяют простые эвристики для

обнаружения отправки нормально выглядящих форм входа: браузер ищет поле `<input type=password>`, а затем, возможно, изучает имена полей формы на строки вроде `user` и `pass`. Когда подходящая форма обнаружена, браузер предложит сохранить связанную информацию для входа в постоянном хранилище на жестком диске,^[6] и, если пользователь согласится, затем будет автоматически извлекать и вставлять эти данные в совпадающие формы, встреченные позднее. В Firefox, Chrome и Safari процесс извлечения сохраненного пароля автоматический; в Internet Explorer и Opera может требоваться дополнительный жест пользователя для подтверждения намерения.

Устройство менеджеров паролей хрупко, но имеет одно явное преимущество: оно работает сразу, даже без официальной поддержки или, если уж на то пошло, информированного согласия со стороны каких-либо сайтов. Веб-приложения, недовольные этой функцией, могут отказаться от нее, добавив плохо названный параметр `autocomplete=off` к проблемному полю пароля,^[7] но за этим пределом процесс почти полностью бесшовен.

Основной способ, которым каждый встроенный в браузер менеджер паролей защищает сохраненные данные, состоит в привязке учетных данных к `SOP origin`, где они были изначально введены, с вниманием к имени хоста, протоколу и порту. Некоторые браузеры также учитывают вторичные индикаторы, такие как порядок или имена полей формы, путь URL к форме или адрес, на который отправляются учетные данные. Как мы знаем из главы 9, такие меры ограничения области не особенно полезны с точки зрения безопасности из-за работы политики одного источника.

В браузерах, которые автоматически заполняют формы входа без необходимости человеческого взаимодействия, разумно смотреть на механизм как на форму привилегированного API: любое содержимое, выполняющееся в подходящем `origin`, сможет запросить хранимые браузером учетные данные, построив правдоподобно выглядящую форму и затем подождет, пока она автоматически заполнится данными входа. Чтобы прочитать эту информацию обратно, скрипту нужно лишь изучить свойство `value` DOM-элемента, связанного с полем пароля.

Примечание. Удаление способности проверять значения полей пароля может показаться простым способом улучшить схему, но это не очень хороший способ. Данные все равно можно украсть, например дождавшись автозаполнения пароля, изменив метод отправки данных с `POST` на `GET`, а затем вызвав `submit()` у формы входа. Эти шаги приведут к навигации на страницу, где пароль явно виден в строке `location.search`. Кроме того, у многих веб-приложений есть легитимные причины читать эти поля на стороне клиента, например чтобы давать советы по надежности пароля.

Как должно быть понятно, самый серьезный риск, связанный с менеджерами паролей, - усиление XSS-багов. В веб-приложениях, использующих `httponly` cookies, успешная эксплуатация XSS-изъяна может дать злоумышленнику только временный доступ к учетной записи пользователя, но если ту же уязвимость можно использовать для кражи пароля пользователя, последствия тяжелее и долговечнее.^[8]

Возможны и более темные побочные эффекты. Например, любое приложение, позволяющее пользователям конструировать пользовательские опросы на основе форм, должно тщательно ограничивать компоновку сгенерированных форм, иначе оно рискует заодно стать инструментом сбора паролей.

Модель зон Internet Explorer

Модель зон Internet Explorer^[9] - проприетарная попытка примирить разные требования безопасности, которые пользователи или системные администраторы могут иметь для разных типов веб-приложений, например банковской страницы и онлайн-игры. Подход Microsoft состоит в установлении нескольких заранее заданных классов сайтов, известных как зоны, каждая со своим набором настраиваемых разрешений безопасности. Поддерживаются пять зон:

- **My computer (она же local machine).** Эта скрытая зона используется для всех локальных ресурсов file: с одним исключением, о котором скоро будет сказано больше. Пользователь не может добавлять или удалять какие-либо элементы из этого набора и не может менять его настройки безопасности через обычный пользовательский интерфейс. Однако администраторы и разработчики могут изменять реестр или использовать hooks urlmon.dll, чтобы переопределять настройки.
- **Local intranet.** Эта зона предназначена для доверенных приложений в локальной сети пользователя. По умолчанию local intranet пользуется многими проблемными привилегиями, такими как неограниченный доступ к системному буферу обмена, способность открывать окна без адресной строки или способность обходить обычные проверки безопасности навигации фреймов, то есть descendant policy, описанную в главе 11. Члены этого набора обнаруживаются автоматически с помощью нескольких настраиваемых эвристик и могут включать адресаты с не полностью квалифицированными именами хостов, адреса в списке исключений HTTP-прокси^[10] или удаленные URL file:, доступные через SMB. Ручное включение сайтов в эту зону также возможно, в дополнение к встроенным эвристикам или вместо них.

Примечание. Зона local intranet устанавливает неявную связь между локальной сетью и доверенной средой. В современной среде эта связь часто сомнительна, особенно с учетом распространенности публичного доступа в интернет через незашифрованный Wi-Fi: другие использования сети не более надежны, чем случайный сайт, размещенный на другом конце света.

- **Trusted sites.** Это номинально пустые зоны, примерно эквивалентные local intranet с точки зрения настроек безопасности, но управляемые исключительно пользователем. Эвристики автообнаружения недоступны, и все записи нужно создавать вручную.
- **Restricted sites.** В эти номинально пустые зоны пользователь может добавлять "недоверенные" адресаты. Настройки по умолчанию для этих зон удаляют многие элементарные и в целом безвредные возможности из загруженного содержимого, например заголовки Refresh не будут работать, при этом предлагая ограниченную пользу для безопасности.

Практичность этой зоны кажется неясной. Из-за необходимости заносить каждый недоверенный сайт в белый список на эту зону очевидно нельзя полагаться как на альтернативу просмотру интернета с разумными настройками по умолчанию для ранее не виденных адресатов.

- **Internet.** Это зона по умолчанию для сайтов, не включенных ни в одну из остальных категорий. Ее настройки по умолчанию соответствуют базовой линии общей модели безопасности браузера, обсуждавшейся ранее в этой книге.

Концепция зон, вместе с некоторыми их средствами контроля безопасности, кажется шагом в правильном направлении. Например, она позволяет системным администраторам тонко настраивать разрешения для документов file:, не затрагивая безопасность или удобство обычного просмотра, или запрещать интернет-сайтам переходить к локальным корпоративным системам, используя настройку с названием "Websites in less privileged web content zone can navigate into this zone". К сожалению, фактическая реализация модели зон замутнена недостатком фокуса, и на практике ею злоупотребляют чаще, чем получают от нее настоящую пользу.

Первая проблема, очевидная каждому, кто пытается освоить механизм зон, - его туповатая терминология и почти комическая сложность многих настроек. У каждой зоны более 100 флажков; некоторые из них глубоко изменяют модель безопасности браузера, тогда как другие вообще не имеют последствий для безопасности. Упомянутая выше настройка Refresh - один пример пустой с точки зрения безопасности операции; способность отключить отправку форм - другой. Эти два класса настроек никак ясно не различаются, и многие почти невозможно понять с первого взгляда. Например, параметр "Binary and script behaviors" можно установить в "enable" или "disable", но подсистема помощи не дает информации о том, что фактически делает каждая настройка. Единственное объяснение предоставлено в официальной документации для разработчиков, размещенной на сайте Microsoft, но даже этот документ может запутать.^[11] Судите сами:

Internet Explorer содержит dynamic binary behaviors: компоненты, инкапсулирующие определенную функциональность для HTML-элементов, к которым они были прикреплены. Эти binary behaviors не контролируются никакой настройкой безопасности Internet Explorer, что позволяет им работать на веб-страницах в зоне Restricted Sites. В Windows Server 2003 Service Pack 1 появилась новая настройка безопасности Internet Explorer для binary behaviors. Эта новая настройка по умолчанию отключает binary behaviors в зоне Restricted Sites. В сочетании с функцией безопасности Local Machine Lockdown она также по умолчанию требует административного одобрения для запуска binary behaviors в зоне Local Machine. Эта новая настройка безопасности binary behaviors предоставляет общее смягчение уязвимостей в binary behaviors Internet Explorer.

Есть много похожих случаев настроек, требующих существенных усилий для понимания. Например, маловероятно, что даже самые опытные администраторы поймут последствия изменения настроек с именами "Access data sources across domains" или "Navigate windows and frames across different domains". У всей этой путаницы есть интересное последствие: доверенные стороны непреднамеренно раздают сомнительные советы. Например, Charles Schwab, заметный инвестиционный банк, говорит клиентам отключить descendant model навигации фреймов,^[12] по сути делая HTML-фреймы небезопасными не только для Charles Schwab, но и для любого другого сайта. Один из сайтов, поддерживаемых Internal Revenue Service, дает тот же крайне невнимательный совет.^[13]

Если оставить в стороне сложность и плохую документацию настроек зон Internet Explorer, другая проблема модели зон - кластеризация несвязанных разрешений. Настройки для контейнеров local intranet и trusted sites включают случайный набор функций, которые могут требоваться некоторым доверенным сайтам, но ни одному доверенному сайту не могут требоваться все разрешения, которые влечет зона. Из-за такого устройства добавление сайтов в привилегированные зоны вновь может иметь неожиданно далеко идущие последствия в случае, скажем, тривиального XSS-изъяна.

Mark of the Web и Zone.Identifier

Чтобы поддерживать целостность модели зон для скачанных файлов, Internet Explorer дополнительно использует два перекрывающихся механизма для отслеживания исходной информации о зоне для любого документа, полученного извне:

- **Mark of the Web (MotW).** Этот простой псевдо-HTML-тег вставляется в начало HTML-документов, скачанных через Internet Explorer, чтобы указать их исходный источник.^[14] Один пример тега MotW может быть таким: `<!-- saved from url=(0024)http://fuzzybunnies.com/ -->`. URL, записанный в этом теге, сопоставляется с подходящей зоной; затем документ открывается в уникальном origin в этой зоне. Самое важное последствие состоит в том, что скачанное содержимое изолируется от других URL file:.

Примечание. Inline-природа MotW - один из его изъянов. Поддельные теги могут быть заранее вставлены вредоносными сторонами в HTML-документы, скачанные через браузеры не Internet Explorer, сохраненные из почтовых клиентов или скачанные Internet Explorer с не HTML-расширением и затем подвергнутые content sniffing. Хотя, справедливости ради, привилегии документов file:, сохраненных без каких-либо тегов MotW, достаточно значительны, чтобы злоумышленники оставались сравнительно незаинтересованными в прыжке из зоны My Computer, скажем, в Local Intranet.

- **Alternate Data Stream (ADS) Zone Identifier.** Это фрагмент NTFS-метаданных, прикрепляемый Internet Explorer и Chrome к каждому скачанному файлу, указывающий числовой код зоны, из которой файл был получен.^[15] Механизм Zone.Identifier менее переносим, чем MotW, и информация теряется, когда файлы сохраняются на не NTFS-файловые системы. Однако он также более универсален, поскольку может применяться к не HTML-документам.

Метаданные Zone.Identifier распознаются самим Internet Explorer, графической оболочкой Windows и некоторыми другими продуктами Microsoft, но стороннее ПО почти повсеместно их игнорирует. Там, где они поддерживаются, это может приводить к применению более ограничительной политики безопасности к документу; чаще это просто вызывает предупреждение безопасности о неуказанных рисках открытия данных, происходящих из интернета.

Памятка по инженерии безопасности

При запросе повышенных разрешений из веб-приложения

Помните, что запрос доступа к данным геолокации, потокам видео или микрофона и другим привилегированным API несет ответственность. Если ваш сайт склонен к XSS-уязвимостям, вы рискуете не только данными, хранящимися в приложении, но и приватностью ваших пользователей. Планируйте соответственно и хорошо разделяйте привилегированную функциональность на изолированные части. Никогда не просите пользователей понижать настройки безопасности Internet Explorer ради вашего приложения и не следуйте слепо такому совету от других, кем бы они ни были.

При написании плагинов или расширений, распознающих привилегированные origins

Вы подвергаете пользователей повышенному риску из-за неизбежных ошибок безопасности веб-приложений. Проектируйте API надежно и пытайтесь использовать вторичные меры безопасности, такие как криптография, чтобы дополнительно защитить коммуникации с вашим сервером. Не заносите в белый список незашифрованные origins, поскольку они подвержены подмене в открытых беспроводных сетях.

Сноски

[1] [\[назад\]](#) Справедливо жаловаться, что браузеры мало делают для снабжения пользователей утвердительными сигналами о надежности посещенного сайта, хотя многие надежные индикаторы вполне можно было бы получить автоматически. Попытки блокировать известные вредоносные сайты на основе черных списков существуют, но с учетом ничтожной стоимости регистрации нового домена или компрометации случайного существующего такие подходы, пожалуй, менее ценны.

[2] [\[назад\]](#) Сейчас эта функциональность поддерживается только плагинами, такими как Adobe Flash, но движется к тому, чтобы стать частью HTML5.

- [3] [\[назад\]](#) Этот API выводит местоположение пользователя из параметров вроде текущего IP-адреса, списка близлежащих беспроводных сетей или сотовых вышек либо данных аппаратного GPS-приемника. За исключением GPS-данных, может потребоваться обращение к внешнему поставщику услуг, чтобы сопоставить эти входные данные с физическими координатами.
- [4] [\[назад\]](#) Публичные записи XSSed о addons.mozilla.org.
- [5] [\[назад\]](#) OpenID - пример федеративного входа ограниченного масштаба.
- [6] [\[назад\]](#) Эти данные могут храниться на диске как plaintext-представление, наивно обфусцированная строка или корректно зашифрованное значение, защищенное "master"-паролем, который нужно предварительно ввести. Все три метода сравнимо уязвимы для решительных злоумышленников с доступом к локальной системе, но plaintext-подход иногда осуждается, поскольку он более открыт для любопытных, но нетехнических пользователей.
- [7] [\[назад\]](#) Несмотря на название, это мешает браузеру записывать пароль, а не только автоматически его заполнять.
- [8] [\[назад\]](#) Такие последствия могут выходить за пределы затронутого приложения: даже при наличии менеджеров паролей повторное использование паролей остается распространенной и печальной тенденцией.
- [9] [\[назад\]](#) Модель зон Internet Explorer.
- [10] [\[назад\]](#) В конфигурациях, где прокси требуется для доступа к защищенным внутренним системам, но не требуется для доступа в интернет, это может иметь непреднамеренный и пугающий эффект классификации всего Web как локальной сети.
- [11] [\[назад\]](#) Документация Microsoft по настройке binary behaviors.
- [12] [\[назад\]](#) Совет Charles Schwab по настройке навигации фреймов.
- [13] [\[назад\]](#) Похожий совет на сайте, поддерживаемом Internal Revenue Service.
- [14] [\[назад\]](#) Mark of the Web, механизм Internet Explorer.
- [15] [\[назад\]](#) ADS Zone.Identifier, metadata NTFS для скачанных файлов.

Часть III. Взгляд на то, что впереди

После почти десятилетия застоя мир браузеров снова стал бушующим полем битвы. В манере, слишком напоминающей Первую войну браузеров конца 1990-х, поставщики конкурируют, выводя на рынок новые функции ежемесячно. Главное различие в том, что безопасность теперь воспринимается как явное торговое преимущество.

Конечно, объективное измерение надежности любого достаточно сложного фрагмента ПО - нерешенная проблема вычислительной техники, тем более если ваша кодовая база несет почти два десятилетия накопленного раздувания. Поэтому значительная часть конкурентных усилий уходит на изобретение, а затем быстрое развертывание новых дополнений с тематикой безопасности, часто без особого учета того, насколько хорошо они на самом деле решают проблему, ради которой должны были появиться.

Тем временем органы стандартизации, помня о своих прежних злключениях, отказались от значительной части академической строгости в пользу того, чтобы просто позволить преданной группе участников настраивать спецификации так, как они считают нужным. Идут разговоры о том, чтобы сделать HTML5 последней нумерованной версией стандарта и перейти к живому документу, который меняется каждый день, часто радикально. Ослабление этого требования помогло поддерживать значительную часть работы вокруг W3C и WHATWG, но также подорвало некоторые преимущества самого наличия центральной организации.

Многие недавние предложения тяготеют к быстрым, узко ограниченным заплаткам, которые даже не пытаются образовать последовательный и хорошо интегрированный фреймворк. Когда это происходит, нет надежного механизма обратной связи, который позволил бы внешним экспертам просмотреть достаточно стабильные спецификации и высказать опасения до начала любой работы над реализацией. Единственный способ оставаться в курсе изменений - погрузиться в повседневную динамику рабочей группы.

Трудно сказать, плох ли этот новый подход к стандартизации. На самом деле его преимущества легко могут перевесить любые умозрительные риски; хотя бы потому, что теперь у нас есть шанс на стандарт, достаточно близкий к тому, что браузеры действительно делают. Тем не менее результаты этого лихорадочного и в значительной степени неконтролируемого процесса могут быть непредсказуемыми, и они требуют от сообщества безопасности большой бдительности.

В этом духе последняя часть книги исследует некоторые из более правдоподобных и продвинутых предложений, которые могут сформировать будущее Web... или с тем же успехом через несколько лет оказаться на свалке истории.

Глава 16. Новые и будущие функции безопасности

Вы скоро увидите, что в том, как все новые браузерные функции сочетаются друг с другом, мало порядка и логики, но нам все же нужно как-то организовать обсуждение. Возможно, лучший подход - посмотреть на их предполагаемые цели и начать со всех механизмов, созданных специально для точной настройки модели безопасности Web ради хорошо определенной выгоды.

Мечта изобрести совершенно новую модель безопасности браузера сильна в сообществе, но за ней всегда приходит осознание, что для этого пришлось бы перестроить весь Web. Поэтому значительная часть практической работы сосредоточена на более скромных расширениях существующего подхода, неизбежно увеличивая сложность критичных для безопасности частей кодовой базы браузера. Эта сложность нежелательна, но ее сторонники неизменно считают ее оправданной: либо потому что они стремятся смягчить класс уязвимостей, либо потому что строят временную подпорку для другой трудной проблемы, которой сейчас никто не хочет заниматься,^[1] либо просто потому что хотят дать возможность строить новые типы приложений в будущем. Все эти выгоды обычно перевешивают расплывчатый риск.

Фреймворки расширения модели безопасности

Некоторые из самых успешных улучшений безопасности, предложенных за последние несколько лет, сводятся к добавлению гибкости к исходным ограничениям, наложенным политикой одного источника и ее родственниками. Например, одно прежде экспериментальное предложение, теперь вошедшее в широкое употребление, - `API postMessage( . . . )` для коммуникации между `origin`, обсуждавшийся в главе 9. Удивительно, но ослабление проверок SOP в некоторых тщательно выбранных сценариях более интуитивно и с меньшей вероятностью вызывает проблемы, чем ужесточение политики. Поэтому, чтобы начать с более легкой ноты, сначала сосредоточимся на этом классе фреймворков.

Междоменные запросы

При исходных ограничениях политики одного источника скрипты, связанные с одним `origin`, не имеют чистого и безопасного способа общаться с клиентскими скриптами, выполняющимися в любом другом `origin`, и не имеют безопасного способа получать потенциально полезные данные от согласного стороннего сервера.

Веб-разработчики давно жаловались на эти ограничения, и в последние годы поставщики браузеров начали прислушиваться к их требованиям. Как вы помните, более срочная задача организации клиентских коммуникаций между скриптами была решена с помощью `postMessage( . . . )`. Сценарий "клиент-сервер" оказался менее срочным и все еще ожидает канонического решения, но некоторый прогресс есть.

Самая успешная попытка создать метод получения документов с серверов не того же `origin` началась в 2005 году. Под эгидой W3C несколько разработчиков, работавших над VoiceXML, малоизвестным форматом документов для построения систем Interactive Voice Response (IVR), подготовили предложение Cross-Origin Resource Sharing (CORS).^[2] Между 2007 и 2009 годами их неуклюжий XML-based дизайн постепенно превратился в куда более простую и широко полезную схему, которая полагалась на сигнализацию уровня HTTP-заголовков для передачи согласия на междоменное получение содержимого через естественное расширение `API XMLHttpRequest`.

Типы запросов CORS

В нынешней спецификации CORS полагается на различие двух типов вызовов API XMLHttpRequest. Когда сайт пытается загрузить документ другого origin через API, браузер сначала должен различить простые запросы, где результирующий HTTP-трафик считается достаточно близким к тому, что можно сгенерировать через другие, уже существующие методы навигации, и непростые запросы, охватывающие все остальное. Работа этих двух классов запросов значительно различается, как мы увидим.

Текущая спецификация говорит, что простые запросы должны иметь метод GET, POST или HEAD. Кроме того, если вызывающая сторона задает какие-либо пользовательские заголовки, они должны принадлежать следующему набору:

- Cache-Control
- Content-Language
- Content-Type
- Expires
- Last-Modified
- Pragma

Сегодня браузеры, поддерживающие CORS, просто не разрешают методы, отличные от GET, POST и HEAD. В то же время они игнорируют рекомендованный белый список заголовков, безусловно понижая любые запросы с пользовательскими значениями заголовков до непростого статуса. Реализация в WebKit также считает любые запросы, несущие полезную нагрузку, непростыми. Неясно, является ли это намеренным проектным решением или багом.

Проверки безопасности для простых запросов

Спецификация CORS позволяет немедленно отправлять простые запросы на целевой сервер, не пытаясь сначала подтвердить, готов ли адресат вообще участвовать в междоменных коммуникациях. Это решение основано на том факте, что злоумышленник может инициировать довольно похожий cookie-аутентифицированный трафик другими средствами, например автоматически отправив форму, и поэтому нет смысла вводить дополнительное рукопожатие специально для CORS.^[3]

Ключевая проверка безопасности выполняется только после того, как ответ получен от сервера: данные раскрываются вызывающей стороне через API XMLHttpRequest только если ответ включает подходящий, правильно сформированный заголовок Access-Control-Allow-Origin. Чтобы помочь серверу, исходный запрос будет включать обязательный заголовок Origin, указывающий origin, связанный с вызывающим скриптом.

Чтобы проиллюстрировать это поведение, рассмотрим следующий междоменный вызов XMLHttpRequest, выполненный с `http://www.bunnyoutlet.com/`:

```
var x = XMLHttpRequest();
x.open('GET', 'http://fuzzybunnies.com/get_message.php?id=42', false);
x.send(null);
```


Результатом будет HTTP-запрос примерно такого вида:

```
GET /get_message.php?id=42 HTTP/1.0
Host: fuzzybunnies.com
Cookie: FUZZYBUNNIES_SESSION_ID=EA7E8167CE8B6AD93D43AC5AA869A920
Origin: http://www.bunnyoutlet.com
```

Чтобы указать, что ответ должен быть читаемым между доменами, сервер должен ответить:

```
HTTP/1.0 200 OK
Access-Control-Allow-Origin: http://www.bunnyoutlet.com

The secret message is: "It's a cold day for pontooning."
```

Примечание. В Access-Control-Allow-Origin можно использовать подстановочный знак (*), но делайте это осторожно. Совершенно неразумно без разбора ставить Access-Control-Allow-Origin: * на все HTTP-ответы, потому что этот шаг в значительной степени устраняет любые гарантии политики одного источника в CORS-совместимых браузерах.

Непростые запросы и preflight

В ранних черновиках протокола CORS почти все запросы предполагалось отправлять без предварительной проверки, действительно ли сервер готов их принять. К сожалению, такой дизайн подрывал интересное свойство, используемое некоторыми веб-приложениями для предотвращения cross-site request forgery: до CORS злоумышленники не могли внедрять произвольные HTTP-заголовки в междоменные запросы, поэтому наличие пользовательского заголовка часто служило доказательством, что запрос пришел из того же origin, что и адресат, и был выпущен через XMLHttpRequest.

Поздние редакции CORS исправили эту проблему, потребовав более сложного двухшагового рукопожатия для запросов, не соответствующих строгим критериям "простого запроса", описанным выше. Рукопожатие для непростых запросов нацелено на подтверждение того, что целевой сервер совместим с CORS и хочет получать нестандартный трафик от этой конкретной вызывающей стороны. Рукопожатие реализуется отправкой обычного запроса OPTIONS ("preflight") на целевой URL, содержащего очерк параметров нижележащего вызова XMLHttpRequest. Самая важная информация передается серверу в трех самоочевидных заголовках: Origin, Access-Control-Request-Method и Access-Control-Request-Headers.

Это рукопожатие считается успешным только если эти параметры корректно подтверждены в ответе с помощью Access-Control-Allow-Origin, Access-Control-Allow-Method и Access-Control-Allow-Headers. После корректного рукопожатия выполняется настоящий запрос. По соображениям производительности результат preflight-проверки для конкретного URL может кэшироваться клиентом на заданный срок.

Текущий статус CORS

На момент написания CORS доступен только в Firefox и браузерах на основе WebKit и заметно отсутствует в Opera или Internet Explorer. Самый важный фактор, мешающий его принятию, может быть просто в том, что API не так критичен, как его клиентский аналог `postMessage(...)`, потому что его часто можно заменить серверным прокси для получения содержимого. Но схема также сталкивается с тремя основными, хотя и слабыми, критическими замечаниями, некоторые из которых приходят напрямую от одного из поставщиков. Очевидно, эта критика делу не помогает.

Первая жалоба, озвученная главным образом разработчиками Microsoft и поддержанная некоторыми академиками, состоит в том, что схема без нужды злоупотребляет неявными полномочиями. Они утверждают, что существует очень мало случаев, где данные, разделяемые между доменами, должны подстраиваться под учетные данные, доступные для целевого сайта. Критики считают, что риск случайной утечки чувствительной информации намного перевешивает любую пользу и что схема, позволяющая выполнять только неаутентифицированные запросы, была бы предпочтительнее. По их мнению, любые сайты, которым нужна форма аутентификации, должны вместо этого полагаться на явно обмененные токены аутентификации.^[4]

Другая, более прагматичная критика CORS состоит в том, что схема без нужды сложна: она расширяет и без того проблемный и подверженный ошибкам API, не объясняя ясно выгоду некоторых настроек. В частности, неясно, стоит ли добавленная сложность `preflight`-запросов периферийной выгоды от возможности выполнять междоменные запросы с нестандартными методами или случайными заголовками.

Последняя из слабых жалоб держится на том факте, что CORS восприимчив к внедрению заголовков. В отличие от некоторых других недавно предложенных браузерных функций, таких как WebSockets (глава 17), CORS не требует, чтобы сервер эхом возвращал непредсказуемую строку-вызов для завершения рукопожатия. Особенно в сочетании с кэшированием `preflight` это может усилить влияние некоторых уязвимостей расщепления заголовков в серверном коде.

XDomainRequest

Возражение Microsoft против CORS, похоже, проистекает из вышеупомянутых опасений по поводу использования неявных полномочий, но также несет тонкие оттенки их неудовлетворенности взаимодействиями с W3C. В 2008 году Сунава Датта, program manager в Microsoft, предложил такое несколько загадочное пояснение:^[5]

Во время Beta 1 Internet Explorer 8 было высказано много основанных на безопасности опасений по поводу междоменного доступа к сторонним данным с использованием `cross site XMLHttpRequest` и фреймворка Access Control. После Beta 1 у нас была возможность работать с другими браузерами и участниками очной встречи W3C, чтобы улучшить серверную сторону и безопасность фреймворка Access Control от W3C.

Вместо того чтобы принять расширения CORS для `XMLHttpRequest`, Microsoft решила реализовать контрпредложение под названием `XDomainRequest`.^[6] Этот примечательно простой новый API отличается от варианта, доступного в других браузерах, тем, что результирующие запросы всегда анонимны, то есть лишены любых управляемых браузером учетных данных, и тем, что он не позволяет использовать какие-либо пользовательские HTTP-заголовки или методы.

Использование API Microsoft в остальном очень похоже на XMLHttpRequest:

```
var x = new XMLHttpRequest();
x.open("GET", "http://www.fuzzybunnies.com/get_data.php?id=1234");
x.send();
```

Заимствуя из предложения W3C, результирующий запрос будет нести заголовок Origin, а данные ответа будут раскрыты вызывающей стороне только если в ответе присутствует совпадающий заголовок Access-Control-Allow-Origin.^[7] Preflight-запросы и кэширование разрешений не являются частью дизайна.

С практической точки зрения решение Microsoft намного разумнее, чем CORS: оно проще, безопаснее и, вероятно, столь же функционально во всех правдоподобных сценариях использования. Тем не менее оно непопулярно. Оно поддерживается только в Internet Explorer 8 и выше, а поскольку W3C поддерживает CORS, у остальных нет причин в ближайшее время принимать XMLHttpRequest.

Тем временем отдельная группа исследователей предложила третье решение, снова действуя под эгидой W3C. Их дизайн, известный как Uniform Messaging Policy, вместе с соответствующим API UniformRequest,^[8] принимает подход, почти идентичный подходу Microsoft. Он не поддерживается ни в одном существующем браузере, но идут разговоры об объединении его с CORS.

Другие применения заголовка Origin

Заголовок Origin - существенная часть CORS, XMLHttpRequest и UMP, но фактически он развивался несколько независимо, с учетом других применений. В своей статье 2008 года Адам Барт, Коллин Джексон и Джон К. Митчелл^[9] выступили за введение нового HTTP-заголовка, который предложил бы более надежную и учитывающую приватность альтернативу Referer. Он также служил бы способом предотвращения cross-site request уязвимостей, предоставляя серверу информацию, необходимую для идентификации SOP-level origin запроса, без раскрытия потенциально более чувствительных данных пути или query.

Конечно, было неясно, сможет ли тонкое улучшение между Referer и его предложенным преемником действительно что-то изменить для небольшой, но ненулевой доли пользователей, блокирующих первый заголовок по соображениям приватности. Поэтому предложение в итоге оказалось в виртуальном подвешенном состоянии: оно не было развернуто ни в одном существующем браузере, но также отговаривало других от продвижения иных решений, таких как XSRF или XSSI.^[10] Справедливости ради, концепция совсем недавно была оживлена под новым именем From-Origin и, возможно, еще не полностью мертва.^[11]

Если оставить в стороне судьбу исходной идеи, полезность заголовка Origin в специализированных случаях вроде CORS была довольно ясна. Около 2009 года это привело к тому, что Барт отправил в IETF черновик, задающий синтаксис заголовка,^[12] но уклонился от каких-либо заявлений о том, когда заголовок должен отправляться или какие конкретные проблемы безопасности он может решать:

User agent MAY include an Origin header in any HTTP request.

[...]

Whenever a user agent issues an HTTP request from a "privacy-sensitive" context, the user agent MUST send the value "null" in the Origin header.

NOTE: This document does not define the notion of a privacy-sensitive context. Applications that generate HTTP requests can designate contexts as privacy-sensitive to impose restrictions on how user agents generate Origin headers.

Суть этой спецификации в том, что каким бы ни был процесс принятия решения, как только клиент выбирает предоставить заголовок, значение должно точно представлять SOP origin, из которого выполняется запрос. Например, когда конкретная операция происходит с `http://www.bunnyoutlet.com:1234/bunny_reports.php`, переданное значение должно быть:

Origin: `http://www.bunnyoutlet.com:1234`

Для origins, которые осмысленно не отображаются на кортеж `protocol-host-port`, браузер вместо этого должен отправить значение `null`.

Несмотря на все эти планы, на момент написания только один браузер включает заголовок `Origin` при не CORS-навигации: реализации на основе WebKit отправляют его при отправке HTML-форм. Firefox, похоже, рассматривает другой подход, но ничего конкретного пока, кажется, не реализовано.

Фреймворки ограничения модели безопасности

Дизайны, расширяющие границы политики одного источника, довольно просты для понимания и обычно отказывают безопасно. Если предложенное изменение не учтено в одном из возможных путей кода или просто не поддерживается конкретным браузером, сработает уже реализованная, более ограничительная логика. По сравнению с этим намного опаснее пытаться возводить новые границы поверх существующей модели безопасности браузера. Это потому, что каждый чувствительный к безопасности путь кода должен быть изменен, чтобы распознавать новую схему, и каждый браузер должен сразу соответствовать ей, иначе возникнут неожиданные проблемы.

В этом разделе мы быстро рассмотрим некоторые из более успешных попыток пойти по этому опасному, но потенциально вознаграждающему пути и исследуем, где они распадаются.

Content Security Policy

Content Security Policy (CSP) - необычно всеобъемлющий фреймворк безопасности, впервые предложенный Брэндоном Стерном из Mozilla в 2008 году.^[13] Изначально фреймворк задумывался как универсальный способ смягчения влияния распространенных веб-уязвимостей, от XSRF до XSS, и как инструмент для владельцев сайтов, позволяющий выполнять различные не связанные с безопасностью задачи контроля содержимого.

В последующие годы CSP быстро развивалась, и несколько раз ее область существенно менялась. Например, автор быстро отказался от плана решать XSRF-уязвимости, делегировав задачу еще нереализованным расширениям заголовка `Origin`. Фактически на момент написания каноническая спецификация Mozilla переписывается как черновик W3C,^[14] что приводит к существенным различиям между реализацией, поставляемой в Firefox, и частичной поддержкой, реализованной в WebKit Адамом Бартом. Internet Explorer и Opera не поддерживают CSP и не объявляли конкретных планов ее принять.

Основные директивы CSP

В своей основе дизайн Стерна позволяет владельцам сайтов задавать политики на уровне документа, ограничивающие способность рассматриваемого документа выполнять действия, которые обычно были бы разрешены политикой одного источника. Например, CSP может помешать странице загружать любые внешние подресурсы, кроме изображений, и ограничить источники изображений только набором доверенных origins:

```
X-Content-Security-Policy: default-src 'none'; img-src http://*.example.com
```

Как видно из этого примера, политики могут кодироваться в HTTP-заголовке. Согласно черновику W3C, их также можно встроить в сам документ с помощью тегов `<meta>` или разместить политику на внешнем URL и указать на нее через `policy-uri`.

Для каждой директивы источников содержимого автор политики может задать любое число полностью квалифицированных origins или выражений с подстановочными знаками, совпадающих с несколькими хостами, протоколами или портами. Три специальных ключевых слова (`none`, `self` и `data:`) соответствуют пустому множеству, origin, связанному со страницей, несущей политику, или всем встроенным URL `data:` соответственно.

На сегодняшний день следующими видами поведения можно управлять с помощью директив CSP:

- **Выполнение скриптов.** Директива `script-src` может использоваться для указания протокола, хоста и порта для допустимых URL `<script src=...>`. Обычно CSP отключает способность встраивать скрипты прямо в документ, будь то через самостоятельные блоки `<script>` или через обработчики событий, а также способность существующих скриптов небрежно передавать строки функциям вроде `eval(...)`, `setTimeout(...)`, `setInterval(...)` и т. д. Благодаря этому директива `script-src` полезна для ограничения влияния XSS-уязвимостей: любая разметка, внедренная злоумышленником, будет ограничена загрузкой скриптов, легитимно размещенных в одном из одобренных origins.^[15]
- **Содержимое плагинов.** Это управляется через `object-src`. Поскольку плагины вроде Java или Flash могут иметь неограниченный доступ к встраиваемой странице, директиву следует считать во многом аналогичной `script-src`, и эти две директивы должны быть ограничены сопоставимым образом, чтобы дать какие-либо выгоды безопасности.
- **Таблицы стилей и шрифты.** Это управляется `style-src` и `font-src`. В отличие от обработки скриптов, CSP изначально не запрещала появление встроенных блоков `<style>` или параметров `style=` на странице. Поэтому любой злоумышленник, эксплуатирующий XSS-изъян, мог драматически изменить внешний вид и функцию уязвимой страницы или хуже,^[16] и эти две директивы служили только не связанным с безопасностью целям, за возможным исключением ограничения ошибок смешанного содержимого. Лишь за несколько мгновений до публикации книги спецификации были изменены, чтобы включить более надежный подход к CSS.
- **Пассивное мультимедиа.** Директивы вроде `img-src` или `media-src` управляют способностью встраивать мультимедийное содержимое из конкретных origins. Как и в исходном дизайне CSS-контролей, это нельзя было считать функцией безопасности. Например, в случае XSS-бага CSP не помешала бы злоумышленнику использовать таблицы стилей, чтобы рисовать произвольные формы на уязвимой странице или даже в некоторой степени анимировать их.
- **Подфреймы.** Директива `frame-src` задает допустимые адресаты для любых тегов `<iframe>`, встреченных на странице; политика родительской страницы не наследуется документом во фрейме. Чтобы сохранить ценность других XSS-смягчений, нужно принять меры и не разрешать здесь URL `data:` (см. главу 10).
- **Политика по умолчанию.** Известная как `default-src` в черновике W3C и под более загадочным именем (`allow`) в документации Mozilla, директива задает fallback-поведение для любого

содержимого, не покрытого более конкретной директивой. Директива обязательна даже в случаях, когда технически она не нужна.

Примечание. Возможно, неудачно, что директивы CSP выбраны так, чтобы очень близко соответствовать отдельным HTML-тегам, вместо группировки функционально похожего поведения. Из-за этого трудно оценить сложные взаимодействия между настройками вроде `script-src`, `frame-src` и `object-src`. Кроме того, подход просто не очень защищен от будущего: уже есть некоторые периферийные классы подресурсов, например "favicons", которые полностью исключены из CSP, и этот список, вероятно, непреднамеренно вырастет.

В необычном отходе от модели, ориентированной на подресурсы, описанной до сих пор, CSP также содержит странную директиву `frame-ancestors`. Этот параметр предназначен для смягчения влияния `clickjacking` путем указания допустимых предков для текущего документа способом, похожим на более устоявшийся заголовок `X-Frame-Options`, описанный в главе 11. Логика `frame-ancestors` полностью независима от `default-src` или любых других частей CSP; ее значение по умолчанию - `*`.

Многие другие возможные расширения политики обсуждаются на момент написания. Они включают директиву `script-nonce`, которую можно было бы использовать для более безопасного встраивания скриптов: каждый блок скрипта должен начинаться с заданного политикой непредсказуемого токена, что часто усложняет эксплуатацию XSS; и директиву `sandbox`, предлагающую альтернативный интерфейс к другому механизму безопасности, обсуждаемому ниже в разделе о `sandboxed frames`.

Нарушения политики

Политика, заданная по этим правилам, ограничивает поведение нижележащего документа. Нарушения обычно приводят к неудачной загрузке подресурса, отказу выполнить встроенный скрипт или подавлению рендеринга страницы в особом случае `frame-ancestors`.

Поскольку CSP управляет широким спектром поведения содержимого, а режим отказа по умолчанию довольно жесток, авторы почувствовали необходимость успокоить вебмастеров. Чтобы сделать CSP более дружелюбной к пользователю и, возможно, также в наивной попытке предложить обнаружение эксплойтов, необязательная функция CSP позволяет браузеру немедленно сообщать обо всех нарушениях политики обратно владельцу сайта. Эту функцию можно включить через ключевое слово `report-uri` в политике. Чтобы дополнительно упростить развертывание, любую политику или ее часть также можно раскатить в "мягком" режиме, где нарушения приводят только к HTTP-уведомлению, но фактически не ломают страницу. Это достигается указанием политики внутри заголовка `X-Content-Security-Policy-Report-Only`.^[17]

Критика CSP

CSP - примечательно разумный и последовательный дизайн по сравнению с большинством одноразовых функций безопасности, предложенных или развернутых в мире браузеров. Тем не менее с момента появления предложение преследуют повторяющиеся опасения по поводу дизайна и реализации.

Пожалуй, самая прозаичная жалоба на CSP не связана с безопасностью: чтобы получить пользу от XSS-защит, предлагаемых фреймворком, вебмастерам приходится переносить все встроенные скрипты на странице, часто сотни отдельных фрагментов кода, в отдельно запрашиваемый документ; в новых черновиках CSP то же потребуется для всех таблиц стилей. Сложность адаптации существующих страниц для работы с CSP и производительный штраф дополнительного HTTP-запроса часто запретительны. Возможно, эту проблему можно решить с помощью расширения `script-nonce`, предложенного в самых последних черновиках.

Более фундаментальная проблема дизайна CSP состоит в том, что задуманная сейчас гранулярность наборов правил на уровне `origin` может не предлагать достаточно надежной защиты от XSS. Учтите, что любой сложный реальный домен вполне может размещать десяток в основном отдельных веб-приложений, каждое из которых состоит из сотен, возможно, несвязанных статических скриптов и JavaScript API. Злоумышленникам, эксплуатирующим XSS-уязвимость на сайте, защищенном CSP, запрещено напрямую выполнять вредоносный скрипт, но они могут привести приложение в несогласованное и возможно опасное состояние, загрузив существующие скрипты в неверном контексте или в неправильной последовательности. История уязвимостей в не веб-ПО показывает, что такие состояния повреждения состояния эксплуатируемы чаще, чем мы можем думать.

Еще более тревожная перспектива состоит в том, что злоумышленник может загрузить подресурс, который на самом деле не является скриптом, но может быть ошибочно принят за него. Крайним примером может быть браузер с поддержкой E4X (см. главу 6): любой допустимый XHTML-документ, в который злоумышленник может поместить номинально безвредную строку, скажем `{alert("Hi mom!")}`, может привести к выполнению кода при загрузке через `<script src=...>`. Осознав эту проблему, разработчики решили требовать `whitelisted` значения `Content-Type` для любых скриптов, загружаемых под CSP, но даже этого подхода часто недостаточно.

Чтобы понять, что может пойти не так, рассмотрим чрезвычайно распространенную практику размещения публичных JSONP API, где клиент может указать имя функции обратного вызова:

```
GET /store_locator_api.cgi?zip=90210&callback=myResultParser HTTP/1.0
...
HTTP/1.0 200 OK
Content-Type: application/x-javascript
...
myResultParser({ "store_name": "Spacely Space Sprockets",
                  "street": ... });
```

Такой API где угодно в разрешенном CSP `origin` может быть использован злоумышленником для вызова произвольных существующих функций в клиентском коде, возможно вместе с контролируемыми злоумышленником параметрами. А если строка функции обратного вызова не ограничена буквенно-цифровыми символами, а почему она должна быть ограничена, указание `callback=alert(1);//` приведет к прямому внедрению кода.

Если оставить в стороне проблемы гранулярности, CSP заслуживает мягкой критики за иногда озадачивающий и вредный недостаток фокуса. С одной стороны, через включение директив вроде `frame-descendants` или `sandbox` она, кажется, заигрывает с идеей построить единый, объединяющий фреймворк безопасности браузера, только чтобы неожиданно исключить XSRF-изъяны из своей области, не предложив жизнеспособной альтернативы кроме туманного

упоминания Origin. С другой стороны, предложение часто стремится быть просто "Content Policy", без особого внимания к достаточно надежным и интуитивным свойствам безопасности. Легкость создания опасных политик скриптов вместе с изначально неэффективным контролем таблиц стилей и изображений свидетельствует об этой тенденции.

Sandboxed frames

Sandboxed frames^[18] - расширение обычного поведения `<iframe>`. Они позволяют владельцу страницы верхнего уровня наложить определенные дополнительные ограничения на встроенный документ вместе со всеми подфреймами этого документа. Цель - сделать более безопасным встраивание потенциально недоверенной рекламы, гаджетов или предварительно отформатированных HTML-документов на иначе чувствительном сайте. Уточнение дизайна и первоначальная реализация этой функции в WebKit, который сейчас является единственным движком, поддерживающим ее, были продвинуты Адамом Бартом.

Примечание. Любопытно, что sandboxed frames - не совсем новая идея: Microsoft почти десятилетием раньше пришла с похожим предложением. Начиная с версии 6, Internet Explorer поддерживал проприетарный параметр `security=restricted`, который принуждает целевой фрейм рендериться в Restricted Zone, фактически убирая его способность выполнять скрипты, переходить в другие места и т. д. Однако никто, похоже, не был заинтересован использовать эту функцию для чего-либо, кроме обхода некоторых клиентских механизмов безопасности JavaScript, наиболее заметно anticlickjacking проверок. Скоро мы узнаем, добьется ли HTML5-преемник большего успеха.

Дизайн sandboxed frames довольно прост: любой фрейм, встроенный в документ, может быть ограничен указанием параметра `sandbox` на соответствующем теге `<iframe>`. По умолчанию документ, подпадающий под это ограничение, не может выполнять скрипты и выполнять некоторые типы навигации. Разрешения можно тонко настроить одним или несколькими разделенными пробелами ключевыми словами, указанными как значение самого параметра `sandbox`:

- `allow-scripts`. При отсутствии этого ключевого слова документ, отображаемый внутри фрейма, не сможет выполнять JavaScript-код. Главная функция этой возможности - помешать встроенному документу выполнять DoS-атаки, открывать браузерные диалоги или применять любую другую сложную автоматизацию страницы.
- `allow-forms`. Когда это ключевое слово отсутствует, любые HTML-формы, встреченные во встроенном документе, не будут работать. Этот механизм предназначен для предотвращения того, чтобы содержимое во фрейме использовало свое размещение на доверенном сайте для phishing чувствительной информации. Заметьте, что при включенном `allow-scripts` в `allow-forms` мало или совсем нет смысла. Скрипты могут легко строить элементы управления, похожие на формы, и автоматически передавать собранную информацию на другой сайт без необходимости в функционирующем теге `<form>`.
- `allow-top-navigation`. Это ключевое слово снова включает способность встроенной страницы навигировать окно верхнего уровня. Такой тип навигации обычно разрешен как одно из исключений из политики одного источника (см. главу 11), и им можно злоупотребить просто для того, чтобы помешать пользователю взаимодействовать с встраиваемым сайтом или выполнять phishing-атаки.
- `allow-same-origin`. Без этого флага содержимому внутри sandboxed frame назначается уникальный, случайно выбранный, синтетический origin. Это не дает странице обращаться к любому привязанному к origin содержимому, которое обычно было бы доступно скриптам, выполняющимся в домене, где она номинально размещена. Включение `allow-same-origin` убирает синтетический origin и разрешает доступ к данным того же origin.

Скрипты, формы и навигация

Первые три ограничения, доступные sandboxed frames, - скрипты, формы и навигация - достаточно интуитивны и безопасны в использовании. Их ценность уменьшается только из-за необходимости также отключать все плагины всякий раз, когда используется атрибут sandbox, потому что фреймворки вроде Flash или Java не признают это расширение и позволили бы любым встроенным апплетам обходить новые браузерные проверки. К сожалению, три самых очевидных сценария использования sandboxed frames - встроенная реклама, видео и игры - сильно полагаются на Flash, тем самым делая этот механизм безопасности намного менее полезным, чем он мог бы быть.

Синтетические origins

Последний механизм в списке, синтетические origins, намного более проблематичен и, вероятно, ошибочен. Он задуман прежде всего как способ сделать возможной отдачу недоверенных документов, таких как входящие HTML-based emails в интерфейсе веб-почты, как есть, вместе с остальной частью приложения, при этом не позволяя этим недоверенным документам обращаться к чувствительным данным.

К сожалению, концепция синтетических origins создает больше проблем, чем решает. Во-первых, если URL встроенного документа не является непредсказуемым, злоумышленник может просто перейти к нему напрямую в новом окне браузера, и в этом случае браузер вообще не увидит атрибут sandbox. В попытке обойти эту проблему авторы спецификации в конце концов предложили использовать специализированный MIME-тип (text/html-sandboxed) для содержимого, предназначенного для показа только в sandboxed frame. Их рассуждение состоит в том, что браузеры обычно не распознают этот MIME-тип и не будут отображать его встроенно, а в коде обработки <iframe> можно создать специальный случай. Конечно, как должно быть ясно из главы 13, такая защита неадекватна, потому что некоторые браузеры и плагины будут отображать ответы text/html-sandboxed встроенно или интерпретировать возвращенные данные другими тревожными способами, скажем как crossdomain.xml.

Концепция синтетических origins также крайне проблематична с учетом фрагментации механизмов безопасности уровня origin или домена в типичном браузере. Например, возможны опасные взаимодействия с менеджерами паролей, которым нужно явно запретить автоматически заполнять формы входа в sandboxed-документах. Также нужно добавить специальную логику в диалоги безопасности, например в диалог, связанный с API геолокации.

После проб и ошибок реализация, сейчас доступная в WebKit, решила многие из этих проблем по отдельности. Тем не менее будущие реализации, вероятно, будут снова и снова попадать в эту ловушку, особенно поскольку спецификация HTML5 считает поведение этих функций выходящим за рамки и никак не задает требуемое поведение.

Примечание. Удаление синтетических origins тоже приводит к неприятностям: если пользователь щелкнет по ссылке того же сайта в sandboxed-рекламе и эта ссылка откроется в новом окне, браузер, вероятно, должен помешать неограниченным скриптам в новом окне пройти по объекту opener, чтобы выполнить действия, которые его родителю запрещено выполнять самостоятельно.

Strict Transport Security

Одна из самых значимых слабостей дизайна HTTPS состоит в том, что пользователи часто начинают навигацию, набирая в адресной строке URL без протокола, например `bankofamerica.com` вместо `https://www.bankofamerica.com`; в таком случае браузер предположит HTTP и отправит первый запрос открытым текстом. Даже если сайт немедленно перенаправит этот трафик на HTTPS, любой активный злоумышленник в сети жертвы может перехватить и изменить этот начальный ответ, не дав пользователю когда-либо перейти на защищенный протокол. В таком случае отсутствие крошечного значка замка в UI браузера очень легко пропустить.

Эта проблема, а также несколько периферийных вопросов, связанных со смешанным содержимым и областью действия cookies, подтолкнули Джеффа Ходжеса и нескольких других исследователей к подготовке предложения HTTP Strict Transport Security (HSTS, или сокращенно STS).^[19] Их подход, сейчас поддерживаемый в WebKit и Firefox, позволяет любому сайту в интернете инструктировать браузер, что все будущие запросы к конкретному имени хоста или домену должны всегда использовать HTTPS и что любой HTTP-трафик должен автоматически повышаться и отправляться только через HTTPS.

Логика дизайна HSTS в том, что первое взаимодействие пользователя с конкретным доменом вряд ли произойдет через соединение, которое активно подделывают, но со временем, по мере того как пользователь перемещается по открытым беспроводным сетям, шансы встретить злоумышленника быстро растут. Поэтому HSTS - несовершенная защита, но на практике обычно достаточно хорошая.

Заголовок opt-in HSTS может появиться в HTTPS-ответах примерно так:

```
Strict-Transport-Security: max-age=3000000; includeSubDomains
```

Примечание. Чтобы HSTS давал разумную защиту, `max-age`, то есть число секунд, в течение которых STS-запись может храниться в браузере, должен быть установлен в значение существенно большее, чем обычное худшее время между посещениями сайта. Поскольку нет простого способа отключить или переопределить HSTS, когда что-то идет не так с HTTPS-сайтом, владельцы сайтов будут склонны выбирать значение достаточно малое, чтобы минимизировать сбои, когда они что-то испортят и им придется откатиться. Неясно, приведет ли этот конфликт интересов к оптимальным решениям веб-программистов.

Отрицательные последствия такого дизайна для безопасности довольно непримечательны: есть немного повышенный риск DoS-атак, потому что злоумышленник мог бы внедрить этот заголовок ответа в домен, который не полностью включен для HTTPS. Также есть возможность использовать уникальную комбинацию настроек HSTS для нескольких ложных имен хостов, чтобы пометить конкретный экземпляр браузера, предлагая еще одну альтернативу отслеживанию пользователя на основе cookies. Ни одно из этих опасений, однако, не особенно выражено.

К сожалению, как и с другими фреймворками, добавляющими ограничения и обсуждавшимися в этом разделе книги, механизм звучит отлично в принципе, но трудно полностью учесть, как он может взаимодействовать с другим унаследованным кодом. В частности, если не используется флаг `includeSubDomains`, HSTS предлагает неожиданно мало защиты для HTTP cookies: cookies, не помеченные как `secure`, все еще могут быть перехвачены простым изобретением несуществующего поддомена и перехватом HTTP-запроса к этому адресату.^[20] Даже `secure` cookies можно затереть похожим образом, просто не прочитав обратно.

В том же духе принуждение HSTS для запросов, исходящих от содержимого на основе плагинов, вряд ли будет хорошо работать.

Режимы приватного просмотра

Приватный просмотр, в разговорной речи известный как "porn mode", - нестандартизованная функция, доступная в большинстве актуальных браузеров. Она предназначена для создания непостоянной песочницы просмотра, изолированной от основной сессии браузера, которая полностью отбрасывается, как только закрыто последнее окно приватного просмотра. В некотором смысле этот механизм можно считать формой изоляции содержимого, добавленной поверх существующих парадигм безопасности браузера, поэтому кажется уместным кратко упомянуть его сейчас.

За исключением Chrome, большинство поставщиков браузеров неточно объясняют гарантии безопасности, связанные с приватным просмотром. К сожалению, интуитивное понимание термина довольно сильно отличается от того, что браузеры фактически могут предоставить.

Пожалуй, самое прямолинейное толкование функции состоит в том, что сессия приватного просмотра должна быть идеально анонимной и что никакие данные об активности пользователя не сохраняются в системе. Эти два предположения уже частично подрываются ограничениями, накладываемыми сетевыми стеками и практиками управления памятью современных операционных систем. Но даже внутри самого браузера цель разумной анонимности почти невозможно достичь. Почти каждый механизм браузера с состоянием, от разрешений геолокации или pop-up до Strict Transport Security, автозаполнения форм и persistent data storage на основе плагинов, должен быть изменен, чтобы правильно учитывать различие между двумя режимами просмотра, и для каждого поставщика достижение этой цели - тяжелая битва. Возможно, еще более раздражающе то, что анонимность также подрывается способностью скриптов уникально снимать отпечаток любой данной системы простым изучением ее характеристик, таких как набор установленных плагинов, шрифтов, разрешений экрана, размеров окна, дрейфа часов или даже поведения некриптографически безопасных PRNG.^[21]

В итоге, вопреки внешнему впечатлению, режим приватного просмотра подходит только для предотвращения случайного раскрытия данных другим нетехническим пользователям той же машины, и даже этой цели иногда трудно достичь.

Другие разработки

Функции безопасности, обсуждавшиеся ранее в этой главе, стремятся сдвинуть границы между веб-приложениями и изменить то, как сайты взаимодействуют друг с другом. Другая группа предлагаемых механизмов ускользает от этой простой классификации, но достаточно важна или зрела, чтобы кратко упомянуть ее здесь. Сейчас мы рассмотрим некоторые из них.

Браузерные HTML-очистители

XSS-уязвимости - безусловно самая распространенная проблема безопасности, встречающаяся в современных веб-приложениях. Поэтому должно удивлять, что так мало предложенных фреймворков безопасности пытаются решать проблему комплексно. Да, CSP - сильный претендент, но она требует радикального изменения того, как пишутся веб-приложения, и ее нельзя разворачивать особенно постепенно или выборочно. Sandboxed frames, с другой стороны, вероятно, слишком ресурсоемки и неудобны для самой распространенной задачи отображения сотен отдельных коротких фрагментов данных, предоставленных пользователем.

Возможно, лучшим решением многих XSS-бед был бы метод, позволяющий веб-фреймворкам предоставить браузеру разобранное, однозначное, бинарное DOM-дерево. Такое решение устранило бы многие проблемы, связанные с экранированием шаблонов и HTML-очисткой. Более приземленной альтернативой могло бы быть снабжение веб-разработчиков надежным инструментом для пометки границ строки, предоставленной злоумышленником, и ограничения поведения или внешнего вида встроенной полезной нагрузки без необходимости ее экранировать или очищать. Можно представить синтаксис вроде:

```
<sandbox token="random_value12345" settings="allow_static_html">
  ...any unsanitized text or HTML...
</sandbox token="random_value12345">
```

Если бы такой инструмент использовался, злоумышленник не смог бы выбраться из такой песочницы и снять ограничение на scripting, не угадав корректное значение случайно сгенерированной token boundary.

К сожалению, такое предложение вряд ли станет частью HTML5 или появится в каком-либо браузере, потому что эта сериализация фундаментально несовместима с XML, а пересмотр самого XML ради малоизвестного сценария использования в HTML - трудный подвиг. Удручающе, но XML уже предлагает похожий метод инкапсуляции произвольных данных внутри блока `<![CDATA[...]]>`, однако без защиты на основе токена из такой песочницы легко выбраться при эксплуатации XSS.

С другой стороны, значительно проще ограничивать привилегии любого HTML, сгенерированного скриптами на стороне клиента. Начиная с Internet Explorer 8 Microsoft предлагает простой и несколько негибкий API `toStaticHTML(...)`,^[22] который обещает удалить JavaScript из любого полностью квалифицированного фрагмента HTML, переданного ему как параметр. Вывод этого метода спроектирован так, чтобы его было безопасно присвоить свойству `innerHTML` где-нибудь в существующем DOM.^[23]

Предложение Microsoft неплохо, но оно обходит самую распространенную и проблемную задачу безопасного отображения документов, предоставленных сервером. И у его API есть небольшая, но совершенно ненужная слабость: оно делает неожиданно опасным обрезание или конкатенацию очищенного вывода `toStaticHTML(...)` после вызова, но до присваивания `innerHTML`, практику, к которой многие веб-разработчики, вероятно, прибегнут. Более разумным подходом было бы разрешить очистку содержимого только при присваивании `innerHTML`. Фактически инженеры WebKit кратко обсуждали предложение такого API, называвшегося попеременно `innerStaticHTML` или `safeInnerHTML`, но усилие, похоже, давно сошло на нет.

XSS Filtering

Снижать частоту межсайтовых уязвимостей трудно, как и ограничивать их влияние. Поэтому некоторые исследователи пришли к выводу, что обнаружение и остановка эксплуатации таких изъянов может быть лучшим выбором. И вот около 2008 года Дэвид Росс из Microsoft объявил о включении логики обнаружения XSS в будущий выпуск Internet Explorer 8,^[24] несколько месяцев спустя Адам Барт реализовал похожую функцию в WebKit. Реализации сравнивают части текущего URL с любыми строками, появляющимися на полученной странице или переданными API вроде `document.write(...)` и `innerHTML`. Если сравнение показывает, что часть JavaScript, присутствующая на странице, могла происходить из неправильно экранированного параметра URL, соответствующая часть страницы может быть заменена безвредной строкой.

К сожалению, эта вроде бы элегантная идея известна серьезными проблемами. Помимо случайных ложных срабатываний, пользователи Internet Explorer 8 неожиданно столкнутся с проблемами при посещении `http://www.google.com/search?q=<script>`, фильтр также можно сработать во вред,

добавив легитимную часть страницы как нефункциональный параметр в URL. В одном крайнем и уже исправленном случае это поведение использовалось для создания XSS-векторов там, где их раньше не было, просто обманом заставляя браузер бессистемно перестраивать разметку.^[25]

Но фундаментальнее то, что для любого сложного веб-приложения рискованно выборочно отключать выбранные злоумышленником блоки скриптов, даже если структура страницы иначе корректно сохранена, и такая настройка может легко привести клиентский код в несогласованное или опасное состояние. Например, рассмотрим онлайн-редактор документов, который реализует каждое из следующих действий в отдельном блоке `<script>`:

1. Инициализирует внутреннее состояние редактора и создает UI с пустым стартовым документом.
2. Загружает текущую версию документа, запрошенную пользователем в параметре URL, с проверкой ошибок для перехвата любых потенциальных сетевых проблем.
3. Если ошибок не обнаружено, входит в интерактивный режим редактирования и автоматически сохраняет текущее состояние документа каждые 30 секунд под ID, полученным из URL.

В этом не совсем неразумном дизайне способность удалить второй шаг может быть катастрофической, потому что следующий шаг может перезаписать существующий, хранимый на сервере документ пустой копией. Упс.

Этой проблемы можно было избежать, используя намного более простой дизайн, при котором любые предполагаемые XSS-атаки приводили бы к тому, что браузер просто отказывался рендерить документ. Увы, сравнительно высокая частота случайных ложных срабатываний помешала авторам пойти этим путем. Только после некоторого обсуждения Microsoft решила предложить "strict" blocking mode на opt-in основе, включаемый таким заголовком ответа:

```
XSS-Protection: 1; mode=block
```

Примечание. Помимо риска ложных срабатываний, XSS filters также склонны к ложным пропускам, ситуации, которую, вероятно, нельзя сильно улучшить. По дизайну эти фильтры никогда не смогут обнаружить, пожалуй, более опасные хранимые XSS-уязвимости, где неправильно экранированные данные приходят из источника, отличного от пройденной ссылки. Но даже помимо этого множество часто неявных схем экранирования входных данных и растущее использование `location.hash` или `pushState` (глава 17) как метода хранения состояния приложения затрудняют формулирование точной связи между тем, что браузер видит в адресной строке, и тем, что приложение делает с полученным URL.

Памятка по инженерии безопасности

Подходите к экспериментальным функциям безопасности браузера осторожно, особенно когда имеете дело с механизмами, создающими более тонкие границы безопасности. Убедитесь, что любое приложение, использующее эти механизмы, будет безопасно деградировать в несовместимом браузере.

- **Cross-domain XMLHttpRequest (CORS).** Довольно безопасен, но им легко злоупотребить. Избегайте непростых запросов и не разрешайте произвольные заголовки или методы. Если вы контролируете серверный фреймворк приложения, рассмотрите автоматическое удаление заголовков `Cookie` во входящих CORS-запросах с не внесенными в белый список значениями `Origin`, чтобы минимизировать риск случайного раскрытия пользовательских данных. Чтобы снизить частоту ошибок смешанного содержимого, рассмотрите отклонение HTTPS-значений `Origin` в любых запросах, полученных через обычный HTTP. Будьте осторожны с

Access-Control-Allow-Origin: *, и если вам нужно его использовать, убедитесь, что он возвращается только для того места, которым вы намерены делиться.

- **XDomainRequest.** Его безопасно использовать. Как и с XMLHttpRequest, ограничение доступа к HTTP API из HTTPS origins может быть хорошим способом искоренить ошибки смешанного содержимого.
- **Content Security Policy.** Ее безопасно использовать как эшелонированную защиту. Пересмотрите предостережения, связанные с взаимодействиями между script-src, object-src и т. д., а также опасности разрешения origins data:. Не допускайте смешанного содержимого случайно: всегда указывайте протоколы в наборах правил и убедитесь, что они соответствуют протоколу, через который отдана запрашивающая страница.
- **Sandboxed frames.** Их безопасно использовать как способ встраивать гаджеты из других origins, но механизм драматически откажет в несовместимых браузерах. Не следует помещать в sandbox документы того же origin.
- **Strict Transport Security.** Ее безопасно использовать как эшелонированную защиту. Обязательно помечайте все релевантные cookies как secure и будьте готовы к возможности cookie injection через подмененные, не STS-локации в вашем домене. Используйте includeSubDomains там, где это осуществимо, чтобы снизить этот риск.
- **toStaticHTML(...).** Его безопасно использовать там, где он доступен, но трудно заменить на клиентской стороне в несовместимых браузерах. Из-за дизайна фильтра уязвимости обхода имеют выше среднего шанс повторяться в API.
- **Приватный просмотр.** Не полагайтесь на этот механизм в целях безопасности.
- **XSS filtering.** Не полагайтесь на этот механизм в целях безопасности. Всегда явно указывайте XSS-Protection: 1; mode=block или XSS-Protection: 0 в HTTP-ответах. Значение по умолчанию довольно небезопасно.

Сноски

[1] [\[назад\]](#) Черные списки вредоносных URL, функция, поддерживаемая и обычно включенная во всех современных браузерах, - яркий пример этой тенденции. Черный список - легкая, грубая замена антивирусу, который, в свою очередь, является плохой заменой актуальному и хорошо спроектированному ПО. Антивредоносные функции не делают отдельные атаки сложнее; они просто предназначены останавливать крупномасштабное распространение несложного malware, исходя из предположения, что большинство пользователей недостаточно интересны, чтобы быть специально выбранными в цель или атакованными чем-то умным.

[2] [\[назад\]](#) Cross-Origin Resource Sharing, предложение W3C.

[3] [\[назад\]](#) Это предположение не полностью верно. Например, до введения этой схемы злоумышленники не могли инициировать междоменный запрос, полностью неотличимый от отправки формы загрузки файла, но при CORS такая подделка возможна.

[4] [\[назад\]](#) То же утверждение можно сделать об использовании HTTP cookies в любой другой обстановке, и оно кажется столь же бесплодным. Верно, что окружающие учетные данные вызывают проблемы чаще, чем некоторые другие формы явной аутентификации, но они также намного удобнее в использовании и просто не исчезнут.

[5] [\[назад\]](#) Sunava Dutta, комментарий о cross-domain access в Internet Explorer 8.

[6] [\[назад\]](#) XDomainRequest, API Microsoft.

[7] [\[назад\]](#) Причина этой проверки, даже если ответ не аутентифицирован, - не дать использовать браузер как прокси, например для сканирования внутренних сетей или рассылки спама.

[8] [\[назад\]](#) Uniform Messaging Policy и UniformRequest.

[9] [\[назад\]](#) Adam Barth, Collin Jackson и John C. Mitchell, работа 2008 года о заголовке Origin.

[10] [\[назад\]](#) Обсуждение альтернативных решений XSRF и XSSI.

[11] [\[назад\]](#) Более позднее имя предложения - From-Origin.

[12] [\[назад\]](#) Черновик IETF, задающий синтаксис заголовка Origin.

[13] [\[назад\]](#) Brandon Sterne, первоначальное предложение CSP в Mozilla, 2008 г.

[14] [\[назад\]](#) Черновик CSP в W3C.

[15] [\[назад\]](#) CSP предлагает несколько способов выстрелить себе в ногу. Во-первых, можно заново включить выполнение скриптов настройками вроде inline-script (название Mozilla, измененное на disable-xss-protection в черновике W3C) или eval-script. Менее очевидно, что можно также ошибиться, разрешив data: или * как допустимый origin либо разрешив HTTP origin на HTTPS-сайте.

[16] [\[назад\]](#) Помните advanced selectors в CSS3? Хитро используя их во внедренных таблицах стилей, часть информации о строках, появляющихся на странице, можно удобно передать стороннему серверу без JavaScript.

[17] [\[назад\]](#) Как побочное замечание, эта функция полезна не только для краткосрочных экспериментов, но и для постоянного обнаружения не критических проблем. Например, владелец сайта может использовать ее для выявления mixed-content issues, создав report-only policy для HTTPS-страниц, которая будет нарушаться любыми HTTP-скриптами.

[18] [\[назад\]](#) Sandboxed frames в HTML5.

[19] [\[назад\]](#) HTTP Strict Transport Security, предложение Jeff Hodges и других исследователей.

[20] [\[назад\]](#) Вспомните из главы 9, что host-scoped cookies довольно трудно создать в некоторых браузерах и совершенно невозможно иметь в Internet Explorer.

[21] [назад](#) Снятие отпечатка системы по набору плагинов, шрифтов, разрешениям экрана, размерам окна, дрейфу часов и PRNG.

[22] [назад](#) API `toStaticHTML(...)` в Internet Explorer 8.

[23] [назад](#) Забавно, что HTML parser в Internet Explorer, по-видимому, настолько туповат, что даже авторам `toStaticHTML(...)` было трудно за ним следовать. С момента появления API страдал от заметного числа уязвимостей обхода, чаще всего связанных с обработкой CSS-данных.

[24] [назад](#) David Ross, объявление XSS-фильтра в Internet Explorer 8.

[25] [назад](#) Пример обхода, где фильтр создавал XSS-векторы за счет перестройки разметки.

Глава 17. Другие браузерные механизмы, заслуживающие внимания

Чтобы завершить третью часть книги, мы кратко перечислим некоторые недавно реализованные или просто планируемые API, которые, хотя и не проектировались ради безопасности, могут существенно изменить ландшафт безопасности в ближайшие годы. Например, одни меняют типы данных, к которым веб-приложения получают доступ, а другие изменяют способ, которым браузер общается с внешним миром.

Следующий список неизбежно неполон: новые, вполне правдоподобные проекты появляются каждую неделю, а старые подходы могут быть выброшены в любой момент, часто задолго до появления в реальном браузере. Тем не менее эта глава должна послужить интересным снимком того, что может принести будущее.

Предложения уровня URL и протоколов

Эти возможности стремятся изменить процессы, окружающие поведение ссылок, адресной строки и обмен данных по сети.

Регистрация протоколов

Веб-приложения часто берут на себя обработку URL-схем, прежде зарезервированных за "настоящими" настольными программами. Один из главных примеров - протокол `mailto:`, изначально предназначенный для запуска отдельного почтового приложения, но сегодня часто более разумно направляемый в интерфейсы веб-почты. Для этой цели Mozilla предложила, а WebKit принял простой API `navigator.registerProtocolHandler(...)`.^[1] Когда этот API вызывается, пользователю показывается простой диалог безопасности, и если действие одобрено, обработчик на основе URL связывается с конкретной схемой. На сегодняшний день связанные с ним диалоги уязвимы к условиям гонки, описанным в главе 14, и, как видно на рисунке 17-1, им, похоже, недостает и других свойств.



Рисунок 17-1. Крайне сбивающий с толку диалог в Firefox. Диалог, показанный в верхней области окна браузера, был создан браузером в ответ на вызов API `registerProtocolHandler(...)`, где имя протокола было задано как "doing really awesome stuff", а имя приложения - как "Firefox (mozilla.org)". Этот конкретный пример безвреден, но более зловещее злоупотребление вполне достижимо.

Манипуляция адресной строкой

Недавно введенный API HTML5 `history.pushState(...)`,^[2] поддерживаемый Firefox, WebKit и Opera, позволяет текущему отображаемому документу менять содержимое адресной строки на любой другой URL того же origin без фактического запуска перехода страницы, обычно связанного с этим шагом. API предлагает более качественную альтернативу широко распространенному злоупотреблению `location.hash` для хранения состояния приложения. Интересно, что, несмотря на свою простоту, он уже привел к заметному числу любопытных ошибок безопасности. Например, некоторые реализации короткое время позволяли не только документу верхнего уровня, но и любым сомнительным сторонним фреймам менять URL верхнего уровня, показанный в адресной строке, а также разрешали таким origins, как `about:blank`, помещать в поле URL почти ничем не ограниченную бессмыслицу.

Бинарный HTTP

SPDY^[3] ("Speedy") - простая, зашифрованная замена HTTP, которую можно подставить вместо него без изменения общей архитектуры. Она сохраняет ключевые принципы дизайна протокола, включая расположение и функцию большинства заголовков. В то же время она минимизирует накладные расходы, связанные с доставкой параллельных запросов или с разбором текстовых запросов и данных ответов. Сейчас протокол поддерживается только в Chrome и, если не считать отдельных сервисов Google, в Web встречается нечасто. Впрочем, скоро он может появиться и в Firefox.

Сетевое взаимодействие без HTTP

WebSocket^[4] - все еще развивающийся API, предназначенный для согласования в основном ничем не ограниченных двунаправленных TCP-потоков в случаях, когда транзакционная природа HTTP мешает, например в приложении чата с малой задержкой. Протокол запускается с помощью рукопожатия "вызов-ответ" с ключом; оно выглядит отчасти как HTTP и, что довольно примечательно, не может быть подделано простой эксплуатацией изъяна расщепления заголовков на целевом сайте. После успешного рукопожатия сырые данные могут двунаправленно обмениваться внутри получившегося долгоживущего TCP-соединения, причем каждое сообщение упаковывается в простой протокольный фрейм. Механизм поддерживается в WebKit и, вероятно, скоро появится в Firefox.

P2P-сетевое взаимодействие

WebRTC^[5] - предлагаемый набор API и сетевых протоколов, рассчитанных на облегчение обнаружения других браузеров и общения с ними без необходимости в централизованной серверной инфраструктуре. Основной сценарий использования такого протокола - реализация IP-телефонии и видеоконференций внутри веб-приложений. Стабильной поддержки в браузерах пока нет.

Офлайн-приложения

Cache manifests^[6] - сравнительно простой способ для веб-сервера указать браузеру, что копии некоторых документов должны храниться неопределенно долго и повторно использоваться всякий раз, когда у клиента, по всей видимости, нет сетевого подключения. В сочетании с клиентскими механизмами хранения, такими как localStorage (глава 9), это позволяет использовать некоторые самодостаточные JavaScript-приложения в офлайн-режиме. Офлайн-работа поддерживается в Firefox, браузерах WebKit и Opera. Как и в случае с localStorage, постоянная природа этого механизма может усугубить долгосрочные последствия посещения недоверенной сети.

Улучшенные cookies

Cake^[7] - ныне истекшее предложение, подготовленное Адамом Бартом, которое стремилось создать более легковесную и безопасную альтернативу HTTP cookies: один привязанный к origin, сгенерированный браузером поспе для каждого целевого сайта. Более современное, но незавершенное предложение, похоже, заигрывает с обычными, но основанными на origin cookies как альтернативой. Ни один из подходов сегодня недоступен ни в одном браузере.

Возможности уровня содержимого

Предложения, описанные в этом разделе, призваны дать возможность создавать новые классы веб-приложений поверх HTML и JavaScript.

Клиентские базы данных

За годы было предложено несколько API для создания локально хранимых баз данных и управления ими, включая печально известный API WebSQL,^[8] который принес бы знаменитый опасностями синтаксис SQL в клиентский JavaScript. Предложение WebSQL было отброшено в пользу более разумного дизайна IndexedDB,^[9] предлагающего чистый API без сериализованных запросов и модель безопасности, сравнимую с localStorage, но только после того, как поддержка WebSQL уже попала в пару браузеров. Тем временем новый API появился в Chrome и ожидается в Firefox.

Фоновые процессы

API Worker,^[10] доступный в Firefox, WebKit и Opera, позволяет создавать фоновые JavaScript-процессы для выполнения вычислительно дорогих задач без опасения заблокировать пользовательский интерфейс браузера. Каждый worker работает в изолированной среде, лишенной обычного DOM окна или документа, и может асинхронно общаться со своим создателем через API `postMessage(...)`. Dedicated workers напрямую доступны только своему создателю, тогда как shared workers могут быть "прикреплены" к нескольким разным сайтам в любой момент времени. Persistent workers, которые работали бы независимо от какого-либо устойчивого спроса на их услуги, были предложены на раннем этапе, но затем отброшены. Концепция worker-потоков поднимает некоторые периферийные опасения DoS, но в остальном не создает очевидных рисков безопасности.

Определение геолокации

API `navigator.geolocation.getCurrentPosition(...)`^[11] позволяет любому веб-сайту запросить сведения о физическом местоположении клиентского устройства при условии пользовательского согласия, которое в значительной степени можно перехватить. Вычисленные данные геолокации могут быть получены из GPS-информации на системе с подходящим аппаратным модулем либо найдены по именам ближайших беспроводных точек доступа, сотовых вышек и так далее. API поддерживается во всех основных браузерах, кроме Internet Explorer.

Ориентация устройства

Неограниченный событийный API `DeviceOrientation`^[12] позволяет веб-сайтам считывать ориентацию устройства на основе данных акселерометра. Этот API, вероятно нацеленный на мобильные игры, доступен в Firefox, WebKit и Opera на системах, оснащенных подходящим оборудованием. Два исследователя из Калифорнийского университета в Дэвисе недавно продемонстрировали роковой изъян: на смартфонах микроскопические движения устройства можно использовать, чтобы надежно реконструировать ввод с экранной клавиатуры, включая пароли, введенные на несвязанных веб-сайтах.^[13]

Предварительный рендеринг страниц

Эта экспериментальная возможность Chrome позволяет заранее загружать страницы в ожидании того, что пользователь перейдет по конкретной ссылке, и дает возможность предварительно отрендерить весь HTML-документ в скрытой вкладке^[14] и на мгновение показать его, когда предсказанное действие навигации действительно происходит. У механизма есть интересные последствия для безопасности браузера, если предварительно отрендеренная страница окажется вредоносной. Реализация в Chrome аккуратно откладывает любые разрушительные действия до момента раскрытия вкладки, но во всех кодовых базах браузеров ошибки будет очень легко допустить.

Измерение времени навигации

Несколько дополняющих друг друга API, сейчас доступных только в Chrome, позволяют очень точно измерять из клиентского JavaScript некоторые типы навигации, включая загрузки междоменных страниц.^[15] Этот интерфейс предназначен для того, чтобы владельцы сайтов могли выявлять очевидные узкие места производительности в том виде, как их переживает типичный посетитель. API позволяет собирать некоторую информацию, связанную с приватностью, профилируя время, необходимое для загрузки определенного стороннего содержимого; но поскольку та же атака возможна множеством других способов, например с обработчиками `onload` на подресурсах, это, вероятно, не так уж важно.

Интерфейсы ввода-вывода

Перечисленные ниже возможности предлагают новые способности ввода и вывода для веб-скриптов.

Уведомления UI

API Notification и `window.notifications`^[16] позволяют создавать текстовые или HTML-основанные всплывающие окна, всегда находящиеся поверх остальных, в углу экрана, давая избранным веб-приложениям возможность мягко уведомлять пользователей о важных событиях, например о новом почтовом сообщении. Согласие пользователя на получение уведомлений требуется отдельно для каждого сайта, что ограничивает риск злоупотреблений. Тем не менее нужно внимательно следить за тем, чтобы правильно сообщать `origin` крошечного окна уведомления и любых диалогов или подсказок, которые оно затем создает, - аспект, на доводку которого ушло некоторое время. Сегодня API доступен только в WebKit.

Полноэкранный режим

Было распространено несколько предложений, позволяющих JavaScript развернуть текущее окно браузера на весь экран и скрыть все браузерное оформление. Такая функциональность необходима для задач вроде просмотра презентаций или фильмов, но с точки зрения безопасности она очевидно очень опасна: получив контроль над всем экраном, любая вредоносная страница может нарисовать поддельное окно браузера с поддельной адресной строкой. Пока, похоже, нет конкретной реализации, доступной для изучения. Также обсуждается раннее предложение по блокировке курсора мыши.

Захват медиа

Предлагаемый набор API `navigator.device.capture`^[17] был выдвинут для предоставления веб-сайтам доступа к данным веб-камеры и микрофона. Вокруг этого механизма возникают очевидные опасения безопасности и приватности, особенно относительно устойчивости любых связанных с ним диалогов безопасности к атакам на условиях гонки. Сегодня у API нет стабильной поддержки в браузерах.

Сноски

- [1] [\[назад\]](#) Предложение Mozilla для `navigator.registerProtocolHandler(...)`.
- [2] [\[назад\]](#) HTML5 History API и `history.pushState(...)`.
- [3] [\[назад\]](#) Экспериментальный протокол SPDY от Google.
- [4] [\[назад\]](#) Спецификация WebSocket для двунаправленных соединений.
- [5] [\[назад\]](#) Раннее предложение WebRTC для межбраузерной связи.
- [6] [\[назад\]](#) Механизм HTML5 cache manifest для офлайн-приложений.
- [7] [\[назад\]](#) Предложение Cake Адама Барта как альтернатива HTTP cookies.
- [8] [\[назад\]](#) WebSQL Database API.
- [9] [\[назад\]](#) Indexed Database API, или IndexedDB.
- [10] [\[назад\]](#) Web Workers API.
- [11] [\[назад\]](#) Geolocation API.
- [12] [\[назад\]](#) DeviceOrientation Event Specification.
- [13] [\[назад\]](#) Исследование о восстановлении ввода с экранной клавиатуры по движениям смартфона.
- [14] [\[назад\]](#) Экспериментальный предварительный рендеринг страниц в Chrome.
- [15] [\[назад\]](#) Navigation Timing API.
- [16] [\[назад\]](#) API Web Notifications.
- [17] [\[назад\]](#) Раннее предложение Media Capture API.

Часть IV. Распространенные веб-уязвимости

До этого момента мы уделяли мало внимания таксономии распространенных веб-уязвимостей. Понимание глубинной механики веб-приложений куда важнее, чем запоминание нескольких тысяч случайных и часто ненужных терминов; номенклатуре вроде `improper restriction of operations within the bounds of a memory buffer` (Common Weakness Enumeration) или `insecure direct object references` (Open Web Application Security Project) нет места в разумном разговоре - и вполне справедливо.

Тем не менее отрасль выработала несколько достаточно точных выражений, которыми исследователи безопасности пользуются каждый день. Теперь, когда внутреннее устройство браузера подробно обсуждено, полезно подытожить и выделить терминологию, которую, скорее всего, увидит средний читатель.

Глава 18. Уязвимости, специфичные для веб-приложений

Термины, описанные в этом разделе, уникальны для технологий, используемых в Web, и часто не имеют непосредственных аналогов в мире "традиционной" безопасности приложений.

Cross-site request forgery (XSRF, CSRF)

Уязвимость, вызванная неспособностью проверить, что конкретный изменяющий состояние HTTP-запрос, полученный серверной частью веб-приложения, был инициирован из ожидаемого клиентского origin. Этот изъян позволяет любому стороннему веб-сайту, загруженному в браузере, выполнять действия от имени жертвы.

См. главу 4 для более подробного обсуждения XSRF.

Cross-site script inclusion (XSSI)

Изъян, вызванный неспособностью защитить чувствительные JSON-подобные ответы от загрузки на сторонних сайтах через `<script src=...>`. Информация в ответе, специфичная для пользователя, может утечь злоумышленникам.

См. главу 6 для обзора проблемы и возможных исправлений.

Cross-site scripting (XSS)

Недостаточная проверка входных данных или экранирование вывода может позволить злоумышленнику разместить собственную HTML-разметку или скрипты на уязвимом сайте. Внедренные скрипты получают доступ ко всему целевому веб-приложению и во многих случаях к HTTP cookies, сохраненным клиентом.

Квалификатор *reflected* относится к случаям, когда внедренная строка является просто результатом неправильного отражения части запроса, тогда как *stored* или *persistent* относится к ситуации, где полезная нагрузка проходит более сложный путь. DOM-based может использоваться, чтобы обозначить, что уязвимость запускается поведением клиентской части веб-приложения, то есть JavaScript.

См. главу 4 для распространенных XSS-векторов в HTML-документах.

См. главу 6 для обзора рисков DOM-based XSS.

См. главу 13 для XSS-векторов, связанных с распознаванием содержимого.

См. главу 9 для обсуждения обычной модели безопасности JS-кода.

Header injection (response splitting)

Недостаточное экранирование переводов строк или эквивалентных символов в HTTP-ответах, создаваемых серверной частью веб-приложения. Такое поведение обычно приводит к XSS, отравлению кэша браузера или прокси и другим последствиям.

См. главу 3 для подробного обсуждения этого изъяна.

Mixed content

Общее название для загрузки не-HTTPS подресурсов на HTTPS-страницах. В случае скриптов и апплетов такое поведение делает приложение тривиально уязвимым для активных злоумышленников, особенно в открытых беспроводных сетях, например в кафе, аэропортах и так далее, и отменяет почти все преимущества HTTPS. Последствия ошибок смешанного содержимого со стилями, шрифтами, изображениями или фреймами обычно также довольно серьезны, но более ограничены.

См. главы 4 и 8 для мер предосторожности на HTTPS-сайтах, специфичных для содержимого.

См. главу 11 для обзора правил обработки смешанного содержимого.

Open redirection

Термин, используемый для приложений, которые выполняют HTTP- или скриптовые запросы к URL, предоставленным пользователем, никак существенно не ограничивая возможные адресаты. Open redirection нежелателен и в некоторых сценариях может быть эксплуатируемым, но сам по себе обычно не особенно опасен.

См. главу 10 для случаев, где неограниченное перенаправление может привести к XSS.

Referer leakage

Случайное раскрытие чувствительного URL через встраивание внешнего подресурса или предоставление внешней ссылки. Любые данные, важные для безопасности или приватности и закодированные в URL родительского документа, утекут в заголовке Referer, за исключением идентификатора фрагмента.

См. главу 3 для обзора логики Referer.

Проблемы, о которых нужно помнить при проектировании веб-приложений

Проблемы, описанные в этом разделе, - неизбежное обстоятельство работы в интернете, и их нужно должным образом учитывать при проектировании или реализации новых веб-приложений.

Cache poisoning

Возможность долгосрочного загрязнения кэша браузера или любых промежуточных прокси сфабрикованной, вредоносной версией целевого веб-приложения. Зашифрованные веб-приложения могут стать целью из-за уязвимостей response splitting. Для незашифрованного трафика активные сетевые злоумышленники также могут модифицировать ответы, получаемые запрашивающим.

См. главу 3 для обзора поведения HTTP-кэширования.

Clickjacking

Возможность поместить во фрейм, украсить или скрыть часть другого веб-приложения так, что жертва при взаимодействии с сайтом злоумышленника не осознает, что отдельные щелчки или нажатия клавиш доставляются другому сайту, в результате чего нежелательные действия выполняются от имени пользователя.

См. главу 11 для обсуждения clickjacking и связанных проблем UI.

Content and character set sniffing

Описывает возможность того, что браузер проигнорирует любые авторитетные сведения о типе содержимого или наборе символов, предоставленные сервером, и неправильно интерпретирует возвращенный документ.

См. главу 13 для обсуждения логики распознавания содержимого.

См. главы 4 и 8 для сценариев, где данные Content-Type игнорируются.

Cookie forcing (or cookie injection)

Возможность вслепую внедрять HTTP cookies в контекст иначе непроницаемого веб-приложения из-за особенностей того, как этот механизм спроектирован и реализован в современных браузерах. Cookie injection особенно важен для HTTPS-приложений. Cookie stuffing - менее распространенный термин, относящийся конкретно к злонамеренному удалению cookies, принадлежащих другому приложению, через переполнение cookie jar.

См. главу 9 для дополнительных сведений об области действия cookies.

См. главу 3 для общего обсуждения работы HTTP cookies.

Denial-of-service (DoS) attacks

Широкий термин, обозначающий любые возможности злоумышленника вывести из строя браузер или сервер либо иным образом заметно затруднить использование конкретного целевого приложения.

См. главу 14 для обзора DoS-соображений в JavaScript.

Framebusting

Возможность страницы во фрейме перевести документ верхнего уровня на новый URL без необходимости удовлетворять проверкам same-origin. Это поведение может использоваться для фишинговых атак или просто для мелкого вредительства.

См. главу 11 для этой и других причуд навигации фреймов.

HTTP downgrade

Способность активных злоумышленников помешать пользователю достичь HTTPS-версии конкретного сайта или понизить существующую HTTPS-сессию до HTTP.

См. главу 3 для обзора HTTPS.

См. главу 16 для Strict Transport Security, предлагаемого решения этой проблемы.

Network fenceposts

Перспектива того, что веб-сайты в интернете используют браузер для взаимодействия с адресатами, напрямую недоступными злоумышленнику, например с системами во внутренней сети жертвы. Такие атаки могут выполняться вслепую, либо, с помощью атак вроде DNS rebinding, злоумышленник может видеть ответы на все запросы.

См. главу 12 для обзора не-SOP границ в браузере.

См. главу 15 для модели зон Internet Explorer, потенциального подхода к этому риску.

Примечание. Остерегайтесь ошибок без модных названий! Не у всех уязвимостей есть броские имена. Веб-разработчикам следует опасаться множества других проблем реализации и дизайна, которые выходят за рамки этой книги, но все равно могут больно ударить. Примеры включают слабые генераторы псевдослучайных чисел, особенно для управления сессиями; недостаточные проверки аутентификации и авторизации, в частности чрезмерное доверие данным, приходящим из браузера; неверное использование криптографии, где изобретение собственных алгоритмов обычно плохая идея; и так далее. Для поразительно подробного обсуждения этих и многих других шаблонов отказа см. *The Art of Software Security Assessment* Дауда, Макдональда и Шуха (Addison-Wesley, 2006).

Распространенные проблемы, уникальные для серверного кода

Следующие проблемы часто встречаются в размещенной на сервере части любого веб-приложения и, поскольку привязаны к конкретным языкам программирования или программным компонентам, вряд ли возникнут на клиентской стороне.

Buffer overflow

Состояние, при котором программа позволяет сохранить в конкретной области памяти больше информации, чем там есть места для входящих данных, что приводит к неожиданной перезаписи других жизненно важных структур данных. Buffer overflows происходят главным образом в низкоуровневых языках программирования, таких как C или C++, и в этих языках их часто можно использовать для выполнения кода, предоставленного злоумышленником.

Command injection (SQL, shell, PHP и так далее)

Проблема, при которой из-за недостаточной фильтрации входных данных или экранирования вывода строки, контролируемые злоумышленником, могут быть непреднамеренно обработаны как операторы в интерпретируемом языке, используемом приложением. В отдаленном смысле это похоже на XSS. Последствия зависят от возможностей языка, но в большинстве случаев итогом становится выполнение кода.

Directory traversal

Проблема, при которой из-за недостаточной фильтрации входных данных, чаще всего из-за неспособности правильно распознать и обработать сегменты `../` в именах файлов, приложение можно обманом заставить читать или писать файлы в произвольных местах на диске. Любые последствия зависят от дополнительных ограничений, но неограниченные ошибки записи файлов обычно легко эксплуатируются для запуска кода, предоставленного злоумышленником.

File inclusion

Если термин используется без квалификатора или с префиксом local (LFI), он в значительной степени синонимичен связанному с чтением directory traversal. Remote file inclusion (RFI), напротив, - альтернативный способ эксплуатации уязвимостей file inclusion через указание URL вместо допустимого пути к файлу. В некоторых скриптовых языках один общий API открывает локальные файлы и получает удаленные URL. В этих случаях возможность получить файл с сервера, контролируемого злоумышленником, может дать существенные преимущества в зависимости от того, как данные затем обрабатываются.

Format-string vulnerability

Ряд широко используемых библиотечных функций принимает шаблоны ("format strings"), за которыми следует набор параметров, которые функция должна вставить в шаблон в заранее заданных местах. Такой подход особенно распространен в C (`printf(...)`, `syslog(...)` и так далее), но не ограничивается этим языком. Format-string vulnerabilities вызваны непреднамеренным разрешением злоумышленникам предоставить шаблон одной из этих функций. В зависимости от возможностей системы шаблонов и особенностей языка эта ошибка может привести к чему угодно - от небольшой утечки данных до выполнения кода.

Integer overflow

Уязвимость, специфичная для языков с ограниченной проверкой диапазонов или без нее. Изъян вызван тем, что разработчик не обнаруживает, что целое число превысило максимально возможное значение и откатилось к нулю, к очень большому отрицательному целому или к какому-либо другому аппаратно-специфичному и неожиданному результату. В зависимости от того, как значение используется, это может привести программу в несогласованное состояние или, что хуже, привести к чтению или записи данных по неверному адресу памяти, что, в свою очередь, может позволить выполнение кода. Integer underflow - противоположный эффект: пересечение минимально допустимого значения и переход к очень большому положительному целому.

Pointer management vulnerabilities

В языках, которые поощряют или требуют использования сырых указателей памяти, главным образом C и C++, можно использовать указатели, которые либо не инициализированы, либо больше не действительны ("висячие"), что приводит к уязвимостям вроде use after free, double free и многим другим. Эти уязвимости повреждают внутреннее состояние программы и обычно позволяют злоумышленнику выполнить предоставленный им код.

Эпилог

Ну кто бы мог подумать. На этом *The Tangled Web* завершается! Надеюсь, вам понравилось читать эту книгу так же, как мне понравилось исследовать мир браузерной безопасности в течение последнего десятилетия или около того. Я также надеюсь, что то, что вы обнаружили на этих страницах, будет направлять вас в будущих путешествиях, куда бы они ни вели.

Что же со всем этим делать? Для меня резкий контраст между удивительной устойчивостью современного Web и необъяснимой ненадежностью его оснований поначалу было трудно примирить. Оглядываясь назад, я думаю, что он дает важное понимание нашего собственного, неразумного отношения к защите онлайн-мира.

Меня преследует неприятное наблюдение: в реальной жизни современные общества построены на поразительно шаткой почве. Каждый день каждый из нас зависит от здравомыслия, моральных норм и сдержанности тысяч случайных незнакомцев - от таксистов до продавцов еды и ремонтников лифтов. Правила игры слабо обеспечиваются серией механизмов сдерживания, но если верить статистике преступности, их эффективность поразительно низка. Проблема не только в том, что большинство мелких преступников думают, будто смогут уйти от ответственности за свои проступки, но и в том, что они обычно правы.

В этом смысле наш мир - не более чем невероятно сложная система чести, в которой большинство из нас добровольно соглашается участвовать. И, вероятно, это нормально: соблюдение самостоятельно принятых норм оказалось разумным эволюционным ходом, и сегодня оно является частью того, кто мы есть. Некоторая степень доверия просто необходима, чтобы наша цивилизация могла продвигаться вперед с разумной скоростью. Кроме того, как ни парадоксально, несмотря на краткосрочные слабости, ускоренный прогресс в долгосрочной перспективе делает всех нас намного сильнее и приспособляемее.

Тогда трудно понять, почему к нашему онлайн-существованию мы относимся настолько иначе. Например, почему мы злимся на разработчиков, которые неправильно используют криптографию, но не возражаем против того, что замки на наших дверях можно открыть английской булавкой? Почему мы презираем веб-разработчиков, которые не могут правильно организовать проверку входных данных, но не проверяем свой завтрак на слабительные или ЛСД?

Единственное объяснение, которое я вижу, состоит в том, что у человечества были тысячи лет, чтобы выработать правила социального взаимодействия в физическом мире. За это время рушились целые общества, возникали новые, и в процессе складывалась все более сложная система поведенческих норм, оптимизированная для сохранения сообществ. К несчастью для нас, нам трудно перенести эти правила в онлайн-экосистему, а этот мир настолько молод, что еще не успел выработать собственный, отдельный кодекс поведения.

Это явление легко увидеть: ваш сосед не станет пытаться тайком проникнуть в ваш дом, но может без особых сомнений пользоваться вашей беспроводной сетью, потому что это ощущается куда меньше похожим на преступление. Он может выступать против кражи, но относиться неоднозначно к незаконному копированию цифрового содержимого. Или он может хмуриться при виде грубых граффити по соседству, но посмеяться над изуродованным сайтом. Параллели есть, но они просто недостаточно хороши.

Что, если наше стремление к совершенству в мире информационной безопасности проистекает лишь из фундаментального непонимания того, как могут возникать и процветать человеческие сообщества? Эксперты моего сорта проповедуют модель сетевого существования, основанную на полном недоверии, но, возможно, ошибочно: по мере того как сложность наших онлайн-взаимодействий приближается к сложности реальной жизни, шансы спроектировать

идеально безопасное ПО стремительно уменьшаются. Тем временем крайняя паранойя начинает брать тяжелую плату с того, как быстро мы можем двигаться вперед.

Возможно, мы продаем рецепт неудачи. Что, если наша настойчивость в требовании абсолютной безопасности лишь приближает нас к судьбе множества других ранних цивилизаций, которые рухнули под тяжестью своих изъянов и в конце концов исчезли? Мне трудно отбросить эту извращенную мысль. К счастью, мы знаем, что из обломков однажды непременно возникнут новые, более просвещенные общества. Какова будет их окончательная природа - остается только гадать.

Примечания

Глава 1

1. D.E. Bell and L.J. La Padula, *Secure Computer System: Unified Exposition and Multics Interpretation* (ESD-TR-75-306), Bedford, MA: MITRE Corporation for US Air Force (1976), <http://csrc.nist.gov/publications/history/bell76.pdf>.
2. C.E. Landwehr, C.L. Heitmeyer, and J.D. McLean, "A Security Model for Military Message Systems: Retrospective," доклад на 17th Annual Computer Security Applications Conference, New Orleans, LA (2001), <http://www.acsa-admin.org/2001/papers/141.pdf>.
3. V. Bush, "As We May Think," *Atlantic Monthly* (July 1945), <http://www.theatlantic.com/doc/194507/bush/>.
4. R. Dhamija, J.D. Tygar, and M. Hearst, "Why Phishing Works," доклад на Conference on Human Factors in Computing Systems, Montreal, Canada (2006), http://people.seas.harvard.edu/~rachna/papers/why_phishing_works.pdf.
5. C. Jackson, D.R. Simon, D.S. Tan, and A. Barth, "An Evaluation of Extended Validation and Picture-in-Picture Phishing Attacks," доклад на Usable Security, Lowlands, Trinidad and Tobago (2007), <http://usablesecurity.org/papers/jackson.pdf>.
6. C. Jackson and A. Barth, "Beware of Finer-Grained Origins," доклад на Web 2.0 Security and Privacy, Oakland, CA (2008), <http://seclab.stanford.edu/websec/origins/fgo.pdf>; C. Jackson and A. Barth, "Beware of Coarser-Grained Origins," доклад на Web 2.0 Security and Privacy, Oakland, CA (2008), <http://seclab.stanford.edu/websec/origins/scheme/>.
7. "Security Exploit Uses Internet Explorer to Attack Mozilla Firefox," MozillaZine (July 11, 2007), <http://www.mozillazine.org/talkback.html?article=22198>.

Страница 19

1. Сайт Net Applications, <http://marketshare.hitslink.com/browser-market-share.aspx?qprid=0>, <http://marketshare.hitslink.com/browser-market-share.aspx?qprid=2> (accessed June 13, 2011).

Глава 2

1. T. Berners-Lee, R. Fielding, and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax," IETF Request for Comments 3986 (2005), <http://www.ietf.org/rfc/rfc3986.txt>.
2. T. Berners-Lee, L. Masinter, and M. McCahill, "Uniform Resource Locators (URL)," IETF Request for Comments 1738 (1994), <http://www.ietf.org/rfc/rfc1738.txt>.
3. R. Fielding, J. Gettys, J. Mogul, H. Frystyk, L. Masinter, P. Leach, and T. Berners-Lee, "Hypertext Transfer Protocol - HTTP/1.1," IETF Request for Comments 2616 (1999), <http://www.ietf.org/rfc/rfc2616.txt>.
4. "Uniform Resource Identifier (URI) Schemes per RFC4395," Internet Assigned Numbers Authority (June 6, 2011), <http://www.iana.org/assignments/uri-schemes.html>.

5. P. Mockapetris, "Domain Names - Implementation and Specification," IETF Request for Comments 1035 (1987), <http://www.ietf.org/rfc/rfc1035.txt>.
6. T. Berners-Lee, "Universal Resource Identifiers in WWW," IETF Request for Comments 1630 (1994), <http://www.w3.org/Addressing/rfc1630.txt>.
7. P. Hoffman, L. Masinter, and J. Zawinski, "The mailto URL Scheme," IETF Request for Comments 2368 (1998), <http://www.ietf.org/rfc/rfc2368.txt>.
8. "HTML 4.01 Specification: Forms," World Wide Web Consortium (1999), <http://www.w3.org/TR/html401/interact/forms.html#h-17.13.4.1>.
9. P. Faltstrom, P. Hoffman, and A. Costello, "Internationalizing Domain Names in Applications (IDNA)," IETF Request for Comments 3490 (2003), <http://www.ietf.org/rfc/rfc3490.txt>.
10. A. Costello, "Punycode: A Bootstring Encoding of Unicode for Internationalized Domain Names in Applications (IDNA)," IETF Request for Comments 3492 (2003), <http://www.ietf.org/rfc/rfc3492.txt>.
11. E. Gabrilovich and A. Gontmakher, "The Homograph Attack," *Communications of the ACM* (2002), http://www.cs.technion.ac.il/~gabr/papers/homograph_full.pdf.
12. E. Rescorla, "HTTP Over TLS," IETF Request for Comments 2818 (2000), <http://www.ietf.org/rfc/rfc2818.txt>.
13. J. Postel and J. Reynolds, "File Transfer Protocol (FTP)," IETF Request for Comments 959 (1985), <http://www.ietf.org/rfc/rfc959.txt>.
14. F. Anklesaria, M. McCahill, P. Lindner, D. Johnson, D. Torrey, and B. Alberti, "The Internet Gopher Protocol," IETF Request for Comments 1436 (1993), <http://www.ietf.org/rfc/rfc1436.txt>.
15. E. Rescorla and A. Schiffman, "The Secure HyperText Transfer Protocol," IETF Request for Comments 2660 (1999), <http://www.ietf.org/rfc/rfc2660.txt>.
16. L. Masinter, "The 'data' URL Scheme," IETF Request for Comments 2397 (1998), <http://www.ietf.org/rfc/rfc2397.txt>.
17. "What Are rss: and feed: Links?" <http://www.brindys.com/winrss/feedformat.html>.
18. M. Zalewski, "A Note on an MHTML Vulnerability," Lcamtuf's blog (March 11, 2011), <http://lcamtuf.blogspot.com/2011/03/note-on-mhtml-vulnerability.html>.

Глава 3

1. T. Berners-Lee, "The Original HTTP as defined in 1991." World Wide Web Consortium archives (1991), <http://www.w3.org/Protocols/HTTP/AsImplemented.html>.
2. T. Berners-Lee, R. Fielding, and H. Frystyk, "Hypertext Transfer Protocol - HTTP/1.0," IETF Request for Comments 1945 (1996), <http://www.ietf.org/rfc/rfc1945.txt>.
3. R. Fielding, J. Gettys, J. Mogul, H. Frystyk, L. Masinter, P. Leach, and T. Berners-Lee, "Hypertext Transfer Protocol - HTTP/1.1," IETF Request for Comments 2616 (1999), <http://www.ietf.org/rfc/rfc2616.txt>.
4. HTTPbis Working Group, "Httpbis Status Pages," <http://tools.ietf.org/wg/httpbis/>.
5. A. Luotonen, "Tunneling TCP-Based Protocols Through Web Proxy Servers," IETF draft (1998), <http://tools.ietf.org/id/draft-luotonen-web-proxy-tunneling-01.txt>.

6. S. Chen, Z. Mao, Y.M. Wang, and M. Zhang, "Pretty-Bad-Proxy: An Overlooked Adversary in Browsers' HTTPS Deployments," Microsoft Research (2009), <http://research.microsoft.com/pubs/79323/pbp-final-with-update.pdf>.
7. "Mozilla Cross-Reference mozilla1.8.0," репозиторий кода Mozilla, <http://mxr.mozilla.org/mozilla1.8.0/source/nsprpub/pr/src/misc/prtime.c#1045>.
8. K. Moore, "MIME (Multipurpose Internet Mail Extensions) Part Three: Message Header Extensions for Non-ASCII Text," IETF Request For Comments 2047 (1996), <http://www.ietf.org/rfc/rfc2047.txt>.
9. N. Freed and K. Moore, "MIME Parameter Value and Encoded Word Extensions: Character Sets, Languages, and Continuations," IETF Request for Comments 2231 (1997), <http://www.ietf.org/rfc/rfc2231.txt>.
10. Mozilla Bug Tracking System, Mozilla bug #418394, https://bugzilla.mozilla.org/show_bug.cgi?id=418394.
11. T. Berners-Lee, "Basic HTTP as defined in 1992: Methods," World Wide Web Consortium archives (1992), <http://www.w3.org/Protocols/HTTP/Methods.html>.
12. L. Dusseault, "HTTP Extensions for Web Distributed Authoring and Versioning (WebDAV)," IETF Request for Comments 4918 (2007), <http://www.ietf.org/rfc/rfc4918.txt>.
13. См. примечание 12 выше.
14. M. Pool, "Meantime: Non-Consensual HTTP User Tracking Using Caches" (2000), <http://sourcefrog.net/projects/meantime/>.
15. L. Montulli, "Persistent Client State HTTP Cookies" (1994), http://curl.haxx.se/rfc/cookie_spec.html.
16. D. Kristol and L. Montulli, "HTTP State Management Mechanism," IETF Request for Comments 2109 (1997), <http://www.ietf.org/rfc/rfc2109.txt>.
17. D. Kristol and L. Montulli, "HTTP State Management Mechanism," IETF Request for Comments 2965 (2000), <http://tools.ietf.org/rfc/rfc2965.txt>.
18. A. Barth, "HTTP State Management Mechanism," IETF Request for Comments 6265 (2011), <http://www.ietf.org/rfc/rfc6265.txt>.
19. J. Franks, P. Hallam-Baker, J. Hostetler, S. Lawrence, P. Leach, A. Luotonen, and L. Stewart, "HTTP Authentication: Basic and Digest Access Authentication," IETF Request for Comments 2617 (1999), <http://www.ietf.org/rfc/rfc2617.txt>.
20. R. Tschalär, "NTLM Authentication Scheme for HTTP" (2003), <http://www.innovation.ch/personal/ronald/ntlm.html>.
21. E. Rescorla, "HTTP Over TLS," IETF Request for Comments 2818 (2000), <http://www.ietf.org/rfc/rfc2818.txt>.
22. P. Hallam-Baker, "The Recent RA Compromise," Comodo IT Security (blog) (March 23, 2011), <http://blogs.comodo.com/it-security/data-security/the-recent-ra-compromise/>.
23. S. Chen, R. Wang, X. F. Wang, and K. Zhang, "Side-Channel Leaks in Web Applications: A Reality Today, a Challenge Tomorrow," Microsoft Research (2010), <http://research.microsoft.com/pubs/119060/WebAppSideChannel-final.pdf>.
24. C. Evans, "Open Redirectors: Some Sanity," Security: Hacking Everything (blog) (June 25, 2010), http://scarybeastsecurity.blogspot.com/2010_06_01_archive.html.

Глава 4

1. T. Berners-Lee, "HTML Tags," World Wide Web Consortium archives (1991), <http://www.w3.org/History/19921103-hypertext/hypertext/WWW/MarkUp/Tags.html>.
2. T. Berners-Lee and D. Connolly, "Hypertext Markup Language - 2.0," IETF Request for Comments 1866 (1995), <http://www.ietf.org/rfc/rfc1866.txt>.
3. D. Raggett, "HTML 3.2 Reference Specification," World Wide Web Consortium (1997), <http://www.w3.org/TR/REC-html32>.
4. D. Raggett, A. Le Hors, and I. Jacobs, "HTML 4.01 Specification," World Wide Web Consortium (1999), <http://www.w3.org/TR/html401/>.
5. I. Hickson, "HTML5," World Wide Web Consortium draft, revision 1.5019 (2011), <http://dev.w3.org/html5/spec/Overview.html>.
6. G. Coldwind, "Too general charset = detection in meta," Mozilla bug 640529 (2011), https://bugzilla.mozilla.org/show_bug.cgi?id=640529.

Глава 5

1. H. Wium Lie and B. Bos, "Cascading Style Sheets, Level 1," World Wide Web Consortium (1996), <http://www.w3.org/TR/CSS1/>.
2. T. Çelik, E.J. Etemad, D. Glazman, I. Hickson, P. Linss, and J. Williams, "Selectors Level 3: Selectors," World Wide Web Consortium (2009), <http://www.w3.org/TR/css3-selectors/#selectors>.
3. I. Hickson, "XML Binding Language (XBL) 2.0," World Wide Web Consortium (2007), <http://www.w3.org/TR/xbl/>.
4. G. Heyes, D. Lindsay, and E.V. Nava, "The Sexy Assassin: Tactical Exploitation Using CSS" (2009), <http://www.scribd.com/doc/54664700/Tactical-Xploit-Css>.

Глава 6

1. Netscape Communications Corporation, "Netscape and Sun Announce JavaScript, the Open, Cross-Platform Object Scripting Language for Enterprise Networks and the Internet" (press release) (December 4, 1995), <http://web.archive.org/web/20070916144913/http://wp.netscape.com/newsref/pr/newsrelease67.html>.
2. ECMA International, "ECMA-262: ECMAScript Language Specification," 3rd ed. (1999), <http://www.ecma-international.org/publications/files/ECMA-ST-ARCH/ECMA-262,%203rd%20edition,%20December%201999.pdf>.
3. ECMA International, "ECMA-262: ECMAScript Language Specification," 5th ed. (2009), <http://www.ecma-international.org/publications/files/ECMA-ST/ECMA-262.pdf>.
4. D. Crockford, "The Application/JSON Media Type for JavaScript Object Notation (JSON)," IETF Request for Comments 4627 (2006), <http://www.ietf.org/rfc/rfc4627.txt>.
5. J. Schneider, R. Yu, and J. Dyer, eds., "Standard ECMA-357: ECMAScript for XML (E4X) Specification," 2nd ed., ECMA International (2005), <http://www.ecma-international.org/publications/standards/Ecma-357.htm>.

6. P. Le Hégarret, R. Whitmer, and L. Wood, "Document Object Model (DOM)," World Wide Web Consortium (2005), <http://www.w3.org/DOM/>.
7. E. Vela Nava, "Bug 38922 - innerHTML decompilation issues in textarea" (WebKit bug-tracking system post) (2010), https://bugs.webkit.org/show_bug.cgi?id=38922.
8. "Windows Scripting 5.8: MsgBox Function," Microsoft Developer Network Platforms (2009), <http://msdn.microsoft.com/en-us/library/sfw6660x%28v=vs.85%29.aspx>.
9. D. Crockford, "JSON in JavaScript," GitHub Social Coding (blog) (March 5, 2011), <https://github.com/douglascrockford/JSON-js/blob/master/json2.js>.

Глава 7

1. J. Ferraiolo, F. Jun, and D. Jackson, "Scalable Vector Graphics (SVG) 1.1 Specification," World Wide Web Consortium (2003), <http://www.w3.org/TR/2003/REC-SVG11-20030114/>.
2. D. Carlisle, P. Ion, and R. Miner, "Mathematical Markup Language (MathML) Version 3.0," World Wide Web Consortium WC3 Recommendation 21 (2010), <http://www.w3.org/TR/MathML3/>.
3. A. Mechelynck, "XUL," Mozilla Developer Network (2011), <https://developer.mozilla.org/en/xul>.
4. Wireless Application Protocol Forum, "Wireless Application Protocol: Wireless Markup Language Specification version 3.0" (1998), <http://www.wapforum.org/what/technical/wml-30-apr-98.pdf>.
5. RSS Advisory Board, "RSS 2.0 Specification version 2.0.11" (2009), <http://www.rssboard.org/rss-specification>.
6. M. Nottingham and R. Sayre, eds., "The Atom Syndication Format," IETF Request for Comments 4287 (2005), <http://www.ietf.org/rfc/rfc4287.txt>.

Глава 8

1. E. Mills, "Security Labs Report: January-June 2010 Recap," M86 Security (2010), http://www.m86security.com/documents/pdfs/security_labs/m86_security_labs_report_1H2010.pdf.
2. B. Rios, "Sun Fixes GIFARs" (December 17, 2008), <http://xs-sniper.com/blog/2008/12/17/sun-fixes-gifars/>.
3. A.K. Sood, "PDF Silent HTTP Form Repurposing Attacks," SecNiche Security Labs (2009), http://secniche.org/papers/SNS_09_03_PDF_Silent_Form_Re_Purp_Attack.pdf.
4. P.D. Petkov, "Universal PDF XSS Afterparty" (January 4, 2007), <http://www.gnucitizen.org/blog/universal-pdf-xss-after-party/>.
5. S. Jobs, "Thoughts on Flash" (2010), <http://www.apple.com/hotnews/thoughts-on-flash/>.
6. "Adobe Shockwave Player," Adobe Systems Incorporated, <http://www.adobe.com/products/shockwaveplayer/>.
7. "ActionScript 3.0 Reference for the Adobe Flash Platform," Adobe Systems Incorporated, http://help.adobe.com/en_US/FlashPlatform/reference/actionscript/3/index.html.
8. "Content Played Back in Flash Player Reaches 99% of Internet Viewers," Adobe Systems Incorporated (March 2011), http://www.adobe.com/products/player_census/flashplayer/.

9. "Web Browser Plugin Market Share," StatOwl (May 2011), http://www.statowl.com/plugin_overview.php.
10. "ActionScript 3.0 Reference for the Adobe Flash Platform: ExternalInterface," Adobe Systems Incorporated, <http://livedocs.adobe.com/flex/3/langref/flash/external/ExternalInterface.html#includeExamplesSummary>.
11. "XAML Overview (WPF)," Microsoft Corporation, <http://msdn.microsoft.com/en-us/library/ms752059.aspx>.
12. "Rich Internet Application Statistics" (July 2011), <http://www.riastats.com/>. См. также StatOwl (глава 8, примечание 9).
13. "Secunia Half Year Report 2010," Secunia (2010), http://secunia.com/gfx/pdf/Secunia_Half_Year_Report_2010.pdf.
14. "WPF XAML Browser Applications Overview," Microsoft Corporation, <http://msdn.microsoft.com/en-us/library/aa970060.aspx>.
15. "Akamai Download Manager Help," Microsoft Corporation, <https://msdn.microsoft.com/en-us/subscriptions/manage/bb153537.aspx>.

Глава 9

1. A. Klein, "IE + Some Popular Forward Proxy Servers = XSS, Defacement (Browser Cache Poisoning)" (May 22, 2006), <http://seclists.org/webappsec/2006/q2/352>; M. Zalewski, "Web 2.0 Backdoors Made Easy with MSIE & XMLHttpRequest" (February 3, 2007), <http://seclists.org/fulldisclosure/2007/Feb/81>.
2. A. van Kesteren, ed., "Cross-Origin Resource Sharing," working draft, World Wide Web Consortium (July 27, 2010), <http://www.w3.org/TR/cors/>.
3. I. Hickson, "Web Storage," editor's draft, World Wide Web Consortium (July 28, 2011), <http://dev.w3.org/html5/webstorage/>.
4. J. Stenback, "Make sessionStorage Use Principals Instead of String Domains," Mozilla bug #495337 (May 28, 2009), https://bugzilla.mozilla.org/show_bug.cgi?id=495337.
5. T. Ormandy, "Common DNS Misconfiguration Can Lead to 'Same Site' Scripting" (January 18, 2008), <http://seclists.org/bugtraq/2008/Jan/270>.
6. R. Singel, "ISPs' Error Page Ads Let Hackers Hijack Entire Web, Researcher Discloses," *Wired* (April 19, 2008), <http://www.wired.com/threatlevel/2008/04/isps-error-page/>.
7. "APSB10-14 Security Update Available for Adobe Flash Player," Adobe Systems Incorporated (June 10, 2010), <http://www.adobe.com/support/security/bulletins/apsb10-14.html>.
8. "Understanding Flash Player 9 April 2008: Security Update Compatibility," Adobe Systems Incorporated (April 8, 2008), http://www.adobe.com/devnet/flashplayer/articles/flash_player9_security_update.html.
9. "ActionScript 3.0 Reference for the Adobe Flash Platform: URLRequestHeader," Adobe Systems Incorporated, http://help.adobe.com/en_US/FlashPlatform/reference/actionscript/3/flash/net/URLRequestHeader.html.
10. "ActionScript 3.0 Reference for the Adobe Flash Platform: Security," Adobe Systems Incorporated, <http://livedocs.adobe.com/flash/9.0/ActionScriptLangRefV3/flash/system/Security.html>.
11. "Adobe Cross Domain Policy File Specification," version 2.0, Adobe Systems Incorporated (August 2, 2010), http://learn.adobe.com/wiki/download/attachments/64389123/CrossDomain_PolicyFile_Specification.pdf?version=1.

12. M. Zalewski, "[RAZOR] Linux Kernel IP Masquerading Vulnerability" (July 30, 2001), <http://seclists.org/bugtraq/2001/Jul/733>.
13. "Silverlight: WebHeaderCollection Class," Microsoft, <http://msdn.microsoft.com/en-us/library/system.net.webheadercollection%28v=VS.95%29.aspx>.
14. "Class HttpURLConnection," Sun Microsystems/Oracle, <http://download.oracle.com/javase/1.4.2/docs/api/java/net/HttpURLConnection.html>.
15. "Class URLConnection," Sun Microsystems/Oracle, <http://download.oracle.com/javase/1.4.2/docs/api/java/net/URLConnection.html>.
16. "Class Socket," Sun Microsystems/Oracle, <http://download.oracle.com/javase/1.4.2/docs/api/java/net/Socket.html>.
17. "Java-to-Javascript Communication," Sun Microsystems/Oracle, http://download.oracle.com/javase/1.4.2/docs/guide/plugin/developer_guide/java_js.html.
18. "Java-to-Javascript Communication: Common DOM API," Sun Microsystems/Oracle, http://download.oracle.com/javase/1.4.2/docs/guide/plugin/developer_guide/java_js.html#common_dom.
19. B. "Snowhare" Franz, "Triple Dot Cookies" (1998), http://snowhare.com/utilities/triple_dot/.
20. "Adobe ActionScript 3.0: Security Sandboxes," Adobe Systems Incorporated, http://help.adobe.com/en_US/ActionScript/3.0_ProgrammingAS3/WS5b3ccc516d4fbf351e63e3d118a9b90204-7e3f.html.

Глава 10

1. L. Masinter, "The 'data' URL Scheme," IETF Request for Comments 2397 (1998), <http://www.ietf.org/rfc/rfc2397.txt>.
2. M. Zalewski, "about:neterror, certerror permit URL spoofing by being same-origin with about:blank," Mozilla bug #602780 (CVE-2010-3774) (2010), https://bugzilla.mozilla.org/show_bug.cgi?id=602780.

Глава 11

1. G. Guninski, "Frame spoofing using loading two frames," Mozilla bug #13871 (1999), https://bugzilla.mozilla.org/show_bug.cgi?id=13871.
2. R. Zilberman, "Frame spoofing is possible within a short time frame while the window is loading," Mozilla bug #381300 (CVE-2007-3089) (2008), https://bugzilla.mozilla.org/show_bug.cgi?id=381300.
3. A. Barth, C. Jackson, and J.C. Mitchell, "Securing Frame Communication in Browsers," *Communications of the ACM* 52, no. 6 (2009): 83-91, <http://www.adambarth.com/papers/2009/barth-jackson-mitchell-cacm.pdf>.
4. R. Hansen and J. Grossman, "Clickjacking" (2008), <http://www.sectheory.com/clickjacking.htm>.
5. M. Zalewski, "Dealing with UI redress vulnerabilities inherent to the current web" (post to whatwg.org list) (September 25, 2008), <http://lists.whatwg.org/htdig.cgi/whatwg-whatwg.org/2008-September/thread.html#16292>.
6. E. Lawrence, "IE8 Security Part VII: ClickJacking Defenses," IEBlog (January 27, 2009), <http://blogs.msdn.com/b/ie/archive/2009/01/27/ie8-security-part-vii-clickjacking-defenses.aspx>.

7. SHODAN, "HTTP Header Survey" (March 14, 2011),
<http://www.shodanhq.com/research/infodisc/report>.
8. P. Stone, "Next Generation Clickjacking," Blackhat Europe (2010),
<http://blog.c22.cc/2010/04/14/blackhat-europe-next-generation-clickjacking-3/>.
9. M. Zalewski, "The curse of inverse strokejacking," Icamtuf's blog (June 8, 2010),
<http://lcamtuf.blogspot.com/2010/06/curse-of-inverse-strokejacking.html>.
10. C. Evans, "Generic cross-browser cross-domain theft," Security (blog) (December 28, 2009),
<http://scarybeastsecurity.blogspot.com/2009/12/generic-cross-browser-cross-domain.html>.
11. C. Evans, "IE8 CSS-based forced tweeting," Security (blog) (September 29, 2010),
<http://scarybeastsecurity.blogspot.com/2010/09/ie8-css-based-forced-tweeting.html>.
12. I. Hickson, "HTML: 4.8.11 The canvas element," WHATWG (2011),
<http://www.whatwg.org/specs/web-apps/current-work/multipage/the-canvas-element.html>.
13. E.W. Felten and M.A. Schneider, "Timing Attacks on Web Privacy," *Proceedings of the 7th ACM Conference on Computer and Communications Security* (2000),
<http://sip.cs.princeton.edu/pub/webtiming.pdf>.
14. C. Evans, "Cross-domain search timing," Security (blog) (December 11, 2009),
<http://scarybeastsecurity.blogspot.com/2009/12/cross-domain-search-timing.html>.
15. C. Wilson, P. Le Hégarret, and V. Apparao, "Document Object Model CSS: 2.2.1 Override and computed style sheet," World Wide Web Consortium (2000),
<http://www.w3.org/TR/DOM-Level-2-Style/css.html#CSS-OverrideAndComputed>.
16. "currentStyle Object," Microsoft Corporation MSDN Library,
<http://msdn.microsoft.com/en-us/library/ms535231%28v=vs.85%29.aspx>.
17. A. Clover, "CSS visited pages disclosure" (February 20, 2002),
<http://seclists.org/bugtraq/2002/Feb/271>.
18. Z. Weinberg, E.Y. Chen, P.R. Jayaraman, and C. Jackson, "I Still Know What You Visited Last Summer" (2011), <http://websec.sv.cmu.edu/visited/visited.pdf>.

Глава 12

1. J. Schwartz, "Giving Web a Memory Cost Its Users Privacy," *New York Times* (September 4, 2001),
<http://www.nytimes.com/2001/09/04/technology/04COOK.html>.
2. N. Wingfield, "Microsoft Quashed Effort to Boost Online Privacy," *Wall Street Journal* (August 2, 2010),
<http://online.wsj.com/article/SB10001424052748703467304575383530439838568.html>.
3. E. Felten, "If You're Going to Track Me, Please Use Cookies," Freedom to Tinker (blog) (July 7, 2009),
<http://www.freedom-to-tinker.com/blog/felten/if-youre-going-track-me-please-use-cookies>.
4. J. Mayer, A. Narayanan, and S. Stamm, "Do Not Track: A Universal Third-Party Web Tracking Opt Out," IETF Request for Comments (2011),
http://datatracker.ietf.org/doc/draft-mayer-do-not-track/?include_text=1.
5. L. Cranor, M. Langheinrich, M. Marchiori, M. Presler-Marshall, and J. Reagle, "The Platform for Privacy Preferences 1.0 (P3P1.0) Specification," World Wide Web Consortium (2002),
<http://www.w3.org/TR/P3P/>.

Глава 13

1. N. Freed and N. Borenstein, "Multipurpose Internet Mail Extensions (MIME) Part Two: Media Types," IETF Request for Comments 2046 (1996), <http://www.ietf.org/rfc/rfc2046.txt>.
2. V. Gupta, "IE Content-Type Logic," IEBlog (February 1, 2005), <http://blogs.msdn.com/b/ie/archive/2005/02/01/364581.aspx>.
3. SHODAN, "HTTP Header Survey" (2011), http://www.shodanhq.com/research/infodisc/download_latest.
4. R. Troost, S. Dorner, and K. Moore, "Communicating Presentation Information in Internet Messages: The Content-Disposition Header Field," IETF Request for Comments 2183 (1997), <http://www.ietf.org/rfc/rfc2183.txt>.
5. G. Heyes, "Inline UTF-7 E4X Javascript Hijacking," The Spanner (blog) (February 24, 2009), <http://www.thespanner.co.uk/2009/02/24/inline-utf-7-e4x-javascript-hijacking/>.

Глава 14

1. M. Zalewski, "URL Spoofing Is Likely Possible Through Address Bar Eliding" (2010), https://bugzilla.mozilla.org/show_bug.cgi?id=581313.
2. R. J. Kosinski, "A Literature Review on Reaction Time," Clemson University (2010), <http://biae.clemson.edu/bpc/bp/Lab/110/reaction.htm#Type%20of%20Stimulus>.
3. M. Zalewski, "Bug 376473: File Action Dialog Controls Vulnerable to Refocus Race" (2007), https://bugzilla.mozilla.org/show_bug.cgi?id=376473.
4. M. Zalewski, "Geolocation Spoofing and Other UI Woes," Bugtraq (mailing list) (August 17, 2010), <http://seclists.org/bugtraq/2010/Aug/201>.
5. D. Simons and C. Chabris, "Selective Attention Test" (1999), http://www.youtube.com/watch?v=vJG698U2Mvo&feature=player_embedded.
6. D.J. Simmons and C.F. Chabris, "Gorillas in our midst: Sustained inattentive blindness for dynamic events," *Perception*, 28, 1059-1074 (1999), http://www.cnbc.cmu.edu/~behrmann/dlpapers/Simons_Chabris.pdf.

Глава 15

1. <http://www.xssed.com/search?key=addons.mozilla.org>
2. <http://openid.net/>
3. "Internet Explorer: Security Zones," Microsoft, <http://technet.microsoft.com/en-us/library/dd361896.aspx>.
4. "Internet Explorer Binary Behaviors Security Setting," Microsoft, [http://technet.microsoft.com/en-us/library/cc776248\(WS.10\).aspx](http://technet.microsoft.com/en-us/library/cc776248(WS.10).aspx).
5. Charles Schwab, "Technical Support," http://www.visualwebcaster.com/charles_schwab/support/ (accessed September 9, 2011).
6. Internal Revenue Service, "Streaming Media System Requirements & Troubleshooting Assistance," http://www.irsvideos.gov/sbv_1099webinar/player/IRS_Webinar_Technical_Support.pdf (accessed September 9, 2011).

7. ".NET Framework 3.0: Mark of the Web," Microsoft,
<http://msdn.microsoft.com/en-us/library/ms537628%28VS.85%29.aspx>.
8. "Persistent Zone Identifier Object," Microsoft,
<http://msdn.microsoft.com/en-us/library/ms537029%28VS.85%29.aspx>.

Глава 16

1. A. van Kesteren, "Cross-Origin Resource Sharing," working draft, World Wide Web Consortium (July 27, 2010), <http://www.w3.org/TR/cors/>.
2. S. Dutta, "Updates for AJAX in IE8 Beta 2," IEBlog (2008),
<http://blogs.msdn.com/b/ie/archive/2008/10/06/updates-for-ajax-in-ie8-beta-2.aspx>.
3. ".NET Framework 3.0: XMLHttpRequest Object," Microsoft Developer Network,
<http://msdn.microsoft.com/en-us/library/cc288060%28v=vs.85%29.aspx>.
4. T. Close and M. Miller, "Uniform Messaging Policy, Level One," working draft, World Wide Web Consortium (January 26, 2010), <http://www.w3.org/TR/UMP/>.
5. A. Barth, C. Jackson, and J.C. Mitchell, "Robust Defenses for Cross-Site Request Forgery," ACM Conference on Computer and Communications Security (2008),
<http://seclab.stanford.edu/websec/csrf/csrf.pdf>.
6. B. Sterne, "Origin Header Proposal,"
<http://people.mozilla.com/~bsterne/content-security-policy/origin-header-proposal.html>.
7. A. van Kesteren, "The From-Origin Header," working draft, World Wide Web Consortium (July 21, 2011), <http://www.w3.org/TR/2011/WD-from-origin-20110721/>.
8. A. Barth, "The Web Origin Concept (v. 9)," IETF Draft (November 26, 2010),
<http://tools.ietf.org/html/draft-abarth-origin-09>.
9. B. Sterne, "Content Security Policy" (2008), <http://people.mozilla.com/~bsterne/content-security-policy/>.
10. B. Sterne, "Content Security Policy," draft, World Wide Web Consortium (March 15, 2011),
<https://dvcs.w3.org/hg/content-security-policy/raw-file/tip/csp-unofficial-draft-20110315.html>.
11. I. Hickson, "HTML Living Standard," WHATWG (2011), <http://www.whatwg.org/specs/web-apps/current-work/multipage/the-iframe-element.html#attr-iframe-sandbox>.
12. J. Hodges, C. Jackson, and A. Barth, "HTTP Strict Transport Security (HSTS)," draft, IETF Request for Comments (August 5, 2011), <http://tools.ietf.org/html/draft-ietf-websec-strict-transport-sec-02>.
13. A. Klein, "Google Chrome 6.0 and Above: Math.random Vulnerability" (2010),
http://www.trusteer.com/sites/default/files/Google_Chrome_6.0_and_7.0_Math.random_vulnerability.pdf.
14. ".NET Framework 3.0: toStaticHTML Method," Microsoft,
<http://msdn.microsoft.com/en-us/library/cc848922%28v=vs.85%29.aspx>.
15. D. Ross, "IE8 Security Part IV: The XSS Filter," IEBlog (2008),
<http://blogs.msdn.com/b/ie/archive/2008/07/02/ie8-security-part-iv-the-xss-filter.aspx>.
16. E. Vela Nava and D. Lindsay, "Abusing Internet Explorer 8's XSS Filters" (2009),
http://p42.us/ie8xss/Abusing_IE8s_XSS_Filters.pdf.

Глава 17

1. "navigator.registerProtocolHandler," Mozilla Developer Network, <https://developer.mozilla.org/en/DOM/window.navigator.registerProtocolHandler>.
2. "Manipulating the Browser History," Mozilla Developer Network, https://developer.mozilla.org/en/DOM/Manipulating_the_browser_history/.
3. A. Langley and M. Belsche, "SPDY: An Experimental Protocol for a Faster Web," The Chromium Projects, <http://www.chromium.org/spdy/spdy-whitepaper/>.
4. I. Fette and A. Melnikov, "The WebSocket Protocol," IETF Request for Comments draft (2011), <http://tools.ietf.org/html/draft-ietf-hybi-thewebsocketprotocol-10/>.
5. J. Rosenberg, M. Kaufman, M. Hiie, and F. Audet, "An Architectural Framework for Browser Based Real-Time Communications," IETF Request for Comments draft (2011), <http://tools.ietf.org/html/draft-rosenberg-rtcweb-framework-00/>.
6. I. Hickson, "HTML5: 5.6 - Offline Web Applications," World Wide Web Consortium (2011), <http://www.w3.org/TR/html5/offline.html>.
7. A. Barth, "Simple HTTP State Management Mechanism," IETF Request for Comments draft (2010), <http://tools.ietf.org/html/draft-abarth-cache-00/>.
8. I. Hickson, "Web SQL Database: W3C Working Group Note 18," World Wide Web Consortium (2010), <http://www.w3.org/TR/webdatabase/>.
9. N. Mehta, J. Sicking, E. Graff, A. Popescu, and J. Orlow, "Indexed Database API: W3C Working Draft 19," World Wide Web Consortium (2011), <http://www.w3.org/TR/IndexedDB/>.
10. I. Hickson, "Web Applications 1.0: Web Workers," WHATWG (2011), <http://www.whatwg.org/specs/web-apps/current-work/complete/workers.html>.
11. A. Popescu, "Geolocation API Specification: Editor's Draft," World Wide Web Consortium (February 10, 2010), <http://dev.w3.org/geo/api/spec-source.html>.
12. "Detecting Device Orientation," Mozilla Developer Network, https://developer.mozilla.org/en/detecting_device_orientation/.
13. L. Cai and H. Chen, "TouchLogger: Inferring Keystrokes on Touch Screen from Smartphone Motion," Usenix HOTSEC (2011), http://www.usenix.org/event/hotsec11/tech/final_files/Cai.pdf.
14. "Web Developer's Guide to Prerendering in Chrome," Google code labs, <http://code.google.com/chrome/whitepapers/prerender.html>.
15. Z. Wang, "Navigation Timing: Editor's Draft," World Wide Web Consortium (July 27, 2011), <https://dvcs.w3.org/hg/webperf/raw-file/tip/specs/NavigationTiming/Overview.html>.
16. J. Gregg, "Web Notifications Overview: W3C Editor's Draft," World Wide Web Consortium (October 12, 2010), <http://dev.w3.org/2006/webapi/WebNotifications/publish/>.
17. D.D. Tran, I. Oksanen, and I. Kliche, "The Media Capture API: W3C Working Draft," World Wide Web Consortium (September 28, 2010), <http://www.w3.org/TR/media-capture-api/>.

Финальные издательские страницы

Еще прямые и практичные книги от No Starch Press

A Bug Hunter's Diary

A Guided Tour Through the Wilds of Software Security

Tobias Klein. Ноябрь 2011, 208 стр., \$39.95. ISBN 978-1-59327-385-9.

Hacking, 2nd Edition

The Art of Exploitation

Jon Erickson. Февраль 2008, 488 стр. с CD, \$49.95. ISBN 978-1-59327-144-2.

The IDA Pro Book, 2nd Edition

The Unofficial Guide to the World's Most Popular Disassembler

Chris Eagle. Июль 2011, 672 стр., \$69.95. ISBN 978-1-59327-289-0.

Metasploit

The Penetration Tester's Guide

David Kennedy, Jim O'Gorman, Devon Kearns и Mati Aharoni. Июль 2011, 328 стр., \$49.95. ISBN 978-1-59327-288-3.

Practical Packet Analysis, 2nd Edition

Using Wireshark to Solve Real-World Network Problems

Chris Sanders. Июль 2011, 280 стр., \$49.95. ISBN 978-1-59327-266-1.

Practical Malware Analysis

The Hands-On Guide to Dissecting Malicious Software

Michael Sikorski и Andrew Honig. Январь 2012, 760 стр., \$59.95. ISBN 978-1-59327-290-6.

Обновления

Посетите <http://nostarch.com/tangledweb.htm>, чтобы получить обновления, исправления и другую информацию.

Телефон: 800.420.7240 или 415.863.9900

Электронная почта: sales@nostarch.com

Сайт: www.nostarch.com

Задняя обложка

\$49.95 (\$52.95 CDN)

Раздел: Computers / Security

Лучшее в развлечениях для гиков

www.nostarch.com

"Я лежу ровно." В этой книге используется RepKover - прочный переплет, который не захлопывается сам.

"Подробный и всесторонний охват от одного из ведущих экспертов по безопасности браузеров." - Tavis Ormandy, Google Inc.

Современные веб-приложения построены на клубке технологий, которые развивались со временем, а затем кое-как были соединены друг с другом. Каждая часть стека веб-приложений, от HTTP-запросов до браузерных скриптов, имеет важные, но тонкие последствия для безопасности. Чтобы пользователи оставались в безопасности, разработчикам необходимо уверенно ориентироваться в этом ландшафте.

В *The Tangled Web* Михал Залевски, один из ведущих мировых экспертов по безопасности браузеров, предлагает увлекательное изложение, которое объясняет, как именно работают браузеры и почему они фундаментально небезопасны. Вместо того чтобы раздавать упрощенные советы об уязвимостях, Залевски рассматривает всю модель безопасности браузера, выявляя слабые места и предоставляя важнейшую информацию для укрепления безопасности веб-приложений. Вы узнаете, как:

- Выполнять распространенные, но неожиданно сложные задачи, такие как разбор URL и очистка HTML.
- Использовать современные функции безопасности, такие как Strict Transport Security, Content Security Policy и Cross-Origin Resource Sharing.
- Использовать множество вариантов политики одного источника, чтобы безопасно разделять сложные веб-приложения на изолированные части и защищать учетные данные пользователей в случае XSS-ошибок.
- Строить mashups и встраивать gadgets, не попадаясь на хитрую политику навигации фреймов.

- Встраивать или размещать содержимое, предоставленное пользователями, не попадая в ловушку распознавания содержимого.

Для быстрого справочника "Security Engineering Cheat Sheets" в конце каждой главы предлагают готовые решения проблем, с которыми вы, скорее всего, столкнетесь. Благодаря охвату вплоть до запланированных возможностей HTML5 *The Tangled Web* поможет вам создавать безопасные веб-приложения, способные выдержать проверку временем.

Об авторе

Михал Залевски - международно признанный эксперт по информационной безопасности с долгой историей передовых исследований. На его счету обнаружение сотен заметных уязвимостей безопасности, и он часто появляется в списках самых влиятельных экспертов по безопасности. Он автор *Silence on the Wire* (No Starch Press), *Browser Security Handbook* от Google и многочисленных важных исследовательских работ.

The Tangled Web

Zalewski

A Guide to Securing Modern Web Applications

Michal Zalewski

A Guide to Securing Modern Web Applications